

Modern .NET Development (2026)

A Guide for Experienced C# Developers

Steve Snively

2026

Copyright © 2026 Steve Snively

Contents

1	The Modern .NET Landscape	11
1.1	Chapter Purpose	11
1.2	Where This Fits	11
1.3	Connection to the Reader's Existing Model	12
1.4	Layer 1 — Conceptual Model	13
1.5	Layer 2 — System Relationships	13
1.6	Layer 3 — Core Mechanics	14
1.7	Layer 4 — Developer Workflow	14
1.8	Layer 5 — Production Usage	15
1.9	Layer 6 — Tradeoffs and Alternatives	16
1.10	Layer 7 — Interview Perspective	17
1.11	Hands-On Lab	18
1.12	Knowledge Check	19
1.13	Summary	19
1.14	Sources	20
1.15	Further Reading	20
2	The Modern Technology Stack	21
2.1	Chapter Purpose	21
2.2	Where This Fits	21
2.3	Connection to the Reader's Existing Model	22
2.4	Layer 1 — Conceptual Model	23
2.5	Layer 2 — System Relationships	23
2.6	Layer 3 — Core Mechanics	24
2.7	Layer 4 — Developer Workflow	25
2.8	Layer 5 — Production Usage	26
2.9	Layer 6 — Tradeoffs and Alternatives	26
2.10	Layer 7 — Interview Perspective	27
2.11	Hands-On Lab	29
2.12	Knowledge Check	30
2.13	Summary	30
2.14	Sources	30
2.15	Further Reading	31
3	Modern Development Workflow	32
3.1	Chapter Purpose	32
3.2	Where This Fits	32
3.3	Connection to the Reader's Existing Model	33
3.4	Layer 1 — Conceptual Model	34
3.5	Layer 2 — System Relationships	34
3.6	Layer 3 — Core Mechanics	35
3.7	Layer 4 — Developer Workflow	36
3.8	Layer 5 — Production Usage	37
3.9	Layer 6 — Tradeoffs and Alternatives	37
3.10	Layer 7 — Interview Perspective	38
3.11	Hands-On Lab	39

3.12	Knowledge Check	41
3.13	Summary	41
3.14	Sources	41
3.15	Further Reading	42
4	Modern .NET	43
4.1	Chapter Purpose	43
4.2	Where This Fits	43
4.3	Connection to the Reader's Existing Model	44
4.4	Layer 1 — Conceptual Model	44
4.5	Layer 2 — System Relationships	45
4.6	Layer 3 — Core Mechanics	45
4.7	Layer 4 — Developer Workflow	47
4.8	Layer 5 — Production Usage	47
4.9	Layer 6 — Tradeoffs and Alternatives	48
4.10	Layer 7 — Interview Perspective	49
4.11	Hands-On Lab	50
4.12	Knowledge Check	51
4.13	Summary	52
4.14	Sources	52
4.15	Further Reading	52
5	ASP.NET Core	53
5.1	Chapter Purpose	53
5.2	Where This Fits	53
5.3	Connection to the Reader's Existing Model	54
5.4	Layer 1 — Conceptual Model	54
5.5	Layer 2 — System Relationships	55
5.6	Layer 3 — Core Mechanics	56
5.7	Layer 4 — Developer Workflow	57
5.8	Layer 5 — Production Usage	58
5.9	Layer 6 — Tradeoffs and Alternatives	59
5.10	Layer 7 — Interview Perspective	60
5.11	Hands-On Lab	61
5.12	Knowledge Check	62
5.13	Summary	62
5.14	Sources	62
5.15	Further Reading	63
6	Data Access	64
6.1	Chapter Purpose	64
6.2	Where This Fits	64
6.3	Connection to the Reader's Existing Model	65
6.4	Layer 1 — Conceptual Model	65
6.5	Layer 2 — System Relationships	66
6.6	Layer 3 — Core Mechanics	67
6.7	Layer 4 — Developer Workflow	69
6.8	Layer 5 — Production Usage	70
6.9	Layer 6 — Tradeoffs and Alternatives	70
6.10	Layer 7 — Interview Perspective	71
6.11	Hands-On Lab	72

6.12	Knowledge Check	73
6.13	Summary	74
6.14	Sources	74
6.15	Further Reading	74
7	Authentication and Security	75
7.1	Chapter Purpose	75
7.2	Where This Fits	75
7.3	Connection to the Reader's Existing Model	76
7.4	Layer 1 — Conceptual Model	76
7.5	Layer 2 — System Relationships	77
7.6	Layer 3 — Core Mechanics	77
7.7	Layer 4 — Developer Workflow	79
7.8	Layer 5 — Production Usage	80
7.9	Layer 6 — Tradeoffs and Alternatives	81
7.10	Layer 7 — Interview Perspective	81
7.11	Hands-On Lab	82
7.12	Knowledge Check	83
7.13	Summary	83
7.14	Sources	84
7.15	Further Reading	84
8	Automated Testing and CI Basics	85
8.1	Chapter Purpose	85
8.2	Where This Fits	85
8.3	Connection to the Reader's Existing Model	86
8.4	Layer 1 — Conceptual Model	86
8.5	Layer 2 — System Relationships	87
8.6	Layer 3 — Core Mechanics	87
8.7	Layer 4 — Developer Workflow	89
8.8	Layer 5 — Production Usage	90
8.9	Layer 6 — Tradeoffs and Alternatives	90
8.10	Layer 7 — Interview Perspective	91
8.11	Hands-On Lab	92
8.12	Knowledge Check	93
8.13	Summary	94
8.14	Sources	94
8.15	Further Reading	94
9	Linux for .NET Developers	95
9.1	Chapter Purpose	95
9.2	Where This Fits	95
9.3	Connection to the Reader's Existing Model	96
9.4	Layer 1 — Conceptual Model	96
9.5	Layer 2 — System Relationships	97
9.6	Layer 3 — Core Mechanics	97
9.7	Layer 4 — Developer Workflow	98
9.8	Layer 5 — Production Usage	99
9.9	Layer 6 — Tradeoffs and Alternatives	100
9.10	Layer 7 — Interview Perspective	100
9.11	Hands-On Lab	101

9.12	Knowledge Check	103
9.13	Summary	103
9.14	Sources	103
9.15	Further Reading	103
10	Docker	104
10.1	Chapter Purpose	104
10.2	Where This Fits	104
10.3	Connection to the Reader's Existing Model	105
10.4	Layer 1 — Conceptual Model	105
10.5	Layer 2 — System Relationships	106
10.6	Layer 3 — Core Mechanics	106
10.7	Layer 4 — Developer Workflow	107
10.8	Layer 5 — Production Usage	108
10.9	Layer 6 — Tradeoffs and Alternatives	109
10.10	Layer 7 — Interview Perspective	109
10.11	Hands-On Lab	111
10.12	Knowledge Check	112
10.13	Summary	112
10.14	Sources	112
10.15	Further Reading	113
11	Docker Compose	114
11.1	Chapter Purpose	114
11.2	Where This Fits	114
11.3	Connection to the Reader's Existing Model	115
11.4	Layer 1 — Conceptual Model	115
11.5	Layer 2 — System Relationships	116
11.6	Layer 3 — Core Mechanics	116
11.7	Layer 4 — Developer Workflow	117
11.8	Layer 5 — Production Usage	118
11.9	Layer 6 — Tradeoffs and Alternatives	119
11.10	Layer 7 — Interview Perspective	119
11.11	Hands-On Lab	120
11.12	Knowledge Check	122
11.13	Summary	122
11.14	Sources	122
11.15	Further Reading	122
12	Deploying ASP.NET Core	123
12.1	Chapter Purpose	123
12.2	Where This Fits	123
12.3	Connection to the Reader's Existing Model	124
12.4	Layer 1 — Conceptual Model	124
12.5	Layer 2 — System Relationships	125
12.6	Layer 3 — Core Mechanics	125
12.7	Layer 4 — Developer Workflow	126
12.8	Layer 5 — Production Usage	127
12.9	Layer 6 — Tradeoffs and Alternatives	128
12.10	Layer 7 — Interview Perspective	128
12.11	Hands-On Lab	129

12.12	Knowledge Check	130
12.13	Summary	131
12.14	Sources	131
12.15	Further Reading	131
13	Cloud Platforms	132
13.1	Chapter Purpose	132
13.2	Where This Fits	132
13.3	Connection to the Reader's Existing Model	133
13.4	Layer 1 — Conceptual Model	133
13.5	Layer 2 — System Relationships	134
13.6	Layer 3 — Core Mechanics	134
13.7	Layer 4 — Developer Workflow	136
13.8	Layer 5 — Production Usage	136
13.9	Layer 6 — Tradeoffs and Alternatives	137
13.10	Layer 7 — Interview Perspective	138
13.11	Hands-On Lab	139
13.12	Knowledge Check	140
13.13	Summary	140
13.14	Sources	140
13.15	Further Reading	140
14	Observability Basics and Production Diagnostics	142
14.1	Chapter Purpose	142
14.2	Where This Fits	142
14.3	Connection to the Reader's Existing Model	143
14.4	Layer 1 — Conceptual Model	143
14.5	Layer 2 — System Relationships	144
14.6	Layer 3 — Core Mechanics	144
14.7	Layer 4 — Developer Workflow	145
14.8	Layer 5 — Production Usage	146
14.9	Layer 6 — Tradeoffs and Alternatives	147
14.10	Layer 7 — Interview Perspective	147
14.11	Hands-On Lab	148
14.12	Knowledge Check	149
14.13	Summary	150
14.14	Sources	150
14.15	Further Reading	150
15	Distributed Applications	151
15.1	Chapter Purpose	151
15.2	Where This Fits	151
15.3	Connection to the Reader's Existing Model	152
15.4	Layer 1 — Conceptual Model	152
15.5	Layer 2 — System Relationships	153
15.6	Layer 3 — Core Mechanics	153
15.7	Layer 4 — Developer Workflow	154
15.8	Layer 5 — Production Usage	155
15.9	Layer 6 — Tradeoffs and Alternatives	156
15.10	Layer 7 — Interview Perspective	156
15.11	Hands-On Lab	157

15.12	Knowledge Check	158
15.13	Summary	159
15.14	Sources	159
15.15	Further Reading	159
16	CI/CD Deployment Pipelines	160
16.1	Chapter Purpose	160
16.2	Where This Fits	160
16.3	Connection to the Reader's Existing Model	161
16.4	Layer 1 — Conceptual Model	161
16.5	Layer 2 — System Relationships	162
16.6	Layer 3 — Core Mechanics	162
16.7	Layer 4 — Developer Workflow	164
16.8	Layer 5 — Production Usage	164
16.9	Layer 6 — Tradeoffs and Alternatives	165
16.10	Layer 7 — Interview Perspective	166
16.11	Hands-On Lab	167
16.12	Knowledge Check	168
16.13	Summary	168
16.14	Sources	168
16.15	Further Reading	169
17	Infrastructure as Code	170
17.1	Chapter Purpose	170
17.2	Where This Fits	170
17.3	Connection to the Reader's Existing Model	171
17.4	Layer 1 — Conceptual Model	171
17.5	Layer 2 — System Relationships	172
17.6	Layer 3 — Core Mechanics	172
17.7	Layer 4 — Developer Workflow	173
17.8	Layer 5 — Production Usage	174
17.9	Layer 6 — Tradeoffs and Alternatives	175
17.10	Layer 7 — Interview Perspective	176
17.11	Hands-On Lab	177
17.12	Knowledge Check	178
17.13	Summary	178
17.14	Sources	178
17.15	Further Reading	178
18	Kubernetes	180
18.1	Chapter Purpose	180
18.2	Where This Fits	180
18.3	Connection to the Reader's Existing Model	181
18.4	Layer 1 — Conceptual Model	181
18.5	Layer 2 — System Relationships	182
18.6	Layer 3 — Core Mechanics	182
18.7	Layer 4 — Developer Workflow	183
18.8	Layer 5 — Production Usage	184
18.9	Layer 6 — Tradeoffs and Alternatives	185
18.10	Layer 7 — Interview Perspective	185
18.11	Hands-On Lab	186

18.12	Knowledge Check	188
18.13	Summary	188
18.14	Sources	188
18.15	Further Reading	188
19	Advanced Observability	189
19.1	Chapter Purpose	189
19.2	Where This Fits	189
19.3	Connection to the Reader's Existing Model	190
19.4	Layer 1 — Conceptual Model	190
19.5	Layer 2 — System Relationships	191
19.6	Layer 3 — Core Mechanics	192
19.7	Layer 4 — Developer Workflow	193
19.8	Layer 5 — Production Usage	194
19.9	Layer 6 — Tradeoffs and Alternatives	195
19.10	Layer 7 — Interview Perspective	195
19.11	Hands-On Lab	196
19.12	Knowledge Check	198
19.13	Summary	198
19.14	Sources	198
19.15	Further Reading	199
20	AI Application Architecture	200
20.1	Chapter Purpose	200
20.2	Where This Fits	200
20.3	Connection to the Reader's Existing Model	201
20.4	Layer 1 — Conceptual Model	201
20.5	Layer 2 — System Relationships	202
20.6	Layer 3 — Core Mechanics	203
20.7	Layer 4 — Developer Workflow	204
20.8	Layer 5 — Production Usage	205
20.9	Layer 6 — Tradeoffs and Alternatives	206
20.10	Layer 7 — Interview Perspective	206
20.11	Hands-On Lab	207
20.12	Knowledge Check	209
20.13	Summary	209
20.14	Sources	209
20.15	Further Reading	209
21	AI Development with .NET	211
21.1	Chapter Purpose	211
21.2	Where This Fits	211
21.3	Connection to the Reader's Existing Model	212
21.4	Layer 1 — Conceptual Model	212
21.5	Layer 2 — System Relationships	213
21.6	Layer 3 — Core Mechanics	213
21.7	Layer 4 — Developer Workflow	215
21.8	Layer 5 — Production Usage	216
21.9	Layer 6 — Tradeoffs and Alternatives	217
21.10	Layer 7 — Interview Perspective	217
21.11	Hands-On Lab	219

21.12	Knowledge Check	220
21.13	Summary	220
21.14	Sources	220
21.15	Further Reading	221
22	Modern Software Architecture and Scale	222
22.1	Chapter Purpose	222
22.2	Where This Fits	222
22.3	Connection to the Reader's Existing Model	223
22.4	Layer 1 — Conceptual Model	223
22.5	Layer 2 — System Relationships	224
22.6	Layer 3 — Core Mechanics	225
22.7	Layer 4 — Developer Workflow	226
22.8	Layer 5 — Production Usage	227
22.9	Layer 6 — Tradeoffs and Alternatives	227
22.10	Layer 7 — Interview Perspective	228
22.11	Hands-On Lab	229
22.12	Knowledge Check	230
22.13	Summary	230
22.14	Sources	231
22.15	Further Reading	231
23	Common Architectural Patterns	232
23.1	Chapter Purpose	232
23.2	Where This Fits	232
23.3	Connection to the Reader's Existing Model	233
23.4	Layer 1 — Conceptual Model	233
23.5	Layer 2 — System Relationships	234
23.6	Layer 3 — Core Mechanics	234
23.7	Layer 4 — Developer Workflow	236
23.8	Layer 5 — Production Usage	237
23.9	Layer 6 — Tradeoffs and Alternatives	237
23.10	Layer 7 — Interview Perspective	238
23.11	Hands-On Lab	239
23.12	Knowledge Check	241
23.13	Summary	241
23.14	Sources	241
23.15	Further Reading	241
24	Interview Preparation and Career Roadmap	242
24.1	Chapter Purpose	242
24.2	Where This Fits	242
24.3	Connection to the Reader's Existing Model	243
24.4	Layer 1 — Conceptual Model	243
24.5	Layer 2 — System Relationships	244
24.6	Layer 3 — Core Mechanics	244
24.7	Layer 4 — Developer Workflow	245
24.8	Layer 5 — Production Usage	247
24.9	Layer 6 — Tradeoffs and Alternatives	247
24.10	Layer 7 — Interview Perspective	248
24.11	Hands-On Lab	249

24.12 Knowledge Check	250
24.13 Summary	250
24.14 Sources	251
24.15 Further Reading	251

1

The Modern .NET Landscape

Chapter Purpose

This chapter rebuilds the map.

If your working model of .NET was formed around Visual Studio 2019, .NET Framework or early modern .NET, Windows Server, IIS, and SQL Server, the core skills still matter. C# still matters. HTTP still matters. SQL still matters. Production discipline still matters.

What changed is the shape of the system around those skills.

Modern .NET development is no longer centered on one Windows server running one IIS-hosted application connected to one SQL Server. That model still exists, and many businesses still run it successfully, but it is no longer the default mental model for new professional .NET systems. Today, the application is often one part of a larger platform made of source control, automated validation, containers, cloud services, managed identity, deployment pipelines, observability, and sometimes AI services.

The purpose of this chapter is not to teach Docker, Kubernetes, CI/CD, cloud hosting, or AI in detail. Those subjects come later. The purpose is to show where they live on the map, why they became important, and how they relate to the enterprise .NET knowledge you already have.

By the end of this chapter, you should be able to explain how modern .NET systems fit together at a high level and why the ecosystem changed after the Visual Studio 2019 and .NET 5 era.

Where This Fits

This is the first chapter because every later chapter depends on the same orientation: modern .NET is not just a runtime upgrade. It is a different way of thinking about the application lifecycle.

A traditional enterprise .NET system might have looked like this:

```
Developer workstation
|
v
Visual Studio build
|
v
Manual or scripted deployment
|
v
Windows Server + IIS
|
v
SQL Server
```

A modern .NET system often looks more like this:

```
Developer workstation or cloud dev environment
|
v
Git repository and pull request
|
v
Automated build and test pipeline
|
v
Container image or deployable package
|
v
Cloud platform, container host, or orchestrator
|
v
Managed databases, caches, queues, identity, and AI services
|
v
Logs, metrics, traces, alerts, and deployment feedback
```

The application code is still central, but it is no longer the whole story. Modern professional development treats the path from source code to production as part of the system.

This book follows that path. First, it rebuilds the mental model. Then it builds applications. Then it introduces Linux, containers, deployment, cloud platforms, production operations, AI, architecture, and interview preparation.

Connection to the Reader's Existing Model

You already know the old landmarks.

You know that an IIS application pool isolates a web application process. You know that configuration files can change behavior without recompiling. You know that SQL Server is both a database engine and an operational responsibility. You know that Windows services run background work. You know that a load-balanced pair of Windows servers introduces session, deployment, and diagnostic concerns that a single server does not.

Modern .NET keeps many of those ideas but changes where the boundaries are.

An ASP.NET Core process still receives HTTP requests and produces responses, but IIS may be only one possible front end, or not involved at all. A container can package the application and its runtime dependencies in a way that feels a little like a disciplined deployment folder, but it also behaves like a standardized process boundary. A cloud platform can replace some server administration work with managed services, but it introduces new ownership questions around identity, cost, networking, and reliability.

The analogy to traditional Windows hosting is useful because the same concerns remain:

- Where does the process run?
- How is it configured?
- How does it connect to the database?
- How is it deployed?
- How do you know whether it is healthy?
- What happens when it fails?

The analogy breaks down because modern systems distribute those concerns across more tools. Instead of one administrator configuring one server, a team may define build rules in source control, publish a container image, provision infrastructure with code, deploy to a managed platform, and monitor behavior through telemetry.

That sounds like more moving parts because it is. The tradeoff is that those parts make repeatability, scale, automation, and team collaboration easier when the system grows.

Layer 1 — Conceptual Model

The modern .NET landscape is the collection of runtimes, tools, platforms, and operating practices used to build and run .NET applications today.

At the center is .NET itself: the runtime, SDK, base class libraries, languages, ASP.NET Core, and related frameworks. Around it is the professional delivery system: source control, automated tests, containerization, cloud hosting, configuration, secrets, observability, security, and operations.

The important conceptual shift is this:

Old center of gravity: the server
New center of gravity: the delivery platform

In the older model, the server was often the durable center of the application. You installed software on it, configured IIS, patched the operating system, managed local folders, copied releases into place, and inspected logs on the machine.

In the modern model, the server may be temporary, abstract, replicated, or hidden behind a managed service. The durable center becomes the combination of source code, build pipeline, deployment definition, container image, configuration system, managed data services, and telemetry.

This does not mean servers disappeared. It means developers are expected to understand platforms that create, replace, scale, observe, and secure runtime environments.

Modern .NET exists to let .NET applications participate naturally in that world:

- cross-platform runtime support instead of Windows-only assumptions;
- command-line and SDK-based workflows instead of IDE-only workflows;
- ASP.NET Core hosting that can run behind IIS, Nginx, cloud load balancers, or container platforms;
- container images for repeatable deployment;
- cloud-native configuration, logging, health checks, and dependency injection;
- libraries and abstractions that integrate with modern identity, telemetry, messaging, and AI services.

It does not solve every problem by itself. Modern .NET does not automatically make an application scalable, secure, observable, or maintainable. It gives you the building blocks and platform alignment to do those things deliberately.

Layer 2 — System Relationships

A modern .NET application participates in a larger system. Understanding the relationships is more important at first than memorizing product names.

The source repository is the system of record for code and increasingly for configuration templates, build definitions, deployment manifests, tests, and architecture notes. In older environments, some of that knowledge lived in server settings or deployment runbooks. In modern environments, teams try to move as much of it as practical into versioned artifacts.

The build system converts source code into something runnable. That might be a published folder, a NuGet package, a container image, or a deployable artifact. The build is not just compilation. It usually includes restore, static checks, tests, packaging, and sometimes vulnerability checks.

The runtime environment is where the application process executes. It might be a developer workstation, a Windows service, IIS, a Linux VM, a container host, a platform-as-a-service environment, or Kubernetes. ASP.NET Core is flexible enough to run in several of these environments, but each environment changes the operational responsibilities.

The application dependencies provide the services the code needs to do useful work: SQL Server, caches, queues, file storage, identity providers, external APIs, observability systems, and AI models. In modern systems, more of these dependencies are managed services rather than software installed on the same server as the application.

The deployment system moves validated changes into an environment. A manual copy to an IIS folder is a deployment system, just a fragile one. A modern pipeline makes the steps explicit, repeatable, reviewable, and observable.

The feedback system tells the team what happened after deployment. Logs, metrics, traces, health checks, alerts, dashboards, and error reports are not extras. They are the way a team sees a system that may be spread across multiple processes, services, regions, and managed platforms.

The main failure boundary also changed. In a simple IIS application, the failure boundary was often the web server, application pool, database, or network. Those still matter. Modern systems add more boundaries: container startup, image registry access, cloud identity, service discovery, queue backlogs, rate limits, region outages, deployment pipeline failures, and telemetry gaps.

The professional .NET developer does not need to be the expert owner of every boundary, but they do need to recognize them.

Layer 3 — Core Mechanics

This chapter keeps mechanics small. Later chapters go deep.

The first mechanic is the modern .NET release model. Microsoft now ships major .NET releases annually, and releases alternate between Long Term Support (LTS) and Standard Term Support (STS). As of July 22, 2026, Microsoft lists .NET 10 as the current LTS release, with .NET 9 as an STS release and .NET 8 as an LTS release still in support. This matters because choosing a target framework is also choosing a support and upgrade rhythm.

The second mechanic is side-by-side installation. Modern .NET versions can be installed alongside one another. That is different from the old feeling of a machine-wide .NET Framework version being part of the Windows server's personality. A developer machine, build agent, or server may have multiple SDKs or runtimes installed.

The third mechanic is SDK-centered development. The .NET SDK supplies the command-line tools used to create, build, test, publish, and package projects. Visual Studio remains important, especially for many Windows-based enterprise teams, but the SDK makes the workflow portable across local machines, CI agents, containers, and cloud build environments.

The fourth mechanic is ASP.NET Core's hosting flexibility. An ASP.NET Core application is a process that can run behind different front ends. IIS can still be part of the story, but it is no longer the only natural host. The same application model can run on Windows, Linux, in a container, or on a managed cloud platform.

The fifth mechanic is containerization. A container image packages the application with the runtime environment expected by the application. Microsoft publishes official .NET container images for different scenarios, including SDK images for building and ASP.NET/runtime images for running applications.

The sixth mechanic is platform integration. Modern .NET applications commonly use dependency injection, configuration providers, structured logging, health checks, and telemetry hooks. These are not decorative framework features. They exist because modern applications must be configured, deployed, monitored, and changed repeatedly across environments.

The seventh mechanic is AI integration. AI is now part of the modern .NET landscape, but in this book it is not magic dust sprinkled over every chapter. For now, treat AI as two related developments: AI-assisted development, where tools help developers write and reason about code, and AI-enabled applications, where .NET systems call models, embedding services, or agent frameworks as part of product behavior.

Layer 4 — Developer Workflow

A typical modern .NET workflow is still recognizable.

You create a project. You write C#. You run it locally. You debug it. You test it. You check it into source control. You deploy it. You fix bugs.

The difference is that the workflow is less centered on one workstation and one server.

At a high level, the workflow looks like this:

```
Create or clone repository
|
v
Restore dependencies
```

```
|  
v  
Run and debug locally  
|  
v  
Add tests and commit changes  
|  
v  
Open pull request  
|  
v  
Automated build and test checks  
|  
v  
Package or publish  
|  
v  
Deploy to an environment  
|  
v  
Observe behavior and iterate
```

In Visual Studio 2019-era development, it was common for the IDE to hide many of these steps. Modern teams still use IDEs, but they also expect the same steps to run outside the IDE. That is why command-line workflows, project files, source-controlled pipeline definitions, and repeatable local setup have become more important.

For this first chapter, the useful commands are discovery commands rather than implementation commands:

```
dotnet --info  
dotnet --list-sdks  
dotnet --list-runtimes
```

On Windows PowerShell, the same commands apply:

```
dotnet --info  
dotnet --list-sdks  
dotnet --list-runtimes
```

These commands answer basic orientation questions: which SDKs are installed, which runtimes are available, and what environment the .NET CLI sees.

The details of creating projects, building APIs, testing, containerizing, and deploying come later. For now, the important workflow shift is that modern .NET development is designed to be repeatable across local development, automated builds, and production environments.

Layer 5 — Production Usage

Production is where the modern landscape becomes real.

In a traditional IIS deployment, production concerns were often concentrated on the server:

- IIS configuration;
- application pool identity;
- installed .NET runtime;
- web.config settings;
- Windows Event Log;
- file permissions;
- SQL Server connection strings;
- load balancer behavior.

Modern .NET production systems still have equivalent concerns, but they are distributed across the application, platform, and delivery process.

Configuration is usually environment-specific. Local development might use user secrets, environment variables, or local configuration files. Production might use a cloud configuration service, secret manager, injected environment variables, or platform-level settings. The principle is the same as protecting connection strings in older systems, but the mechanisms are broader.

Security extends beyond application login. Authentication and authorization still matter, but so do service identities, secret rotation, container image scanning, dependency updates, cloud permissions, TLS, and deployment access.

Reliability is no longer just “keep the server up.” It includes health checks, restart behavior, retries, timeouts, queue handling, horizontal scaling, database failover, and safe deployment strategies.

Deployment becomes a repeatable process. A production release should not depend on a person remembering which files to copy or which server setting to change. The goal is not automation for its own sake. The goal is reducing variation between releases.

Observability replaces server guessing. When an application runs in multiple instances, containers, or managed platforms, logging into the server is often not the first diagnostic move. You need structured logs, metrics, traces, health checks, alerts, and dashboards to understand what the system is doing.

Scaling moves from buying a bigger server to choosing the right boundary. Sometimes the answer is a larger database. Sometimes it is more application instances. Sometimes it is caching, background processing, or queue-based work. Sometimes the correct answer is to simplify the design.

Persistence remains central. SQL Server is still a major part of many .NET systems, including modern ones. The difference is that it may be hosted as a managed database, run in a container for local development, accessed through Entity Framework Core or Dapper, and deployed alongside migration automation.

Cost becomes a design concern. In a traditional environment, cost was often hidden behind infrastructure budgets. In cloud environments, architectural decisions can affect monthly bills directly. A chatty service, oversized database, unnecessary Kubernetes cluster, or inefficient AI call can have a visible cost.

The recurring production distinction is simple:

Local development optimizes for fast feedback.
Production optimizes for reliability, security, repeatability, and visibility.

Modern .NET gives you tools that can support both, but the design must be intentional.

Layer 6 — Tradeoffs and Alternatives

Modern does not always mean better. It means better aligned with current professional expectations and platform realities. Use modern .NET practices when the application needs cross-platform hosting, cloud deployment, automated delivery, containerized environments, strong observability, frequent change, or integration with managed services.

Do not assume every application needs every modern tool. A small internal application may not need Kubernetes. A batch utility may not need a distributed architecture. A stable line-of-business system may benefit more from automated tests and clear deployment scripts than from a wholesale platform migration.

Simpler alternatives remain valid:

- IIS hosting can still be appropriate for some ASP.NET Core applications.
- A single well-designed application can be better than a premature set of microservices.
- A managed platform-as-a-service option can be simpler than running Kubernetes.
- SQL Server can remain the right database.
- Manual approval gates can be appropriate even inside automated pipelines.

More advanced alternatives exist for systems that need them:

- container orchestration;
- infrastructure as code;
- service meshes;
- event-driven architecture;
- distributed tracing;
- multi-region deployment;
- AI agents and retrieval-augmented generation.

The common overengineering mistake is treating the modern stack as a checklist. Docker, Kubernetes, Redis, queues, cloud platforms, and AI services each solve specific problems. They also add operational responsibility. A strong modern .NET developer knows both sides of that tradeoff.

The better question is not “Are we using the newest thing?” It is “Which system boundary is causing pain, and which tool makes that boundary clearer, safer, or easier to change?”

Layer 7 — Interview Perspective

Interviewers do not usually expect a returning .NET developer to be a deep expert in every modern platform tool. They do expect an accurate map.

Concepts commonly tested:

- the difference between .NET Framework, .NET Core, and modern .NET;
- why ASP.NET Core is not tied to IIS in the same way classic ASP.NET was;
- what containers solve and what they do not solve;
- why CI/CD exists;
- how cloud hosting changes deployment and operations;
- the difference between logs, metrics, and traces;
- why microservices are not automatically better than monoliths;
- how AI services fit into ordinary application architecture.

Representative questions:

- “What changed in .NET after .NET Framework?”
- “Why would a .NET team run an application on Linux?”
- “What problem does Docker solve for a .NET API?”
- “When would you choose a managed cloud service instead of a VM?”
- “What is the difference between deploying code and operating a production system?”
- “How would you modernize an older IIS-hosted application without rewriting everything?”

A strong answer connects concepts. For example:

“Docker does not make the application scalable by itself. It packages the application and its runtime environment in a repeatable way. Scaling depends on where the container runs, whether the app is stateless, how configuration and secrets are handled, and what the database can support.”

Common misconceptions:

- “Modern .NET means cloud only.”
- “ASP.NET Core requires Linux.”
- “Containers replace deployment pipelines.”
- “Kubernetes is the default place to run every production app.”
- “Microservices are the modern version of layered architecture.”
- “AI integration means replacing normal application design.”

Small design scenario:

You inherit an ASP.NET application hosted on Windows Server with IIS and SQL Server. Releases are manual, configuration differs between environments, and production issues are diagnosed by remote desktop access and log files on the server.

A good modernization discussion would not begin with “move it to Kubernetes.” It would begin by identifying the current pain:

- Is deployment unreliable?
- Is the application hard to test?
- Are environments inconsistent?
- Is the server difficult to patch?
- Are production failures hard to diagnose?
- Is scaling actually required?

Then it would propose a sequence: get the code building consistently, add automated tests, clarify configuration, improve logging and health checks, choose a deployment target, and only then consider containers, cloud services, or orchestration if they solve a real problem.

Hands-On Lab

Objective:

Create a personal map of the modern .NET landscape and inspect the .NET installation on your machine.

Prerequisites:

- A development machine with the .NET SDK installed.
- A terminal: Windows Terminal, PowerShell, Command Prompt, macOS Terminal, or a Linux shell.
- No application code is required yet.

Steps:

1. Open a terminal.
2. Run the following command:
3. Identify the installed SDK version or versions.
4. Identify the installed runtime version or versions.
5. Note the operating system and architecture reported by the command.
6. Run:

```
dotnet --info
```

7. Run:
8. Draw a simple architecture map for a modern .NET application using these boxes:

```
dotnet --list-sdks
```

```
dotnet --list-runtimes
```

```
Developer
Source control
Build and test pipeline
Application package or container image
Runtime host
Database
Observability
Users
```

9. Add arrows showing how code moves from development to production and how production feedback returns to the team.

Expected results:

- You can name the .NET SDKs and runtimes installed on your machine.
- You can explain the difference between local development, build automation, deployment, runtime hosting, and production feedback.
- You have a first-pass map of the modern .NET system this book will fill in.

Validation commands:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

Troubleshooting notes:

- If `dotnet` is not recognized, the SDK may not be installed or may not be on your `PATH`.
- If only runtimes are installed, you may be able to run .NET applications but not build them.
- If several SDKs are installed, that is normal. Modern .NET versions can live side by side.

Knowledge Check

1. Why is “modern .NET” more than a newer runtime version?
2. In what ways is a container similar to a disciplined deployment package, and where does that analogy break down?
3. Why did the center of gravity move from individual servers toward delivery platforms?
4. What production concerns remain the same whether an application runs under IIS, in a container, or on a cloud platform?
5. Why is Kubernetes not the natural starting point for every modernization effort?
6. How does AI-assisted development differ from building AI-enabled application features?
7. Why should a returning .NET developer understand CI/CD even before becoming a deployment specialist?
8. What does observability provide that server access alone does not?

Summary

Modern .NET development begins with the same durable skills you already have: C#, HTTP, SQL, configuration, deployment judgment, and production discipline. The landscape around those skills has changed.

The old model placed the Windows server and IIS near the center. The modern model places the application inside a delivery platform made of source control, automated validation, repeatable packaging, cloud or container hosting, managed dependencies, security boundaries, and production telemetry.

.NET adapted to that world by becoming cross-platform, SDK-centered, cloud-friendly, container-friendly, and better integrated with modern configuration, logging, dependency injection, hosting, and AI patterns.

The rest of this book fills in the map one layer at a time. First you will learn the major technologies and workflow. Then you will build .NET applications. Then you will run them on Linux and in containers. Then you will deploy, observe, secure, scale, and reason about them as production systems.

The goal is not to chase novelty. The goal is to become fluent again in the professional environment where modern .NET systems are built and operated.

Sources

- Microsoft Learn, “.NET releases, patches, and support”: <https://learn.microsoft.com/en-us/dotnet/core/releases-and-support>
- Microsoft, “.NET Support Policy”: <https://dotnet.microsoft.com/en-us/platform/support/policy>
- Microsoft Learn, “Microsoft .NET and .NET Core - Microsoft Lifecycle”: <https://learn.microsoft.com/en-us/lifecycle/products/microsoft-net-and-net-core>
- Microsoft Learn, “.NET container images”: <https://learn.microsoft.com/en-us/dotnet/core/docker/container-images>
- Microsoft Learn, “Develop .NET apps with AI features”: <https://learn.microsoft.com/en-us/dotnet/ai/overview>

Further Reading

- Microsoft Learn, “What’s new in .NET 10”: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview>
- Microsoft Learn, “Install .NET on Windows”: <https://learn.microsoft.com/en-us/dotnet/core/install/windows>
- Microsoft Learn, “AI apps for .NET developers”: <https://learn.microsoft.com/en-us/dotnet/ai/>

2

The Modern Technology Stack

Chapter Purpose

Chapter 1 rebuilt the map. This chapter labels the major landmarks.

The modern .NET stack is not a single product. It is a working combination of runtime, web framework, operating system, packaging model, deployment platform, data stores, source-control system, delivery pipeline, and increasingly AI services. If you come from a Windows, IIS, SQL Server, and Visual Studio background, the older stack may have felt vertically integrated. Microsoft supplied the language, runtime, IDE, web server integration, database, operating system, and deployment target. You could build a serious enterprise application while staying almost entirely inside one vendor ecosystem.

That world still exists, but the center widened.

Modern .NET remains a Microsoft-led ecosystem, but professional .NET work now commonly touches Linux, Docker, Kubernetes, Redis, GitHub, cloud platforms, open-source libraries, managed services, and AI providers. Some of these tools were adopted because they solved problems Microsoft products already faced. Some became industry standards outside the Microsoft ecosystem and .NET had to meet developers where the industry had moved. That is not a weakness. It is one of the reasons modern .NET is viable in cloud-native systems.

The purpose of this chapter is to introduce the major technologies without teaching their implementation details yet. You should finish this chapter able to explain what each technology is for, how it relates to the technologies around it, and where it fits in a complete application system.

This chapter deliberately stays at the map level. Later chapters teach the mechanics.

Where This Fits

Chapter 1 described the shift from server-centered development to platform-centered development. Chapter 2 turns that platform into named components.

A typical modern .NET application stack might look like this:

```
Users
|
v
Web or API edge
|
v
ASP.NET Core application
|
+-- SQL Server or managed relational database
|
+-- Redis cache
|
+-- Queue, background worker, or external service
```

```

|
+-- AI service when the application needs model behavior
|
v
Logs, metrics, traces, and alerts

Source code -> GitHub -> CI/CD -> package or container image -> host platform

```

The technologies in the roadmap do not all sit at the same layer:

- .NET is the runtime and developer platform.
- ASP.NET Core is the web and API framework.
- Linux is a common operating-system host.
- Docker is a packaging and local runtime tool for containers.
- Kubernetes is an orchestration platform for running containers at scale.
- SQL Server is the relational database many enterprise .NET systems already know.
- Redis is a fast in-memory data store often used for caching and coordination.
- Cloud platforms provide managed compute, databases, identity, networking, and operations.
- GitHub is commonly the source-control and collaboration hub.
- CI/CD is the automated path from code change to validated artifact and, eventually, deployment.
- AI services provide model-backed behavior that applications can call like other external dependencies.

This chapter introduces all of them together so later chapters can zoom in without forcing you to learn the system vocabulary from scratch.

Connection to the Reader's Existing Model

The easiest way to understand the modern stack is to compare each layer to something familiar.

.NET is still the platform you write C# against. The difference is that modern .NET is cross-platform, SDK-centered, open-source, and released on a regular cadence. It is less tied to a particular Windows installation than .NET Framework felt.

ASP.NET Core still handles HTTP requests, routing, application services, configuration, logging, and responses. The familiar IIS mental model helps, but ASP.NET Core is not just “classic ASP.NET with new project files.” It can run behind IIS, behind a Linux reverse proxy, inside a container, or on a managed cloud platform.

Linux is the operating system you are most likely to encounter when .NET code runs outside Windows. Think of it as another production host with different filesystem, process, permission, service, and shell conventions. You do not need to become a Linux administrator to be productive, but you do need enough fluency to read paths, inspect processes, understand permissions, and interpret logs.

Docker is closest to a disciplined deployment package plus a lightweight process boundary. It is not a virtual machine in the traditional Hyper-V sense. It packages an application and its runtime dependencies as an image, then runs that image as an isolated process called a container.

Kubernetes is closest to a large-scale operations control plane. If a load-balanced IIS farm answers “which server receives the request?”, Kubernetes also asks “which containers should exist, where should they run, how should they be replaced, how are they reached, and how is desired state maintained?” That analogy is useful, but Kubernetes is broader and more abstract than a web farm.

SQL Server is already familiar. The modern shift is not that relational databases disappeared. It is that SQL Server may run on Windows, Linux, in a container for development, in Azure SQL Database, in Azure SQL Managed Instance, or on a VM. The database remains a strong consistency and persistence boundary.

Redis is not a replacement for SQL Server. Think of it as a very fast in-memory structure server that can hold cached data, counters, distributed locks, session-like state, streams, or temporary coordination data. It solves a different class of problems from a relational database.

Cloud platforms are the modern equivalent of renting a data center where many infrastructure concerns are exposed as APIs and managed services. They replace some hardware and server administration with service selection, identity, networking, cost management, and operational design.

GitHub is more than a remote Git folder. In many teams it is the collaboration surface for pull requests, code review, security scanning, issue tracking, automation, and package publishing.

CI/CD replaces memory-based release discipline with executable release discipline. If you previously relied on a build machine, deployment checklist, and human care, CI/CD moves more of that process into source-controlled automation.

AI services are external intelligence dependencies. From an architecture point of view, they are closer to calling a payment processor, search service, or document service than to adding an ordinary class library. They introduce latency, cost, security, prompt design, evaluation, and correctness concerns.

Layer 1 — Conceptual Model

The modern technology stack can be understood as five cooperating layers:

```
Application layer
.NET, ASP.NET Core, C#, application libraries

Data and dependency layer
SQL Server, Redis, queues, files, APIs, AI services

Runtime and packaging layer
Windows, Linux, Docker, container images

Platform and operations layer
Cloud platforms, Kubernetes, identity, networking, observability

Delivery layer
GitHub, pull requests, CI/CD, artifact registries, release automation
```

The application layer is where most business code lives. This is where you write APIs, validation, business rules, background services, and integrations.

The data and dependency layer is where the application stores durable state, reads fast temporary state, communicates with other systems, and calls external capabilities.

The runtime and packaging layer answers the question, “What exactly runs, and what does it require from the machine?” This includes the OS, installed runtime or container image, process environment, ports, files, certificates, and startup behavior.

The platform and operations layer answers the question, “Where does it run, how is it secured, how does it scale, and how do we know it is healthy?”

The delivery layer answers the question, “How does a code change become a validated production change?”

No single layer is the whole stack. A developer who only knows the application layer may write good C# but struggle to diagnose production. A developer who only knows platform tools may ship elaborate infrastructure for a simple problem. Modern .NET fluency means understanding how the layers cooperate.

Layer 2 — System Relationships

Each technology has inputs, outputs, dependencies, ownership boundaries, and failure modes.

.NET takes source code, project files, NuGet packages, SDKs, and configuration as inputs. It produces assemblies, applications, packages, and runtime behavior. The development team usually owns the code and package choices; the platform or operations team may own installed runtimes, base images, and runtime policies.

ASP.NET Core takes HTTP requests, configuration, dependency registrations, and middleware as inputs. It produces HTTP responses, logs, metrics, and calls to dependencies. It fails when routing is wrong, configuration is missing, dependencies are unavailable, startup fails, or runtime exceptions escape.

Linux takes processes, files, users, groups, sockets, services, and environment variables as operating-system concepts. A .NET developer usually does not own the whole Linux estate, but they may own enough of the application process to read logs, understand paths, inspect environment variables, and troubleshoot container behavior.

Docker takes a Dockerfile, application output, base images, and configuration as inputs. It produces container images and containers. The development team often owns the application image. A platform team may own the registry, base image policy, runtime host, and security scanning.

Kubernetes takes declarative resource definitions that describe desired state. It attempts to keep the actual cluster state aligned with those definitions. The application team may own deployment definitions for its service; a platform team often owns the cluster, networking, ingress, policies, secrets integration, and shared observability.

SQL Server takes schema, queries, transactions, indexes, data files, memory, storage, and connections. It produces durable state and query results. Its failure boundaries include locking, blocking, disk, memory, schema migration, backup, recovery, permissions, and network access.

Redis takes keys, values, data structures, expiration rules, memory, and client connections. It produces fast reads, writes, and coordination behavior. Its failure boundaries include memory pressure, eviction policy, persistence configuration, clustering, and the risk of treating cached data as if it were durable relational truth.

Cloud platforms take resource definitions, subscriptions or accounts, permissions, regions, networks, and billing rules. They produce hosted services. Their failure boundaries include identity, quota, cost, region availability, misconfiguration, provider limits, and shared-responsibility misunderstandings.

GitHub takes commits, branches, pull requests, issues, workflow files, and repository permissions. It produces review history, source history, automation events, and collaboration records. Its failure boundaries include branch-policy gaps, secret leaks, broken workflows, weak review practices, and permissions that are too broad.

CI/CD takes repository events, workflow definitions, build agents, secrets, tests, package feeds, and deployment credentials. It produces build results, test reports, packages, images, deployments, and audit trails. Its failure boundaries include flaky tests, missing environment parity, credential problems, incomplete rollback plans, and automation that hides rather than reduces risk.

AI services take prompts, input data, context, model choices, safety settings, tools, embeddings, and credentials. They produce generated text, structured outputs, classifications, embeddings, or tool calls. Their failure boundaries include latency, cost, hallucination, data leakage, model drift, evaluation gaps, and ambiguous ownership of generated behavior.

The relationships matter because production failures rarely respect chapter boundaries. A slow request might involve ASP.NET Core routing, Redis cache misses, SQL Server query plans, cloud network latency, and missing telemetry. The stack is a system.

Layer 3 — Core Mechanics

The smallest useful example is a web API with a database, cache, and delivery pipeline:

```
GitHub repository
|
v
CI builds and tests the .NET solution
|
v
Docker image is produced
|
v
Cloud platform runs the ASP.NET Core container
|
+-- SQL Server stores orders
|
+-- Redis caches product lookups
|
```

```
  +-- AI service summarizes support notes
  |
  v
Observability records logs, metrics, and traces
```

The core terminology starts here.

A runtime executes compiled application code. In modern .NET, the runtime is not the same thing as the SDK. The SDK builds and publishes applications; the runtime runs them.

A framework provides reusable application structure. ASP.NET Core is the web framework used to build HTTP APIs, web applications, real-time apps, and services.

An operating system hosts processes, files, networking, permissions, and resource limits. Windows and Linux can both host modern .NET applications.

A container image is a packaged filesystem and metadata template used to create containers. A container is a running instance of that image.

An orchestrator manages many containers across machines. Kubernetes is the dominant open-source orchestrator and is widely used in cloud-native environments, but it is not required for every application.

A relational database stores structured durable data using tables, relationships, indexes, transactions, and queries. SQL Server remains a major enterprise relational database and, according to DB-Engines in July 2026, ranked third overall among database management systems.

An in-memory data store keeps data primarily in memory for speed. Redis is a prominent example and ranked eighth overall in the July 2026 DB-Engines ranking.

A cloud platform provides infrastructure and application capabilities as services: compute, databases, storage, networking, identity, observability, and AI.

A repository stores source history. Git is the version-control system; GitHub is a collaboration platform built around Git repositories and automation.

Continuous integration means automatically building and testing code changes after they are committed or proposed. Continuous delivery and deployment extend that automation toward release artifacts and environments.

An AI service exposes model behavior through an API or SDK. In modern .NET, Microsoft.Extensions.AI provides common abstractions for interacting with models and related AI capabilities across providers.

Layer 4 — Developer Workflow

Chapter 3 will teach the modern workflow in more detail. For this chapter, the workflow is about recognizing which tool participates at each stage.

Early in development, you mostly touch .NET, ASP.NET Core, SQL Server or a local database, maybe Redis, and GitHub. You run locally, test quickly, and commit changes.

As the application grows, Docker often enters the workflow to make the local environment more repeatable. Instead of every developer installing the same database, cache, and supporting services by hand, a team can describe a local multi-container environment.

When the team starts sharing changes, CI becomes important. A pull request should not depend only on “it works on my machine.” It should trigger a build and tests in a clean environment.

When the team starts deploying repeatedly, CD becomes important. The question shifts from “Can we build it?” to “Can we safely move this exact validated artifact into an environment?”

As production needs grow, cloud services and observability become part of the daily workflow. Developers need to know how configuration reaches the application, where logs go, how database access is secured, how health is reported, and how deployments can be rolled back.

Kubernetes usually appears after these basics. It is most useful when a team needs a consistent platform for many containerized workloads, not when it is trying to learn containers for the first time.

AI services may appear in two places. AI coding assistants may help during development. AI application services may become runtime dependencies when the product needs summarization, classification, chat, extraction, embeddings, or agentic workflows.

Useful orientation commands:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

On Windows PowerShell, the commands are the same:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

These commands are not a setup requirement for this chapter. They simply show which parts of the stack are already present on your machine.

Layer 5 — Production Usage

In production, the technology stack becomes a set of responsibility boundaries.

.NET and ASP.NET Core define the application runtime boundary. Production questions include which .NET version is supported, whether the app is patched, how configuration is loaded, how logs are emitted, how health is exposed, and how dependency failures are handled.

Linux or Windows defines the host operating-system boundary. Production questions include patching, file permissions, service identity, certificates, network access, process limits, and diagnostic access.

Docker defines the packaging boundary. Production questions include base-image trust, image size, vulnerability scanning, registry access, immutable tags, startup behavior, and whether runtime configuration is separated from the image.

Kubernetes defines the orchestration boundary. Production questions include replicas, health probes, rolling updates, secrets integration, network policy, resource limits, storage, cluster ownership, and operational complexity.

SQL Server defines the durable data boundary. Production questions include backup and restore, high availability, transaction isolation, migration strategy, index health, query performance, permissions, and cost.

Redis defines the fast state boundary. Production questions include whether the data is disposable or durable, how memory is sized, how eviction works, whether clustering is needed, and how cache misses affect the database.

Cloud platforms define the managed-service boundary. Production questions include region choice, identity, networking, quotas, compliance, platform service-level agreements, cost controls, and shared responsibility.

GitHub and CI/CD define the delivery boundary. Production questions include who can approve changes, which checks are required, how secrets are protected, how artifacts are versioned, and how deployments are audited.

AI services define the model boundary. Production questions include data privacy, prompt injection, output validation, model selection, cost per call, evaluation, latency, fallback behavior, and observability of AI decisions.

The state-of-the-art direction across these areas is not “use more tools.” It is platform engineering: make the correct path easy for teams. A mature organization gives developers paved paths for common tasks such as creating an API, adding a database, creating a pipeline, publishing a container, connecting to secrets, emitting telemetry, and deploying safely.

That is the modern production goal: not heroic deployment knowledge, but repeatable systems that make the safe thing normal.

Layer 6 — Tradeoffs and Alternatives

The modern stack is full of good tools, and good tools can still be wrong for the job.

.NET competes most often with ecosystems such as Java, Go, Node.js, Python, Rust, and Kotlin. .NET is strong for enterprise applications, APIs, cloud services, Windows integration, high-performance server workloads, and teams that value C# and mature tooling. It may not be the natural first choice when a team is already deeply invested in another ecosystem or when a specialized runtime dominates a domain.

ASP.NET Core competes with frameworks such as Spring Boot, Express, FastAPI, Django, Rails, Laravel, and Go HTTP frameworks. ASP.NET Core is strong for typed, high-performance APIs and enterprise web systems. It can be excessive for a tiny script-like endpoint, and it may be unfamiliar to teams centered on JavaScript or Python.

Linux competes with Windows Server as a host for .NET workloads. Linux is common for containers and cloud-native hosting. Windows remains appropriate for applications with Windows-specific dependencies, legacy COM integration, classic .NET Framework, or operational teams built around Windows tooling.

Docker competes with direct host installation, virtual machines, language-specific packaging, and platform-native deployment packages. Docker is useful when environment repeatability matters. It can be unnecessary for a simple internal utility or a platform-as-a-service deployment that already abstracts packaging well.

Kubernetes competes with simpler options: Azure App Service, Azure Container Apps, AWS Elastic Beanstalk, AWS ECS, Google Cloud Run, ordinary VMs, and managed platform services. Kubernetes is powerful when many services need a shared orchestration model. It is often overengineering for one or two modest applications.

SQL Server competes with PostgreSQL, MySQL, Oracle, SQLite, cloud-native managed databases, document databases, and analytical stores. SQL Server is a strong choice for enterprise relational workloads, T-SQL expertise, Microsoft tooling, and existing estates. PostgreSQL is a major open-source alternative; SQLite is excellent for embedded and local scenarios; document stores fit different data shapes.

Redis competes with in-process memory caches, Memcached, database caching, message brokers, and specialized streaming systems. Redis is useful when fast shared state or data structures matter. It is not a substitute for relational integrity, durable event logs, or careful database design.

Cloud platforms compete with on-premises hosting, colocation, private cloud, and hybrid designs. Azure is often natural for Microsoft-centered enterprises, AWS is broad and deeply established, and Google Cloud has strengths in data, analytics, Kubernetes heritage, and AI. The right choice is usually shaped by organizational skills, contracts, compliance, existing workloads, and service fit.

GitHub competes with Azure DevOps, GitLab, Bitbucket, and self-hosted Git systems. GitHub is widely used for open source and increasingly common in enterprise collaboration. Azure DevOps remains common in Microsoft enterprise shops with established Boards, Repos, and Pipelines usage.

CI/CD tools include GitHub Actions, Azure Pipelines, GitLab CI, Jenkins, TeamCity, CircleCI, Buildkite, and others. The best tool is the one your team can operate reliably with clear secrets management, repeatable builds, and useful feedback.

AI services compete across providers and hosting models: OpenAI, Azure OpenAI, Azure AI Foundry, GitHub Models, Amazon Bedrock, Google Gemini, local models through tools such as Ollama, and traditional ML through ML.NET. Modern .NET's direction is to provide abstractions that let applications use model services without binding every part of the code to one provider.

The most common overengineering mistake is stacking all of these choices into the first version of an application. A strong architecture grows the stack when the system has earned the complexity.

Layer 7 — Interview Perspective

Interviewers use stack questions to test whether you understand boundaries, not whether you can recite logos.

Concepts commonly tested:

- the role of .NET versus ASP.NET Core;
- why Linux matters to modern .NET;
- the difference between containers and virtual machines;
- what Kubernetes adds on top of containers;

- why SQL Server and Redis are complementary rather than interchangeable;
- how GitHub, pull requests, and CI relate to team quality;
- what cloud platforms manage and what the team still owns;
- how AI services fit into an ordinary application architecture.

Representative questions:

- “Where does ASP.NET Core end and the hosting platform begin?”
- “Why would a .NET team use Docker?”
- “When would you avoid Kubernetes?”
- “What belongs in SQL Server versus Redis?”
- “How is GitHub Actions different from Git itself?”
- “What does a cloud provider manage for you, and what remains your responsibility?”
- “How would you add an AI feature without making the whole application depend on one model provider?”

A strong answer names the boundary and the tradeoff.

For example:

“Redis can reduce load on SQL Server by caching frequently read data, but SQL Server remains the system of record. If Redis is unavailable, the application should either fall back to SQL Server or fail in a controlled way, depending on the feature.”

Common misconceptions:

- “.NET is only a Windows platform.”
- “ASP.NET Core requires IIS.”
- “Docker is the same as Kubernetes.”
- “Kubernetes makes applications cloud-native automatically.”
- “Redis is just a faster database.”
- “CI/CD is only a deployment script.”
- “Cloud means no operations work.”
- “AI services can be treated like deterministic libraries.”

Small design scenario:

You are asked to design the first modern version of an internal order-tracking API. The company knows C#, SQL Server, and Windows Server. It wants to become more modern but does not yet have a large platform team.

A sensible stack might be:

- .NET and ASP.NET Core for the API;
- SQL Server for orders and durable business data;
- GitHub or Azure DevOps for source control and pull requests;
- CI to build and test every change;
- Docker for repeatable local dependencies and possibly packaging;
- a managed cloud app service or container app for hosting;
- basic logging, health checks, and metrics;
- Redis only if caching becomes necessary;
- Kubernetes only if the organization later needs orchestration at scale;
- AI services only when there is a concrete product feature, such as support note summarization or order-message classification.

The strong answer is not maximal. It is staged.

Hands-On Lab

Objective:

Create a stack map for a modern .NET application and classify each technology by responsibility.

Prerequisites:

- Chapter 1 completed.
- A text editor or notebook.
- Optional local installations of .NET, Git, Docker, and Kubernetes CLI tools.

Steps:

1. Draw five columns:

```
Application | Data | Packaging/Runtime | Platform/Ops | Delivery
```

2. Place each technology from this chapter into the most natural column:

```
.NET
ASP.NET Core
Linux
Docker
Kubernetes
SQL Server
Redis
Cloud platform
GitHub
CI/CD
AI services
```

3. For each technology, write one sentence explaining what problem it solves.
4. For each technology, write one sentence explaining what it does not solve.
5. Mark which technologies are essential for a first ASP.NET Core API.
6. Mark which technologies are optional until the system grows.
7. If you have the tools installed, run:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

8. Rewrite the stack as a simple architecture diagram for an order-tracking API.

Expected results:

- You can describe the role of each major technology.
- You can separate application concerns from data, runtime, platform, and delivery concerns.
- You can identify which technologies are foundational and which are situational.

Validation commands:

```
dotnet --info
git --version
docker --version
kubectl version --client
```

Troubleshooting notes:

- If `docker` is not installed, complete the lab conceptually. Docker is taught later.
- If `kubectl` is not installed, that is fine. Kubernetes is intentionally not required yet.

- If you cannot decide where a technology belongs, ask what boundary it owns: code, data, runtime, platform, or delivery.

Knowledge Check

1. Why is ASP.NET Core not the same layer as Docker or Kubernetes?
2. What problem does Linux solve for modern .NET teams if Windows Server still exists?
3. Why is Docker useful even before an application runs in Kubernetes?
4. What does Kubernetes add beyond running a single container?
5. Why should SQL Server usually remain the system of record when Redis is added?
6. What is the difference between Git and GitHub?
7. Why is CI/CD part of the technology stack rather than only a team process?
8. What does a cloud platform manage, and what does it not manage for your application team?
9. Why should AI services be treated as runtime dependencies rather than ordinary deterministic libraries?
10. Which technologies in this chapter would you choose for the first version of a small internal API, and which would you postpone?

Summary

The modern .NET technology stack is a layered system.

.NET and ASP.NET Core provide the application foundation. SQL Server and Redis serve different data needs: durable relational state and fast shared in-memory state. Linux and Docker shape how applications run and how their environments are packaged. Kubernetes manages containerized workloads when scale and platform consistency justify its complexity. Cloud platforms provide managed compute, data, identity, networking, and operations services. GitHub and CI/CD turn source changes into validated, reviewable, repeatable delivery. AI services add model-backed behavior when the product needs capabilities that ordinary deterministic code does not provide.

The stack is not a checklist. It is a set of choices around boundaries. The professional skill is knowing which boundary you are addressing and whether the tool adds more clarity than complexity.

The next chapter moves from the technology map to the day-to-day workflow: how modern teams create, review, test, and move .NET changes through the software lifecycle.

Sources

- Microsoft Learn, “.NET releases, patches, and support”: <https://learn.microsoft.com/en-us/dotnet/core/releases-and-support>
- Microsoft Learn, “ASP.NET documentation”: <https://learn.microsoft.com/en-us/aspnet/core/>
- Docker Docs, “What is Docker?”: <https://docs.docker.com/get-started/docker-overview/>
- Kubernetes Documentation, “Concepts”: <https://kubernetes.io/docs/concepts/>
- Microsoft Learn, “What is SQL Server?”: <https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server>
- Redis, “What is Redis?: An Overview”: <https://redis.io/tutorials/what-is-redis/>
- GitHub Docs, “GitHub Actions documentation”: <https://docs.github.com/en/actions>
- Microsoft Learn, “Use continuous integration”: <https://learn.microsoft.com/en-us/devops/develop/what-is-continuous-integration>
- Microsoft Azure, “What is cloud computing?”: <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing/>
- Microsoft Learn, “Develop .NET apps with AI features”: <https://learn.microsoft.com/en-us/dotnet/ai/overview>
- DB-Engines, “DB-Engines Ranking”: <https://db-engines.com/en/ranking/>

- CNCF, “2025 Annual Cloud Native Survey” announcement: <https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/>
- Stack Overflow, “2025 Developer Survey: Technology”: <https://survey.stackoverflow.co/2025/technology/>

Further Reading

- Microsoft Learn, “.NET + AI ecosystem tools and SDKs”: <https://learn.microsoft.com/en-us/dotnet/ai/dotnet-ai-ecosystem>
- Google Cloud, “Google Cloud overview”: <https://docs.cloud.google.com/docs/overview>
- AWS, “What is cloud computing?”: <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/what-is-cloud-computing.html>
- Docker Docs, “What is a container?”: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

3

Modern Development Workflow

Chapter Purpose

The previous chapter named the major technologies in the modern .NET stack. This chapter shows how work moves through that stack during an ordinary professional development cycle.

If your baseline is Visual Studio 2019, Team Foundation Server or an older Git setup, manual QA handoffs, and deployments coordinated through people and servers, the modern workflow can look busier than the old one. There are pull requests, automated checks, cloud development environments, code review tools, pipeline statuses, package feeds, container images, security scans, and AI coding assistants.

The purpose of all that machinery is not ceremony. It is controlled feedback.

Modern development workflow exists because professional software teams learned the cost of late integration, unclear review, environment drift, manual deployment, and production surprises. The newer workflow tries to make each change small enough to understand, visible enough to review, automated enough to validate, and traceable enough to operate.

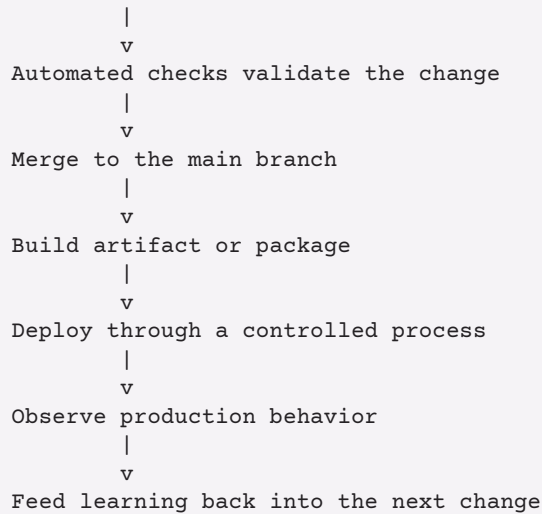
This chapter does not teach full CI/CD implementation. Chapter 8 covers automated testing and CI basics, and Chapter 16 covers deployment pipelines. Here, the goal is to understand the complete lifecycle at a high level: local development, source control, pull requests, testing, code reviews, AI coding assistants, development environments, and how code moves toward production.

Where This Fits

Chapter 1 shifted the mental model from server-centered development to platform-centered development. Chapter 2 identified the technologies in that platform. Chapter 3 connects them through time.

A modern workflow is a loop:

```
Plan a small change
  |
  v
Create or update code locally
  |
  v
Run local build and tests
  |
  v
Commit to a short-lived branch
  |
  v
Open a pull request
  |
  v
Review, discuss, and revise
```



The key idea is that development does not end when code compiles. In a modern team, the workflow connects development, delivery, and operations. Microsoft describes DevOps as uniting people, process, and technology across planning, development, delivery, and operations. That is the lifecycle this chapter is mapping.

This workflow is now common across ecosystems, not only .NET. Java, Node.js, Python, Go, and .NET teams all use variants of this pattern because it solves a team coordination problem, not a language-specific problem.

Connection to the Reader's Existing Model

The familiar workflow probably had these landmarks:

- open the solution in Visual Studio;
- make a change;
- run locally with IIS Express or a local IIS site;
- maybe run unit tests;
- check in code;
- wait for a build or manually produce one;
- hand off to QA or operations;
- deploy by copying files, running a script, or using a release tool;
- troubleshoot on the server if something failed.

Modern development keeps the same broad intent but makes more of the workflow explicit and repeatable.

Visual Studio is still a powerful local environment, but it is no longer the only place where the project must build. A build should work from the command line, on another developer's machine, and on a clean build agent.

Source control is no longer just a backup and history mechanism. It is the coordination point for branches, pull requests, review, automated checks, security scanning, release notes, and sometimes infrastructure definitions.

A pull request is similar to a formal code review or change request, but it is attached directly to the code diff, tests, comments, approvals, and automation results.

Automated tests are similar to regression test scripts, but they execute as part of the normal change process instead of waiting for a separate late-stage test phase.

A CI check is similar to a build-server validation, but modern teams expect it to run frequently, consistently, and close to the code review.

A development environment is still a workstation-like place where you write and run code. The difference is that the environment may be local, container based, cloud hosted, or standardized through repository configuration.

An AI coding assistant is not a replacement for the developer. It is closer to an always-available pair programmer that can suggest code, explain APIs, draft tests, summarize changes, and help navigate unfamiliar areas. The analogy is useful as long as you remember that the assistant does not own correctness, security, or design judgment.

The old model often relied on expert memory: the senior developer knew how to build it, the release engineer knew how to deploy it, and the production admin knew which server setting mattered. The modern model tries to turn that memory into visible workflow.

Layer 1 — Conceptual Model

Modern development workflow is the repeatable path by which a team turns an idea into a safe, reviewed, validated, deployable software change.

It exists to solve five recurring problems.

First, integration delay. When developers work separately for too long, changes conflict, assumptions diverge, and bugs are discovered late. Short-lived branches, frequent commits, pull requests, and CI reduce the cost of bringing work together.

Second, environment drift. If every developer's machine, test server, and production server is slightly different, success in one place does not predict success in another. SDK-based builds, containerized dependencies, documented setup, and cloud development environments reduce drift.

Third, review ambiguity. A hallway conversation or email thread is easy to lose. Pull requests connect the code, conversation, tests, and approval record.

Fourth, manual validation. Humans are good at judgment and design review. Humans are poor at repeatedly remembering every mechanical check. Automated builds, tests, linters, formatters, and scans handle repeatable validation so people can focus on meaning.

Fifth, production disconnect. In older workflows, developers could finish a change and lose sight of it once it crossed into QA or operations. Modern workflow keeps feedback connected through deployment status, telemetry, incidents, and post-release learning.

What the workflow does not solve by itself:

- It does not guarantee good architecture.
- It does not make tests valuable just because they are automated.
- It does not replace code review judgment.
- It does not remove the need for production ownership.
- It does not make AI-generated code correct.
- It does not make a broken team healthy through tools alone.

The workflow is a support system for disciplined engineering. It makes good habits easier to repeat.

Layer 2 — System Relationships

The modern workflow has several connected actors and boundaries.

The developer owns the local change. Inputs include a work item, branch, existing code, local environment, tests, and context from previous chapters or documentation. Outputs include commits, tests, documentation updates, and a pull request.

The repository owns shared history. It receives commits and branches. It outputs version history, diffs, merge records, tags, and sometimes release artifacts. Its failure boundaries include bad branching practices, weak permissions, missing history, and unclear ownership.

The pull request owns review coordination. It receives a proposed diff, description, linked work item, review comments, automated check results, and updates from the author. It outputs an approved, rejected, or revised change. Its failure boundaries include oversized changes, vague descriptions, rubber-stamp review, stale branches, and unresolved discussion.

The test suite owns repeatable validation. It receives compiled code, test data, dependencies, and environment configuration. It outputs pass/fail signals and diagnostics. Its failure boundaries include flaky tests, missing coverage for important behavior, over-mocked tests, slow feedback, and tests that assert implementation details instead of behavior.

The CI system owns clean validation. It receives repository events such as commits or pull requests. It checks out the code, restores dependencies, builds the solution, runs tests, and reports status. Its failure boundaries include broken agents, missing secrets, unavailable package feeds, nondeterministic tests, and mismatched SDK versions.

The code review process owns human judgment. It receives the author's intent, the diff, test evidence, design context, and automated results. It outputs feedback, suggestions, approval, or a request for changes. Its failure boundaries include reviewing style while missing behavior, deferring too much to automation, or treating review as personal criticism instead of shared ownership.

The development environment owns local productivity. It receives repository configuration, SDKs, tools, containers, secrets for local development, and IDE settings. It outputs fast feedback. Its failure boundaries include slow setup, hidden machine-specific assumptions, missing dependencies, and local-only success.

The AI coding assistant owns suggestion and acceleration, not authority. It receives prompts, visible code context, chat history, repository files, and possibly tool access depending on the environment. It outputs suggestions, explanations, edits, tests, or summaries. Its failure boundaries include plausible but wrong code, outdated assumptions, insecure suggestions, data exposure, and over-trust.

The deployment process owns movement toward runtime environments. In this chapter, deployment remains a high-level concept. Later chapters teach the mechanics. For now, understand that the output of development should be a validated change that can become a deployable artifact through a controlled path.

Layer 3 — Core Mechanics

The smallest useful workflow is a branch and pull request with automated validation.

```
main branch
|
+-- feature branch
    |
    v
    commits
    |
    v
    pull request
    |
    +-- human review
    |
    +-- automated build and tests
    |
    v
    merge
    |
    v
main branch with validated change
```

The main branch represents the shared integration line. Some teams call it `main`, some use `master`, and some use trunk-based development language. The name matters less than the rule: the shared line should be kept healthy.

A branch is an isolated line of work. Modern teams generally prefer short-lived branches because they reduce integration delay.

A commit is a recorded change. Good commits make the history understandable. They are not just save points; they are communication.

A pull request proposes that branch changes be merged into another branch. It contains the diff, discussion, review state, and automated checks.

A code review is a human evaluation of the proposed change. GitHub pull request reviews support comments, suggestions, approval, and requests for changes.

A check is an automated result attached to the proposed change. Checks might build the solution, run tests, enforce formatting, scan dependencies, or produce preview artifacts.

Continuous integration is the practice of automatically building and testing code when changes are committed to version control. It exists to catch integration problems early.

Continuous delivery is the broader practice of automating build, test, configuration, and deployment movement toward environments. This chapter mentions it only as part of the lifecycle. The detailed release pipeline comes later.

A development environment is the place where the developer writes and runs the application. It can be a local machine, a local container setup, a cloud environment such as GitHub Codespaces, or a managed developer workstation such as Microsoft Dev Box.

An AI coding assistant is a tool that uses large language models to assist with coding tasks. Examples include inline suggestions, chat-based code explanation, refactoring help, test generation, and debugging support. The state-of-the-art direction in 2026 is broader than autocomplete: assistants are increasingly integrated into IDEs, repositories, pull requests, terminals, cloud consoles, and agent-like workflows. That power makes review and judgment more important, not less.

Layer 4 — Developer Workflow

Here is the modern workflow in practical terms.

Start with a small piece of work. It might come from a backlog item, bug report, incident follow-up, architecture decision, or direct user need. The unit of work should be small enough that someone else can review it without reconstructing the entire system.

Update your local repository:

```
git status
git pull
```

Create a branch:

```
git switch -c feature/add-order-status
```

Run the application or tests before changing code when practical. This gives you a baseline:

```
dotnet build
dotnet test
```

Make the change in your editor or IDE. Visual Studio, Visual Studio Code, JetBrains Rider, Codespaces, and Dev Box can all participate in modern .NET workflows. The specific tool matters less than whether the workflow is repeatable outside one tool.

Use an AI coding assistant carefully if available. Good uses include asking it to explain unfamiliar code, draft a test shape, identify edge cases, summarize a diff, or suggest a refactoring. Risky uses include accepting large generated changes without understanding them, pasting secrets into prompts, or treating generated code as reviewed code.

Run local validation:

```
dotnet format --verify-no-changes
dotnet build
dotnet test
```

The exact commands vary by repository. Some teams use scripts, `make`, `just`, PowerShell, npm, Docker Compose, or custom build tools. The principle is that the repository should expose a known way to validate the change.

Commit the change:

```
git status
git add .
git commit -m "Add order status validation"
```

Push the branch:

```
git push -u origin feature/add-order-status
```

Open a pull request. The pull request description should explain what changed, why it changed, how it was tested, and any risk or follow-up work. If the change has screenshots, logs, migration notes, or API examples, include them.

Respond to review. Review is not a ceremony after the real work; it is part of the work. A reviewer may find a missed case, unclear name, fragile test, hidden security issue, or simpler design.

Wait for automated checks. If checks fail, fix the branch rather than hoping the failure is unrelated. If the failure is flaky, treat that as a workflow problem worth improving.

Merge after review and checks pass. Depending on team policy, the merge may create a merge commit, squash commits, or rebase. That policy is less important than keeping shared history understandable and the main branch healthy.

After merge, the change moves toward deployment. In a mature environment, the same change can be traced from work item to pull request, commit, build artifact, deployment, and production telemetry.

Layer 5 — Production Usage

Workflow choices affect production quality.

Configuration should be separated from code. Local development may use local settings or user secrets. CI may use test configuration. Production may use environment variables, managed secret stores, or platform configuration. Workflow should make it hard to accidentally commit secrets.

Security should shift earlier without pretending development can catch everything. Modern workflows often include dependency scanning, secret scanning, branch protection, least-privilege repository access, required reviews, and controlled deployment credentials. These practices reduce risk before the application reaches production.

Reliability starts before deployment. Small changes are easier to review, test, deploy, and roll back. Automated tests reduce regression risk. Pull requests capture design judgment. CI confirms that the change works outside the author's machine.

Deployment should consume validated artifacts. A common failure in older systems was rebuilding or modifying files during release in ways that made it unclear what actually reached production. Modern workflows try to build once, identify the artifact, and move that artifact through environments.

Observability closes the loop. Production logs, metrics, traces, errors, and alerts tell the team whether the change behaved as expected. Without feedback, deployment is a cliff.

Scaling affects workflow because larger teams need clearer boundaries. A three-person team can coordinate with conversation. A thirty-person team needs branch policies, ownership rules, review expectations, consistent local setup, and automation that prevents accidental damage.

Persistence affects workflow because database changes must be coordinated with application changes. A schema migration, data correction, or index change is not just “code.” It has deployment ordering, rollback, locking, backup, and performance implications.

Cost affects workflow when builds, cloud development environments, preview environments, AI tools, and test infrastructure are metered. A modern workflow should give fast feedback without silently creating waste.

Local-development arrangements optimize for fast feedback. Production designs optimize for reliability, security, auditability, and recoverability. A good workflow keeps those goals connected without pretending they are identical.

Layer 6 — Tradeoffs and Alternatives

There is no single correct workflow for every team.

Small teams may use lightweight GitHub repositories, short branches, pull requests, and a few required checks. Larger enterprises may add issue tracking, change approval, security gates, compliance evidence, release trains, and environment promotion.

GitHub is widely used, especially for open source and many modern product teams. Azure DevOps remains common in Microsoft-oriented enterprises. GitLab, Bitbucket, Jenkins, TeamCity, CircleCI, and Buildkite are also valid choices. The workflow principles matter more than the brand: versioned code, review, automated validation, traceable delivery, and feedback from production.

Pull requests are not the only collaboration model. Some high-performing teams use trunk-based development with very short-lived branches and strong automation. Some regulated environments require heavier approval processes. Some open-source projects rely on maintainer review across forks. The best choice depends on team size, risk, deployment frequency, and compliance needs.

Local development is not always enough. A standard workstation may be fastest for many developers. A containerized setup may reduce dependency drift. GitHub Codespaces can provide cloud-hosted, repository-configured development environments. Microsoft Dev Box can provide secure, managed cloud development workstations. These options solve setup and consistency problems, but they add cost and administrative decisions.

AI coding assistants can accelerate exploration, routine code, test drafts, documentation, and unfamiliar API usage. They can also produce confident mistakes. Use them as collaborators under review, not as authorities.

Common overengineering mistakes:

- requiring a pull request for every tiny experimental spike;
- creating so many pipeline gates that developers stop trusting the system;
- running slow checks on every keystroke-level change;
- adding cloud development environments before local setup is understood;
- treating code review as permission bureaucracy instead of design feedback;
- accepting AI-generated code because it looks polished;
- confusing a sophisticated workflow with a healthy engineering culture.

Simpler alternatives are sometimes better. A small internal tool may need Git, a README, a build command, and a modest test suite before it needs preview environments and advanced deployment rings. More advanced alternatives are worthwhile when the risk and scale justify them: trunk-based development, ephemeral environments, progressive delivery, feature flags, policy-as-code, supply-chain signing, and platform-engineered paved paths.

The state-of-the-art direction is not heavier process. It is shorter feedback loops with stronger guardrails.

Layer 7 — Interview Perspective

Interviewers use workflow questions to learn how you work with a team.

Concepts commonly tested:

- the purpose of source control beyond backup;
- branch and pull request workflow;
- what belongs in a pull request description;
- the difference between local tests and CI checks;
- the purpose of code review;
- how to handle failed builds;
- how AI coding assistants should and should not be used;
- how a code change moves toward production.

Representative questions:

- “Walk me through your development workflow from task to merge.”

- “What makes a pull request easy to review?”
- “What should happen before code is merged to the main branch?”
- “How do you respond when CI fails but the code works locally?”
- “What is the difference between continuous integration and continuous delivery?”
- “How do you use AI coding tools responsibly?”
- “How would you improve a team that deploys manually and often has release surprises?”

A strong answer describes feedback loops. For example:

“I try to keep changes small, run local validation before opening the pull request, explain the intent and test evidence, respond to review, and treat CI failures as real until understood. After merge, the change should be traceable through build and deployment so production feedback can be tied back to the code.”

Common misconceptions:

- “Source control is mainly a backup.”
- “A pull request is only for catching syntax mistakes.”
- “If it works locally, CI is optional.”
- “Code review is less important when tests pass.”
- “AI-generated code does not need the same review as human-written code.”
- “DevOps means developers do all operations work.”
- “Continuous delivery means every commit must go directly to production.”

Small design scenario:

You join a team maintaining a .NET line-of-business application. Developers work directly on a shared branch, releases are manual, tests are mostly run by QA near the end, and production fixes often happen under pressure.

A good modernization plan would be incremental:

- move to short-lived branches and pull requests;
- require a clear description and reviewer before merge;
- add a clean build check;
- add a small but meaningful automated test set;
- document local setup;
- protect the main branch;
- make release artifacts identifiable;
- improve production logging before attempting aggressive deployment automation;
- introduce AI coding tools with review and security expectations.

The answer should improve feedback before adding ceremony.

Hands-On Lab

Objective:

Practice the shape of a modern workflow using a small repository-level change.

Prerequisites:

- Git installed.
- .NET SDK installed.
- A local repository.
- No remote repository is required for the local parts of this lab.

Steps:

1. Inspect the repository state:

```
git status
```

2. Create a branch:

```
git switch -c workflow-practice
```

3. Inspect the installed .NET SDK:

```
dotnet --info
```

4. If the repository has a solution or project, run:

```
dotnet build
dotnet test
```

If the repository does not yet contain buildable code, write down the commands you expect the repository to support once code exists.

5. Create a short pull request draft in a text file or notebook with these headings:

```
Summary
Why
Testing
Risks
Follow-up
```

6. Write a sample review comment against your own hypothetical change. Make it specific, respectful, and actionable.
7. Write one AI-assistant prompt that would be useful and safe for this work. For example:

```
Review this small diff for edge cases and missing tests. Do not rewrite the
implementation unless you find a specific issue.
```

8. Return to your original branch when finished:

```
git switch -
```

Expected results:

- You can describe the workflow from branch to pull request to validation.
- You can distinguish local validation from CI validation.
- You can write a useful pull request description.
- You can state how AI assistance fits into review rather than replacing it.

Validation commands:

```
git status
git branch --show-current
dotnet --info
dotnet build
dotnet test
```

Troubleshooting notes:

- If `git switch -c` fails because the branch already exists, choose another branch name.
- If `dotnet build` fails because there is no project file, that is acceptable for this chapter. The next chapters begin building code.
- If `dotnet test` reports no test projects, note that as a future workflow gap rather than an error.
- If you use an AI assistant, do not paste secrets, private credentials, or confidential production data into the prompt.

Knowledge Check

1. Why is modern development workflow best understood as a feedback loop?
2. What problems do short-lived branches reduce?
3. What information should a pull request description provide to reviewers?
4. Why are automated checks useful even when reviewers are experienced?
5. How is continuous integration different from continuous delivery?
6. Why is “works on my machine” weaker evidence than a passing CI build?
7. What kinds of comments make code review more useful?
8. How can cloud development environments reduce onboarding friction?
9. What risks do AI coding assistants introduce into the workflow?
10. Why should production telemetry feed back into development planning?

Summary

Modern .NET development workflow is the path from idea to validated change to production feedback.

The core practices are not mysterious: work in small increments, keep source history clear, review proposed changes, automate repeatable validation, protect the shared branch, package changes consistently, and learn from production behavior. The tools have changed, but the goal is familiar: make software change safer.

Compared with older workstation-and-server workflows, the modern workflow places more responsibility in shared systems: Git repositories, pull requests, CI checks, documented development environments, controlled delivery, and observability. AI coding assistants now participate in that workflow, but they do not replace the developer’s responsibility for correctness, security, and design.

The next chapter begins the practical .NET layer. With the workflow map in place, you can now look at modern .NET versions, SDKs, project structure, cross-platform development, and language improvements as parts of a larger professional system.

Sources

- Microsoft Learn, “What is DevOps?”: <https://learn.microsoft.com/en-us/devops/what-is-devops>
- Microsoft Learn, “Use continuous integration”: <https://learn.microsoft.com/en-us/devops/develop/what-is-continuous-integration>
- Microsoft Learn, “What is continuous delivery?”: <https://learn.microsoft.com/en-us/devops/deliver/what-is-continuous-delivery>
- GitHub Docs, “Quickstart for reviewing pull requests”: <https://docs.github.com/en/pull-requests/get-started/reviewing-pull-requests-quickstart>
- GitHub Docs, “Requesting a pull request review”: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/requesting-a-pull-request-review>
- GitHub Docs, “What are GitHub Codespaces?”: <https://docs.github.com/en/codespaces/about-codespaces/what-are-codespaces>
- Microsoft Learn, “Open a dev box in VS Code”: <https://learn.microsoft.com/en-us/azure/dev-box/how-to-set-up-dev-tunnels>
- Microsoft Learn, “GitHub Copilot Fundamentals”: <https://learn.microsoft.com/en-us/training/paths/copilot/>

Further Reading

- Microsoft Learn, “Deliver with DevOps”: <https://learn.microsoft.com/en-us/training/modules/deliver-with-devops/>
- GitHub Docs, “About pull requests”: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests>
- GitHub Docs, “GitHub Actions documentation”: <https://docs.github.com/en/actions>
- Microsoft Learn, “Adopt, extend and build Copilot experiences across the Microsoft Cloud”: <https://learn.microsoft.com/en-us/microsoft-cloud/dev/copilot/overview>

4

Modern .NET

Chapter Purpose

The first three chapters established the modern .NET landscape, the technology stack, and the development workflow. This chapter moves into the platform at the center of that system: modern .NET itself.

For an experienced developer returning from the .NET 5 era, the important shift is not that C# suddenly became unfamiliar. The shift is that .NET is now a fast-moving, cross-platform, SDK-driven platform with a regular support cadence, simpler project files, stronger command-line workflows, first-class container and cloud integration, and ongoing language evolution. Modern .NET exists because the old .NET world had reached a boundary. .NET Framework was powerful and deeply integrated with Windows, IIS, Visual Studio, and enterprise infrastructure, but that same integration made it hard to move at cloud speed. The industry was moving toward open source, Linux servers, containers, command-line automation, microservice-friendly runtimes, and frequent platform updates. Java, Node.js, Go, Python, and other ecosystems had already made cross-platform server development feel ordinary.

.NET Core began as Microsoft's answer to that reality: a modular, cross-platform, open-source version of .NET. .NET 5 unified the branding around "one .NET" for modern workloads. By 2026, modern .NET is no longer a side path. It is the main .NET line.

This chapter explains modern .NET versions, SDKs, project structure, cross-platform development, and language improvements. It gives enough practical mechanics to orient you without turning into a full API reference.

Where This Fits

Modern .NET is the application platform layer underneath the rest of the book.

```
Developer workflow
|
v
.NET SDK and project system
|
v
C# source code, project files, NuGet packages
|
v
Build, test, publish
|
v
Runtime process
|
+-- ASP.NET Core
+-- worker services
+-- console tools
```

```
+-- libraries
|
v
Windows, Linux, containers, cloud platforms
```

Chapter 5 builds on this foundation with ASP.NET Core. Chapter 6 applies it to data access. Chapter 8 uses it for automated testing and CI. Chapters 9-13 use it across Linux, Docker, deployment, and cloud hosting.

The key point: modern .NET is both a runtime and a development platform. The runtime runs applications. The SDK builds, tests, publishes, packages, and tools them. The project system describes how source code becomes output.

Connection to the Reader's Existing Model

You already understand .NET as a managed platform: C# compiles to assemblies, the runtime executes managed code, libraries provide reusable APIs, and project files describe how the application is built.

That model still works, but several boundaries have moved.

In .NET Framework, the installed framework felt like part of the Windows machine. Applications targeted a framework version that was installed on the server. Visual Studio was often the center of build and publish behavior. Project files were verbose. IIS was the natural web host. Windows was assumed.

In modern .NET, the SDK is the center of development. The command line can create, build, test, run, publish, and package applications. Visual Studio, Visual Studio Code, Rider, GitHub Actions, Azure Pipelines, Docker builds, and cloud development environments all use the same underlying project and SDK model.

The modern project file is much smaller because SDK-style projects infer many defaults. A project file no longer needs to list every source file by default. NuGet package references are first-class. Target frameworks are explicit.

Side-by-side installation is normal. A machine can have several SDKs and runtimes installed. A repository can use `global.json` to select the SDK range used by command-line builds. This is very different from thinking of the server as having one blessed framework personality.

Cross-platform hosting is normal. A .NET application may be developed on Windows, built on Linux, tested in CI, packaged into a container, and deployed to a managed cloud environment. That does not mean Windows is obsolete. It means Windows is one supported environment rather than the assumption under every layer.

The IIS analogy remains useful for understanding process hosting, ports, configuration, and deployment. It breaks down when you assume IIS owns the application lifecycle. In modern .NET, an ASP.NET Core application is itself a process. IIS may reverse proxy to it, but so may Nginx, a cloud load balancer, a container platform, or a local development server.

Layer 1 — Conceptual Model

Modern .NET is the current, cross-platform .NET implementation for building applications, services, libraries, tools, cloud workloads, desktop apps, mobile apps, games, and AI-enabled systems.

It solves several problems:

- it provides a common managed runtime for C# and other .NET languages;
- it supports cross-platform development and hosting;
- it supplies a consistent SDK and command-line workflow;
- it uses a simpler project format;
- it supports side-by-side versions and regular releases;
- it integrates naturally with packages, tests, containers, cloud platforms, telemetry, and modern tooling.

It does not solve every application concern:

- it does not choose your architecture;
- it does not make your code maintainable automatically;

- it does not remove the need to understand deployment;
- it does not replace SQL Server, Redis, or cloud platforms;
- it does not guarantee cross-platform behavior if your code uses platform-specific APIs;
- it does not make every old .NET Framework application portable without work.

The conceptual center is this:

```
SDK describes how to build.  
Project file describes what to build.  
Target framework describes which APIs are available.  
Runtime executes the built application.
```

Once that model is clear, most modern .NET mechanics become easier to place.

Layer 2 — System Relationships

Modern .NET interacts with the rest of the system through a few important relationships.

The SDK receives project files, source files, package references, target frameworks, analyzers, tool manifests, and build properties. It outputs compiled assemblies, test results, NuGet packages, publish folders, container images in some scenarios, and diagnostic information.

The runtime receives compiled assemblies, dependencies, configuration, command line arguments, environment variables, and operating-system services. It outputs process behavior: HTTP responses, console output, logs, files, network calls, database calls, and exit codes.

The project file owns build intent. It tells the SDK which project SDK to use, which target framework to compile against, which packages to reference, and which build options matter. In SDK-style projects, many conventions are implicit, which keeps the file small but makes it important to understand the defaults.

NuGet owns package distribution. A modern .NET project usually depends on NuGet packages from nuget.org, private feeds, or local package sources. Package restore is therefore part of build reliability and supply-chain security.

The operating system owns process execution, files, environment variables, networking, certificates, native dependencies, and permissions. If an application uses only cross-platform .NET APIs, it can often run on Windows, Linux, and macOS. If it depends on Windows-specific APIs, COM components, registry behavior, GDI, or old framework libraries, portability becomes a design constraint.

The CI system owns clean reproducibility. It uses the SDK to restore, build, test, and publish without relying on a developer's machine. If the build works only inside one person's IDE, the project is not yet modern in workflow terms.

The deployment platform owns runtime placement. It may run the app as a Windows service, IIS-hosted process, Linux systemd service, container, managed app service, serverless function, or orchestrated workload. .NET supports many of these targets, but each target changes configuration and operations.

Failure boundaries include mismatched SDK versions, unsupported target frameworks, missing runtimes, package restore failures, platform-specific code, native dependency issues, incorrect runtime identifiers, broken publish settings, and assuming local development behavior will match production.

Layer 3 — Core Mechanics

The modern .NET mechanics begin with versions.

Microsoft ships major .NET releases annually. Releases alternate between Long Term Support (LTS) and Standard Term Support (STS). As of July 22, 2026, Microsoft lists .NET 10 as an LTS release supported until November 2028, .NET 9 as an STS release supported until November 2026, and .NET 8 as an LTS release supported until November 2026.

That version choice matters. For production enterprise applications, an LTS release is often the conservative default. An STS release may be useful when a team wants newer platform capabilities and accepts a faster upgrade rhythm.

The next mechanic is the distinction between SDK and runtime.

The runtime runs .NET applications. The SDK creates and builds them. A server may need only the runtime if it runs framework-dependent applications. A developer machine and CI agent need the SDK.

The target framework tells the compiler which API surface the project uses. For modern .NET, this appears as a target framework moniker:

```
<TargetFramework>net10.0</TargetFramework>
```

A library can target multiple frameworks when it needs to support more than one consumer:

```
<TargetFrameworks>net8.0;net10.0</TargetFrameworks>
```

Multi-targeting is useful for libraries. It is usually unnecessary for a simple application where the team controls the runtime environment.

The project SDK appears at the top of an SDK-style project file:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

For a web application, the project uses the web SDK:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

`ImplicitUsings` reduces repetitive `using` directives for common namespaces. `Nullable` enables nullable reference type analysis, which helps identify possible null-related bugs at compile time. If you are returning from older C# habits, nullable reference types are one of the most important language-era changes to take seriously.

Runtime identifiers, or RIDs, identify target platforms such as `win-x64`, `linux-x64`, and `osx-arm64`. They matter when publishing for a specific platform or using packages with native assets.

`global.json` can select the SDK version used by CLI commands in a repository:

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  }
}
```

This does not choose the runtime your project targets. The target framework does that. `global.json` chooses the SDK used to build.

C# evolves with .NET. As of .NET 10, C# 14 is the latest C# release. It includes features such as extension members, null-conditional assignment, `field` backed properties, improved `Span<T>` conversions, and file-based app support. You do not need to memorize every new feature immediately. The important production habit is to distinguish language convenience from design clarity.

Layer 4 — Developer Workflow

The modern .NET workflow is SDK-first. You can still use Visual Studio, and many enterprise developers should, but the project should also behave predictably from the command line.

Check your environment:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

Create a console application:

```
dotnet new console -n ModernDotnetDemo
cd ModernDotnetDemo
dotnet run
```

Inspect the project file:

```
cat ModernDotnetDemo.csproj
```

On Windows PowerShell:

```
Get-Content .\ModernDotnetDemo.csproj
```

Build and run:

```
dotnet build
dotnet run
```

Create a solution and add the project:

```
cd ..
dotnet new sln -n ModernDotnetDemo
dotnet sln ModernDotnetDemo.sln add ModernDotnetDemo/ModernDotnetDemo.csproj
```

Create a local tool manifest when a repository needs repeatable local tools:

```
dotnet new tool-manifest
dotnet tool restore
```

Local tools matter because they let a repository define tool dependencies without requiring every developer to install the same global tool manually. Examples later in the book may include formatters, EF Core tools, or other project-specific commands. Create a `global.json` when a repository needs predictable SDK selection:

```
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestFeature
```

The exact SDK version should match the version your team supports. In CI, being explicit avoids surprising builds when an agent image updates.

Publish the application:

```
dotnet publish -c Release
```

This produces output that can be run on a machine with an appropriate runtime or packaged into a deployment artifact. Later chapters cover ASP.NET Core, containers, and deployment in detail.

The workflow principle is simple: the same basic commands should work locally, in CI, and in deployment automation.

Layer 5 — Production Usage

Production .NET usage starts with support.

Choose a supported .NET version. For most enterprise production applications, that usually means choosing an LTS version unless a specific STS feature justifies the shorter support window. Keep applications patched with servicing updates because Microsoft support expects supported releases to be on supported servicing levels.

Separate SDK concerns from runtime concerns. Build agents need SDKs. Runtime hosts may need only runtimes, or no preinstalled runtime if you publish self-contained or run in a container image that includes the runtime. Do not assume the production machine has the same SDKs as a developer workstation.

Configuration belongs outside the compiled application. Modern .NET supports configuration from JSON files, environment variables, command-line arguments, secret stores, and platform providers. Local development can use local settings. Production should use mechanisms appropriate to the host platform.

Secrets should not be committed to source control. User secrets can be useful for local development. Production secrets should come from managed secret stores, environment injection, platform identity, or deployment systems.

Reliability depends on predictable builds and repeatable publishes. A production artifact should be identifiable. You should know which commit, target framework, SDK, package versions, and deployment settings produced it.

Observability starts in the application. Even before advanced telemetry, modern .NET applications should emit meaningful logs and expose health where appropriate. ASP.NET Core expands on this in the next chapter.

Cross-platform production requires discipline. File paths, case sensitivity, line endings, certificates, native libraries, time zones, globalization, process signals, and filesystem permissions can differ between Windows and Linux. Code that assumes Windows behavior may compile but still fail when hosted elsewhere.

Cost enters through runtime choices. Self-contained deployments can be larger. Containers introduce base-image and registry management. Cloud hosts charge for compute, memory, storage, networking, and sometimes build minutes. Newer features such as Native AOT can reduce startup time and footprint in some workloads, but they also introduce compatibility and diagnostic tradeoffs.

Local development optimizes for speed and convenience. Production optimizes for supportability, security, repeatability, and clear ownership.

Layer 6 — Tradeoffs and Alternatives

Modern .NET is a strong choice when a team values C#, mature tooling, enterprise libraries, high-performance server workloads, Microsoft ecosystem integration, and cross-platform deployment.

It competes with Java, Go, Node.js, Python, Rust, Kotlin, Ruby, PHP, and other ecosystems. Java remains a major enterprise platform with Spring. Go is common for small, fast cloud-native services and infrastructure tooling. Node.js is natural for JavaScript-centered teams. Python dominates many data and automation workflows. Rust is compelling when memory safety and systems-level control matter. Modern .NET's advantage is the combination of C#, runtime performance, enterprise fit, tooling, and platform breadth.

Use modern .NET for new .NET applications unless you have a specific reason not to. Use .NET Framework only when the application depends on technologies that require it, such as some legacy ASP.NET, WCF server scenarios, Windows-specific desktop dependencies, or older third-party libraries that cannot move.

Choose LTS for conservative production systems. Choose STS only when the team has an explicit upgrade plan and wants newer features sooner.

Use SDK-style projects for modern projects. Avoid manually expanding project files into old-style file lists unless a special build requirement demands it.

Use multi-targeting for reusable libraries that must support multiple consumers. Avoid multi-targeting applications without a clear reason; it increases testing and support work.

Use self-contained deployment when you need to carry the runtime with the app. Use framework-dependent deployment when the host environment manages the runtime. Use containers when repeatable packaging and runtime environment consistency matter.

Use Native AOT selectively. It can improve startup time and deployment size for some workloads, especially small services and command-line tools, but it can limit reflection-heavy libraries and change diagnostics.

Common overengineering mistakes:

- upgrading target frameworks without checking package and hosting support;
- treating every new C# feature as a reason to rewrite old code;
- multi-targeting applications unnecessarily;
- pinning SDK versions so tightly that servicing becomes painful;
- ignoring nullable warnings because the project still compiles;
- assuming cross-platform means every API behaves identically everywhere;
- choosing self-contained, containerized, trimmed, and AOT deployment all at once before understanding the tradeoffs.

The state-of-the-art direction in modern .NET is pragmatic platform breadth: fast runtime, minimal hosting, container support, cloud integration, better diagnostics, optional Native AOT, strong language evolution, and AI-friendly libraries. The skill is knowing which capabilities the application actually needs.

Layer 7 — Interview Perspective

Interviewers use modern .NET questions to test whether you understand the platform model, not just C# syntax.

Concepts commonly tested:

- .NET Framework versus .NET Core versus modern .NET;
- LTS versus STS releases;
- SDK versus runtime;
- target frameworks and target framework monikers;
- SDK-style project files;
- NuGet package references;
- `global.json`;
- framework-dependent versus self-contained deployment;
- cross-platform hosting;
- nullable reference types;
- when to use newer language features.

Representative questions:

- “What changed between .NET Framework and modern .NET?”
- “What is the difference between the .NET SDK and the runtime?”
- “Why would a project use `global.json`?”
- “What does `net10.0` mean?”
- “When would you choose an LTS release?”
- “What makes SDK-style project files different from older project files?”
- “What should you check before running a .NET application on Linux?”
- “When would self-contained deployment be useful?”

A strong answer connects versioning, build, and runtime:

“The target framework controls which APIs the project compiles against. The SDK builds the project, and `global.json` can influence which SDK is selected for CLI commands. The runtime is what actually executes the application. In production, those choices affect support, deployment size, patching, and compatibility.”

Common misconceptions:

- “.NET is just a renamed .NET Framework.”
- “The SDK and runtime are the same thing.”
- “`global.json` chooses the target framework.”
- “If it compiles on Windows, it is automatically Linux-ready.”
- “LTS means the application never needs patching.”
- “New C# syntax is always a net improvement.”
- “Self-contained deployment is always better.”

Small design scenario:

You inherit a .NET 5 internal API. It builds only in Visual Studio, has no `global.json`, targets an out-of-support framework, and is deployed manually to a Windows server.

A good modernization plan would:

- move to a supported LTS target framework;
- verify package compatibility;
- make the build work through `dotnet build`;
- add or update `global.json` if the team needs SDK predictability;
- enable nullable analysis thoughtfully if it is not already enabled;
- define a repeatable publish command;
- confirm whether the app has Windows-specific assumptions;
- defer containers, cloud migration, and advanced deployment until the platform baseline is healthy.

The strong answer upgrades the foundation before changing the hosting model.

Hands-On Lab

Objective:

Create a small modern .NET console application and inspect the SDK-style project model.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A terminal.
- A text editor or IDE.

Steps:

1. Create a working folder:

```
mkdir ModernDotnetLab
cd ModernDotnetLab
```

2. Check installed SDKs and runtimes:

```
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

3. Create a console app:

```
dotnet new console -n HelloModernDotnet
cd HelloModernDotnet
```

4. Open `HelloModernDotnet.csproj` and identify:

- the project SDK;
- the target framework;

- whether nullable reference types are enabled;
- whether implicit usings are enabled.

5. Run the app:

```
dotnet run
```

6. Build in Release configuration:

```
dotnet build -c Release
```

7. Publish the app:

```
dotnet publish -c Release
```

8. Create a solution one directory above the project:

```
cd ..  
dotnet new sln -n ModernDotnetLab  
dotnet sln ModernDotnetLab.sln add HelloModernDotnet/HelloModernDotnet.csproj
```

9. Create a `global.json`:

```
dotnet new globaljson --roll-forward latestFeature
```

10. Re-run:

```
dotnet --info
```

Expected results:

- You can identify the SDK and runtime versions installed.
- You can explain the project SDK and target framework in the project file.
- You can build, run, and publish from the command line.
- You can explain what `global.json` does and does not control.

Validation commands:

```
dotnet --info  
dotnet build  
dotnet run  
dotnet publish -c Release
```

Troubleshooting notes:

- If `dotnet` is not recognized, install the .NET SDK and reopen the terminal.
- If `dotnet new console` targets a different framework than expected, inspect which SDK is installed and selected.
- If `global.json` causes an SDK error, the requested SDK may not be installed or the roll-forward setting may be too restrictive.
- If publish output is hard to find, inspect the `bin/Release` folder under the project.

Knowledge Check

1. Why did modern .NET move away from the Windows-centered assumptions of .NET Framework?
2. What is the difference between the .NET SDK and the .NET runtime?
3. Why does the target framework matter to both compilation and production support?
4. When is an LTS release usually preferable to an STS release?
5. What does an SDK-style project file hide through convention?

6. What does `global.json` control, and what does it not control?
7. Why can cross-platform development still fail when the code compiles?
8. When would multi-targeting be useful?
9. What production tradeoff does self-contained deployment make?
10. Why should nullable reference type warnings be treated as design feedback instead of noise?

Summary

Modern .NET is the main .NET platform for current professional development. It grew out of the need for a cross-platform, open-source, cloud-ready, SDK-driven successor to the Windows-centered .NET Framework world.

The central mechanics are straightforward once the layers are separated. The SDK builds and tools applications. The runtime executes them. The project file describes build intent. The target framework describes the API surface. NuGet packages provide dependencies. `global.json` can select the SDK used by command line builds. Runtime identifiers describe target platforms when platform-specific publishing or native assets matter.

Modern .NET's power is not only newer APIs. It is that the same project model can participate in local development, command-line builds, CI, containers, Linux hosting, cloud platforms, and production deployment.

The next chapter builds on this foundation with ASP.NET Core, where modern .NET becomes a web and API application platform.

Sources

- Microsoft Learn, “.NET releases, patches, and support”: <https://learn.microsoft.com/en-us/dotnet/core/releases-and-support>
- Microsoft Learn, “What’s new in .NET 10”: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview>
- Microsoft Learn, “.NET project SDKs”: <https://learn.microsoft.com/en-us/dotnet/core/project-sdk/overview>
- Microsoft Learn, “Target frameworks in SDK-style projects”: <https://learn.microsoft.com/en-us/dotnet/standard/frameworks>
- Microsoft Learn, “.NET Runtime Identifier catalog”: <https://learn.microsoft.com/en-us/dotnet/core/rid-catalog>
- Microsoft Learn, “global.json overview”: <https://learn.microsoft.com/en-us/dotnet/core/tools/global-json>
- Microsoft Learn, “Install .NET on Windows”: <https://learn.microsoft.com/en-us/dotnet/core/install/windows>
- Microsoft Learn, “What’s new in C# 14”: <https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-14>

Further Reading

- Microsoft Learn, “.NET CLI overview”: <https://learn.microsoft.com/en-us/dotnet/core/tools/>
- Microsoft Learn, “.NET tools”: <https://learn.microsoft.com/en-us/dotnet/core/tools/global-tools>
- Microsoft Learn, “C# language reference”: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/>
- Microsoft Learn, “.NET application publishing overview”: <https://learn.microsoft.com/en-us/dotnet/core/deploying/>

5

ASP.NET Core

Chapter Purpose

Modern .NET becomes most visible when it is used to build web applications and HTTP APIs. ASP.NET Core is the framework that makes that happen.

If your previous web development model was ASP.NET MVC or Web API hosted in IIS, ASP.NET Core will feel familiar in some places and very different in others. It still handles HTTP requests, routing, controllers, configuration, dependency injection, logging, authentication, and responses. But it is no longer built around System.Web, IIS, or a Windows-only hosting model.

ASP.NET Core exists because the classic ASP.NET stack had become too tightly coupled to Windows, IIS, and a large framework surface. The industry had moved toward lightweight HTTP services, Linux hosting, containers, reverse proxies, cloud platforms, and high-throughput APIs. Frameworks such as Node.js Express, Ruby on Rails, Spring Boot, Django, FastAPI, and Go's HTTP ecosystem shaped developer expectations around simple startup, explicit routing, middleware, and portable hosting.

ASP.NET Core was Microsoft's answer: a redesigned, cross-platform, high-performance web framework for modern .NET. By 2026, it is the standard web framework for current .NET applications.

This chapter introduces ASP.NET Core's place in the system and its core building blocks: Minimal APIs, controllers, dependency injection, configuration, middleware, and logging. Later chapters go deeper into data, security, testing, deployment, observability, and cloud hosting.

Where This Fits

ASP.NET Core sits between users or clients and the rest of the application system.

```

Browser, mobile app, service client, or integration partner
|
v
HTTP request
|
v
ASP.NET Core host
|
+-- Middleware pipeline
|
+-- Routing
|
+-- Minimal API endpoint or controller action
|
+-- Application services through dependency injection
|

```

```
+-- Configuration and logging
|
v
Database, cache, queue, file storage, external API, or AI service
```

This chapter is the first point where the book turns the platform concepts from Chapters 1-4 into a running server application. It prepares for Chapter 6 on data access, Chapter 7 on authentication and security, Chapter 8 on testing and CI, and later deployment chapters.

ASP.NET Core is not the whole application. It is the HTTP boundary. A good ASP.NET Core application still needs clear business logic, data access, validation, security, deployment, and observability. The framework gives the request-processing structure where those concerns meet.

Connection to the Reader's Existing Model

Classic ASP.NET applications were naturally understood through IIS.

IIS received the request, selected the site and application pool, loaded the application, applied modules, handed the request into ASP.NET, and wrote the response. Configuration often lived in `web.config`. The application pool identity affected file and database access. IIS logs and Windows Event Log were primary diagnostic tools.

ASP.NET Core changes the center.

An ASP.NET Core application is a .NET process. That process contains a web host, configuration, services, middleware, routes, endpoints, and logging. IIS can still be involved, especially on Windows, but it is often a reverse proxy or hosting integration layer rather than the conceptual owner of the application.

The analogy to IIS application pools is still useful for process isolation. The analogy breaks down because ASP.NET Core can run behind IIS, Nginx, Apache, a cloud load balancer, a container platform, or directly through Kestrel. Kestrel is ASP.NET Core's cross-platform web server.

HTTP modules and handlers from classic ASP.NET map conceptually to middleware and endpoints. Middleware handles cross-cutting request behavior such as exception handling, HTTPS redirection, static files, routing, authentication, authorization, compression, and custom request processing. Endpoints handle the application-specific request.

Controllers remain familiar if you used ASP.NET MVC or Web API. They group related actions into classes, use attributes for routing and behavior, and work well for larger APIs with conventions, filters, and model binding.

Minimal APIs are newer. They let you define HTTP endpoints directly in code with less ceremony. They are especially useful for small APIs, microservices, and services where endpoint behavior is easier to read inline or in small route groups.

Configuration files still matter, but configuration is broader than files. ASP.NET Core can read configuration from JSON files, environment variables, command-line arguments, user secrets, and external providers. This matches modern deployment where production settings often come from the hosting platform.

Dependency injection is built in. If you previously used a third-party IoC container or manually constructed services, ASP.NET Core makes service registration and constructor injection part of the default application model.

Logging is built in as a framework abstraction. Instead of writing only to a local file or Windows Event Log, the application can emit structured events that local tools, cloud platforms, or observability systems can collect.

Layer 1 — Conceptual Model

ASP.NET Core is a framework for building HTTP-based applications on modern .NET.

It solves these problems:

- accepting HTTP requests;
- routing requests to application code;
- binding request data to .NET types;
- producing HTTP responses;

- composing cross-cutting request behavior;
- managing application services through dependency injection;
- reading configuration from multiple sources;
- emitting logs and diagnostics;
- hosting on Windows, Linux, containers, and cloud platforms.

It does not solve these problems by itself:

- it does not design your domain model;
- it does not choose your database strategy;
- it does not make endpoints secure unless security is configured;
- it does not guarantee API usability;
- it does not replace tests;
- it does not remove deployment and operations concerns;
- it does not make distributed systems simple.

The conceptual center is the request pipeline.

```
Request -> Middleware -> Routing -> Endpoint -> Response
```

Middleware is software assembled into a pipeline. Each middleware component can work before the next component, work after the next component, pass the request forward, or stop the pipeline early. Microsoft calls this stopping behavior short-circuiting.

An endpoint is the application code selected to handle a request. In ASP.NET Core APIs, the endpoint is often a Minimal API handler or a controller action.

Services are application dependencies registered with the dependency injection container. Endpoints, controllers, middleware, and other services can request those dependencies through constructors or parameters.

Configuration provides values the application needs without hard-coding them: connection strings, feature flags, external service URLs, logging levels, security settings, and environment-specific behavior.

Logging provides a record of what the application is doing. In modern systems, logs are part of production visibility, not just debugging.

Layer 2 — System Relationships

ASP.NET Core interacts with several surrounding components.

The client sends an HTTP request. The client might be a browser, mobile app, JavaScript frontend, another service, a scheduled job, a webhook provider, or an integration partner. The request includes method, path, headers, body, query string, cookies, and connection information.

The host starts the application process. It loads configuration, sets up logging, registers services, builds the middleware pipeline, maps endpoints, and begins listening for requests. In the minimal hosting model, this startup code usually lives in `Program.cs`.

The web server receives HTTP traffic. Kestrel is the built-in cross-platform web server used by ASP.NET Core. In production, Kestrel is often placed behind a reverse proxy, load balancer, or platform ingress that handles internet edge concerns.

The middleware pipeline owns cross-cutting HTTP behavior. Inputs include the request and current application state. Outputs include a modified request, response, logs, authentication state, errors, or an early response.

Routing selects the endpoint. It compares the HTTP method and path with mapped routes. It may also consider route constraints and endpoint metadata.

Minimal API handlers or controller actions own request-specific application behavior. They should validate input, call application services, and return clear responses. They should not become dumping grounds for database and business logic.

Application services own business operations. They are registered with dependency injection and consumed by endpoints. This keeps HTTP details from swallowing the entire application design.

Configuration providers own environment-specific values. Local development may use `appsettings.Development.json` and user secrets. Production may use environment variables, secret stores, or cloud configuration services.

Logging providers own output destinations. The application writes through the ASP.NET Core logging abstraction. Providers send those events to console, debug output, files, cloud systems, or observability backends.

Failure boundaries include startup failure, missing configuration, dependency registration errors, middleware ordering mistakes, route conflicts, model binding failures, invalid input, unhandled exceptions, downstream service failures, thread-pool exhaustion, connection limits, and incomplete logging.

The most common learning mistake is treating ASP.NET Core as one object called “the web app.” It is better understood as a host, pipeline, endpoint model, service container, configuration system, and logging system working together.

Layer 3 — Core Mechanics

The smallest useful ASP.NET Core application is a Minimal API.

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/", () => "Hello from ASP.NET Core");

app.Run();
```

This small program contains the basic shape.

`WebApplication.CreateBuilder(args)` creates the builder. The builder gathers configuration, logging, services, and host settings.

`builder.Services` is where application services are registered:

```
builder.Services.AddSingleton<Clock>();
```

`builder.Build()` creates the application.

`app.MapGet("/", ...)` maps an HTTP GET request for `/` to an endpoint handler.

`app.Run()` starts the application and blocks until shutdown.

Middleware is added before the application starts:

```
app.UseHttpsRedirection();

app.MapGet("/time", (Clock clock) =>
{
    return new { utc = clock.GetUtcNow() };
});
```

The order matters. Middleware runs in the order it is registered. A middleware component can affect everything after it.

A simple service might look like this:

```
public sealed class Clock
{
    public DateTimeOffset GetUtcNow() => DateTimeOffset.UtcNow;
}
```

ASP.NET Core can provide that service to the endpoint because it was registered with dependency injection.

Controllers use a class-based model:

```
using Microsoft.AspNetCore.Mvc;
```

```
[ApiController]
[Route("api/[controller]")]
public sealed class TimeController : ControllerBase
{
    private readonly Clock _clock;

    public TimeController(Clock clock)
    {
        _clock = clock;
    }

    [HttpGet]
    public IActionResult Get()
    {
        return Ok(new { utc = _clock.GetUtcNow() });
    }
}
```

To use controllers, the app registers controller services and maps controller endpoints:

```
builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

Minimal APIs and controllers are not enemies. They are two endpoint styles. Microsoft currently recommends Minimal APIs for new HTTP API projects because they provide a simplified, high-performance approach with minimal code and configuration. Controllers remain useful for larger APIs, MVC conventions, filters, and teams that prefer class-based organization. Configuration is available through `builder.Configuration`:

```
var connectionString = builder.Configuration.GetConnectionString("Orders");
```

Logging is available through `ILogger<T>`:

```
public sealed class OrderReporter
{
    private readonly ILogger<OrderReporter> _logger;

    public OrderReporter(ILogger<OrderReporter> logger)
    {
        _logger = logger;
    }

    public void ReportCreated(int orderId)
    {
        _logger.LogInformation("Order {OrderId} was created", orderId);
    }
}
```

The `{orderId}` placeholder is a structured logging property. This is more useful than building one long string because logging systems can preserve the property as searchable data.

Layer 4 — Developer Workflow

Create a new ASP.NET Core web API:

```
dotnet new webapi -n Orders.Api
cd Orders.Api
dotnet run
```

The template creates an ASP.NET Core project using the web SDK:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
```

Run the app and look for the listening URLs in the console output. Local development typically uses HTTPS and a development certificate.

Inspect the available project templates:

```
dotnet new list web
```

Build the project:

```
dotnet build
```

Run with automatic rebuild during development:

```
dotnet watch
```

`dotnet watch` is useful because web development often involves small edits and fast feedback.

Call an endpoint from another terminal. The exact URL and endpoint depend on the template and launch profile:

```
curl https://localhost:5001/weatherforecast
```

On Windows PowerShell, use:

```
Invoke-RestMethod https://localhost:5001/weatherforecast
```

If the template uses a different port, use the URL printed by `dotnet run`.

Add a small endpoint in `Program.cs`:

```
app.MapGet("/health/basic", () => Results.Ok(new { status = "ok" }));
```

Run again and call the endpoint:

```
curl https://localhost:5001/health/basic
```

The developer workflow is intentionally simple at this stage:

```
Create project -> run locally -> add endpoint -> call endpoint -> inspect logs
```

Later chapters add data access, authentication, tests, containers, deployment, and observability.

Layer 5 — Production Usage

Production ASP.NET Core begins with hosting clarity.

Kestrel is the ASP.NET Core web server. In production it may run behind IIS, Nginx, Apache, a cloud load balancer, or a container ingress. The outer server or platform often handles TLS termination, request forwarding, static edge concerns, and network exposure. The ASP.NET Core app handles application routing and behavior.

Configuration should be environment-specific. Local development can use `appsettings.Development.json` and user secrets. Production should use environment variables, managed configuration, secret stores, or platform settings. Never commit production secrets.

Security includes more than login. HTTPS, forwarded headers, CORS, authentication, authorization, anti-forgery for browser forms, secure cookies, rate limiting, input validation, dependency patching, and safe error handling all matter. Chapter 7 covers authentication and authorization in detail.

Reliability depends on startup behavior, health checks, timeouts, cancellation tokens, retry policy, graceful shutdown, and clear failure modes when dependencies are unavailable. An API that hangs under dependency failure is harder to operate than one that fails clearly.

Deployment should use a repeatable artifact. ASP.NET Core apps can be published as framework-dependent deployments, self-contained deployments, container images, or platform-specific packages. Chapter 12 covers deployment more fully.

Observability starts with logs. ASP.NET Core emits framework logs and lets the application emit structured logs through `ILogger<T>`. Production systems should also expose health and later add metrics and tracing. Chapter 14 covers observability basics.

Scaling requires statelessness where possible. A load-balanced ASP.NET Core API should not assume that all requests from one user reach the same process. Session-like data, cache data, background work, and file storage need explicit design.

Persistence belongs behind clear service and data-access boundaries. Do not put SQL queries, migrations, and transaction logic directly into endpoint handlers unless the application is deliberately tiny. Chapter 6 covers data access.

Cost is affected by hosting choice, memory use, request volume, logging volume, dependency calls, and scaling settings. An inefficient endpoint can cost money in cloud environments even if it seems harmless locally.

Local development optimizes for fast iteration and clear debugging. Production optimizes for secure configuration, predictable startup, repeatable deployment, visibility, reliability, and controlled scale.

Layer 6 — Tradeoffs and Alternatives

Use ASP.NET Core when you need to build HTTP APIs, web applications, server-rendered pages, real-time services, gRPC services, backend-for-frontend services, or cloud-hosted .NET workloads.

Do not use ASP.NET Core when a simpler process is enough. A scheduled job, command-line import, Windows service, worker service, or serverless function may be a better fit if there is no HTTP boundary.

Minimal APIs are a strong default for new HTTP APIs, especially when endpoints are small, routing is explicit, and the team values low ceremony.

Controllers are strong when the API benefits from class-based organization, filters, conventions, attribute-heavy behavior, shared base controller patterns, or a team already has a mature controller-based style.

Razor Pages, MVC views, and Blazor are alternatives for applications that render user interfaces rather than only HTTP APIs. They are part of the ASP.NET Core family, but this chapter focuses on APIs because later samples build from API foundations.

Competing web frameworks include Spring Boot for Java, Express and Fastify for Node.js, Django and FastAPI for Python, Ruby on Rails, Laravel for PHP, and Go HTTP frameworks. ASP.NET Core is especially strong for C# teams that need performance, mature tooling, enterprise integration, and cloud-ready APIs. Stack Overflow's 2025 Developer Survey lists ASP.NET Core among the web frameworks and technologies used by professional developers, though JavaScript and Python web ecosystems remain very common across the broader industry.

State-of-the-art ASP.NET Core development in 2026 emphasizes Minimal APIs, OpenAPI descriptions, built-in dependency injection, structured logging, configuration from the host environment, container-friendly hosting, health-oriented operations, and careful security defaults.

Common overengineering mistakes:

- creating controllers, services, repositories, factories, and wrappers before the endpoint has real complexity;
- putting all business logic directly in Minimal API lambdas;
- treating middleware order as incidental;
- using global mutable state for request-specific behavior;
- returning inconsistent error shapes from related endpoints;
- logging secrets or entire request bodies in production;
- assuming local HTTPS, ports, and configuration match production hosting.

The right choice is usually staged: start with clear endpoints, simple service boundaries, good configuration, useful logs, and tests. Add framework features when they solve an actual pressure.

Layer 7 — Interview Perspective

Interviewers use ASP.NET Core questions to test whether you understand the modern web application model.

Concepts commonly tested:

- Kestrel and hosting;
- middleware pipeline and ordering;
- routing and endpoints;
- Minimal APIs versus controllers;
- dependency injection lifetimes;
- configuration providers;
- logging and structured logging;
- environment-specific behavior;
- production concerns such as HTTPS, health checks, and reverse proxies.

Representative questions:

- “How is ASP.NET Core different from classic ASP.NET?”
- “What is middleware?”
- “Why does middleware order matter?”
- “When would you choose Minimal APIs versus controllers?”
- “How does dependency injection work in ASP.NET Core?”
- “Where should configuration come from in production?”
- “What is Kestrel?”
- “How would you structure a small API so business logic does not live entirely in endpoint handlers?”

A strong answer connects the request pipeline to application boundaries:

“ASP.NET Core receives a request through the host and passes it through an ordered middleware pipeline. Routing selects an endpoint, such as a Minimal API handler or controller action. That endpoint should coordinate application services through dependency injection rather than contain all business and data-access logic directly.”

Common misconceptions:

- “ASP.NET Core requires IIS.”
- “Minimal APIs are only for toy applications.”
- “Controllers are obsolete.”
- “Middleware order rarely matters.”
- “Dependency injection is optional plumbing with no design impact.”
- “Logging is only for debugging locally.”
- “Configuration files are the only configuration source.”
- “A passing local request proves the API is production-ready.”

Small design scenario:

You need to build an internal order-status API. It has five endpoints, calls SQL Server, and will later require authentication. A good first design might use Minimal APIs with route groups, a small application service for order status operations, configuration for the database connection, structured logging, and simple health reporting. It would avoid Kubernetes, custom middleware, caching, or complex architecture until the API has a real need.

If the API grows to dozens of endpoints with shared filters, complex request models, and strong team conventions around controller organization, controllers may become the more natural endpoint style.

Hands-On Lab

Objective:

Create a small ASP.NET Core API, add an endpoint, register a service, and call the endpoint locally.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A terminal.
- A browser, `curl`, or PowerShell.

Steps:

1. Create the API:

```
dotnet new webapi -n Chapter05.Api
cd Chapter05.Api
```

2. Run the application:

```
dotnet run
```

3. Note the HTTPS URL printed in the console.

4. In another terminal, call the default endpoint. Adjust the URL and path to match the template output:

```
curl https://localhost:5001/weatherforecast
```

5. Stop the application.

6. Add this service class:

```
public sealed class SystemClock
{
    public DateTimeOffset GetUtcNow() => DateTimeOffset.UtcNow;
}
```

7. Register the service in `Program.cs` before `builder.Build()`:

```
builder.Services.AddSingleton<SystemClock>();
```

8. Add an endpoint after the app is built:

```
app.MapGet("/time", (SystemClock clock) =>
{
    return Results.Ok(new { utc = clock.GetUtcNow() });
});
```

9. Run the application again:

```
dotnet run
```

10. Call the new endpoint:

```
curl https://localhost:5001/time
```

11. On Windows PowerShell, use:

```
Invoke-RestMethod https://localhost:5001/time
```

Expected results:

- The API starts locally.
- The default endpoint responds.
- The `/time` endpoint returns a JSON response containing a UTC timestamp.
- The service is provided through dependency injection.
- The console shows ASP.NET Core logging output.

Validation commands:

```
dotnet build
dotnet run
curl https://localhost:5001/time
```

Troubleshooting notes:

- If the port is different, use the URL printed by `dotnet run`.
- If HTTPS fails locally, trust the development certificate or use the HTTP URL printed by the application.
- If dependency injection fails, confirm the service was registered before `builder.Build()`.
- If the endpoint returns 404, confirm the route string and that the app was restarted after editing.

Knowledge Check

1. Why is ASP.NET Core better understood as a request-processing framework than as “IIS for modern .NET”?
2. What role does Kestrel play?
3. What is middleware, and why does order matter?
4. How is a Minimal API endpoint different from a controller action?
5. When might controllers be a better fit than Minimal APIs?
6. Why should application services be separated from endpoint handlers as an API grows?
7. What kinds of values belong in configuration rather than code?
8. Why is structured logging more useful than string-only logging?
9. What production concerns appear before data access or authentication are added?
10. How does ASP.NET Core’s hosting model support Windows, Linux, containers, and cloud platforms?

Summary

ASP.NET Core is the modern .NET framework for HTTP applications. It replaced the old System.Web-centered mental model with a cross-platform host, Kestrel, an ordered middleware pipeline, endpoint routing, built-in dependency injection, flexible configuration, and integrated logging.

Minimal APIs provide a low-ceremony way to build HTTP APIs and are the modern default for many new API projects. Controllers remain valuable for larger APIs and teams that benefit from class-based organization and conventions.

The most important idea is the pipeline: requests enter the host, pass through middleware, route to endpoints, use services through dependency injection, and return responses while emitting logs and diagnostics.

The next chapter adds the data layer. With ASP.NET Core as the HTTP boundary, the application now needs durable state, queries, migrations, and performance discipline.

Sources

- [ASP.NET documentation](#)
- [ASP.NET Core fundamentals overview](#)

- [Tutorial: Create a Minimal API with ASP.NET Core](#)
- [APIs overview](#)
- [ASP.NET Core Middleware](#)
- [Overview of OpenAPI support in ASP.NET Core API apps](#)
- [Stack Overflow 2025 Developer Survey: Technology](#)

Further Reading

- [Understand ASP.NET Core fundamentals](#)
- [Configure services with dependency injection in ASP.NET Core](#)
- [Customize ASP.NET Core behavior with middleware](#)
- [Create web APIs with ASP.NET Core](#)

6

Data Access

Chapter Purpose

Most business applications eventually become data applications.

Chapter 5 introduced ASP.NET Core as the HTTP boundary. That boundary is useful only when the application can read and change durable state: customers, orders, invoices, users, permissions, audit records, product catalogs, and workflow history. Chapter 6 explains how modern .NET applications usually reach that data.

For a returning Windows, SQL Server, and C# developer, this chapter begins from familiar ground. SQL Server is still a major enterprise database. T-SQL, indexes, transactions, constraints, query plans, backup, restore, and security still matter. Modern .NET did not make relational thinking obsolete.

What changed is the application side of the boundary.

Modern .NET applications usually do not treat database access as a few connection strings hidden in `web.config` and a set of ad hoc ADO.NET calls spread through pages or controllers. They tend to use explicit data-access libraries, dependency injection, environment-based configuration, migrations, async APIs, source-controlled schema changes, and production observability.

Entity Framework Core, or EF Core, is Microsoft's modern object-relational mapper for .NET. It exists to let developers work with database-backed data through .NET objects while still using a relational database underneath. Dapper is a popular micro-ORM originally developed for Stack Overflow. It exists for teams that want to write SQL directly while avoiding repetitive mapping code.

This chapter introduces SQL Server, EF Core, Dapper, migrations, and performance from a modern .NET perspective. It does not try to teach database administration or every ORM feature. The goal is to help you place data access correctly in an ASP.NET Core application and understand the tradeoffs.

Where This Fits

Data access sits behind the HTTP layer and inside the application boundary.

```

HTTP client
  |
  v
ASP.NET Core endpoint or controller
  |
  v
Application service
  |
  v
Data access boundary
  |
  +-- EF Core DbContext
  
```

```

|
+-- Dapper query object
|
+-- raw ADO.NET when needed
|
v
SQL Server or another database

```

The endpoint should not need to know every table, join, transaction detail, or connection-management rule. It should coordinate request handling. The application service should express the business operation. The data-access layer should own the database conversation.

This chapter prepares for several later chapters. Chapter 7 adds security. Chapter 8 validates code with tests and CI. Chapter 11 uses Docker Compose to run multi-container development environments, including local SQL Server. Chapter 12 discusses deployment. Chapter 14 covers diagnostics. Chapter 15 adds caching and distributed application patterns.

Connection to the Reader's Existing Model

You already understand SQL Server as a durable system of record.

You know that tables, keys, constraints, indexes, transactions, stored procedures, views, and query plans are not implementation trivia. They are the shape of business data. You also know that a database is not just a file the application happens to use. It has its own lifecycle, permissions, backups, capacity, and operational failure modes.

That knowledge remains valuable.

The modern shift is how the application describes and reaches the database. Instead of connection strings in `web.config`, ASP.NET Core usually reads connection strings from configuration providers. In local development, that may be `appsettings.Development.json` or user secrets. In production, it should be a secret store, environment variable, or managed platform setting.

Instead of creating `SqlConnection` everywhere, modern applications usually centralize data access through dependency injection. EF Core uses a `DbContext` as the unit that tracks entities, builds queries, and saves changes. Dapper uses an open database connection and extension methods such as `QueryAsync` and `ExecuteAsync`.

Instead of manually remembering which schema changes were applied to which database, EF Core migrations can record schema evolution in source control. This does not remove the need for DBA judgment. It gives the team a repeatable artifact to review, test, script, and deploy.

The analogy to stored procedures is useful. Stored procedures centralize database behavior close to the data. EF Core centralizes database mapping and query generation close to the application model. Dapper centralizes explicit SQL close to application code. Each style chooses a different place for database knowledge to live.

The analogy breaks down when an ORM is treated as if the relational database disappeared. It did not. EF Core still produces SQL. SQL Server still executes queries. Indexes still determine performance. Transactions still define consistency. The database is still real.

Layer 1 — Conceptual Model

Data access is the boundary between application behavior and durable or shared state.

It solves these problems:

- connecting to a database;
- translating application operations into database operations;
- querying data efficiently;
- saving changes consistently;
- managing transactions;

- evolving schema over time;
- keeping data-access details out of HTTP endpoints;
- making database behavior testable and observable.

It does not solve these problems by itself:

- it does not design a correct data model;
- it does not make slow queries fast automatically;
- it does not remove the need for indexes;
- it does not make distributed transactions simple;
- it does not protect secrets unless configuration is handled correctly;
- it does not replace backup, restore, monitoring, or operational planning.

The main modern .NET data-access choices are:

```
EF Core
Higher-level object-relational mapping
Strong integration with LINQ, DbContext, migrations, and change tracking

Dapper
SQL-first micro-ORM
Strong control over SQL with lightweight object mapping

ADO.NET
Lowest-level database API
Maximum explicit control with more repetitive code
```

EF Core is often the default starting point for modern .NET applications because it integrates well with .NET, ASP.NET Core dependency injection, LINQ, migrations, and multiple database providers.

Dapper is often chosen when SQL control is more important than ORM modeling, when queries are highly tuned, or when a team already thinks naturally in SQL.

ADO.NET remains underneath both approaches and is still available when direct control matters.

Layer 2 — System Relationships

The application sends data-access intent. It asks to create an order, load a customer, check a permission, search products, or record an audit event.

The data-access code turns that intent into database commands. With EF Core, LINQ queries and tracked entity changes become SQL commands. With Dapper, explicit SQL text and parameters become commands. With ADO.NET, the developer builds the command directly.

The database provider owns the database-specific connection and translation details. EF Core supports multiple providers, including SQL Server, SQLite, PostgreSQL, MySQL, and others. Dapper works through ADO.NET connections, so it can work with many databases that expose .NET data providers.

The database owns durable state, constraints, query execution, locking, transactions, indexes, storage, backup, and recovery. The configuration system owns connection information. The application should not hard-code production connection strings. The connection string is an input to the application, not part of the compiled binary.

The dependency injection container owns service lifetimes. In ASP.NET Core, EF Core `DbContext` instances are commonly registered with a scoped lifetime, meaning one context instance is used for a request scope. Dapper-based designs often register a connection factory or data-access service rather than a single shared connection.

The migration history owns schema evolution. EF Core migrations generate C# files that describe schema changes and a model snapshot used to detect later changes. Those files should be reviewed and committed like other source files.

Failure boundaries include connection failures, authentication failures, missing permissions, deadlocks, blocking, timeouts, migration mistakes, transaction conflicts, inefficient queries, model/schema drift, missing indexes, connection pool exhaustion, and serialization problems when data is returned through an API.

One practical rule helps: the database is a dependency, but it is not a minor dependency. It is often the most important production boundary in the system.

Layer 3 — Core Mechanics

Start with a small entity:

```
public sealed class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public decimal Total { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
}
```

In EF Core, a `DbContext` represents a session with the database:

```
using Microsoft.EntityFrameworkCore;

public sealed class OrdersDbContext : DbContext
{
    public OrdersDbContext(DbContextOptions<OrdersDbContext> options)
        : base(options)
    {
    }

    public DbSet<Order> Orders => Set<Order>();
}
```

The ASP.NET Core application registers the context:

```
builder.Services.AddDbContext<OrdersDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("Orders"));
});
```

An endpoint or service can then query:

```
app.MapGet("/orders/{id:int}", async (
    int id,
    OrdersDbContext db) =>
{
    var order = await db.Orders.FindAsync(id);

    return order is null
        ? Results.NotFound()
        : Results.Ok(order);
});
```

For creation:

```
app.MapPost("/orders", async (
    CreateOrderRequest request,
    OrdersDbContext db) =>
{
    var order = new Order
    {
```

```

        CustomerName = request.CustomerName,
        Total = request.Total,
        CreatedAt = DateTimeOffset.UtcNow
    };

    db.Orders.Add(order);
    await db.SaveChangesAsync();

    return Results.Created($"/orders/{order.Id}", order);
});

public sealed record CreateOrderRequest(
    string CustomerName,
    decimal Total);

```

This is useful for learning, but production code often moves business rules out of the endpoint and into application services. With Dapper, the SQL is explicit:

```

using Dapper;
using Microsoft.Data.SqlClient;

public sealed class OrderQueries
{
    private readonly string _connectionString;

    public OrderQueries(IConfiguration configuration)
    {
        _connectionString =
            configuration.GetConnectionString("Orders")
            ?? throw new InvalidOperationException(
                "Missing Orders connection string.");
    }

    public async Task<Order?> GetOrderAsync(int id)
    {
        const string sql = """
            select Id, CustomerName, Total, CreatedAt
            from dbo.Orders
            where Id = @Id;
            """;

        await using var connection =
            new SqlConnection(_connectionString);

        return await connection.QuerySingleOrDefaultAsync<Order>(
            sql,
            new { Id = id });
    }
}

```

Notice the tradeoff. EF Core hides most SQL generation and gives change tracking, LINQ, and migrations. Dapper keeps SQL visible and gives lightweight mapping.

Migrations describe schema changes:

```

dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate
dotnet ef database update

```

For production, do not blindly apply migrations because a command exists. Microsoft recommends generating SQL scripts for production deployment so they can be reviewed and adjusted before being applied.

```

dotnet ef migrations script -o migrations.sql

```

Performance begins with query awareness. EF Core LINQ is not magic; it is translated to SQL. Dapper SQL is not automatically fast; it still needs good indexes, parameters, and query plans. The database engine always has the final say.

Layer 4 — Developer Workflow

A typical EF Core workflow in an ASP.NET Core API looks like this:

```
Define entity and DbContext
|
v
Add provider package
|
v
Configure connection string
|
v
Register DbContext with dependency injection
|
v
Create migration
|
v
Apply migration to local database
|
v
Write endpoint or service
|
v
Run and test
```

Create a web API:

```
dotnet new webapi -n Orders.Api
cd Orders.Api
```

Add EF Core packages for SQL Server:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Install or restore EF Core tools:

```
dotnet new tool-manifest
dotnet tool install dotnet-ef
dotnet tool restore
```

Create a migration:

```
dotnet ef migrations add InitialCreate
```

Apply it to a local development database:

```
dotnet ef database update
```

List migrations:

```
dotnet ef migrations list
```

Check whether model changes are missing a migration:

```
dotnet ef migrations has-pending-model-changes
```

For Dapper:

```
dotnet add package Dapper
dotnet add package Microsoft.Data.SqlClient
```

Dapper usually does not manage schema. Teams using Dapper often manage schema with SQL scripts, a migration tool, database projects, Flyway, Liquibase, DbUp, Roundhouse, or another database-change process.

Local development may use SQL Server installed directly, SQL Server in a container, LocalDB on Windows, or a shared development instance. Chapter 11 returns to local multi-container environments.

The important workflow habit is that schema changes, code changes, and tests move together. A pull request that changes data shape should include the application change, migration or SQL script, test evidence, and deployment notes.

Layer 5 — Production Usage

Production data access is where small shortcuts become expensive.

Configuration should keep connection strings out of source control. Local development can use user secrets or local settings. Production should use a secret store, environment variable, managed identity pattern, or hosting platform configuration.

Security includes database authentication, least-privilege database users, encrypted connections, parameterized queries, secret rotation, auditing, and careful handling of personally identifiable or regulated data. Dapper and EF Core both support parameterized commands; string-concatenated SQL remains a security risk.

Reliability depends on timeouts, retries, transaction boundaries, connection pooling, idempotent operations where possible, and clear behavior when the database is unavailable. Retrying every failure is not wisdom. Some failures are transient. Some indicate bad data, deadlocks, or broken design.

Deployment must respect schema and application compatibility. A database migration can break older application versions if columns are renamed, constraints are changed, or data is transformed in one step. Safer production changes often use expand-and-contract patterns: add compatible schema, deploy application changes, backfill data, then remove old schema later.

Observability should include slow queries, command failures, connection pool pressure, timeout rates, migration history, and database health. Application logs should contain enough context to diagnose failures without logging secrets or sensitive data.

Scaling data access is often harder than scaling web servers. Adding more API instances can increase database load. Caching, read replicas, query tuning, pagination, background processing, and data partitioning can help, but each adds tradeoffs.

Persistence is the point of the database. Do not treat cache or in-memory data as durable. SQL Server remains the system of record unless the architecture explicitly defines another durable store.

Cost is affected by database tier, storage, backup retention, DTUs or vCores, IO, memory, licensing, managed-service choices, inefficient queries, and over-eager logging. A slow query can become a cloud bill.

Local development optimizes for convenience and reset capability. Production optimizes for durability, backup, auditability, security, tested schema changes, and predictable performance.

Layer 6 — Tradeoffs and Alternatives

EF Core is a strong default when the application has a rich domain model, needs LINQ queries, benefits from change tracking, wants migrations, and fits well with a model-first or code-first workflow.

Dapper is strong when SQL control is central, queries are hand-tuned, stored procedures are important, the data model is close to result shapes, or the team wants minimal abstraction over SQL.

ADO.NET is appropriate when maximum control matters, when avoiding extra dependencies is important, or when building infrastructure-level libraries. It requires more repetitive code.

Stored procedures remain valid. They can centralize data rules, support security patterns, improve plan stability in some cases, and align with DBA-led database ownership. They can also hide behavior from application source control if not managed carefully.

SQL Server competes with PostgreSQL, MySQL, Oracle, SQLite, MariaDB, cloud relational databases, document databases, key-value stores, and analytical systems. According to DB-Engines in July 2026, Microsoft SQL Server ranked third among

relational database management systems, behind Oracle and MySQL and just ahead of PostgreSQL. That ranking does not decide your architecture, but it confirms SQL Server remains a major professional database.

PostgreSQL is a major open-source alternative and is especially prominent in cloud-native and open-source-heavy systems. SQLite is excellent for embedded, local, test, and single-file scenarios. Document databases fit data that is naturally document-shaped. Analytical stores fit reporting and large-scale analysis. Redis fits fast in-memory state, not relational truth.

Common overengineering mistakes:

- creating a repository abstraction over EF Core that adds no useful boundary;
- putting all queries directly in controllers or Minimal API handlers;
- treating migrations as production-safe without review;
- ignoring generated SQL because the LINQ looks clean;
- loading entire tables and filtering in memory;
- using lazy loading without understanding query volume;
- adding Redis before fixing obvious query and index problems;
- treating database changes as less important than C# changes.

State-of-the-art .NET data access in 2026 is pragmatic: use EF Core where its modeling and migrations help, use Dapper where explicit SQL is clearer, review schema changes, observe query behavior, and respect the database as a production system.

Layer 7 — Interview Perspective

Interviewers use data-access questions to test whether you understand both .NET and databases.

Concepts commonly tested:

- EF Core `DbContext` ;
- entity classes and `DbSet<T>` ;
- LINQ-to-SQL translation;
- change tracking;
- migrations;
- Dapper versus EF Core;
- parameterized queries;
- connection strings and secrets;
- transactions;
- indexes and query performance;
- avoiding business logic in controllers.

Representative questions:

- “What problem does EF Core solve?”
- “When would you choose Dapper instead of EF Core?”
- “What is a migration?”
- “Why should generated migration SQL be reviewed before production?”
- “What is the risk of lazy loading?”
- “How do you avoid SQL injection?”
- “What should not go in an ASP.NET Core endpoint handler?”
- “Why can adding more web servers make database performance worse?”

A strong answer keeps the database visible:

“EF Core lets us work with entities and LINQ, but it still generates SQL that the database must execute. I use it where change tracking and migrations help, inspect generated SQL for important paths, use indexes and pagination, and review migrations before production.”

Common misconceptions:

- “EF Core means I do not need to understand SQL.”
- “Dapper is always faster, therefore always better.”
- “Migrations are just development convenience.”
- “A repository layer automatically makes data access clean.”
- “Redis can replace SQL Server for ordinary business data.”
- “Async database calls make slow queries fast.”
- “If an endpoint is simple, data access can stay in the controller forever.”

Small design scenario:

You are building an order API. The first version needs to create orders, read orders by ID, list recent orders, and update order status.

A good design might use SQL Server as the system of record, EF Core for ordinary CRUD and migrations, application services for business operations, and carefully indexed queries for common reads. If a reporting endpoint later needs a tuned join, Dapper can be introduced for that query without replacing EF Core everywhere.

The strong answer chooses tools per boundary instead of turning one data-access style into a religion.

Hands-On Lab

Objective:

Add EF Core data access to a small ASP.NET Core API and understand the shape of a migration.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- A SQL Server development database, SQL Server container, LocalDB on Windows, or another SQL Server-compatible development option.
- Basic familiarity with Chapter 5’s ASP.NET Core API.

Steps:

1. Create a new API:

```
dotnet new webapi -n Chapter06.Api
cd Chapter06.Api
```

2. Add EF Core packages:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
```

3. Create a local tool manifest and install EF Core tools:

```
dotnet new tool-manifest
dotnet tool install dotnet-ef
```

4. Add an `order` entity and `ordersDbContext`.
5. Add a development connection string named `orders`.
6. Register the context:

```
builder.Services.AddDbContext<OrdersDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("Orders"));
});
```

7. Add a migration:

```
dotnet ef migrations add InitialCreate
```

8. Inspect the generated migration files before applying them.

9. Apply the migration to the local database:

```
dotnet ef database update
```

10. Add one endpoint to create an order and one endpoint to retrieve an order.

11. Build and run:

```
dotnet build
dotnet run
```

Expected results:

- The project builds.
- EF Core packages are referenced.
- A migration is generated and visible in source.
- The local database schema is created or updated.
- The API can save and retrieve an order.

Validation commands:

```
dotnet build
dotnet ef migrations list
dotnet ef migrations has-pending-model-changes
dotnet ef database update
dotnet run
```

Troubleshooting notes:

- If `dotnet ef` is not found, run `dotnet tool restore`.
- If the connection fails, verify the connection string and SQL Server instance.
- If the migration cannot be created, confirm the `DbContext` is registered and the design package is referenced.
- If `has-pending-model-changes` reports changes, create a migration or review whether the model changed unintentionally.
- If the API runs but database calls fail, check database permissions and whether the migration was applied.

Knowledge Check

1. Why is data access a boundary rather than just a utility function?
2. What does EF Core's `DbContext` represent?
3. Why does EF Core not remove the need to understand SQL Server?
4. When is Dapper a better fit than EF Core?
5. What problem do migrations solve?
6. Why should migrations be reviewed before production deployment?
7. What is the risk of putting database code directly in endpoint handlers?
8. Why can adding more application instances increase database pressure?

9. What is the difference between durable state and cached state?
10. How should connection strings and secrets be handled differently in local development and production?

Summary

Modern .NET data access begins with the same truth experienced SQL Server developers already know: the database is a durable, operationally important system. Tables, indexes, transactions, constraints, and query plans still matter.

EF Core adds a modern object-relational mapping layer with `DbContext`, entities, LINQ, change tracking, providers, and migrations. Dapper offers a lighter SQL-first path for teams that want explicit queries and simple object mapping. ADO.NET remains available underneath both.

The modern practice is not to hide the database. It is to create a clear, testable, observable boundary between application behavior and persistent state. Schema changes should be source-controlled, reviewed, tested, and deployed deliberately. Query performance should be measured against the real database, not assumed from clean C#.

The next chapter adds authentication and security, because once an API can read and write data, the next question is who is allowed to do so.

Sources

- [What is SQL Server?](#)
- [Overview of Entity Framework Core](#)
- [Managing Database Schemas](#)
- [Migrations Overview](#)
- [Managing Migrations](#)
- [Applying Migrations](#)
- [Efficient Querying](#)
- [What's New in EF Core 10](#)
- [Dapper GitHub repository](#)
- [DB-Engines Ranking of Relational DBMS](#)

Further Reading

- [Entity Framework Core documentation](#)
- [SQL Server documentation](#)
- [Dapper documentation repository](#)
- [EF Core performance documentation](#)
- [Connection strings and configuration in ASP.NET Core](#)

7

Authentication and Security

Chapter Purpose

Chapter 6 gave the application durable state. Once an API can read and change business data, the next question is unavoidable: who is allowed to do it?

Authentication and security exist because useful systems have boundaries. Some callers are anonymous. Some are employees. Some are customers. Some are services. Some can read. Some can write. Some can approve payments, reset passwords, view audit records, or administer tenants. The application must know the difference.

For a developer coming from Windows, IIS, SQL Server, and enterprise application development, the familiar model may involve Windows Authentication, Active Directory groups, IIS settings, SQL logins, service accounts, and connection strings protected on a server. Those ideas still matter, but modern .NET applications commonly run behind reverse proxies, in containers, on cloud platforms, and across identity providers. Security is no longer only a server setting. It is an application, platform, identity, deployment, and operations concern.

ASP.NET Core provides authentication and authorization building blocks. OAuth 2.0 and OpenID Connect provide common protocol patterns. JSON Web Tokens, or JWTs, are commonly used to carry identity and authorization claims to APIs. Secrets management keeps credentials and sensitive configuration out of source control and ordinary application files. This chapter introduces authentication, authorization, JWT, OAuth, OpenID Connect, and secrets management at the level needed for modern .NET API development. It does not make you an identity specialist. It gives you the map, terminology, and basic mechanics so later chapters can build securely.

Where This Fits

Security surrounds the HTTP and data boundaries introduced in Chapters 5 and 6.

```
Client
|
| obtains token or session through identity system
v
ASP.NET Core API
|
|-- Authentication: who is the caller?
|
|-- Authorization: what may this caller do?
|
|-- Application service
|
|-- Data access
|
v
SQL Server, cache, queue, external API, or AI service
```

Security also touches deployment:

```
Source control -> CI/CD -> runtime platform
      |           |           |
      v           v           v
    no secrets   protected   managed identity,
    in code      variables    secret store,
                                TLS, audit
```

The application cannot treat authentication as a decorative endpoint attribute added at the end. Authentication affects request handling. Authorization affects business rules. Secrets affect configuration. Identity affects database, cloud, queue, and external API access.

Connection to the Reader's Existing Model

Windows enterprise systems often used a perimeter-oriented security model.

The application ran on a known Windows server. IIS might require Windows Authentication. Users belonged to Active Directory groups. The application pool identity accessed files or SQL Server. Connection strings lived in configuration files protected by server permissions. The network boundary did some of the work.

Modern systems use many of the same ideas with different shapes.

An application pool identity maps conceptually to a service identity or managed identity. It answers, “What is this running application allowed to access?”

An Active Directory group maps conceptually to claims, roles, and policies. It answers, “What does this authenticated user or service represent?”

IIS authentication settings map conceptually to ASP.NET Core authentication schemes and middleware. The request still needs an identity before protected resources can be accessed, but the mechanism may be cookies, JWT bearer tokens, OpenID Connect, certificates, API keys, or platform-provided identity.

SQL Server permissions still matter. Application authorization should not be the only protection around data. The database account or managed identity should have only the access the application needs.

The analogy breaks down when the application is no longer protected mainly by a corporate network or one IIS server. Public APIs, mobile clients, cloud services, service-to-service calls, and external identity providers require explicit trust decisions. “It is inside the network” is not a complete security model.

Layer 1 — Conceptual Model

Authentication determines identity.

Authorization determines access.

Secrets management protects sensitive values the application needs to operate.

Those three concepts are related, but they are not interchangeable.

Authentication answers:

Who is calling?

Authorization answers:

What is this caller allowed to do?

Secrets management answers:

How does the application safely obtain credentials and sensitive settings?

Authentication solves the problem of establishing a caller's identity. In an ASP.NET Core API, the result is usually a `ClaimsPrincipal`: an identity plus claims about that identity.

Authorization solves the problem of making access decisions. It can use roles, claims, policies, resource ownership, tenant boundaries, business state, or a combination of those.

JWT bearer authentication solves a common API problem: a client can send a token in the `Authorization` header, and the API can validate that token without storing server-side session state for every request.

OAuth 2.0 solves delegated authorization. It lets a client obtain access to a resource with scoped permission, often without giving the client the user's password.

OpenID Connect builds on OAuth 2.0 to provide authentication and sign-in. It adds ID tokens and standardized identity information.

Secrets management solves the problem of keeping passwords, connection strings, API keys, certificates, signing keys, and client secrets out of code, logs, and casual configuration files.

Security does not solve these problems automatically:

- it does not make poor authorization rules correct;
- it does not protect secrets that are logged or committed;
- it does not make JWT contents trustworthy unless the token is validated;
- it does not replace input validation;
- it does not make production safe without patching and monitoring;
- it does not remove the need for least privilege.

Layer 2 — System Relationships

The client initiates a request. It may be a browser, mobile app, JavaScript frontend, server-side app, scheduled job, webhook sender, or another API.

The identity provider authenticates users or services and issues tokens or session information. Examples include Microsoft Entra ID, Auth0, Okta, Duende IdentityServer, Keycloak, cloud identity services, and custom identity systems. The identity provider owns sign-in policy, token issuance, keys, claims, and often multi-factor authentication.

The ASP.NET Core authentication middleware examines the incoming request. It uses configured authentication handlers, called schemes, to validate cookies, bearer tokens, certificates, or other credentials. If successful, it creates the user identity for the request.

The authorization system evaluates whether that identity can access the resource. ASP.NET Core supports role-based and policy-based authorization. Policies can evaluate claims and custom requirements.

The endpoint or controller performs application behavior only after the security boundary allows it. Some endpoints may allow anonymous access, such as health checks or public metadata. Others require an authenticated user or a specific policy.

The data layer receives the result of these decisions indirectly. It should not assume the endpoint already made every correct access decision. Important business rules, such as tenant ownership or row-level access, should be enforced near the application service or data boundary as well.

The configuration system provides non-secret and secret settings. In development, user secrets can hold local values. In production, a managed secret store or platform configuration should provide sensitive values.

Failure boundaries include expired tokens, wrong issuer, wrong audience, missing signing keys, clock skew, missing claims, incorrect policies, confused roles, overbroad permissions, leaked secrets, broken redirect URIs, CORS misconfiguration, and authorization checks that protect the endpoint but not the underlying business operation.

Layer 3 — Core Mechanics

In ASP.NET Core, authentication and authorization are configured in `Program.cs`.

A JWT bearer API has this broad shape:

```
builder.Services
    .AddAuthentication()
    .AddJwtBearer(options =>
    {
        options.Authority = builder.Configuration["Auth:Authority"];
        options.Audience = builder.Configuration["Auth:Audience"];
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/orders", () => Results.Ok())
    .RequireAuthorization();

app.Run();
```

`AddAuthentication` registers authentication services. `AddJwtBearer` registers a bearer-token authentication scheme. The authority identifies the issuer trusted by the API. The audience identifies the API the token is meant for.

`AddAuthorization` registers authorization services.

`UseAuthentication` runs authentication middleware. `UseAuthorization` runs authorization middleware. The order matters: authorization depends on the identity created by authentication.

`RequireAuthorization` marks the endpoint as protected.

A policy expresses a named access rule:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanReadOrders", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.RequireClaim("scope", "orders.read");
    });
});
```

The endpoint can require that policy:

```
app.MapGet("/orders/{id:int}", (int id) =>
{
    return Results.Ok(new { id });
})
.RequireAuthorization("CanReadOrders");
```

For controllers, the same idea appears with attributes:

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
[Authorize(Policy = "CanReadOrders")]
public sealed class OrdersController : ControllerBase
{
    [HttpGet("{id:int}")]
    public IActionResult Get(int id)
    {
        return Ok(new { id });
    }
}
```

JWT stands for JSON Web Token. In an API scenario, a bearer token is commonly sent like this:

```
Authorization: Bearer eyJhbGciOi...
```

The API should validate the token's signature, issuer, audience, expiration, and relevant claims. It should not merely decode the token and trust the contents.

Secrets should come from configuration, not constants:

```
var connectionString =  
    builder.Configuration.GetConnectionString("Orders");
```

For local development, user secrets can keep values out of the repository:

```
dotnet user-secrets init  
dotnet user-secrets set "ConnectionStrings:Orders" "Server=..."
```

User secrets are for development convenience. They are not a production secret store.

Layer 4 — Developer Workflow

A safe development workflow begins before code is written.

Identify the protected resources:

```
Orders  
Invoices  
User profile  
Administration  
Reports  
Health checks
```

Identify the callers:

```
Anonymous user  
Authenticated customer  
Employee  
Administrator  
Background service  
Partner API
```

Define access rules in ordinary language:

```
Customers can read their own orders.  
Employees can read orders for assigned regions.  
Administrators can change order status.  
Background services can write fulfillment events.  
Anonymous users can read public health metadata only.
```

Then translate those rules into authentication schemes, policies, claims, roles, and application checks.

Create an API:

```
dotnet new webapi -n Chapter07.Api  
cd Chapter07.Api
```

Add JWT bearer support:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Initialize user secrets for local values:

```
dotnet user-secrets init  
dotnet user-secrets set "Auth:Authority" "https://login.example.com/"  
dotnet user-secrets set "Auth:Audience" "chapter07-api"
```


Build and run:

```
dotnet build
dotnet run
```

The exact identity-provider configuration depends on the provider. Microsoft Entra ID, Auth0, Okta, Keycloak, and other providers all require application registration, redirect URI or API audience settings, and token configuration. Those details belong to the provider setup. The ASP.NET Core application should treat the provider as the trusted authority and validate tokens accordingly.

During development, avoid these habits:

- do not commit tokens;
- do not paste production client secrets into local files;
- do not log authorization headers;
- do not turn off token validation to “get unstuck”;
- do not rely on frontend checks as the only authorization enforcement.

Use local test tokens, development identity providers, or integration-test helpers when available. Chapter 8 returns to testing; this chapter’s goal is to establish the secure workflow mindset.

Layer 5 — Production Usage

Production security starts with trust boundaries.

Configuration should separate local, test, staging, and production identity settings. A token issued for a development audience should not work against production. A production API should validate issuer, audience, lifetime, and signature. Secrets should be stored in a managed secret store or platform secret system. Examples include Azure Key Vault, AWS Secrets Manager, Google Secret Manager, Kubernetes secrets with appropriate encryption and access controls, or a dedicated enterprise vault. Environment variables can be a delivery mechanism, but they are not a complete secret-management strategy by themselves.

Security requires least privilege. The application identity should have only the database, storage, queue, and cloud permissions it needs. Users and service clients should receive only the scopes, roles, or claims required for their work.

Reliability includes key rollover, token expiration, identity-provider availability, clock synchronization, and fallback behavior. If the identity provider has an outage, existing token validation may continue for a period depending on cached metadata and keys, but new sign-ins may fail.

Deployment must protect credentials. CI/CD systems should use secure variable stores, federated identity, managed identity, or short-lived credentials when possible. Long-lived client secrets copied between systems are operational debt.

Observability should record security-relevant events without exposing secrets. Log authentication failures, authorization failures, suspicious request patterns, and administrative actions. Do not log bearer tokens, passwords, client secrets, full connection strings, or sensitive claims.

Scaling affects security because multiple application instances must agree on token validation, data-protection keys for cookie scenarios, clock behavior, and policy configuration. Stateless JWT validation helps APIs scale, but revocation and permission changes require careful design.

Persistence matters because authorization often depends on data ownership. “Can read orders” may not be enough. The application may need to check whether the order belongs to the current tenant, customer, region, or account.

Cost appears through identity-provider licensing, secret store operations, audit-log retention, security scanning, and compliance requirements. Security costs are usually cheaper than incident costs.

Local development optimizes for learning and fast feedback. Production optimizes for verified trust, least privilege, credential hygiene, auditing, and controlled access.

Layer 6 — Tradeoffs and Alternatives

Use ASP.NET Core authentication and authorization for application-level security decisions. Use platform security for infrastructure-level access. Use database security for data-level protection. Strong systems use all three.

Cookies are natural for browser-based web applications where the server and browser maintain an authenticated session. JWT bearer tokens are natural for APIs called by clients, single-page applications, mobile apps, or other services. API keys can be useful for simple service integration, but they do not identify users well and require careful rotation and scope control.

OAuth 2.0 is for delegated authorization. OpenID Connect is for sign-in and identity. Do not describe every token flow as “OAuth login” without knowing which problem the flow solves.

Microsoft Entra ID is common in Microsoft-centered enterprises. Auth0, Okta, Keycloak, Duende IdentityServer, AWS Cognito, Google Identity, and other providers may be better fits depending on organization, hosting platform, customer identity needs, open-source preference, compliance, and operations skills.

ASP.NET Core Identity is useful when an application owns its own user store, especially for traditional web applications. External identity providers are often preferable for enterprise SSO, customer identity, multi-factor authentication, federation, and centralized account governance.

Common overengineering mistakes:

- building a custom identity provider when a managed one would do;
- using roles for every fine-grained business rule;
- trusting decoded JWT payloads without validation;
- putting authorization only in the frontend;
- storing production secrets in `appsettings.json`;
- logging tokens during debugging and forgetting to remove the log;
- using one overpowered service account for every dependency;
- treating authentication as complete security.

State-of-the-art security in modern .NET favors standards-based identity, short-lived tokens, policy-based authorization, managed identities, external secret stores, least privilege, strong telemetry, and threat modeling early in the design.

Layer 7 — Interview Perspective

Interviewers use security questions to test whether you know the difference between identity, access, and secret handling.

Concepts commonly tested:

- authentication versus authorization;
- claims, roles, and policies;
- JWT bearer authentication;
- OAuth 2.0 versus OpenID Connect;
- access tokens versus ID tokens;
- middleware order;
- secret storage;
- least privilege;
- tenant or ownership checks;
- avoiding token and secret leakage.

Representative questions:

- “What is the difference between authentication and authorization?”
- “How does JWT bearer authentication work in an ASP.NET Core API?”
- “Why should an API validate issuer and audience?”
- “What is OpenID Connect’s relationship to OAuth 2.0?”

- “When would you use policies instead of roles?”
- “Where should production secrets be stored?”
- “Why is frontend authorization not enough?”
- “How would you secure an endpoint that returns customer orders?”

A strong answer separates concerns:

“Authentication establishes the caller’s identity, often through a token or cookie. Authorization decides whether that identity can perform the requested action. For an orders API, I would validate the token, require a policy such as `orders.read`, and still check that the requested order belongs to the caller’s tenant or account.”

Common misconceptions:

- “If a user is authenticated, they can use the endpoint.”
- “JWTs are secure because they are encoded.”
- “OAuth and OpenID Connect are the same thing.”
- “Roles are always enough.”
- “Secrets in environment variables are automatically safe.”
- “HTTPS removes the need for token validation.”
- “The frontend can hide unauthorized buttons, so the API is protected.”

Small design scenario:

You are adding security to the order API from Chapter 6. Customers can read their own orders. Employees can read orders for assigned regions. Administrators can update order status. A background service can add shipment events.

A good design would use a trusted identity provider, JWT bearer authentication for the API, policies for scopes or permissions, application-level ownership checks for customer and region access, least-privilege database access, and a secret store for connection strings and identity-provider settings.

The strong answer protects the business operation, not just the URL.

Hands-On Lab

Objective:

Add basic authorization structure and local secret storage to an ASP.NET Core API.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Basic ASP.NET Core experience from Chapter 5.
- No real identity provider is required for the conceptual parts of this lab.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter07.Api
cd Chapter07.Api
```

2. Add JWT bearer support:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

3. Initialize user secrets:

```
dotnet user-secrets init
```

4. Store placeholder development settings:

```
dotnet user-secrets set "Auth:Authority" "https://login.example.com/"
dotnet user-secrets set "Auth:Audience" "chapter07-api"
```

5. Add authentication and authorization services in `Program.cs`.
6. Add `UseAuthentication()` before `UseAuthorization()`.
7. Add a policy named `CanReadOrders`.
8. Protect an endpoint with `.RequireAuthorization("CanReadOrders")`.
9. Leave one simple endpoint anonymous, such as `/health/basic`.
10. Build the project:

```
dotnet build
```

Expected results:

- The project builds.
- Authentication and authorization are configured.
- Protected endpoints require authorization.
- Local secret values are stored outside source-controlled files.

Validation commands:

```
dotnet build
dotnet user-secrets list
dotnet run
```

Troubleshooting notes:

- If the app fails on startup, check that configuration keys match the code.
- If all endpoints are protected, confirm the intended anonymous endpoint does not call `RequireAuthorization`.
- If authorization always fails, remember that a real request needs a valid token from the configured authority.
- If `dotnet user-secrets` fails, confirm the command is run from the project directory.

Knowledge Check

1. Why is authentication not enough to protect an API?
2. What does authorization decide?
3. Why should a JWT be validated instead of merely decoded?
4. What is the difference between an access token and an ID token?
5. How does OpenID Connect relate to OAuth 2.0?
6. Why does middleware order matter for authentication and authorization?
7. When are policies better than roles?
8. Why should authorization sometimes check resource ownership, not only claims?
9. What kinds of values should be treated as secrets?
10. Why are user secrets appropriate for development but not production?

Summary

Authentication identifies the caller. Authorization decides what the caller may do. Secrets management protects the sensitive values the application needs in order to run.

ASP.NET Core provides authentication schemes, middleware, authorization policies, endpoint protection, and integration points for modern identity providers. JWT bearer authentication is common for APIs. OAuth 2.0 addresses delegated authorization. OpenID Connect extends OAuth 2.0 for sign-in and identity.

The most important security habit is to protect business operations, not just routes. Validate tokens. Require policies. Check ownership and tenant boundaries. Keep secrets out of code. Use least privilege for users, services, databases, and cloud resources.

The next chapter adds automated testing and CI basics so application behavior, including security-sensitive behavior, can be validated repeatedly before deployment.

Sources

- [Overview of ASP.NET Core authentication](#)
- [Introduction to authorization in ASP.NET Core](#)
- [Configure JWT bearer authentication in ASP.NET Core](#)
- [OpenID Connect on the Microsoft identity platform](#)
- [Safe storage of app secrets in development in ASP.NET Core](#)
- [OWASP API Security Top 10](#)

Further Reading

- [Authentication and authorization in Minimal APIs](#)
- [Policy-based authorization in ASP.NET Core](#)
- [Microsoft identity platform documentation](#)
- [Azure Key Vault documentation](#)
- [OWASP Cheat Sheet Series](#)

8

Automated Testing and CI Basics

Chapter Purpose

The previous chapters built the first usable application shape: modern .NET, ASP.NET Core, data access, and security. That is enough to write meaningful software. It is also enough to break meaningful software.

Automated testing and continuous integration exist because professional teams need confidence before change reaches production. Manual testing, careful code review, and developer judgment still matter, but they are not enough by themselves. People forget steps. Local machines drift. Integration problems hide until code meets other code. Security-sensitive behavior can regress quietly. A database change can compile but fail when the application starts.

If your earlier model involved a local Visual Studio build, a QA phase, and a build server that produced occasional releases, the modern model pulls validation earlier. Every proposed change should be able to prove basic health: the solution restores, builds, and passes the relevant test suite in a clean environment.

This chapter introduces automated validation, unit tests, integration tests, test projects, GitHub Actions, pull request checks, and build pipelines. It focuses on validating application changes before deployment. Production release automation comes later in Chapter 16.

Where This Fits

Automated testing and CI sit between developer workflow and deployment.

```
Developer branch
  |
  v
Local build and tests
  |
  v
Pull request
  |
  v
CI workflow
  |
  +-- restore packages
  +-- build solution
  +-- run unit tests
  +-- run selected integration tests
  +-- report status
  |
  v
Merge decision
```

This chapter builds directly on Chapter 3's workflow loop. It uses the application structure from Chapters 4-7 but does not require Linux, Docker, cloud hosting, or production pipeline knowledge yet.

That ordering matters. Before learning deployment automation, the reader should know what a pipeline is supposed to validate. A fast, repeatable build and test process is the foundation. Without it, continuous deployment simply makes broken releases move faster.

Connection to the Reader's Existing Model

You probably already know several older validation patterns:

- compile the solution in Visual Studio;
- run the application locally;
- execute a few manual smoke tests;
- rely on QA test plans;
- use a build server to produce artifacts;
- deploy to a test server;
- fix bugs found after integration.

Modern testing and CI keep the same goal but move feedback closer to the code change.

A unit test is like a small repeatable check around one piece of logic. Instead of opening the application and manually exercising a case, the test runner does it every time.

An integration test is closer to exercising a slice of the application the way the runtime uses it. In ASP.NET Core, integration tests can start a test host and send HTTP requests through the request pipeline.

A CI build is like a clean build machine with discipline. It does not care that the code worked in your Visual Studio session. It restores dependencies, builds, and tests from source in a separate environment.

A pull request check is like an automated reviewer that handles mechanical questions: does it compile, do tests pass, did formatting drift, did a basic security scan fail? It cannot judge whether the design is good, but it can protect the team from many repeatable mistakes.

The analogy breaks down when CI is treated as a final QA department. CI is a feedback system, not a substitute for thoughtful design, exploratory testing, threat modeling, performance testing, or production monitoring.

Layer 1 — Conceptual Model

Automated testing is executable evidence that the software behaves as expected in specific scenarios.

Continuous integration is the practice of automatically validating changes when they are committed or proposed for merge.

Together, they solve these problems:

- catching regressions early;
- proving that the project builds outside one developer's machine;
- reducing integration surprises;
- documenting expected behavior in executable form;
- giving reviewers confidence;
- protecting the main branch;
- making later deployment automation safer.

They do not solve these problems automatically:

- they do not prove the absence of bugs;
- they do not replace human review;
- they do not guarantee useful coverage;

- they do not validate production performance by default;
- they do not make flaky tests trustworthy;
- they do not remove the need for observability after deployment.

The conceptual model is:

```
Unit tests answer: does this small behavior work?
Integration tests answer: do these components work together?
CI answers: does this change work from a clean shared process?
Pull request checks answer: is this change ready for human merge judgment?
```

The goal is not to maximize test count. The goal is to create a fast, trustworthy signal about the risks that matter.

Layer 2 — System Relationships

The production project contains application code. It might be an ASP.NET Core API, class library, worker service, or console application.

The test project contains test code. It references the production project and uses a test framework such as MSTest, xUnit, or NUnit.

The test framework provides attributes, assertions, fixtures, and conventions. MSTest, xUnit, and NUnit are common choices in .NET. The best choice is often the one your team already understands.

The test runner discovers and executes tests. `dotnet test` builds the project and runs tests using the configured runner. In .NET 10, test runner selection can be configured, with VSTest as the traditional default and Microsoft Testing Platform available for newer scenarios.

The system under test, often shortened to SUT, is the application or component being tested. In an ASP.NET Core integration test, the SUT is usually the web application started by the test host.

Test doubles replace dependencies when isolation is useful. Fakes, stubs, mocks, and in-memory implementations can make unit tests fast and focused. They can also hide integration problems when overused.

The CI system runs validation away from the developer's workstation. GitHub Actions, Azure Pipelines, GitLab CI, Jenkins, TeamCity, CircleCI, and others can all perform this role.

The pull request receives CI status. Branch protection can require selected checks to pass before merge.

Failure boundaries include missing SDKs, package restore failures, flaky tests, tests that depend on local machine state, slow integration tests, secrets missing in CI, inconsistent databases, incorrect test data, weak assertions, and test suites that are ignored because they are noisy.

Layer 3 — Core Mechanics

A unit test starts with a small behavior.

```
public sealed class OrderTotalCalculator
{
    public decimal Calculate(decimal subtotal, decimal tax)
    {
        if (subtotal < 0)
        {
            throw new ArgumentOutOfRangeException(nameof(subtotal));
        }

        return subtotal + tax;
    }
}
```

An MSTest unit test might look like this:


```
using Microsoft.VisualStudio.TestTools.UnitTesting;

[TestClass]
public sealed class OrderTotalCalculatorTests
{
    [TestMethod]
    public void Calculate_WithSubtotalAndTax_ReturnsTotal()
    {
        var calculator = new OrderTotalCalculator();

        var result = calculator.Calculate(100m, 8.25m);

        Assert.AreEqual(108.25m, result);
    }
}
```

The familiar pattern is Arrange, Act, Assert:

```
Arrange: create the situation
Act: run the behavior
Assert: verify the result
```

An ASP.NET Core integration test checks a broader slice. It can start the app in a test host and make HTTP requests through the pipeline:

```
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.VisualStudio.TestTools.UnitTesting;

[TestClass]
public sealed class HealthEndpointTests
{
    [TestMethod]
    public async Task HealthEndpoint_ReturnsOk()
    {
        await using var factory =
            new WebApplicationFactory<Program>();

        using var client = factory.CreateClient();

        var response = await client.GetAsync("/health/basic");

        response.EnsureSuccessStatusCode();
    }
}
```

This type of test can catch routing, middleware, dependency injection, and configuration problems that a small unit test would miss.

A basic GitHub Actions workflow for .NET validation has the same shape as the local commands:

```
name: dotnet-ci

on:
  pull_request:
  push:
    branches: [ "main" ]

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6
```

```

- uses: actions/setup-dotnet@v4
  with:
    dotnet-version: "10.0.x"

- run: dotnet restore

- run: dotnet build --no-restore --configuration Release

- run: dotnet test --no-build --configuration Release

```

The YAML is not magic. It checks out code, installs or selects .NET, restores packages, builds, and runs tests.

Layer 4 — Developer Workflow

Start by creating a solution with production and test projects:

```

dotnet new sln -n Chapter08
dotnet new classlib -n Chapter08.Domain
dotnet new mstest -n Chapter08.Domain.Tests
dotnet sln add Chapter08.Domain/Chapter08.Domain.csproj
dotnet sln add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj
dotnet add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj reference Chapter08.Domain/Chapter08.Domain.csproj

```

Run tests locally:

```
dotnet test
```

Run with Release configuration:

```
dotnet test --configuration Release
```

Filter tests when diagnosing:

```
dotnet test --filter "FullyQualifiedName~Order"
```

Create an ASP.NET Core integration test project when testing the request pipeline:

```

dotnet new mstest -n Chapter08.Api.Tests
dotnet add Chapter08.Api.Tests package Microsoft.AspNetCore.Mvc.Testing
dotnet add Chapter08.Api.Tests reference Chapter08.Api/Chapter08.Api.csproj
dotnet sln add Chapter08.Api.Tests/Chapter08.Api.Tests.csproj

```

Keep unit and integration tests separated when practical. This makes it easier to run fast tests frequently and broader tests intentionally.

Create a workflow file:

```
.github/workflows/dotnet-ci.yml
```

Use it to run restore, build, and test on pull requests and pushes to `main`.

The daily workflow becomes:

```

Change code
|
v
Run focused tests locally
|
v
Run full local validation before opening PR
|
v

```

```
Open PR
|
v
CI validates in clean environment
|
v
Reviewer evaluates behavior and design
```

When CI fails, assume the signal matters until you prove otherwise. A flaky test is not harmless. It teaches the team to distrust the pipeline.

Layer 5 — Production Usage

This chapter is not about production deployment, but test and CI choices shape production risk.

Configuration should distinguish local test settings from production settings. Tests should not accidentally connect to production databases, production queues, production identity providers, or paid AI services.

Secrets should be avoided in tests when possible. When CI needs secrets, use the CI platform's secure secret store and restrict access. Never commit secrets to test configuration.

Security-sensitive behavior deserves tests. Authorization policies, ownership checks, input validation, and secret-loading behavior should not rely only on manual testing. Chapter 7's security boundaries are good candidates for focused integration tests.

Reliability improves when CI protects the main branch. A main branch that regularly fails builds becomes background noise. A healthy main branch gives the team a trustworthy integration point.

Deployment depends on validated artifacts. Later CD pipelines should consume artifacts produced after restore, build, and test. If the validation path is unclear, deployment automation inherits that uncertainty.

Observability starts in CI too. Failed tests, test duration, flaky test patterns, code coverage trends, and build times are signals about the health of the engineering system.

Scaling the test suite requires layers. Thousands of slow integration tests on every pull request may block development. A better shape is many fast unit tests, focused integration tests for important boundaries, and heavier tests scheduled or gated where appropriate.

Persistence in tests requires caution. Tests that write to a real database need clean setup and teardown, isolated data, or disposable databases. Shared test databases often create order-dependent failures.

Cost appears through CI minutes, hosted runners, cloud test environments, database instances, browser tests, and AI-assisted test generation. Fast feedback is valuable; wasteful feedback is still waste.

Local development optimizes for speed. CI optimizes for clean reproducibility. Production optimizes for safety. The three should align, but they are not the same environment.

Layer 6 — Tradeoffs and Alternatives

Use unit tests for deterministic business rules, validation logic, mapping logic, calculations, authorization decisions, and small service behavior.

Use integration tests for ASP.NET Core routing, middleware, dependency injection, database access, configuration, authentication behavior, and important cross-component flows.

Use end-to-end tests sparingly for complete user or system flows. They are valuable, but slower and more fragile. Browser automation tools such as Playwright are useful when the UI matters.

MSTest, xUnit, and NUnit are all credible .NET test frameworks. MSTest has first-party Microsoft positioning and integrates naturally with Visual Studio. xUnit is common in open-source and ASP.NET Core samples. NUnit is mature and widely used. Pick one deliberately and avoid mixing frameworks casually.

GitHub Actions is a natural CI choice for repositories hosted on GitHub. Azure Pipelines fits many Azure DevOps organizations. GitLab CI, Jenkins, TeamCity, CircleCI, and Buildkite are valid alternatives depending on existing platform, compliance, hosting, and team skill.

Common overengineering mistakes:

- testing trivial property getters while missing business rules;
- mocking every dependency until the test no longer represents reality;
- writing only integration tests and accepting slow feedback;
- depending on test execution order;
- allowing flaky tests to remain unresolved;
- treating code coverage percentage as proof of quality;
- running production-like tests against production resources;
- adding CI gates no one understands.

The state-of-the-art direction is faster, more reliable feedback: clear test layers, parallel execution where safe, disposable infrastructure for integration tests, test impact analysis in large systems, and CI workflows that produce useful diagnostics instead of only red or green lights.

Layer 7 — Interview Perspective

Interviewers use testing and CI questions to learn whether you can work safely on a team.

Concepts commonly tested:

- unit tests versus integration tests;
- Arrange, Act, Assert;
- test project structure;
- `dotnet test`;
- test doubles;
- CI workflow basics;
- pull request checks;
- branch protection;
- flaky tests;
- testing security and data-access behavior.

Representative questions:

- “What is the difference between a unit test and an integration test?”
- “What should run in CI for a .NET API?”
- “How would you structure test projects in a solution?”
- “What do you do when a test passes locally but fails in CI?”
- “Why are flaky tests dangerous?”
- “How would you test an authorization policy?”
- “What belongs in a pull request check versus a deployment pipeline?”

A strong answer connects tests to risk:

“I use unit tests for fast feedback on isolated business behavior and focused integration tests for boundaries such as HTTP routing, dependency injection, database access, and authorization. CI should restore, build, and run the relevant tests in a clean environment before merge.”

Common misconceptions:

- “Unit tests prove the whole application works.”

- “Integration tests are always better because they test more.”
- “CI is only useful after deployment automation exists.”
- “Flaky tests are acceptable if they usually pass.”
- “Code coverage means the behavior is correct.”
- “Tests should connect to whatever database is easiest.”
- “A pull request check replaces code review.”

Small design scenario:

You inherit an ASP.NET Core order API with EF Core and JWT authorization. The team manually tests endpoints before release and often finds broken authorization or migration problems late.

A good first validation plan would add unit tests for order business rules, integration tests for protected endpoints, a migration check for pending model changes, and a GitHub Actions workflow that restores, builds, and runs tests on pull requests. It would not begin by building a complex production release pipeline.

Hands-On Lab

Objective:

Create a test project, write a unit test, and add a basic GitHub Actions CI workflow for a .NET solution.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Git installed.
- A GitHub repository if you want to run the workflow remotely.

Steps:

1. Create a solution and projects:

```
dotnet new sln -n Chapter08
dotnet new classlib -n Chapter08.Domain
dotnet new mstest -n Chapter08.Domain.Tests
dotnet sln add Chapter08.Domain/Chapter08.Domain.csproj
dotnet sln add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj
dotnet add Chapter08.Domain.Tests/Chapter08.Domain.Tests.csproj reference
Chapter08.Domain/Chapter08.Domain.csproj
```

2. Add a small class to the domain project:

```
public sealed class OrderTotalCalculator
{
    public decimal Calculate(decimal subtotal, decimal tax)
    {
        if (subtotal < 0)
        {
            throw new ArgumentOutOfRangeException(nameof(subtotal));
        }

        return subtotal + tax;
    }
}
```

3. Add a test for the calculator.

4. Run tests:

```
dotnet test
```

5. Create `.github/workflows/dotnet-ci.yml`.

6. Add this workflow:

```
name: dotnet-ci

on:
  pull_request:
  push:
    branches: [ "main" ]

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6

      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"

      - run: dotnet restore
      - run: dotnet build --no-restore --configuration Release
      - run: dotnet test --no-build --configuration Release
```

7. Commit the workflow and open a pull request if using GitHub.

Expected results:

- The solution builds.
- The test project references the production project.
- `dotnet test` runs the test locally.
- GitHub Actions can restore, build, and test the solution on pull requests.

Validation commands:

```
dotnet restore
dotnet build
dotnet test
git status
```

Troubleshooting notes:

- If the test project cannot see the production type, check the project reference.
- If CI fails but local tests pass, compare SDK versions and operating systems.
- If the workflow cannot find a solution, confirm it runs from the repository root.
- If `actions/setup-dotnet` cannot find the version, check the supported SDK version syntax.

Knowledge Check

1. Why does automated validation belong before deployment automation?
2. What is the difference between a unit test and an integration test?
3. Why should unit tests usually avoid real databases and network calls?
4. When is an integration test worth its extra cost?
5. What does `dotnet test` do?
6. Why is a clean CI environment more trustworthy than one developer's machine?
7. What should a basic .NET pull request workflow validate?
8. Why are flaky tests harmful to team behavior?
9. What security-sensitive behavior should be tested in an API?

10. Why is code coverage useful but insufficient?

Summary

Automated testing turns expected behavior into executable evidence. Continuous integration runs that evidence in a clean shared process before changes are merged.

Unit tests are fast checks around small behavior. Integration tests verify that important components work together, such as ASP.NET Core routing, middleware, dependency injection, configuration, data access, and authorization. CI ties those checks to pull requests and the main branch.

The goal is not a giant test suite for its own sake. The goal is trustworthy feedback. A healthy validation system is fast enough to use, broad enough to catch meaningful regressions, and reliable enough that the team believes it.

The next chapter shifts operating systems. With a validated .NET application workflow in place, you can now learn the Linux concepts modern .NET developers need before containers and cloud hosting.

Sources

- [dotnet test command](#)
- [Write tests with MSTest](#)
- [Integration tests in ASP.NET Core](#)
- [Building and testing .NET with GitHub Actions](#)
- [GitHub Actions documentation](#)
- [Order unit tests](#)

Further Reading

- [Testing in .NET](#)
- [Unit testing C# with MSTest and .NET](#)
- [ASP.NET Core testing documentation](#)
- [Workflow syntax for GitHub Actions](#)

9

Linux for .NET Developers

Chapter Purpose

Modern .NET is cross-platform, which means a .NET developer can no longer assume that production is always Windows Server and IIS.

That does not mean you must become a full-time Linux administrator. This chapter has a narrower goal: give an experienced Windows and .NET developer enough Linux fluency to understand where modern .NET applications run, how to navigate a Linux environment, how processes and permissions work, how basic networking is inspected, and why Linux matters before Docker and cloud hosting.

Linux exists because the software industry needed an open, Unix-like operating system that could run across many hardware and server environments. It became the dominant operating-system foundation for cloud infrastructure, containers, DevOps tooling, and open-source server software. Microsoft did not make .NET cross-platform by accident. .NET had to meet the operating system where much of modern server computing already lived.

For the reader of this book, Linux is not a replacement for everything you know about Windows. It is a second operating-system mental model. You already understand processes, services, files, permissions, networking, logs, and deployment. This chapter translates those ideas.

Where This Fits

Linux is the operating-system layer underneath many modern .NET runtime environments.

```
ASP.NET Core application
  |
  v
.NET runtime
  |
  v
Linux process
  |
  +-- filesystem
  +-- users and permissions
  +-- environment variables
  +-- sockets and ports
  +-- logs and service manager
  |
  v
VM, container host, cloud app platform, or Kubernetes node
```

Chapter 10 introduces Docker. Docker will make much more sense if Linux processes, filesystems, users, ports, and environment variables are already familiar. Chapter 12 returns to hosting. Chapter 18 introduces Kubernetes, where Linux concepts appear again underneath pods and containers.

This chapter is about orientation, not mastery. You need enough Linux to read commands, diagnose common hosting problems, and avoid Windows assumptions.

Connection to the Reader's Existing Model

Windows and Linux solve similar operating-system problems with different conventions.

A Windows process maps to a Linux process. Both have process IDs, memory, threads, environment variables, open files, and network connections.

A Windows service maps roughly to a `systemd` service on many Linux distributions. Both define long-running background processes that start, stop, restart, and write logs. The analogy is useful, but Linux distributions can vary, and containers often run a process directly without `systemd`.

An IIS application pool identity maps to a Linux user account or service user. Both determine what the running process can access. The difference is that Linux file permissions and ownership are visible almost everywhere and are central to daily troubleshooting.

`C:\inetpub\wwwroot` maps conceptually to an application directory such as `/var/www/myapp`, `/opt/myapp`, or a container working directory. The Linux filesystem does not use drive letters. Everything hangs from `/`, the root of the filesystem.

Windows Event Log maps partly to `journalctl`, partly to application console logs, and partly to files under `/var/log`. In containers and cloud platforms, writing structured logs to standard output is often preferred.

Windows environment variables map directly to Linux environment variables, but syntax differs. PowerShell uses `$env:ASPNETCORE_ENVIRONMENT`; shells such as Bash use `$ASPNETCORE_ENVIRONMENT`.

The analogy breaks down when you assume the same defaults. Linux paths are case-sensitive on most filesystems. `/app/config.json` and `/app/Config.json` are different names. File execution depends on permission bits. Services often run as non-root users. Package installation, certificates, process management, and shell behavior are different enough to deserve attention.

Layer 1 — Conceptual Model

Linux is an operating-system family built around the Linux kernel plus a userspace of tools, libraries, shells, package managers, and distributions.

For .NET developers, Linux solves these practical problems:

- it provides a common server runtime environment;
- it underpins most container hosts;
- it is widely used by cloud platforms;
- it supports command-line automation well;
- it gives lightweight server images for APIs and services;
- it provides a common environment for CI runners and deployment targets.

It does not solve these problems by itself:

- it does not make an application cloud-native;
- it does not remove the need for deployment discipline;
- it does not make security automatic;
- it does not replace SQL Server, IIS, or Windows where those are still the right fit;
- it does not make every Windows-specific .NET application portable.

The conceptual center is this:

Linux hosts processes.
Processes read files, environment variables, ports, and permissions.
.NET applications are processes.

Once you understand that, Linux becomes less mysterious. An ASP.NET Core app on Linux is still a process listening on a port, reading configuration, calling dependencies, writing logs, and exiting with a status code.

Layer 2 — System Relationships

The Linux kernel owns low-level resources: CPU scheduling, memory, processes, filesystems, devices, networking, and permissions.

The distribution packages the kernel with userspace tools and conventions. Ubuntu, Debian, Red Hat Enterprise Linux, Fedora, Alpine, SUSE, and others are Linux distributions. They differ in package managers, default tools, support policies, filesystem layout details, and operational conventions.

The shell receives commands from the user or script. Bash is common, though many shells exist. The shell expands variables, runs programs, redirects input/output, and connects commands.

The package manager installs and updates software. Debian and Ubuntu commonly use `apt`. Red Hat-family systems commonly use `dnf` or `yum`. Alpine uses `apk`. .NET installation instructions vary by distribution.

The filesystem stores application files, configuration, logs, certificates, temporary data, and mounted volumes. Linux uses `/` as the root and paths such as `/etc`, `/var`, `/usr`, `/home`, `/tmp`, and `/opt`.

Users and groups define ownership and access. A process runs as a user. That user's permissions determine whether the process can read a file, write a directory, bind to certain ports, or access mounted resources.

Networking exposes sockets and ports. An ASP.NET Core process may listen on `http://0.0.0.0:8080` inside a Linux host or container. A firewall, reverse proxy, container port mapping, or cloud load balancer may determine whether clients can reach it.

The .NET runtime executes the application. Microsoft provides .NET downloads and installation guidance for Windows, macOS, and many Linux distributions, and .NET 10 downloads include Linux SDK/runtime builds for multiple architectures.

Failure boundaries include missing packages, wrong CPU architecture, file permission errors, case-sensitive path bugs, missing environment variables, ports already in use, certificate trust problems, line-ending issues, native library dependencies, and differences between a full VM and a minimal container image.

Layer 3 — Core Mechanics

The filesystem starts at `/`.

<code>/</code>	root of the filesystem
<code>/home</code>	user home directories
<code>/etc</code>	system configuration
<code>/var</code>	variable data such as logs and application state
<code>/tmp</code>	temporary files
<code>/usr</code>	installed user-space programs and libraries
<code>/opt</code>	optional or vendor application software

Paths use forward slashes:

```
/home/steve/appsettings.json
/var/log/myapp/app.log
/opt/orders-api/Orders.Api.dll
```

Permissions are commonly shown with `ls -l`:

```
-rw-r--r--  1 app app  1024 Jul 22 appsettings.json
drwxr-xr-x  2 app app  4096 Jul 22 logs
```

The first character indicates type: `-` for file, `d` for directory. The next groups represent read, write, and execute permissions for owner, group, and others.

Processes can be inspected:

```
ps aux
ps -ef
```

Ports can be inspected:

```
ss -tulpn
```

Environment variables can be viewed:

```
printenv
echo "$ASPNETCORE_ENVIRONMENT"
```

A .NET app can run directly:

```
dotnet Orders.Api.dll
```

Or a published self-contained app can run as an executable:

```
./Orders.Api
```

The execute bit matters. If Linux says “permission denied” when running a file, it may not be executable:

```
chmod +x Orders.Api
```

Logs may come from the process output:

```
dotnet Orders.Api.dll
```

Or from `systemd` when the app is configured as a service:

```
journalctl -u orders-api
```

These commands are not the whole Linux world. They are the beginning of the Linux vocabulary a .NET developer needs.

Layer 4 — Developer Workflow

Start by identifying the environment:

```
uname -a
cat /etc/os-release
whoami
pwd
```

Navigate the filesystem:

```
ls
ls -la
cd /tmp
mkdir chapter09
cd chapter09
```

Create and inspect a file:

```
echo "hello from linux" > message.txt
cat message.txt
ls -l message.txt
```

Inspect environment variables:

```
printenv | sort
export ASPNETCORE_ENVIRONMENT=Development
echo "$ASPNETCORE_ENVIRONMENT"
```

Check .NET:

```
dotnet --info
dotnet --list-runtimes
```

Create and run a .NET app:

```
dotnet new console -n LinuxDotnetDemo
cd LinuxDotnetDemo
dotnet run
```

Publish it:

```
dotnet publish -c Release
```

Find files:

```
find . -name "*.dll"
```

Inspect running processes:

```
ps -ef | grep dotnet
```

The workflow habit is to ask four questions:

```
Where am I in the filesystem?
Which user am I?
Which process is running?
Which port or file is it trying to use?
```

Those four questions solve a surprising number of first-level Linux hosting problems.

Layer 5 — Production Usage

Production Linux hosting starts with ownership.

Do not run application processes as `root` unless there is a specific and justified reason. Create a service user or use the platform's non-root runtime model. File permissions should allow the application to read what it needs and write only where it is expected to write.

Configuration should come from files, environment variables, secret stores, or platform configuration, depending on the host. Avoid editing production application binaries or source files on the server.

Secrets should not be stored in shell history, world-readable files, or checked-in scripts. Linux permissions help, but a readable secret file is still a secret exposure if too many users or processes can read it.

Security includes patching, user permissions, firewall rules, SSH access, certificate trust, dependency updates, and process isolation. Linux is not secure merely because it is Linux.

Reliability depends on service management, restart behavior, logs, health checks, disk space, memory pressure, CPU pressure, and dependency availability. On VM-style Linux hosting, `systemd` commonly owns service restart behavior. In containers, the container platform usually owns restart behavior.

Deployment should be repeatable. Copying files manually into `/opt` may be acceptable for learning, but production should use packages, artifacts, containers, or deployment automation.

Observability depends on where logs go. A VM service may write to `journald` and application logs. A containerized app should usually write logs to standard output so the platform can collect them.

Scaling is usually handled outside the individual Linux process: multiple VMs, containers, platform instances, or Kubernetes pods. The application still needs to be stateless enough to scale safely.

Persistence should be explicit. Local Linux disk may be temporary in cloud or container environments. Durable state belongs in databases, managed storage, or mounted volumes designed for persistence.

Cost is affected by VM size, disk, network traffic, operational labor, support subscriptions, and cloud platform choices. Linux can reduce licensing cost in some scenarios, but it does not eliminate operations cost.

Local development optimizes for learning and convenience. Production optimizes for least privilege, patching, repeatability, monitoring, and recoverability.

Layer 6 — Tradeoffs and Alternatives

Use Linux hosting when the application benefits from container alignment, cloud-native platform support, lower server footprint, open-source tooling, or team familiarity with Linux operations.

Use Windows hosting when the application depends on Windows-specific APIs, classic .NET Framework, COM components, Windows Authentication patterns, desktop automation, vendor software, or an operations team and compliance model built around Windows Server.

Use managed platforms when you do not want to administer Linux VMs directly. Azure App Service, Azure Container Apps, AWS Elastic Beanstalk, AWS ECS, Google Cloud Run, and similar services can run .NET workloads while hiding some operating-system management.

Use containers when repeatable runtime packaging matters. Containers usually run on Linux hosts, though Windows containers exist for Windows-specific workloads.

Linux distributions are alternatives too. Ubuntu is common in cloud and developer contexts. Debian is common for stability and base images. Red Hat Enterprise Linux is common in enterprises with support requirements. Alpine is common for small container images, though its musl C library can introduce compatibility considerations. The right choice depends on support, image size, package availability, security policy, and team experience.

Common overengineering mistakes:

- trying to become a Linux administrator before learning the basics needed for .NET hosting;
- running everything as `root` to avoid permission errors;
- assuming paths are case-insensitive;
- editing production files manually instead of redeploying;
- storing secrets in shell scripts;
- ignoring logs because they are not in Windows Event Viewer;
- choosing a minimal container image before understanding missing dependencies.

The state-of-the-art direction is not “Linux instead of Windows everywhere.” It is platform fit: Linux for cloud-native and container-friendly workloads, Windows where Windows-specific value remains, and managed platforms where the team should not own the OS layer directly.

Layer 7 — Interview Perspective

Interviewers usually do not expect a .NET developer to be a senior Linux administrator. They do expect comfort with the basics.

Concepts commonly tested:

- why Linux matters to modern .NET;
- filesystem layout and path differences;
- case-sensitive paths;
- users, groups, and permissions;

- processes and services;
- environment variables;
- ports and networking;
- logs and standard output;
- Linux in containers and cloud hosting;
- Windows versus Linux hosting tradeoffs.

Representative questions:

- “Why would an ASP.NET Core app run on Linux?”
- “What problems can case-sensitive paths cause?”
- “How would you check whether a .NET process is running?”
- “How would you inspect environment variables?”
- “What does `permission denied` usually suggest?”
- “Where would you expect logs to go on Linux?”
- “When would Windows Server still be the better host?”

A strong answer translates from concepts:

“An ASP.NET Core app on Linux is still a process. I would check which user it runs as, which directory it uses, whether required environment variables are set, whether it can read its files, and whether it is listening on the expected port.”

Common misconceptions:

- “Linux is required for modern .NET.”
- “Linux knowledge means memorizing every command.”
- “Containers mean I do not need to understand Linux.”
- “If a file path works on Windows, it will work the same on Linux.”
- “Running as root is fine if the server is private.”
- “Linux logs all appear in one obvious place.”

Small design scenario:

You deploy an ASP.NET Core API to a Linux VM. It starts locally on Windows but fails in production. The logs say it cannot find `AppSettings.json`.

A good investigation would check the deployed filename, the code’s expected path, Linux case sensitivity, working directory, file permissions, and whether configuration should come from environment variables instead of a copied file.

The strong answer avoids blaming Linux and inspects the operating-system boundary.

Hands-On Lab

Objective:

Practice the Linux commands and concepts most relevant to running a .NET application.

Prerequisites:

- A Linux shell. This may be WSL, a Linux VM, a cloud shell, a container shell, or a native Linux machine.
- .NET SDK installed if you want to run the .NET steps.

Steps:

1. Identify the system:

```
uname -a
cat /etc/os-release
```

2. Identify your user and location:

```
whoami  
pwd
```

3. Create a working directory:

```
mkdir -p ~/chapter09  
cd ~/chapter09
```

4. Create and inspect files:

```
echo "hello" > message.txt  
ls -la  
cat message.txt
```

5. Inspect permissions:

```
ls -l message.txt
```

6. Work with an environment variable:

```
export ASPNETCORE_ENVIRONMENT=Development  
echo "$ASPNETCORE_ENVIRONMENT"
```

7. Check .NET:

```
dotnet --info
```

8. Create and run a console app:

```
dotnet new console -n LinuxDotnetDemo  
cd LinuxDotnetDemo  
dotnet run
```

9. Inspect processes:

```
ps -ef | grep dotnet
```

Expected results:

- You can identify the Linux distribution.
- You can navigate directories and inspect files.
- You can read permission output.
- You can set and read an environment variable.
- You can run a .NET application from a Linux shell.

Validation commands:

```
uname -a  
cat /etc/os-release  
whoami  
pwd  
ls -la  
dotnet --info  
dotnet run
```

Troubleshooting notes:

- If `dotnet` is not found, install the .NET SDK for your distribution or use a shell where it is already installed.
- If a command says `permission denied`, check file permissions and the current user.
- If a path is not found, check spelling and capitalization.
- If `grep dotnet` shows only the `grep` command, no matching .NET process may be running.

Knowledge Check

1. Why does Linux matter to modern .NET development?
2. What is the practical difference between `c:\apps\api` and `/opt/api`?
3. Why can filename capitalization break a .NET app on Linux?
4. What does a Linux user account control for a running process?
5. How are environment variables useful for ASP.NET Core configuration?
6. Why should production services usually avoid running as `root`?
7. How do Linux logs differ between VM-style hosting and container hosting?
8. What is the relationship between Linux and Docker?
9. When is Windows Server still the right hosting choice?
10. What four questions should you ask first when diagnosing a Linux-hosted .NET app?

Summary

Linux is not a detour from modern .NET. It is one of the operating-system foundations underneath modern .NET hosting, containers, CI runners, cloud platforms, and Kubernetes.

For a Windows developer, the most useful first step is translation. Linux has processes, services, files, permissions, environment variables, ports, and logs just as Windows does, but the conventions are different. Paths start at `/`, filenames are usually case-sensitive, permissions are visible and central, and many hosting workflows assume shell-based automation.

You do not need to become a Linux administrator before writing .NET applications. You do need enough Linux fluency to understand where the process runs, which user owns it, which files it can read, which port it listens on, and where logs and configuration come from.

The next chapter builds on this by introducing Docker, where Linux process and filesystem concepts become the foundation for images and containers.

Sources

- [Install .NET on Windows, Linux, and macOS](#)
- [Download .NET 10.0](#)
- [The Linux Foundation: It's not just the Linux operating system](#)
- [About the Linux Foundation](#)
- [Stack Overflow Developer Survey 2025](#)

Further Reading

- [Install .NET on Ubuntu](#)
- [Install .NET on Debian](#)
- [Install .NET on Red Hat Enterprise Linux](#)
- [Linux Foundation Training](#)
- [Linux.com](#)

10

Docker

Chapter Purpose

Chapter 9 introduced Linux as an operating-system environment for modern .NET. Docker builds directly on that foundation.

Docker exists because teams needed a repeatable way to package and run applications across machines. Before containers became common, a deployment often depended on a server's installed runtime, patch level, environment variables, folders, certificates, native dependencies, and hand-maintained configuration. "Works on my machine" was not a joke; it was a real boundary.

Docker changed that boundary by packaging an application and its runtime environment into an image. A container is a running instance of that image. The same image can run on a developer machine, CI runner, test host, cloud container service, or Kubernetes cluster, assuming compatible CPU and OS support.

Docker was not invented for .NET. It became popular first in the broader Linux, cloud, and open-source world. Modern .NET adapted to that world by becoming cross-platform, publishing official container images, supporting container-friendly configuration and logging, and eventually adding SDK support for publishing container images directly.

This chapter introduces why containers exist, images, containers, volumes, networks, Dockerfiles, and container registries. Chapter 11 builds on this with Docker Compose for multi-container development environments.

Where This Fits

Docker sits between the application build and the runtime host.

```
.NET source code
|
v
dotnet publish
|
v
Docker image
|
v
Container
|
+-- filesystem from image
+-- environment variables
+-- exposed port
+-- mounted volumes when needed
+-- network connections
|
```

v
Docker host, cloud container service, or Kubernetes

Docker does not replace ASP.NET Core, SQL Server, CI/CD, cloud platforms, or Kubernetes. It provides a packaging and runtime boundary that those systems can use.

For this book's progression, Docker is the bridge from "I can run the app" to "I can run the same packaged app consistently in different places."

Connection to the Reader's Existing Model

The closest Windows-era analogy is a disciplined deployment folder plus a process boundary.

In an IIS deployment, you might publish an application to a folder, configure an application pool, set environment-specific configuration, and point IIS at the result. The server still supplied many assumptions: installed framework, native dependencies, IIS configuration, machine-level certificates, directory permissions, and OS patch level.

A Docker image moves more of that runtime assumption into a versioned package. The image can include the application files, runtime base image, startup command, working directory, exposed port metadata, and non-root user settings.

A container is not the same as a virtual machine. A VM includes a full guest operating system. A Linux container is an isolated process using the host's kernel with separate filesystem, process, network, and resource views. That is why Chapter 9 matters: a containerized ASP.NET Core app is still a Linux process.

An image is similar to an immutable deployment artifact. A container is similar to running that artifact as a process. A registry is similar to a package feed, but for container images.

Volumes are somewhat like mounted folders or attached disks. They let data outlive a container or be shared into a container.

Docker networks are somewhat like private virtual networks for containers. They let containers reach each other by name without exposing every internal port to the host.

The analogy breaks down when you treat a container like a server you log into and maintain. Containers are meant to be replaced, not lovingly patched in place. If the app changes, rebuild the image and run a new container.

Layer 1 — Conceptual Model

Docker is a platform for developing, shipping, and running applications using containers.

It solves these problems:

- packaging application runtime dependencies consistently;
- reducing environment drift between developer machines and CI;
- giving applications a clear process and filesystem boundary;
- making local development dependencies easier to run;
- producing artifacts that cloud platforms and orchestrators can run;
- standardizing image distribution through registries.

It does not solve these problems by itself:

- it does not make the application scalable;
- it does not replace tests;
- it does not make secrets safe automatically;
- it does not make the database durable unless storage is designed correctly;
- it does not replace production monitoring;
- it does not remove the need to understand Linux, networking, or security.

The conceptual model is:

```
Dockerfile describes how to build an image.  
Image is the packaged template.  
Container is a running instance of the image.  
Registry stores and distributes images.  
Volume stores data outside the container filesystem.  
Network connects containers.
```

Images are immutable in spirit. Containers are disposable in spirit. Data that must survive should live outside the container or in a real database.

Layer 2 — System Relationships

The Docker client receives commands such as `docker build`, `docker run`, `docker ps`, and `docker logs`.

The Docker engine builds images and runs containers. Docker Desktop provides a developer-friendly environment on Windows and macOS, usually backed by a Linux virtual machine for Linux containers.

The Dockerfile describes image construction. It identifies a base image, copies files, runs build steps, sets the working directory, exposes ports, and defines the startup command.

The base image provides the starting filesystem and runtime dependencies. Microsoft publishes official .NET SDK, ASP.NET Core runtime, and .NET runtime images. In .NET 10, Microsoft's default Linux container tags use Ubuntu-based images instead of the older Debian default.

The application image layers your app on top of the base image. A typical ASP.NET Core image uses a multi-stage build: one stage uses the SDK to build and publish, and a smaller runtime stage runs the published app.

The container receives runtime inputs: environment variables, port mappings, mounted volumes, network connections, CPU and memory limits, and secrets or configuration supplied by the host.

The registry stores images. Docker Hub, GitHub Container Registry, Azure Container Registry, Amazon Elastic Container Registry, and Google Artifact Registry are common examples.

Failure boundaries include wrong base image, wrong CPU architecture, missing files, bad working directory, incorrect port binding, case-sensitive path errors, missing environment variables, container exits, permission problems, unmounted volumes, network name resolution, large images, vulnerable base images, and secrets baked into image layers.

Layer 3 — Core Mechanics

A minimal ASP.NET Core Dockerfile often uses multiple stages:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build  
WORKDIR /src  
  
COPY *.csproj .  
RUN dotnet restore  
  
COPY . .  
RUN dotnet publish -c Release -o /app/publish  
  
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime  
WORKDIR /app  
  
COPY --from=build /app/publish .  
  
EXPOSE 8080  
  
ENTRYPOINT ["dotnet", "Orders.Api.dll"]
```

The build stage uses the SDK image because it needs build tools. The runtime stage uses the ASP.NET Core runtime image because it only needs to run the published app.

Build an image:

```
docker build -t orders-api:dev .
```

Run a container:

```
docker run --rm -p 8080:8080 orders-api:dev
```

The `-p 8080:8080` option maps host port `8080` to container port `8080`.

Set an environment variable:

```
docker run --rm -p 8080:8080 \
  -e ASPNETCORE_ENVIRONMENT=Development \
  orders-api:dev
```

List containers:

```
docker ps
docker ps -a
```

Inspect logs:

```
docker logs <container-name-or-id>
```

Stop a container:

```
docker stop <container-name-or-id>
```

Create a volume:

```
docker volume create orders-data
```

Run with a volume:

```
docker run --rm \
  -v orders-data:/data \
  orders-api:dev
```

Create a network:

```
docker network create orders-net
```

The modern .NET SDK can also publish container images without a Dockerfile in some scenarios:

```
dotnet publish /t:PublishContainer
```

That is useful, but Dockerfiles remain important because many teams still use them to express explicit build, runtime, and base-image choices.

Layer 4 — Developer Workflow

The basic Docker workflow is:

```
Create or update app
|
v
Write Dockerfile
|
v
Build image
|
v
Run container locally
|
```

```
      v
Inspect logs and behavior
      |
      v
Tag image
      |
      v
Push image to registry
```

Create a sample API:

```
dotnet new webapi -n Chapter10.Api
cd Chapter10.Api
```

Add a `.dockerignore` file so build output and local files are not copied into the image context:

```
bin/
obj/
.git/
.vs/
.vscode/
```

Add a Dockerfile.

Build:

```
docker build -t chapter10-api:dev .
```

Run:

```
docker run --rm -p 8080:8080 chapter10-api:dev
```

Call the app:

```
curl http://localhost:8080/weatherforecast
```

If the app listens on a different port, align the container's ASP.NET Core URL setting and the Docker port mapping:

```
docker run --rm -p 8080:8080 \
  -e ASPNETCORE_URLS=http://+:8080 \
  chapter10-api:dev
```

Tag for a registry:

```
docker tag chapter10-api:dev registry.example.com/chapter10-api:dev
```

Push:

```
docker push registry.example.com/chapter10-api:dev
```

You do not need to push images for every local experiment. The key development habit is to make the runtime environment explicit and repeatable.

Layer 5 — Production Usage

Production container usage starts with image discipline.

Choose supported base images. For .NET 10, default Microsoft Linux tags use Ubuntu-based images. If your organization requires a specific distribution, test it deliberately and maintain custom images when needed.

Configuration should be supplied at runtime, not baked into the image. The same image should usually run in development, test, staging, and production with different environment-specific settings.

Secrets should not be copied into images, written into Dockerfiles, or passed casually on command lines that end up in shell history. Use platform secret stores, secret mounts, managed identities, or CI/CD secret mechanisms.

Security includes base-image patching, vulnerability scanning, non-root containers, minimal images, signed images where appropriate, restricted capabilities, and least-privilege access to registries and runtime platforms.

Reliability depends on predictable startup, clear exit behavior, health checks, resource limits, graceful shutdown, and logs written to standard output. A container that silently fails or writes logs only to an internal file is harder to operate.

Deployment should use immutable image tags or digests. Avoid deploying whatever `latest` happens to mean at the moment. A production deployment should identify the exact image it runs.

Observability should treat containers as replaceable. Logs should go to standard output and standard error. Metrics and traces should flow to the platform or observability system. Do not depend on logging into a container to read local files.

Scaling requires stateless application containers. Durable state should live in databases, managed storage, or carefully designed volumes. Scaling a containerized API does not scale SQL Server automatically.

Persistence must be explicit. Container writable layers are temporary and tied to the container. Use volumes for data that must survive container replacement, but prefer managed databases for business data.

Cost comes from image size, registry storage, network transfer, build minutes, runtime CPU and memory, vulnerability management, and operational complexity. Small, well-built images are easier and cheaper to move.

Local development optimizes for repeatability and fast iteration. Production optimizes for immutable artifacts, security, traceability, and recoverability.

Layer 6 — Tradeoffs and Alternatives

Use Docker when you need repeatable runtime packaging, consistent local dependencies, container-based deployment, CI parity, or preparation for cloud container platforms.

Do not use Docker just because it is modern. A small internal console tool, a desktop app, or an application deployed cleanly to a managed platform may not need a custom container image.

Docker competes with direct host installation, virtual machines, platform-native deployment packages, language-specific packaging, and serverless deployment models. It is strongest when the runtime environment is part of the problem.

Podman is an open-source daemonless container alternative and can run OCI containers. Some organizations prefer it for security or Linux-native workflows.

.NET SDK container publishing can reduce Dockerfile boilerplate. Dockerfiles remain useful when the image requires custom OS packages, explicit build steps, special users, or detailed control.

Kubernetes is not an alternative to Docker in the simple sense. Kubernetes is an orchestrator for running containers across machines. Docker helps build and run containers; Kubernetes manages containerized workloads at scale.

Common overengineering mistakes:

- containerizing before the app builds cleanly from the command line;
- baking secrets into images;
- using `latest` for production deployments;
- treating containers like long-lived servers;
- storing business data in the container writable layer;
- running as root because permissions are inconvenient;
- creating huge images by copying the whole repository;
- ignoring base-image updates and vulnerability scans.

The state-of-the-art direction is secure, minimal, traceable images; explicit runtime configuration; non-root execution; SBOMs and vulnerability scanning; and simple local workflows that match CI and production where it matters.

Layer 7 — Interview Perspective

Interviewers use Docker questions to test whether you understand packaging, runtime boundaries, and tradeoffs.

Concepts commonly tested:

- image versus container;
- Dockerfile;
- layers;
- base images;
- port mapping;
- environment variables;
- volumes;
- networks;
- registries;
- containers versus virtual machines;
- Docker versus Kubernetes.

Representative questions:

- “What problem does Docker solve?”
- “What is the difference between an image and a container?”
- “Why use a multi-stage Dockerfile for .NET?”
- “How does port mapping work?”
- “Where should secrets be provided to a container?”
- “What happens to data written inside a container?”
- “When would Docker be unnecessary?”
- “How is Docker different from Kubernetes?”

A strong answer connects repeatability and boundaries:

“A Docker image packages the application and runtime environment. A container is a running instance of that image. The image helps reduce environment drift, but persistent data, secrets, networking, scaling, and monitoring still need explicit design.”

Common misconceptions:

- “A container is a lightweight virtual machine.”
- “Docker makes applications scalable.”
- “Docker replaces CI/CD.”
- “A container should be patched by logging into it.”
- “Volumes are optional for durable data.”
- “Kubernetes and Docker are the same layer.”
- “If it runs in Docker locally, it is production-ready.”

Small design scenario:

You have an ASP.NET Core API that runs locally but fails often on test servers because the runtime version, environment variables, and installed dependencies differ between machines.

A good Docker plan would define a Dockerfile, use official .NET base images, set runtime configuration through environment variables, write logs to standard output, run as a non-root user where possible, tag images with commit-based versions, and push images to a registry for test deployment.

The strong answer uses Docker to reduce environment drift without pretending it solves database, security, or deployment design by itself.

Hands-On Lab

Objective:

Containerize a small ASP.NET Core API and run it locally.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Docker Desktop or another OCI-compatible container runtime.
- A terminal.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter10.Api
cd Chapter10.Api
```

2. Add `.dockerignore`:

```
bin/
obj/
.git/
.vs/
.vscode/
```

3. Add a Dockerfile:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY *.csproj .
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app

COPY --from=build /app/publish .

EXPOSE 8080

ENTRYPOINT ["dotnet", "Chapter10.Api.dll"]
```

4. Build the image:

```
docker build -t chapter10-api:dev .
```

5. Run the container:

```
docker run --rm -p 8080:8080 \
  -e ASPNETCORE_URLS=http://+:8080 \
  chapter10-api:dev
```

6. In another terminal, call the API:

```
curl http://localhost:8080/weatherforecast
```

7. List running containers:

```
docker ps
```

8. Stop the container with `ctrl+c` in the terminal where it is running.

Expected results:

- Docker builds an image.
- Docker runs a container from that image.
- The ASP.NET Core API responds through a mapped host port.
- Logs appear in the container's console output.

Validation commands:

```
docker build -t chapter10-api:dev .
docker run --rm -p 8080:8080 -e ASPNETCORE_URLS=http://+:8080 chapter10-api:dev
docker ps
curl http://localhost:8080/weatherforecast
```

Troubleshooting notes:

- If Docker cannot find the Dockerfile, run the command from the project directory.
- If the app does not respond, confirm the port mapping and `ASPNETCORE_URLS`.
- If the endpoint path differs, use the route created by the template.
- If restore fails during build, check network access and NuGet package sources.
- If the container exits immediately, inspect the console output or run `docker logs` with the container ID.

Knowledge Check

1. Why did containers become important for modern application delivery?
2. What is the difference between a Docker image and a container?
3. Why is a container not the same as a full virtual machine?
4. What problem does a Dockerfile solve?
5. Why are multi-stage Dockerfiles useful for .NET applications?
6. What does port mapping do?
7. Why should secrets not be baked into images?
8. What happens to data written only to a container's writable layer?
9. What role does a registry play?
10. Why does Docker not automatically make an application production-ready?

Summary

Docker gives modern .NET teams a repeatable packaging and runtime boundary. An image is the packaged template. A container is a running instance. A Dockerfile describes how the image is built. Volumes provide storage outside the container filesystem. Networks connect containers. Registries store and distribute images.

For a Windows developer, Docker is best understood as a disciplined deployment artifact plus a Linux process boundary, not as a small VM. The app still needs configuration, secrets, logging, health, persistence, and security decisions.

The next chapter adds Docker Compose, which lets a team run an ASP.NET Core API, SQL Server, Redis, and other development dependencies together as a multi-container environment.

Sources

- What is Docker?
- Containerize a .NET application

- [.NET language-specific guide for Docker](#)
- [Containerize an app with Docker tutorial](#)
- [Containerize an app with dotnet publish](#)
- [Run an ASP.NET Core app in Docker containers](#)
- [Default .NET images use Ubuntu](#)

Further Reading

- [Dockerfile reference](#)
- [Docker storage overview](#)
- [Docker networking overview](#)
- [.NET container images](#)

11

Docker Compose

Chapter Purpose

Chapter 10 showed how to package and run one application container. Real development rarely stops at one process.

An ASP.NET Core API usually needs dependencies: SQL Server, Redis, a message broker, a mock identity provider, a local storage emulator, or another service. Before containers, every developer might install these dependencies directly, point at a shared development server, or maintain a long setup document. Those approaches work until they do not: versions drift, ports conflict, credentials differ, shared databases get messy, and onboarding slows down.

Docker Compose exists to describe and run a multi-container application with one configuration file and a small set of commands. It lets a team say, “For local development, this API runs with this SQL Server container, this Redis container, these environment variables, these ports, these volumes, and this private network.”

Compose was not created specifically for .NET. It became popular across the container ecosystem because developers needed a simple local orchestration tool before reaching for full platforms such as Kubernetes. Modern .NET fits well into that model because ASP.NET Core reads configuration from environment variables, logs to console, and runs cleanly in Linux containers.

This chapter introduces Docker Compose, application configuration, local SQL Server, networking, and development environments. It focuses on local and team development. Production deployment comes later.

Where This Fits

Docker Compose sits above individual containers and below production orchestration.

```
compose.yaml
|
+-- api service
|   +-- ASP.NET Core container
|   +-- environment variables
|   +-- exposed host port
|
+-- sql service
|   +-- SQL Server container
|   +-- password from development config
|   +-- named volume for data
|
+-- redis service
    +-- Redis container
    +-- internal network name
```

```
Docker Compose
|
```

v Local Docker host

Chapter 11 completes the Linux and container development section. Chapter 12 then moves toward deploying ASP.NET Core outside the local machine. Later chapters discuss cloud platforms, observability, distributed applications, and Kubernetes.

Compose is important because it creates a realistic local system without requiring cloud infrastructure on day one.

Connection to the Reader's Existing Model

Think back to a traditional development environment.

You might have installed SQL Server Developer Edition locally, configured IIS Express, used a shared test database, installed a Windows service dependency, and kept a setup checklist in a wiki. A senior developer knew the order: install this, set that port, change this connection string, run that script, restart that service.

Docker Compose turns much of that setup into a versioned file.

A Compose service is roughly like a named local server process. One service might run the API. Another might run SQL Server. Another might run Redis.

A Compose network is like a private development network where those services can reach each other by name. The API does not need to know the SQL Server container's changing IP address. It can connect to `sql`.

A Compose volume is like a named local disk managed by Docker. It lets SQL Server data survive container replacement during development.

Environment variables in Compose are like application settings supplied by the host. They are especially important because ASP.NET Core can bind environment variables into its configuration system.

The analogy breaks down if you treat Compose as a production cluster. Compose is excellent for local development, demos, integration testing, and simple single-host scenarios. It is not a full production orchestrator with multi-node scheduling, self-healing, policy, ingress, autoscaling, and platform operations.

Layer 1 — Conceptual Model

Docker Compose is a tool for defining and running multi-container applications.

It solves these problems:

- starting multiple related containers together;
- giving services stable names on a private network;
- describing local ports and environment variables;
- attaching volumes for local persistence;
- reducing setup drift between developers;
- making local integration testing more repeatable.

It does not solve these problems by itself:

- it does not replace production orchestration;
- it does not make local secrets production-safe;
- it does not guarantee cloud parity;
- it does not remove the need for database migrations;
- it does not make distributed systems simple;
- it does not automatically wait until dependencies are truly ready unless readiness is designed.

The conceptual model is:

```

Service: one container definition
Project: a group of services
Network: how services find each other
Volume: data that survives container replacement
Environment: configuration supplied to containers
Compose file: versioned description of the local system

```

The most important idea is service names. Inside a Compose network, one service can usually reach another by its service name. An API service can use `Server=sql` in its SQL Server connection string because `sql` is the Compose service name.

Layer 2 — System Relationships

The Compose file is the source-controlled description of the local container system. It is commonly named `compose.yaml` or `docker-compose.yml`.

The Compose CLI reads that file and asks the Docker engine to create networks, volumes, and containers.

Each service describes one container role: image, build context, ports, environment variables, volumes, dependencies, health checks, and command overrides.

The Docker network connects services. By default, Compose creates a network for the project and attaches services to it. Services can resolve each other by name on that network.

The host port mapping exposes selected container ports to the developer's machine. A SQL Server container might listen on port `1433` internally, while the host maps it to `14333` to avoid conflicts with an installed SQL Server.

The named volume stores data outside the container writable layer. This is useful for a local SQL Server database that should survive `docker compose down` unless volumes are removed.

The application configuration receives values from Compose environment variables. ASP.NET Core uses double underscores, such as `ConnectionStrings__Orders`, to represent nested configuration keys in environment variables.

Failure boundaries include incorrect indentation, wrong image tags, port conflicts, missing environment variables, weak local passwords, dependency containers that start but are not ready, volume state that hides migration problems, network name mistakes, and assuming a Compose setup is identical to production.

Layer 3 — Core Mechanics

A small Compose file for an ASP.NET Core API and SQL Server might look like this:

```

services:
  api:
    build:
      context: ./Chapter11.Api
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    environment:
      ASPNETCORE_URLS: "http://+:8080"
      ASPNETCORE_ENVIRONMENT: "Development"
      ConnectionStrings__Orders: "Server=sql,1433;Database=Orders;User
Id=sa;Password=#{SQL_PASSWORD};TrustServerCertificate=True"
    depends_on:
      sql:
        condition: service_healthy

  sql:
    image: mcr.microsoft.com/mssql/server:2022-latest
    ports:
      - "14333:1433"

```

```

environment:
  ACCEPT_EULA: "Y"
  MSSQL_SA_PASSWORD: "${SQL_PASSWORD}"
volumes:
  - sql-data:/var/opt/mssql
healthcheck:
  test: ["CMD-SHELL", "/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P \"${MSSQL_SA_PASSWORD}\" -C -Q \"select 1\" || exit 1"]
  interval: 10s
  timeout: 5s
  retries: 10

volumes:
  sql-data:

```

The `api` service builds from a local Dockerfile.

The `sql` service uses Microsoft's SQL Server container image.

The `ports` entry for the API maps host port `8080` to container port `8080`. The SQL Server mapping exposes host port `14333` to avoid colliding with a local SQL Server using `1433`.

The `ConnectionStrings__Orders` environment variable becomes the ASP.NET Core configuration key `ConnectionStrings:Orders`.

The SQL Server password is read from an environment variable named `SQL_PASSWORD`. In local development, that can come from a `.env` file:

```
SQL_PASSWORD=Your_strong_password123
```

Do not commit real secrets. For teaching and local-only development, a `.env` file may be acceptable if it contains disposable values and the team agrees on the risk. Production secrets require a different mechanism.

Start the environment:

```
docker compose up --build
```

Run in the background:

```
docker compose up --build -d
```

Inspect services:

```
docker compose ps
```

View logs:

```
docker compose logs api
docker compose logs sql
```

Stop containers:

```
docker compose down
```

Stop and remove volumes:

```
docker compose down -v
```

That last command deletes the named volume and therefore local SQL Server data. Use it deliberately.

Layer 4 — Developer Workflow

A practical Compose workflow begins with deciding which dependencies belong locally.

For an ASP.NET Core API, a good first Compose setup might include:

```
api
sql
redis later, if caching is being developed
```

Create the API:

```
dotnet new webapi -n Chapter11.Api
```

Add or reuse the Dockerfile from Chapter 10.

Create `compose.yaml` at the repository root.

Create a local `.env` file if needed:

```
SQL_PASSWORD=Your_strong_password123
```

Start the stack:

```
docker compose up --build
```

Call the API:

```
curl http://localhost:8080/weatherforecast
```

Connect to SQL Server from the host using port `14333` if you have a local SQL tool:

```
Server: localhost,14333
User: sa
Password: value from SQL_PASSWORD
Trust server certificate: true for local development
```

When you change C# code, rebuild the image unless your setup uses bind mounts or a development container workflow:

```
docker compose up --build
```

When the database state gets messy during development:

```
docker compose down -v
docker compose up --build
```

That reset should feel a little like recreating a local development database, not like touching production.

Layer 5 — Production Usage

Compose is often used in local development, demos, integration tests, and small single-host deployments. For this book, treat it primarily as a development tool.

Configuration should be explicit and environment-specific. Local Compose values should not be copied into production blindly. Development passwords, trusted certificates, broad ports, and debug logging are convenient locally and dangerous in production.

Secrets should not live in source-controlled Compose files. Compose supports environment variables and secret-related features, but production secret handling should be designed with the target platform: cloud secret stores, managed identities, Kubernetes secrets with proper controls, or CI/CD secret injection.

Security requires careful port exposure. Services that only the API needs do not always need host ports. SQL Server can be reachable inside the Compose network without being exposed to the developer machine unless a local tool needs it.

Reliability in Compose is limited. `depends_on` controls startup ordering and, with health checks, can wait for a healthy dependency, but it is not a complete resilience strategy. The application should still handle dependency failures.

Deployment to production should use the concepts Compose teaches: explicit services, environment configuration, networks, volumes, and health. The tool used in production may be different.

Observability should follow container habits. Logs should go to standard output. Compose can show logs locally, while production platforms collect logs centrally.

Scaling with Compose is limited to a single Docker environment. It can run multiple replicas in some scenarios, but it does not provide the full multi-node scheduling and service management expected from Kubernetes or managed container platforms. Persistence through volumes is useful locally, but production persistence needs backup, restore, encryption, monitoring, and capacity planning. A named Docker volume on a developer laptop is not a production data strategy.

Cost is mostly local machine resource cost: CPU, memory, disk, battery, and developer time. SQL Server and other containers can be heavy. Keep the local stack useful without making every developer run the entire company on a laptop.

Layer 6 — Tradeoffs and Alternatives

Use Docker Compose when a developer needs several containers to work together: an API and database, API and cache, web frontend and backend, or a small service set for integration tests.

Do not use Compose when a single process is enough, when a managed development environment already supplies dependencies, or when the team needs production orchestration features.

Alternatives include direct local installation, shared development servers, Dev Containers, GitHub Codespaces, Microsoft Dev Box, Testcontainers for integration tests, Kubernetes-based development environments, and managed cloud resources. Direct local installation can be simpler for one dependency but becomes harder to standardize across a team.

Shared development servers reduce laptop load but create coupling between developers and often accumulate state that is hard to reset.

Dev Containers and Codespaces can use Compose under the hood while also standardizing editor and tool setup.

Testcontainers is useful when tests should start disposable dependencies programmatically rather than relying on a manually started Compose stack.

Kubernetes is more advanced and appears later. It is a production-grade orchestration model, not the natural first step for local .NET development.

Common overengineering mistakes:

- putting every company service into one local Compose file;
- exposing every container port to the host;
- treating `.env` files as safe secret storage;
- forgetting that named volumes preserve old database state;
- using `depends_on` as a substitute for retry and health-aware application logic;
- making local startup so heavy that developers avoid it;
- assuming Compose is the same as production.

The state-of-the-art direction is focused local environments: enough dependencies to develop and test the feature, disposable where possible, configured through environment variables, and close enough to production to catch meaningful integration problems.

Layer 7 — Interview Perspective

Interviewers use Compose questions to test whether you understand multi-container development without confusing it with production orchestration.

Concepts commonly tested:

- service definitions;
- default networks and service-name DNS;
- host port versus container port;
- environment variables;
- named volumes;
- health checks;

- `depends_on` ;
- local SQL Server containers;
- Compose versus Docker;
- Compose versus Kubernetes.

Representative questions:

- “What problem does Docker Compose solve?”
- “How does one service reach another service in Compose?”
- “What is the difference between `8080:8080` and an internal container port?”
- “Why use a named volume for SQL Server?”
- “What happens when you run `docker compose down -v`?”
- “Why is `depends_on` not a complete readiness strategy?”
- “Would you use Compose for production?”

A strong answer distinguishes local composition from production operations:

“Compose is great for defining a local multi-container environment. The API can reach SQL Server by the service name `sql`, and SQL Server can persist local data in a named volume. But Compose does not replace production orchestration, secret management, backup, monitoring, or resilience design.”

Common misconceptions:

- “Compose is just a longer Dockerfile.”
- “Compose networking uses localhost between containers.”
- “A container is ready when it starts.”
- “A named volume is the same as a production database backup.”
- “If Compose works locally, production will work the same way.”
- “Kubernetes is required for local multi-container development.”

Small design scenario:

Your ASP.NET Core order API needs SQL Server and Redis for local development. New developers spend half a day installing dependencies and often point at the wrong shared database.

A good Compose setup would define `api`, `sql`, and `redis` services, provide development-only configuration through environment variables, attach services to the default network, store SQL Server data in a named volume, expose only the ports developers need, and document reset commands.

The strong answer improves developer repeatability without pretending local containers are production.

Hands-On Lab

Objective:

Run an ASP.NET Core API and SQL Server together with Docker Compose.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Docker Desktop or another Compose-compatible Docker environment.
- Basic Docker knowledge from Chapter 10.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter11.Api
```

2. Add a Dockerfile to `chapter11.Api` using the pattern from Chapter 10.
3. Create `compose.yaml` next to the `chapter11.Api` folder.
4. Add an `api` service that builds `chapter11.Api`.
5. Add a `sql` service using `mcr.microsoft.com/mssql/server:2022-latest`.
6. Add a named volume for SQL Server data.
7. Add environment variables:

```
ConnectionStrings__Orders: "Server=sql,1433;Database=Orders;User
Id=sa;Password=${SQL_PASSWORD};TrustServerCertificate=True"
```

8. Create a local `.env` file with a strong development password:

```
SQL_PASSWORD=Your_strong_password123
```

9. Start the stack:

```
docker compose up --build
```

10. Call the API:

```
curl http://localhost:8080/weatherforecast
```

11. Inspect services and logs:

```
docker compose ps
docker compose logs sql
```

12. Stop the stack:

```
docker compose down
```

Expected results:

- Compose starts the API and SQL Server containers.
- The API is reachable from the host.
- The API can be configured with a SQL Server connection string using the Compose service name `sql`.
- SQL Server data is stored in a named volume.

Validation commands:

```
docker compose config
docker compose up --build
docker compose ps
docker compose logs api
docker compose logs sql
docker compose down
```

Troubleshooting notes:

- If YAML parsing fails, check indentation.
- If SQL Server exits, check password complexity and license acceptance.
- If the API cannot connect to SQL Server, confirm it uses `sql`, not `localhost`, inside the Compose network.
- If the host port is already in use, change the left side of the port mapping.
- If old database state causes confusion, run `docker compose down -v` to remove volumes, understanding that this deletes local container data.

Knowledge Check

1. What problem does Docker Compose solve that `docker run` alone does not?
2. What is a Compose service?
3. How does the API container find the SQL Server container?
4. Why does `localhost` mean different things inside and outside a container?
5. What is the difference between a host port and a container port?
6. Why use a named volume for SQL Server in local development?
7. What does `docker compose down -v remove`?
8. Why are local `.env` files not a production secret-management strategy?
9. What does a health check add to a Compose setup?
10. Why should Compose not be confused with Kubernetes?

Summary

Docker Compose lets a team describe and run a local multi-container environment. Instead of every developer hand-installing SQL Server, Redis, and other dependencies, the repository can define services, networks, volumes, ports, and environment variables.

For modern .NET development, Compose is especially useful because ASP.NET Core configuration maps cleanly to environment variables, containers provide repeatable runtime boundaries, and dependencies such as SQL Server can run locally without becoming permanent machine state.

Compose is a development and local integration tool first. It teaches the service, network, volume, and configuration concepts that production platforms also use, but it does not replace production orchestration, secret management, backup, monitoring, or resilience.

The next chapter moves from local containers to deploying ASP.NET Core, including publishing, reverse proxies, hosting choices, configuration, and production deployment concerns.

Sources

- Docker Compose overview
- Docker Compose file reference
- Docker Compose networks
- Docker Compose volumes
- Docker Compose environment variables
- Run SQL Server Linux container images with Docker
- Configure ASP.NET Core

Further Reading

- Docker Compose quickstart
- Docker Compose command-line reference
- Docker healthcheck reference
- Microsoft SQL Server container images
- Use Docker Compose with Visual Studio

12

Deploying ASP.NET Core

Chapter Purpose

The previous chapters taught how to build, secure, test, and containerize an ASP.NET Core application. This chapter asks the next question: how does that application leave the developer machine and run somewhere users can reach it?

Deployment exists because a running local application is not the same thing as a production system. A deployed application needs a supported runtime, a host, configuration, secrets, network exposure, TLS, process management, logging, health, rollback strategy, and a clear way to know which version is running.

For a developer with an IIS background, deployment may bring back familiar ideas: publish output, application pools, site bindings, reverse proxies, configuration files, Windows services, and load balancers. ASP.NET Core keeps many of those concerns, but the hosting choices are broader. The same application can run behind IIS, behind Nginx on Linux, in a container, in a managed app platform, or later under Kubernetes.

This chapter introduces publishing, reverse proxies, hosting options, configuration, and production deployment concerns. It does not teach every cloud provider or full CI/CD release automation. Chapter 13 covers cloud platforms, and Chapter 16 covers deployment pipelines.

Where This Fits

Deployment sits between validated code and production operation.

```
Source code
  |
  v
Build and test
  |
  v
Publish output or container image
  |
  v
Hosting environment
  |
  +-- process manager
  +-- reverse proxy or load balancer
  +-- configuration and secrets
  +-- TLS and networking
  +-- logs and health checks
  |
  v
Users and dependent systems
```

Chapter 12 is the bridge from local container development to cloud-native deployment. It explains the deployment vocabulary before Chapter 13 introduces Azure, AWS, Google Cloud, managed databases, storage, networking, and identity. The central distinction is:

Publishing prepares files or images.
Deployment places a specific artifact into a runtime environment.
Operations keeps that environment healthy.

Confusing those three steps is a common source of fragile releases.

Connection to the Reader's Existing Model

Traditional IIS deployment gives a useful starting point.

You may have published a web application, copied files to a server, configured an IIS site, set an application pool, updated `web.config`, checked bindings, installed certificates, and restarted the site or app pool. You knew that deployment was more than compilation.

ASP.NET Core preserves the idea of a published application, but it changes who owns the process.

In classic ASP.NET, IIS was deeply tied to the request pipeline. In ASP.NET Core, the application runs as a .NET process using Kestrel. IIS can still host ASP.NET Core through the ASP.NET Core Module, but it usually acts as a front end that forwards requests to the app process.

The same pattern appears on Linux. Nginx or Apache commonly receives public traffic and forwards requests to Kestrel. In cloud platforms, a managed load balancer, app gateway, ingress, or platform proxy may play that role.

An IIS application pool maps conceptually to process ownership and restart behavior. On Linux, that might be `systemd`. In containers, it might be the container runtime or orchestrator. In a managed platform, it might be hidden behind platform settings.

`web.config` maps partly to ASP.NET Core configuration and partly to hosting configuration. ASP.NET Core can read JSON files, environment variables, command-line values, user secrets, and external providers. Production should not depend on hand-editing files on the server.

The analogy breaks down when deployment is treated as server mutation. Modern deployment prefers identifiable artifacts: a publish folder, package, or container image built from a known commit and moved through environments.

Layer 1 — Conceptual Model

Deploying ASP.NET Core means preparing an application artifact and running it under a host that can serve traffic reliably and securely.

It solves these problems:

- creating deployable output;
- selecting a hosting model;
- starting and restarting the app process;
- exposing HTTP traffic safely;
- supplying environment-specific configuration;
- protecting secrets;
- enabling logs, health, and diagnostics;
- making releases repeatable and traceable.

It does not solve these problems by itself:

- it does not make bad code reliable;
- it does not replace tests or CI;
- it does not choose the right cloud service;

- it does not automatically secure secrets;
- it does not make database migrations safe;
- it does not remove the need for monitoring and rollback.

ASP.NET Core deployment has several common forms:

Framework-dependent publish

App files depend on a compatible .NET runtime installed on the host.

Self-contained publish

App carries the .NET runtime for a specific OS and architecture.

Container image

App and runtime environment are packaged as an OCI image.

Managed platform deployment

Platform accepts code, package, or image and manages more hosting details.

The right model depends on who owns the host, how repeatable the environment must be, how patching is handled, and what operational skills the team has.

Layer 2 — System Relationships

The build system creates the artifact. For folder-based deployment, it runs `dotnet publish`. For container deployment, it builds or publishes a container image. The output should be tied to a source commit and build record.

The artifact contains application files. A `dotnet publish` output commonly includes assemblies, dependencies, a `.deps.json` file, a `.runtimeconfig.json` file, configuration files, and content files needed by the app.

The runtime host executes the artifact. It might be Windows Server with IIS, Linux with Nginx and `systemd`, a container host, Azure App Service, AWS Elastic Beanstalk, Google Cloud Run, or another platform.

Kestrel serves the ASP.NET Core application. In production, Kestrel commonly runs behind a reverse proxy or load balancer that handles public edge concerns.

The reverse proxy receives client traffic and forwards it to the app. It may terminate TLS, enforce request size limits, serve static files, add forwarded headers, and provide buffering or routing.

The configuration system supplies environment-specific settings: connection strings, feature flags, logging levels, allowed origins, external URLs, and identity-provider settings.

The secret system supplies sensitive settings: passwords, certificates, API keys, client secrets, signing keys, and connection strings.

The operations layer observes and controls the deployment: logs, metrics, health checks, restarts, alerts, backup coordination, rollback, and incident response.

Failure boundaries include missing runtimes, wrong runtime identifier, bad publish output, file permission errors, port conflicts, TLS misconfiguration, forwarded-header mistakes, missing environment variables, unavailable databases, failed migrations, unhealthy instances, and unclear rollback.

Layer 3 — Core Mechanics

The primary publish command is:

```
dotnet publish -c Release
```

Microsoft describes `dotnet publish` as the command that compiles the application, reads project dependencies, and publishes files to a directory for deployment to a hosting system.

A framework-dependent publish relies on a compatible .NET runtime on the host:

```
dotnet publish -c Release --self-contained false
```

A self-contained publish includes the runtime for a target platform:

```
dotnet publish -c Release -r linux-x64 --self-contained true
```

A container publish can create an image through the .NET SDK:

```
dotnet publish /t:PublishContainer
```

Or the team can use a Dockerfile, as shown in Chapter 10:

```
docker build -t orders-api:2026.07.22 .
```

Kestrel listens on configured URLs. In containers, a common pattern is:

```
ASPNETCORE_URLS=http://+:8080
```

When a reverse proxy terminates HTTPS, the request reaching Kestrel may appear as HTTP. ASP.NET Core needs forwarded headers so features such as redirects, link generation, authentication callbacks, and client IP handling can use the original request information.

In non-IIS reverse proxy scenarios, configure forwarded headers deliberately:

```
using Microsoft.AspNetCore.HttpOverrides;

builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders =
        ForwardedHeaders.XForwardedFor |
        ForwardedHeaders.XForwardedProto;
});

var app = builder.Build();

app.UseForwardedHeaders();
```

Only trusted proxies should be allowed to forward headers. Otherwise, a client could spoof values such as original IP address or scheme.

A Linux service might be managed by `systemd`:

```
[Unit]
Description=Orders API

[Service]
WorkingDirectory=/opt/orders-api
ExecStart=/usr/bin/dotnet /opt/orders-api/Orders.Api.dll
Restart=always
RestartSec=10
User=orders
Environment=ASPNETCORE_URLS=http://localhost:5000

[Install]
WantedBy=multi-user.target
```

That file is not the only deployment model. It simply shows the production concepts: working directory, startup command, restart policy, user, and environment.

Layer 4 — Developer Workflow

A basic deployment workflow begins locally:

```
dotnet restore
dotnet build -c Release
dotnet test -c Release
dotnet publish -c Release -o ./artifacts/orders-api
```

Inspect the output:

```
ls ./artifacts/orders-api
```

Run the published app locally:

```
dotnet ./artifacts/orders-api/Orders.Api.dll
```

For a Linux target from a Windows or macOS machine, publish with a runtime identifier when needed:

```
dotnet publish -c Release -r linux-x64 --self-contained false
```

For a container target:

```
docker build -t orders-api:dev .
docker run --rm -p 8080:8080 \
  -e ASPNETCORE_URLS=http://+:8080 \
  orders-api:dev
```

Before deploying anywhere shared, answer these questions:

```
Which commit produced this artifact?
Which .NET version does it target?
Which host will run it?
How is configuration supplied?
Where are secrets stored?
Which port does Kestrel listen on?
Which proxy or load balancer fronts it?
How are logs collected?
How is health checked?
How do we roll back?
```

This workflow still can be manual for learning. In a team, Chapter 16's deployment pipelines should automate the repeatable parts.

Layer 5 — Production Usage

Production deployment begins with supported platform choices.

Use a supported .NET version. Use a supported operating system or container base image. Know whether the runtime is installed on the host, included in the artifact, or supplied by a container image.

Configuration should be external to the artifact. The same artifact should move through environments with different settings supplied by the platform. Avoid editing published files to create staging or production variants.

Secrets should come from managed secret systems, secure platform settings, or short-lived identity mechanisms. Do not deploy secrets in source, Docker image layers, zip packages, or shell scripts.

Security includes TLS, forwarded headers, trusted proxies, least-privilege process identities, patched runtimes, restricted filesystem permissions, secure cookies, CORS configuration, authentication settings, and dependency updates.

Reliability requires process management. The app should start automatically, restart after failure when appropriate, stop gracefully, report health, and fail clearly when configuration or dependencies are missing.

Deployment should be repeatable. Prefer immutable artifacts and identifiable versions. Avoid manual server changes that cannot be traced back to source control or deployment records.

Observability should be present before the first serious release. Logs should reach a place the team can search. Health endpoints should support platform checks. Basic metrics and diagnostics arrive in Chapter 14, but deployment should already prepare for them.

Scaling affects deployment shape. A single VM can be enough for a small internal app. Multiple instances require stateless application design, shared configuration, centralized logs, load balancing, and careful database capacity planning. Persistence must not depend on local app instance storage unless that is an explicit design. Uploaded files, logs, and user data need durable storage, databases, or mounted volumes with backup.

Cost depends on hosting choice, runtime size, instance count, logging volume, network traffic, managed services, and operational labor. A simple managed platform can be cheaper than a custom VM setup once maintenance is counted.

Local development optimizes for fast feedback. Production deployment optimizes for repeatability, security, supportability, and recovery.

Layer 6 — Tradeoffs and Alternatives

Use IIS hosting when the organization has strong Windows Server operations, existing IIS infrastructure, Windows-specific integration, or a gradual modernization path from older ASP.NET systems.

Use Linux with Nginx or Apache when the team wants direct VM control and is comfortable owning Linux operations, patching, service management, TLS, and reverse proxy configuration.

Use containers when repeatable runtime packaging matters and the deployment target accepts container images.

Use managed app platforms when the team wants to reduce server management. Azure App Service, Azure Container Apps, AWS Elastic Beanstalk, AWS App Runner, Google App Engine, and Google Cloud Run are examples. Chapter 13 compares cloud platform concepts.

Use Kubernetes only when orchestration complexity is justified by scale, platform standardization, or operational requirements. It is not required just to deploy an ASP.NET Core API.

Framework-dependent deployment is smaller and lets the host patch shared runtimes centrally. Self-contained deployment is more isolated and does not require a preinstalled runtime, but it is larger and shifts runtime patching responsibility toward the application artifact.

Common overengineering mistakes:

- building a Kubernetes platform for one small app;
- copying files manually and calling it modern deployment;
- storing production configuration in the artifact;
- ignoring reverse proxy forwarded headers;
- using `latest` container tags in production;
- deploying database schema changes with no compatibility plan;
- creating deployment automation before the manual deployment steps are understood;
- treating health checks as optional.

The state-of-the-art direction is not one universal host. It is artifact-based deployment, platform-supplied configuration and secrets, trusted proxy setup, observable releases, and simple hosting choices that match the application's risk and scale.

Layer 7 — Interview Perspective

Interviewers use deployment questions to test whether you understand the path from code to production.

Concepts commonly tested:

- `dotnet publish`;
- framework-dependent versus self-contained deployment;
- Kestrel;
- IIS, Nginx, and reverse proxies;
- forwarded headers;
- environment-specific configuration;

- secret handling;
- process management;
- container deployment;
- rollback and health checks.

Representative questions:

- “What does `dotnet publish` produce?”
- “How is deploying ASP.NET Core different from classic ASP.NET?”
- “Why is Kestrel often behind a reverse proxy?”
- “What are forwarded headers and why do they matter?”
- “When would you choose self-contained deployment?”
- “How should production configuration be supplied?”
- “What should you know before deploying a database-backed API?”
- “How would you roll back a bad deployment?”

A strong answer separates publish, deploy, and operate:

“`dotnet publish` prepares files. Deployment moves a known artifact to a host and supplies configuration and secrets. Operations keeps it running, observes health, and supports rollback. For ASP.NET Core, I also need to understand how Kestrel is exposed, usually through IIS, Nginx, a load balancer, or a managed platform.”

Common misconceptions:

- “Publishing is the same as deployment.”
- “Kestrel means no reverse proxy is ever needed.”
- “Self-contained deployment is always better.”
- “Environment variables make secrets automatically safe.”
- “If the app runs locally in Docker, deployment is solved.”
- “Forwarded headers are only a networking detail.”
- “A restart policy replaces health checks and observability.”

Small design scenario:

You have an ASP.NET Core order API with SQL Server. The team wants the first production deployment but does not yet have a full platform team.

A good answer might choose a managed app platform or a simple IIS/Linux host, publish a Release artifact or container image, externalize configuration, store secrets in the platform, configure TLS and forwarded headers, add a basic health endpoint, run database migrations through a reviewed process, and define rollback before the first release.

The strong answer keeps the first deployment boring and traceable.

Hands-On Lab

Objective:

Publish an ASP.NET Core API and run the published output locally as if it were a deployment artifact.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Basic ASP.NET Core knowledge from Chapter 5.
- Optional Docker knowledge from Chapter 10.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter12.Api
cd Chapter12.Api
```

2. Build and test:

```
dotnet build -c Release
dotnet test -c Release
```

If the project has no tests, note that as a validation gap.

3. Publish to an artifacts folder:

```
dotnet publish -c Release -o ../artifacts/chapter12-api
```

4. Inspect the publish output:

```
ls ../artifacts/chapter12-api
```

On Windows PowerShell:

```
Get-ChildItem ../artifacts\chapter12-api
```

5. Run the published application:

```
dotnet ../artifacts/chapter12-api/Chapter12.Api.dll
```

6. In another terminal, call the API using the URL printed by the app:

```
curl http://localhost:5000/weatherforecast
```

7. Publish for Linux x64 as a framework-dependent artifact:

```
dotnet publish -c Release -r linux-x64 --self-contained false -o ../artifacts/chapter12-api-linux
```

8. Optional: build and run a container image using the Dockerfile pattern from Chapter 10.

Expected results:

- The API builds in Release configuration.
- A publish folder is created.
- The published app runs outside the project source folder.
- You can explain what still must be supplied by a real production host.

Validation commands:

```
dotnet build -c Release
dotnet publish -c Release
dotnet ../artifacts/chapter12-api/Chapter12.Api.dll
```

Troubleshooting notes:

- If `dotnet test` fails because no test project exists, continue and record the gap.
- If the published app cannot find configuration, check copied files and environment variables.
- If the app listens on an unexpected port, inspect launch settings versus runtime environment variables.
- If publishing to a folder inside the project creates nested publish output, publish outside the project directory.

Knowledge Check

1. Why is publishing not the same as deployment?
2. What does `dotnet publish` prepare?

3. What is the difference between framework-dependent and self-contained deployment?
4. Why is Kestrel commonly placed behind a reverse proxy?
5. What information do forwarded headers preserve?
6. Why should production configuration live outside the artifact?
7. What makes a deployment artifact traceable?
8. Why does database migration strategy matter during deployment?
9. What should a health check prove?
10. When is a managed hosting platform simpler than a VM?

Summary

Deploying ASP.NET Core means preparing a known artifact and running it in a hosted environment with configuration, secrets, networking, process management, logs, health, and rollback.

`dotnet publish` creates deployable output. Framework-dependent deployment relies on a shared runtime. Self-contained deployment carries the runtime. Container deployment packages the application and runtime environment as an image. Hosting may involve IIS, Nginx, Apache, containers, managed platforms, or eventually Kubernetes.

The most important deployment habit is to keep artifacts and environments separate. Build the artifact once. Supply environment-specific configuration and secrets through the host. Know which version is running. Make the release observable and reversible.

The next chapter expands the hosting discussion into cloud platforms: Azure, AWS, Google Cloud, managed databases, storage, networking, and identity.

Sources

- `dotnet publish` command
- ASP.NET Core host and deploy overview
- Configure ASP.NET Core to work with proxy servers and load balancers
- Host ASP.NET Core on Linux with Nginx
- Run an ASP.NET Core app in Docker containers
- What is ASP.NET Core?

Further Reading

- .NET application publishing overview
- Runtime configuration options for .NET
- Host ASP.NET Core on Windows with IIS
- ASP.NET Core environments
- ASP.NET Core configuration

13

Cloud Platforms

Chapter Purpose

Chapter 12 explained how an ASP.NET Core application becomes a deployable artifact and runs under a host. Chapter 13 expands the hosting question into the cloud.

Cloud platforms exist because organizations wanted infrastructure and application capabilities without buying, installing, and operating every piece of hardware and platform software themselves. Instead of procuring servers, storage arrays, load balancers, firewall appliances, backup systems, and data center space months in advance, teams can provision compute, databases, storage, networking, identity, observability, and AI services on demand.

For a Windows, IIS, and SQL Server developer, the cloud can feel like a data center with an API. That analogy is useful. Azure, AWS, and Google Cloud all provide virtual machines, networks, disks, databases, firewalls, identity systems, logs, and management consoles. But the analogy breaks down because the cloud is not just rented servers. It is a catalog of managed services with different responsibility boundaries, pricing models, regional behavior, identity systems, and operational tradeoffs.

This chapter introduces Azure, AWS, Google Cloud, managed databases, storage, networking, and identity. It explains how the major cloud providers compare without pretending that a single chapter can make you a cloud architect.

Where This Fits

Cloud platforms sit underneath and around deployed applications.

```
ASP.NET Core artifact or container image
  |
  v
Cloud compute service
  |
  +-- managed database
  +-- object storage
  +-- virtual network
  +-- identity and secrets
  +-- load balancing
  +-- logs and metrics
  +-- deployment integration
  |
  v
Users, partners, internal systems, and operations teams
```

Chapter 13 follows deployment because cloud services are easier to understand once publishing, hosting, reverse proxies, configuration, and production deployment are clear. It prepares for Chapter 14 on observability, Chapter 16 on deployment pipelines, Chapter 17 on infrastructure as code, and Chapter 18 on Kubernetes.

The purpose here is not to memorize product names. Product names change, services evolve, and teams standardize differently. The purpose is to understand the major cloud categories and the responsibility tradeoffs.

Connection to the Reader's Existing Model

A traditional enterprise environment might have had:

- Windows Server VMs;
- IIS;
- SQL Server;
- file shares;
- Active Directory;
- load balancers;
- firewall rules;
- backup jobs;
- monitoring tools;
- deployment scripts;
- server administrators and DBAs.

Cloud platforms provide equivalents, but they are exposed as services.

A virtual machine maps directly to a server VM. You still patch, configure, secure, monitor, and deploy to it. It is the most familiar cloud starting point, but also the least transformative.

A managed app platform maps roughly to “I want to run my application without owning the server.” Azure App Service, Azure Container Apps, AWS App Runner, AWS Elastic Beanstalk, Google App Engine, and Google Cloud Run are examples in this family.

A managed database maps to SQL Server operated as a service. Azure SQL Database, Amazon RDS for SQL Server, and Google Cloud SQL for SQL Server all reduce some database server administration, though schema design, indexing, query performance, security, and cost remain application responsibilities.

Object storage maps loosely to file shares, but it is not a normal filesystem. Azure Blob Storage, Amazon S3, and Google Cloud Storage store objects in buckets or containers and expose APIs for durable storage.

Cloud identity maps partly to Active Directory and service accounts. Microsoft Entra ID, AWS IAM, and Google Cloud IAM all answer questions about who or what can access a resource.

Cloud networking maps to VLANs, subnets, firewalls, DNS, public IPs, and load balancers, but the implementation is virtual and API-driven.

The old model asked, “Which server owns this?” The cloud model asks, “Which service owns this responsibility, in which region, under which identity, with which cost and failure boundary?”

Layer 1 — Conceptual Model

A cloud platform is a provider-operated set of infrastructure and application services that customers provision and configure on demand.

It solves these problems:

- provisioning compute without buying hardware;
- using managed databases and storage;
- scaling resources up and down;
- deploying across regions and zones;
- integrating identity and access control;
- automating infrastructure through APIs;

- paying for measured usage instead of only fixed capital expense;
- accessing specialized services such as messaging, analytics, and AI.

It does not solve these problems automatically:

- it does not choose the right architecture;
- it does not make applications secure by default;
- it does not eliminate operations work;
- it does not make databases cheap or fast automatically;
- it does not prevent outages;
- it does not remove the need to understand networking, identity, and cost.

The basic cloud model is:

```
Compute runs code.  
Databases store structured durable data.  
Object storage stores files and blobs.  
Networking connects and isolates resources.  
Identity controls access.  
Observability reports behavior.  
Billing measures usage.
```

The state-of-the-art direction in 2026 is managed platforms, container services, serverless functions, managed databases, integrated identity, platform engineering, AI services, and infrastructure as code. The trend is not “never use VMs.” The trend is to choose the highest useful level of management without losing required control.

Layer 2 — System Relationships

The cloud account or tenant is the top-level administrative boundary. Azure uses tenants, subscriptions, management groups, and resource groups. AWS uses accounts, organizations, regions, and resources. Google Cloud uses organizations, folders, projects, regions, zones, and resources.

The region is the geographic placement boundary. It affects latency, data residency, service availability, disaster recovery, and cost.

The availability zone is a failure-isolation boundary inside many regions. Not every service uses zones the same way. Some resources are global, some are regional, and some are zonal.

The compute service runs application code. Options include VMs, app platforms, container services, serverless functions, and Kubernetes.

The database service stores durable structured data. Managed relational databases reduce server management, but the team still owns schema, query behavior, access patterns, and migration safety.

The storage service stores objects, files, disks, backups, and artifacts. Object storage is commonly used for uploads, exports, static files, logs, backups, and data exchange.

The virtual network controls private communication, public exposure, subnets, routes, firewall rules, DNS, load balancers, private endpoints, and service connectivity.

The identity system controls access for users, applications, services, build pipelines, and administrators. Strong cloud design relies on least privilege and short-lived or managed credentials where possible.

Failure boundaries include region outages, identity misconfiguration, public network exposure, quota limits, wrong service tier, missing backups, overly broad permissions, secret leakage, service limits, unexpected cost, and provider-specific behavior hidden behind familiar names.

Layer 3 — Core Mechanics

The smallest useful cloud design for an ASP.NET Core API looks like this:

```

Public HTTPS endpoint
    |
    v
Managed app or container service
    |
    +-- managed SQL database
    +-- object storage for uploaded files
    +-- secret store for connection strings and keys
    +-- identity for service-to-service access
    +-- logs and metrics

```

The three major providers offer similar categories with different names and different strengths.

For compute:

```

Azure: Azure App Service, Azure Container Apps, Azure Functions, Azure VMs,
      Azure Kubernetes Service

AWS: Elastic Beanstalk, App Runner, Lambda, EC2, ECS, EKS

Google Cloud: App Engine, Cloud Run, Cloud Functions, Compute Engine,
             Google Kubernetes Engine

```

For relational databases:

```

Azure: Azure SQL Database, Azure SQL Managed Instance,
      Azure Database for PostgreSQL

AWS: Amazon RDS, Amazon Aurora

Google Cloud: Cloud SQL, AlloyDB for PostgreSQL

```

For object storage:

```

Azure: Azure Blob Storage
AWS: Amazon S3
Google Cloud: Cloud Storage

```

For identity and secrets:

```

Azure: Microsoft Entra ID, managed identities, Azure Key Vault
AWS: IAM, IAM roles, AWS Secrets Manager, AWS Systems Manager Parameter Store
Google Cloud: Cloud IAM, service accounts, Secret Manager

```

For a .NET team, the most important mechanics are not the names but the questions:

```

Where does the app run?
Where is data stored?
How does the app authenticate to dependencies?
Which network paths are public and private?
How is configuration supplied?
How are logs and metrics collected?
How is cost measured?

```

Cloud development uses provider CLIs as well as portals:

```

az version
aws --version
gcloud version

```

Those commands are not required for every developer, but teams that deploy to cloud platforms should understand that cloud resources are scriptable and eventually should be defined with repeatable automation.

Layer 4 — Developer Workflow

At the developer level, the cloud workflow should begin with a target environment and a responsibility map.

For an ASP.NET Core API:

```
Application artifact: publish folder or container image
Compute target: managed app service or container service
Database: managed SQL database
Secrets: provider secret store
Identity: app-managed identity or service role
Networking: HTTPS public endpoint, private database path if possible
Observability: provider logs and metrics
```

A simple early workflow:

```
Build and test locally
  |
  v
Publish or build container image
  |
  v
Provision cloud resources
  |
  v
Configure app settings and secrets
  |
  v
Deploy artifact
  |
  v
Verify health and logs
  |
  v
Record cost and cleanup plan
```

Do not start by clicking every interesting cloud service. Start with the minimum runtime path: app, database, configuration, identity, logs, and network exposure.

Provider-specific tools are useful:

```
az login
az group list
```

```
aws configure sso
aws sts get-caller-identity
```

```
gcloud auth login
gcloud config list
```

These commands answer the same question in different ecosystems: which cloud identity and context am I using?

For team work, avoid unmanaged portal-only resources. Manual exploration is fine for learning, but production resources should move toward infrastructure as code in Chapter 17.

Layer 5 — Production Usage

Production cloud usage starts with the shared responsibility model. The cloud provider owns the physical data centers and managed service internals. The customer still owns application code, configuration, identity assignments, data classification, access policy, network exposure, cost management, and many security decisions.

Configuration should be environment-specific and centrally controlled. Do not use the same database, storage account, secret, or identity across development, test, and production unless there is a deliberate and reviewed reason.

Secrets should live in managed secret stores or be replaced by managed identity patterns when possible. A cloud platform makes secret storage easier; it does not make hard-coded secrets safe.

Security includes IAM, network isolation, private endpoints, TLS, managed identity, key rotation, audit logs, policy enforcement, patching choices, and secure deployment credentials.

Reliability depends on regions, zones, backups, replication, health checks, autoscaling, service quotas, retry behavior, and disaster recovery plans. Managed services reduce some work but introduce provider-specific limits and failure modes.

Deployment should use traceable artifacts and controlled promotion. A cloud portal deployment can teach the platform, but production should eventually be automated and repeatable.

Observability is a first-class cloud concern. Each provider offers logging, metrics, tracing, alerting, and dashboards. Chapter 14 introduces the basic diagnostics needed for deployed applications.

Scaling should follow bottlenecks. Scaling app instances is easy on many cloud platforms. Scaling a database, cache, or downstream API may be harder and more expensive.

Persistence requires backup and retention decisions. Managed databases and object storage are durable services, but accidental deletion, bad migrations, and corrupt writes remain application and operations risks.

Cost must be designed, not discovered by surprise. Use budgets, tags or labels, alerts, right-sized service tiers, cleanup automation, and review of logs, storage, network transfer, and idle resources.

Local development optimizes for quick feedback. Cloud production optimizes for secure identity, durable services, controlled network exposure, observable operation, and accountable cost.

Layer 6 — Tradeoffs and Alternatives

Azure is often the most natural cloud for Microsoft-centered enterprises. It integrates deeply with .NET, Visual Studio, GitHub, Microsoft Entra ID, Windows Server, SQL Server, Azure SQL, and Microsoft security and governance tools. It is not limited to Microsoft technologies; Azure also supports Linux, containers, Kubernetes, Terraform, PostgreSQL, MySQL, Java, Python, JavaScript, Go, and more.

AWS has the longest history as a broad public cloud platform and a very large service catalog and community. It is a common choice for organizations with existing AWS investment, broad cloud skills, mature platform teams, or service needs that align with AWS strengths.

Google Cloud is strong in data, analytics, Kubernetes heritage, global networking, and AI-related services. It can be attractive when those strengths fit the organization or when teams already use Google Cloud foundations.

Gartner's 2025 Magic Quadrant for Strategic Cloud Platform Services named AWS, Microsoft, and Google as Leaders according to provider announcements and the published Gartner abstract. That does not make the choice automatic. It means all three are credible strategic providers.

Alternatives include on-premises infrastructure, private cloud, hybrid cloud, Oracle Cloud, IBM Cloud, sovereign clouds, managed hosting providers, and platform-as-a-service products outside the hyperscalers.

Common overengineering mistakes:

- choosing Kubernetes before a managed app service has been considered;
- exposing databases publicly for convenience;
- using one cloud account or subscription for every environment;
- granting administrator permissions to application identities;
- ignoring egress and logging costs;
- assuming managed means maintenance-free;
- using portal clicks as the only production documentation;
- copying provider reference architectures without understanding the workload.

The best provider is not always the provider with the most services. It is the provider your organization can operate securely, reliably, and economically for the system you are building.

Layer 7 — Interview Perspective

Interviewers use cloud questions to test whether you understand tradeoffs and responsibility boundaries.

Concepts commonly tested:

- cloud versus traditional hosting;
- IaaS, PaaS, containers, and serverless;
- regions and availability zones;
- managed databases;
- object storage;
- virtual networking;
- IAM and managed identity;
- secrets management;
- cost awareness;
- provider comparison.

Representative questions:

- “Why use a managed app platform instead of a VM?”
- “What is the difference between object storage and a file share?”
- “What does a managed database manage, and what does the team still own?”
- “Why do regions and zones matter?”
- “How should an app authenticate to cloud resources?”
- “When would Azure be a natural fit for a .NET team?”
- “Why might a team choose AWS or Google Cloud instead?”
- “What cloud costs surprise teams?”

A strong answer names ownership:

“A managed database removes much of the server administration, but the team still owns schema design, indexes, queries, migrations, permissions, backup retention choices, and cost. Managed means the boundary moved, not that the responsibility disappeared.”

Common misconceptions:

- “Cloud means no operations.”
- “Managed services are always cheaper.”
- “A VM in the cloud is automatically cloud-native.”
- “Object storage is just a normal disk.”
- “The provider secures everything.”
- “Any cloud provider choice is only a technical decision.”
- “The best cloud is the one with the longest feature list.”

Small design scenario:

You need to host an ASP.NET Core order API with SQL Server, file uploads, authentication, and basic monitoring. The company already uses Microsoft Entra ID and SQL Server.

A strong first design might use Azure App Service or Azure Container Apps, Azure SQL Database, Azure Blob Storage, managed identity, Azure Key Vault, and Azure Monitor. It would keep the design simple, use private connectivity where appropriate, avoid public database exposure, and introduce infrastructure as code once the resource model is understood.

If the company already had mature AWS or Google Cloud operations, the same architecture categories would map to that provider's services instead.

Hands-On Lab

Objective:

Create a provider-neutral cloud architecture map for an ASP.NET Core API.

Prerequisites:

- Chapters 1-12 completed.
- No cloud account is required.

Steps:

1. Draw these boxes:

```
ASP.NET Core API
Managed compute
Managed SQL database
Object storage
Secret store
Identity system
Virtual network
Logs and metrics
```

2. For Azure, map each box to a likely service.
3. For AWS, map each box to a likely service.
4. For Google Cloud, map each box to a likely service.
5. For each provider, answer:

```
Which service runs the API?
Which service stores relational data?
Which service stores uploaded files?
How does the app get secrets?
How does the app authenticate to dependencies?
What is public?
What should be private?
Where do logs and metrics go?
```

6. Choose one provider and write a one-paragraph justification based on team skills, existing systems, security, operations, and cost.

Expected results:

- You can map the same application architecture to Azure, AWS, and Google Cloud.
- You can explain why managed services change responsibility boundaries.
- You can identify provider-specific names without confusing them with the underlying concepts.

Validation commands:

```
az version
aws --version
gcloud version
```

These commands are optional and only validate whether provider CLIs are installed.

Troubleshooting notes:

- If a CLI is not installed, complete the lab conceptually.

- If service names differ by region or release, use current provider documentation.
- If two services seem to fit, compare operational responsibility, networking, scaling, and cost rather than choosing by name.

Knowledge Check

1. Why is the cloud more than rented virtual machines?
2. What does a managed app platform manage for an ASP.NET Core application?
3. What does a managed database not manage for the application team?
4. Why do regions and zones matter?
5. How is object storage different from a normal application folder?
6. Why is identity central to cloud architecture?
7. What should usually be private in a database-backed API architecture?
8. Why is cost a design concern in cloud systems?
9. When is Azure a natural fit for a .NET organization?
10. Why might AWS or Google Cloud still be the right choice for a .NET system?

Summary

Cloud platforms provide compute, databases, storage, networking, identity, observability, AI, and operations capabilities as services. They reduce the need to own physical infrastructure and some platform operations, but they do not remove engineering responsibility.

For modern .NET developers, Azure, AWS, and Google Cloud are best understood through categories first and product names second. An ASP.NET Core API needs a place to run, a database, storage, configuration, secrets, identity, network boundaries, logs, metrics, and cost controls. Each provider supplies those capabilities differently.

Azure is often natural for Microsoft-centered enterprises. AWS is broad, mature, and deeply established. Google Cloud is strong in data, Kubernetes, networking, and AI-oriented platform capabilities. The right choice depends on the organization's existing skills, contracts, compliance, architecture, and operational maturity.

The next chapter introduces observability basics so deployed applications can be understood after they leave the developer machine.

Sources

- [Why Azure?](#)
- [What is cloud computing? - AWS](#)
- [Overview of Amazon Web Services](#)
- [Google Cloud overview](#)
- [Gartner Magic Quadrant for Strategic Cloud Platform Services](#)
- [AWS named as a Leader in 2025 Gartner Magic Quadrant for Strategic Cloud Platform Services](#)
- [Google is a Leader in the Gartner Magic Quadrant for Strategic Cloud Platform Services](#)

Further Reading

- [Microsoft Cloud Adoption Framework for Azure](#)

- AWS Well-Architected Framework
- Google Cloud Architecture Framework
- Azure Well-Architected Framework
- Google Cloud products
- AWS products

14

Observability Basics and Production Diagnostics

Chapter Purpose

Chapter 13 showed that modern .NET applications often run on cloud platforms, managed hosts, containers, and services the team does not log into directly. That changes how production problems are understood.

In a traditional IIS environment, diagnostics often began with Remote Desktop, Event Viewer, IIS logs, Windows Performance Monitor, SQL Server Management Studio, and maybe a shared logging folder. Those tools were useful because the server was visible and relatively stable.

Modern production systems are less server-centered. An ASP.NET Core API may run in several instances, inside containers, behind a load balancer, connected to managed databases, using cloud identity, and writing logs to a centralized system. A failed request may not be reproducible on a single machine. The container that handled it may already be gone.

Observability exists so teams can understand a running system from the outside using signals the system emits: logs, metrics, health checks, traces, events, alerts, and diagnostics artifacts. This chapter introduces the basics: ASP.NET Core logging, health checks, structured logging, basic metrics, local diagnostics, and production monitoring concepts.

Advanced distributed tracing and multi-service monitoring come later in Chapter 19. This chapter gives you enough observability knowledge to understand a deployed application before the system becomes distributed.

Where This Fits

Observability sits beside the running application and feeds information back to the team.

```
Users and clients
|
v
ASP.NET Core application
|
+-- logs: what happened?
+-- health checks: can it serve traffic?
+-- metrics: how much, how fast, how many errors?
+-- diagnostics: what is happening inside the process?
|
v
Monitoring and alerting system
|
v
Developers and operators
```

Chapter 14 follows cloud platforms because deployed applications need visibility immediately. It prepares for Chapter 15 on distributed applications, where failures can involve caches, queues, and background services, and for Chapter 19 on advanced observability.

The goal is not to install every monitoring product. The goal is to understand which signals a modern .NET application should emit and how those signals help the team diagnose production behavior.

Connection to the Reader's Existing Model

You already know production diagnostics in older environments.

Windows Event Log recorded system and application events. IIS logs recorded requests. SQL Server had query plans, wait stats, and error logs. Performance Monitor exposed counters. A load balancer could show whether a node was healthy. Remote Desktop gave direct access to a server.

Modern observability keeps the same intent but changes the collection model.

ASP.NET Core logging maps to Event Log and application logs, but it is provider-based and often writes to console, cloud logging systems, or third-party sinks. In containers, console output is usually the right default because the platform collects it. Health checks map to load balancer probes and service-monitor checks. They answer whether an instance should receive traffic or whether a dependency is available.

Metrics map to performance counters, but they are usually collected by monitoring systems and visualized in dashboards. They answer rate, duration, count, and resource questions.

Structured logging improves on plain text logs by preserving named values such as `OrderId`, `UserId`, `StatusCode`, or `ElapsedMilliseconds`. That makes logs searchable and aggregatable.

Local diagnostics tools such as `dotnet-counters`, `dotnet-dump`, `dotnet-trace`, and `dotnet-monitor` map to the familiar idea of inspecting a process. The tools are different, but the question is the same: what is the process doing right now?

The analogy breaks down when you assume you can always inspect the server after the fact. In cloud and container systems, instances are replaceable and ephemeral. If the application did not emit useful signals while the problem happened, the evidence may be gone.

Layer 1 — Conceptual Model

Observability is the ability to understand a system's behavior from the signals it produces.

It solves these problems:

- discovering whether the application is healthy;
- diagnosing failures after deployment;
- detecting slow responses and error spikes;
- understanding dependency failures;
- alerting humans when action is needed;
- supporting rollback and incident response;
- creating feedback for future development.

It does not solve these problems by itself:

- it does not prevent bugs;
- it does not replace tests;
- it does not make logs useful if the app logs the wrong things;
- it does not fix performance problems automatically;
- it does not remove the need for security and privacy discipline;
- it does not make every alert actionable.

The basic observability model is:

```
Logs explain events.
Health checks report readiness or liveness.
Metrics quantify behavior over time.
Diagnostics tools inspect process internals.
Alerts turn important signals into action.
```

.NET documentation commonly describes logs, metrics, and distributed traces as the three pillars of observability. This chapter emphasizes logs, health, and metrics first. Distributed tracing becomes more important once requests cross multiple services.

Layer 2 — System Relationships

The application emits signals. ASP.NET Core produces framework logs, request metrics, and health check responses. Application code adds business-specific logs and measurements.

The logging provider decides where logs go. Built-in providers include console and debug output. Other providers can send logs to Azure, Application Insights, OpenTelemetry collectors, files, or third-party systems.

The health check endpoint responds to probes. Load balancers, container platforms, cloud services, uptime monitors, and deployment systems can call that endpoint.

The metrics pipeline collects numeric measurements. Metrics can come from ASP.NET Core, the .NET runtime, libraries, and application code. Monitoring systems store and graph those measurements.

The diagnostics tool connects to a running .NET process. Tools can collect counters, dumps, traces, stacks, and other artifacts. These are especially useful when logs and metrics show a problem but not the cause.

The alerting system evaluates signals against thresholds, anomaly rules, or service-level objectives. It notifies people or automation when the signal requires action.

The operations process owns response. A dashboard is not enough. Someone must know what the alert means, how to investigate it, and when to escalate.

Failure boundaries include missing logs, overbroad log levels, sensitive data in logs, health checks that always return healthy, metrics with high cardinality, alerts that are too noisy, dashboards no one uses, local-only diagnostics, and telemetry costs that grow without review.

Layer 3 — Core Mechanics

ASP.NET Core logging uses `ILogger<T>`:

```
public sealed class OrderService
{
    private readonly ILogger<OrderService> _logger;

    public OrderService(ILogger<OrderService> logger)
    {
        _logger = logger;
    }

    public void MarkShipped(int orderId)
    {
        _logger.LogInformation(
            "Order {OrderId} was marked shipped",
            orderId);
    }
}
```

`{orderId}` is a structured property. A logging backend can search or filter by `orderId` more reliably than it can parse arbitrary text.

Log levels communicate severity:

Trace	very detailed diagnostics
Debug	development and troubleshooting detail
Information	normal application events
Warning	unexpected but handled situations
Error	failures in an operation
Critical	severe failures requiring immediate attention

Health checks are registered and mapped:

```
builder.Services.AddHealthChecks();

var app = builder.Build();

app.MapHealthChecks("/health");
```

A basic health endpoint proves the app can respond. More advanced checks can probe dependencies such as SQL Server, queues, or external services. Do not make every health check depend on every downstream service unless that is really the desired traffic-routing behavior.

ASP.NET Core includes built-in metrics. Examples include request duration, request counts, and response status information. Metrics answer questions such as:

```
How many requests are arriving?
How long do requests take?
How many fail?
Which status codes are increasing?
Is memory rising?
Is the thread pool saturated?
```

Local diagnostics tools provide process-level insight:

```
dotnet-counters monitor --process-id <pid>
dotnet-dump collect --process-id <pid>
dotnet-trace collect --process-id <pid>
```

`dotnet-monitor` can collect diagnostic artifacts such as dumps, traces, logs, and metrics from .NET applications, including production environments when configured carefully.

Use these tools with care in production. Dumps and traces can contain sensitive data and can affect performance.

Layer 4 — Developer Workflow

Start with local visibility before production visibility.

Create an API:

```
dotnet new webapi -n Chapter14.Api
cd Chapter14.Api
```

Add a health check endpoint:

```
builder.Services.AddHealthChecks();

var app = builder.Build();

app.MapHealthChecks("/health");
```

Run the app:

```
dotnet run
```

Call the health endpoint:

```
curl http://localhost:5000/health
```

Add structured logging to an endpoint:

```
app.MapGet("/orders/{id:int}", (
    int id,
    ILogger<Program> logger) =>
{
    logger.LogInformation("Loading order {OrderId}", id);

    return Results.Ok(new { id });
});
```

Run and inspect console logs.

Find the process ID:

```
dotnet-trace ps
```

Monitor counters:

```
dotnet-counters monitor --process-id <pid>
```

For a deployed app, the workflow changes:

```
Check health endpoint
    |
    v
Check recent logs
    |
    v
Check request rate, duration, and error metrics
    |
    v
Check dependency status
    |
    v
Collect deeper diagnostics only if needed
```

This order matters. Do not collect a memory dump before checking whether the app is simply missing a connection string.

Layer 5 — Production Usage

Production observability starts before deployment.

Configuration should set appropriate log levels per environment. Development can use more verbose logs. Production should default to useful information, warnings, errors, and targeted debugging when needed.

Secrets must not appear in logs, metrics tags, health responses, dumps shared carelessly, or exception messages shown to clients. Treat telemetry as data that requires security review.

Security monitoring should include authentication failures, authorization failures, suspicious request patterns, administrative actions, and unexpected configuration changes. Avoid logging full tokens, passwords, or sensitive claims.

Reliability depends on health checks that mean something. A liveness check can answer “is the process alive?” A readiness check can answer “should this instance receive traffic?” Those are different questions.

Deployment needs observability. A release should be watched through health, error rate, request duration, and key business indicators. A rollback decision should be based on signals, not vibes.

Observability data should be centralized for multi-instance applications. If each instance keeps local logs only, diagnosis becomes slow and incomplete.

Scaling requires metrics. Autoscaling decisions often use CPU, memory, request rate, queue length, or custom metrics. Bad metrics can cause bad scaling behavior.

Persistence matters because logs and metrics need retention rules. Keeping everything forever is expensive. Keeping too little makes incidents impossible to investigate.

Cost must be managed. High-volume logs, high-cardinality metrics, long retention, and verbose traces can become expensive. Observability should be designed like any other production dependency.

Local diagnostics optimize for detail. Production monitoring optimizes for continuous, secure, actionable signals.

Layer 6 — Tradeoffs and Alternatives

Use built-in ASP.NET Core logging and health checks for nearly every production application. They are foundational.

Use provider-native monitoring when running on a cloud platform and the team benefits from integration: Azure Monitor and Application Insights, Amazon CloudWatch, or Google Cloud Observability.

Use open standards such as Open Telemetry when the team wants portability across backends or needs consistent instrumentation across languages and services.

Use third-party platforms such as Datadog, New Relic, Splunk, Elastic, Grafana, Honeycomb, or Sentry when their workflow, querying, alerting, error tracking, or organizational standard fits better.

Use local diagnostics tools when investigating process-level behavior: memory, CPU, thread pool, deadlocks, dumps, and traces.

Common overengineering mistakes:

- adding a monitoring platform before deciding what questions must be answered;
- logging every request body;
- using `Error` for ordinary business validation;
- creating health checks that always return healthy;
- creating health checks that fail whenever any optional dependency is slow;
- adding user IDs, email addresses, or tenant names as high-cardinality metric tags without understanding cost;
- alerting on everything until everyone ignores alerts;
- collecting dumps without protecting sensitive contents.

The state-of-the-art direction is practical observability: structured logs, meaningful health checks, built-in and custom metrics, secure telemetry, OpenTelemetry-compatible instrumentation, and alerts tied to user impact.

Layer 7 — Interview Perspective

Interviewers use observability questions to test whether you understand production responsibility.

Concepts commonly tested:

- logs versus metrics versus traces;
- structured logging;
- log levels;
- health checks;
- liveness versus readiness;
- basic ASP.NET Core metrics;
- local .NET diagnostics tools;
- alerting;
- sensitive data in telemetry;
- production troubleshooting sequence.

Representative questions:

- “What would you log for a failed API request?”
- “What is structured logging?”
- “How is a metric different from a log?”
- “What should a health check prove?”
- “Why are liveness and readiness different?”
- “How would you diagnose high CPU in a .NET app?”
- “What should trigger an alert?”
- “What telemetry should not contain?”

A strong answer connects signals to action:

“I would start with health, recent errors, request duration, request rate, and dependency failures. Logs explain specific events, metrics show trends, and diagnostics tools help inspect the process when the higher-level signals point to CPU, memory, or thread-pool issues.”

Common misconceptions:

- “Logging is observability.”
- “More logs are always better.”
- “A health check should test every dependency.”
- “Metrics are only for infrastructure teams.”
- “Alerts should fire for every exception.”
- “A dump is safe because it is only diagnostic data.”
- “If the cloud platform has monitoring, the app does not need instrumentation.”

Small design scenario:

You deploy an ASP.NET Core order API to a cloud app platform. Users report intermittent slow responses after a release.

A good investigation would check health status, request duration metrics, error rates, recent logs around the release, database dependency failures, connection timeouts, instance resource usage, and deployment version. If those signals suggest CPU, memory, or thread pool issues, collect deeper diagnostics.

The strong answer starts broad, uses signals, and avoids guessing.

Hands-On Lab

Objective:

Add basic observability to an ASP.NET Core API with structured logs, a health endpoint, and local diagnostics.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Basic ASP.NET Core knowledge from Chapter 5.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter14.Api  
cd Chapter14.Api
```

2. Add health checks in `Program.cs`:

```
builder.Services.AddHealthChecks();
```

3. Map the health endpoint:

```
app.MapHealthChecks("/health");
```

4. Add an endpoint that writes a structured log:

```
app.MapGet("/orders/{id:int}", (
    int id,
    ILogger<Program> logger) =>
{
    logger.LogInformation("Loading order {OrderId}", id);

    return Results.Ok(new { id });
});
```

5. Run the app:

```
dotnet run
```

6. Call the endpoints:

```
curl http://localhost:5000/health
curl http://localhost:5000/orders/42
```

7. Find the process:

```
dotnet-trace ps
```

8. Monitor counters:

```
dotnet-counters monitor --process-id <pid>
```

Expected results:

- The health endpoint returns a healthy response.
- The order endpoint writes a structured log containing `orderId`.
- The console shows ASP.NET Core request and application logs.
- `dotnet-counters` shows runtime counters for the running process.

Validation commands:

```
dotnet build
dotnet run
curl http://localhost:5000/health
curl http://localhost:5000/orders/42
dotnet-trace ps
dotnet-counters monitor --process-id <pid>
```

Troubleshooting notes:

- If the port differs, use the URL printed by `dotnet run`.
- If `dotnet-counters` is not installed, install it as a .NET tool or skip the local diagnostics step.
- If the health endpoint returns 404, confirm `MapHealthChecks` appears after `builder.Build()`.
- If logs do not show your message, confirm the endpoint is being called and the production log level has not filtered it out.

Knowledge Check

1. Why is observability more important in cloud and container environments?
2. What is the difference between a log and a metric?
3. Why is structured logging useful?
4. What should a basic health check prove?

5. How are liveness and readiness different?
6. Why can too much logging be harmful?
7. What kinds of data should never appear in telemetry?
8. How do metrics support scaling decisions?
9. When would you use `dotnet-counters`, `dotnet-dump`, or `dotnet-trace`?
10. Why should alerts be tied to action?

Summary

Observability is how a team understands a running system from the signals it emits. For ASP.NET Core applications, the first signals are logs, health checks, metrics, and local diagnostics.

Logs explain events. Structured logs preserve named values so production systems can search and group them. Health checks tell platforms whether an instance should receive traffic. Metrics show behavior over time: request rate, duration, errors, CPU, memory, and other measurements. Diagnostics tools inspect process internals when higher-level signals are not enough.

The goal is not to collect everything. The goal is to collect the signals that help the team detect, diagnose, and respond to real problems without leaking secrets or creating unmanageable noise.

The next chapter introduces distributed applications, where observability becomes even more important because requests begin crossing caches, queues, background services, and other system boundaries.

Sources

- Logging in .NET and ASP.NET Core
- Health checks in ASP.NET Core
- ASP.NET Core metrics
- Diagnostics in .NET
- Diagnostic monitoring and collection utility `dotnet-monitor`

Further Reading

- `dotnet-counters` diagnostic tool
- `dotnet-dump` diagnostic tool
- `dotnet-trace` diagnostic tool
- OpenTelemetry .NET
- Azure Monitor overview

15

Distributed Applications

Chapter Purpose

So far, the book has built toward a deployable ASP.NET Core application with data access, security, containers, cloud hosting, and basic observability. Chapter 15 introduces what happens when one application process is no longer the whole system. Distributed applications exist because some work does not fit cleanly inside a single request to a single process and a single database. A web request may need cached data. A long-running task may need to continue after the user gets a response. A spike in traffic may need buffering. A dependency may fail temporarily. A background service may need to process messages independently from the API.

If your existing model is a load-balanced IIS application with SQL Server, scheduled tasks, Windows services, and maybe MSMQ, the ideas are not entirely new. You already know that systems can have more than one moving part. Modern .NET uses different tools and cloud-native patterns, but the core questions remain: where does work happen, where does state live, what happens when a component fails, and how does the team know?

This chapter introduces caching, Redis, messaging, background services, queues, and resilience. It does not teach microservices architecture in full. That comes later. The goal here is to understand the first production boundaries that make a .NET system distributed.

Where This Fits

A distributed .NET application adds more runtime components around the API.

```
Client
|
v
ASP.NET Core API
|
+-- SQL Server for durable business data
|
+-- Redis for cache or fast shared state
|
+-- Queue for work that can happen later
|
+-- Background worker for queued processing
|
+-- External HTTP service
|
v
Logs, metrics, health checks, and alerts
```


Chapter 14 matters here because distributed systems are harder to understand without observability. Chapter 16 follows with deployment pipelines, because more moving parts require safer release practices. Chapter 19 later returns to advanced observability, including distributed tracing.

This chapter's theme is simple: every new component solves a problem and adds a failure mode.

Connection to the Reader's Existing Model

Several older Windows concepts translate well.

A Redis cache is conceptually similar to using ASP.NET cache, memory cache, or a shared lookup table to avoid repeated expensive work. The difference is that Redis runs as an external process and can be shared across application instances.

A background service maps to a Windows service or scheduled task. It runs outside the immediate request/response path and performs work over time. Modern .NET uses hosted services and worker services for this pattern.

A queue maps to MSMQ or a SQL-backed work table. It stores work until a consumer is ready. The sender and receiver do not need to run at the same moment.

Resilience patterns map to the judgment you already used when a database, file share, or downstream service was unavailable. The difference is that modern .NET provides explicit libraries for timeouts, retries, circuit breakers, rate limiting, fallback, and related strategies.

The analogy breaks down when distributed components are treated as invisible plumbing. A cache has memory limits and eviction behavior. A queue can build a backlog. A background worker can fail. Retries can duplicate work. A message can be processed twice. A downstream dependency can throttle you.

Distributed does not mean safer by default. It means the failure boundaries are more explicit.

Layer 1 — Conceptual Model

A distributed application is an application system whose behavior depends on multiple processes, services, or networked components.

It solves these problems:

- reducing repeated expensive reads with caching;
- sharing temporary state across multiple app instances;
- moving slow work out of the request path;
- buffering spikes with queues;
- processing work asynchronously;
- surviving transient dependency failures;
- scaling parts of the system independently.

It does not solve these problems automatically:

- it does not remove the need for durable data design;
- it does not guarantee consistency;
- it does not make failures disappear;
- it does not make debugging easier;
- it does not eliminate the need for idempotency;
- it does not replace observability.

The basic model is:

```
Cache: fast temporary data
Queue: durable or semi-durable work buffer
Worker: process that performs background work
```

Message: data describing work or an event
Resilience policy: planned response to expected failure

The key distinction is durable truth versus derived or temporary state. SQL Server is usually the system of record for business data. Redis might hold a cached copy. A queue might hold work to perform. A worker might update durable state after processing. Confusing those roles creates bugs.

Layer 2 — System Relationships

The API receives client requests and coordinates application behavior. It should keep request latency reasonable and avoid doing long-running work inline when the user does not need to wait.

SQL Server owns durable business state. It remains the source of truth for orders, users, invoices, and other critical records unless the architecture explicitly assigns that role elsewhere.

Redis owns fast shared state. It may store cached query results, session-like data, counters, locks, or pub/sub messages. It can improve performance, but it is a network dependency with its own memory, persistence, and availability settings.

The queue owns pending work. It receives messages from producers and exposes them to consumers. Azure Service Bus, Amazon SQS, RabbitMQ, Kafka, Redis Streams, and database-backed queues can all play queue-like roles with different tradeoffs.

The background worker owns asynchronous processing. It reads messages or performs scheduled work, calls services, updates databases, and emits logs and metrics.

The external dependency owns behavior outside your process: payment, email, identity, search, shipping, AI, or another internal API. Network calls can fail even when your code is correct.

The resilience layer owns planned failure behavior: timeouts, retries, circuit breakers, fallback, rate limiting, and cancellation. Failure boundaries include stale cache entries, cache stampedes, queue backlogs, duplicate messages, poison messages, worker crashes, partial updates, retries that repeat side effects, slow dependencies, missing timeouts, and insufficient telemetry to know which component is failing.

Layer 3 — Core Mechanics

Caching begins with a key and a value.

```
public sealed class ProductLookup
{
    private readonly IDistributedCache _cache;

    public ProductLookup(IDistributedCache cache)
    {
        _cache = cache;
    }

    public async Task<string?> GetNameAsync(int productId)
    {
        var key = $"product:{productId}:name";

        return await _cache.GetStringAsync(key);
    }
}
```

`IDistributedCache` is a .NET abstraction for distributed caching. Microsoft documents Redis as one available implementation through `Microsoft.Extensions.Caching.StackExchangeRedis`.

Register Redis caching:

```
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
        builder.Configuration.GetConnectionString("Redis");
});
```

A background service uses `BackgroundService` :

```
public sealed class ShipmentWorker : BackgroundService
{
    private readonly ILogger<ShipmentWorker> _logger;

    public ShipmentWorker(ILogger<ShipmentWorker> logger)
    {
        _logger = logger;
    }

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            _logger.LogInformation("Checking for shipment work");

            await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);
        }
    }
}
```

Register it:

```
builder.Services.AddHostedService<ShipmentWorker>();
```

This is useful for learning, but production workers usually read from a queue or scheduled source instead of polling blindly. Resilience for HTTP calls uses explicit policies. Modern .NET provides `Microsoft.Extensions.Http.Resilience`, built on Polly:

```
builder.Services.AddHttpClient<ShippingClient>()
    .AddStandardResilienceHandler();
```

That one line is not a design. It is a starting point. You still need to understand timeouts, retries, idempotency, and what happens when the downstream service remains unavailable.

Layer 4 — Developer Workflow

Start by identifying the pressure.

```
Repeated expensive reads -> consider cache
Slow noninteractive work -> consider background processing
Traffic bursts -> consider queue
Transient dependency failure -> consider resilience policy
Independent scale needs -> consider separate worker or service
```

Add Redis for local development with Docker Compose:

```
services:
  redis:
    image: redis:8
    ports:
      - "6379:6379"
```

Configure the app:

```
ConnectionStrings__Redis=redis:6379
```

Add the package:

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

Create a worker project:

```
dotnet new worker -n Orders.Worker
```

Run it locally:

```
dotnet run --project Orders.Worker
```

For queue-backed work, choose a real queue before production. For early local learning, a bounded in-memory channel can teach the producer/consumer shape, but it is not durable and does not survive process restart.

The workflow habit is to document the new boundary:

```
What component owns the data?  
Is the data durable?  
Can work be processed twice?  
What happens if the dependency is down?  
How is backlog measured?  
How is failure alerted?  
How is local development configured?
```

If you cannot answer those questions, the distributed design is not ready.

Layer 5 — Production Usage

Production distributed systems require explicit ownership.

Configuration should separate cache endpoints, queue names, retry settings, timeouts, and worker concurrency by environment. Development settings should not leak into production.

Secrets include Redis passwords, queue credentials, service bus connection strings, API keys, and managed identity settings. Use the platform secret system or managed identity where possible.

Security includes network isolation, TLS, authentication to Redis and queues, least-privilege worker identities, message validation, and avoiding sensitive data in messages unless protection is designed.

Reliability depends on timeouts, retries, idempotent handlers, dead-letter queues, poison-message handling, graceful shutdown, backpressure, and clear recovery procedures.

Deployment must coordinate producers and consumers. A new API version may send messages that an old worker cannot understand. Message contracts need version discipline.

Observability is essential. Monitor cache hit rate, cache latency, queue length, oldest message age, worker success/failure counts, retry counts, dead-letter counts, and downstream dependency latency.

Scaling changes bottlenecks. More API instances can increase queue volume. More workers can increase database load. More cache usage can hide database pressure until cache failure.

Persistence must be designed. Redis can be configured for persistence, but it is often used for temporary data. Queues may be durable, but they are not databases. SQL Server remains the system of record unless the design says otherwise.

Cost includes managed cache tiers, queue operations, worker compute, database load, retries, logging volume, and idle infrastructure.

Local development optimizes for ease of reset. Production optimizes for durability, controlled failure, backpressure, and visibility.

Layer 6 — Tradeoffs and Alternatives

Use caching when repeated reads are expensive, data can tolerate staleness, and the cache behavior is observable.

Do not use caching to hide a broken data model, missing indexes, or inefficient queries before those basics have been addressed.

Use Redis when the application needs shared cache state across instances, fast counters, distributed coordination, pub/sub, streams, or data structures beyond simple in-memory caching.

Use in-memory cache for single-instance, noncritical local optimization. Do not rely on it for load-balanced consistency.

Use queues when work can happen asynchronously, when bursts need buffering, or when producers and consumers should be decoupled.

Do not use queues when the user truly needs an immediate result or when the team is not prepared to handle duplicate, delayed, or failed processing.

Messaging alternatives include Azure Service Bus, RabbitMQ, Amazon SQS, Apache Kafka, Redis Streams, NATS, and database-backed queues. Kafka is strong for event streams and high-throughput log-like data. RabbitMQ is strong for brokered messaging patterns. Service Bus and SQS are managed cloud queue options. SQL-backed queues can be simple but may add load to the database.

Use background services for long-running processes, scheduled work, and queue consumers. Use platform-native jobs, serverless functions, or managed schedulers when they fit better.

Common overengineering mistakes:

- adding Redis before measuring database performance;
- using cache as the source of truth;
- sending huge messages instead of references to durable data;
- ignoring duplicate message delivery;
- retrying non-idempotent operations;
- running background work inside the API process when it needs independent scale;
- creating microservices before understanding queues, workers, and failure.

The state-of-the-art direction is measured distribution: add a cache, queue, worker, or resilience policy when it solves a specific boundary, then observe that boundary in production.

Layer 7 — Interview Perspective

Interviewers use distributed application questions to test whether you respect failure.

Concepts commonly tested:

- cache versus database;
- Redis use cases;
- cache invalidation;
- queues and asynchronous work;
- background services;
- message idempotency;
- retries and timeouts;
- circuit breakers;
- dead-letter queues;
- observability for distributed components.

Representative questions:

- “When would you add Redis to a .NET application?”
- “Why is cache invalidation hard?”
- “What problem does a queue solve?”

- “What is a poison message?”
- “Why must message handlers be idempotent?”
- “How do retries make systems worse?”
- “When should background work run outside the API process?”
- “What metrics matter for a queue-backed worker?”

A strong answer names the tradeoff:

“A queue decouples the API from slow work and buffers spikes, but it means work may be delayed, duplicated, or fail later. The handler must be idempotent, failures need a dead-letter path, and we need metrics for queue depth and message age.”

Common misconceptions:

- “Redis is just a faster SQL Server.”
- “A queue guarantees exactly-once processing.”
- “Retries are always safe.”
- “Background services are just scheduled tasks with a new name.”
- “Distributed systems improve reliability automatically.”
- “If a message is small, it does not need a contract.”
- “Observability can be added after distribution.”

Small design scenario:

An order API sends confirmation emails during the request. When the email provider is slow, checkout slows down. When the provider fails, orders are created but users see errors.

A better design would commit the order to SQL Server, enqueue an email command or event, return a successful response, and let a worker send the email. The worker should retry transient failures, avoid duplicate emails, dead-letter poison messages, and emit metrics for queue length and failures.

The strong answer improves user latency while making delayed failure visible.

Hands-On Lab

Objective:

Add Redis caching and a simple background worker concept to a .NET solution.

Prerequisites:

- .NET 10 SDK installed, or another currently supported .NET SDK.
- Docker or Docker Compose.
- Basic ASP.NET Core experience.

Steps:

1. Create an API:

```
dotnet new webapi -n Chapter15.Api
cd Chapter15.Api
```

2. Add Redis distributed cache support:

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

3. Register Redis caching:

```
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
```

```
builder.Configuration.GetConnectionString("Redis");  
});
```

4. Add an endpoint that reads and writes a cache value using `IDistributedCache`.
5. Run Redis locally:

```
docker run --rm -p 6379:6379 redis:8
```

6. Set a local connection string:

```
dotnet user-secrets init  
dotnet user-secrets set "ConnectionStrings:Redis" "localhost:6379"
```

7. Run the API:

```
dotnet run
```

8. Create a worker project:

```
cd ..  
dotnet new worker -n Chapter15.Worker  
dotnet run --project Chapter15.Worker
```

9. Observe worker logs and stop it with `ctrl+c`.

Expected results:

- Redis runs locally.
- The API can use `IDistributedCache`.
- The worker service starts and writes logs.
- You can explain why this local setup is not yet a production queue design.

Validation commands:

```
docker run --rm -p 6379:6379 redis:8  
dotnet build  
dotnet run  
dotnet run --project Chapter15.Worker
```

Troubleshooting notes:

- If Redis connection fails, confirm the container is running and the port is mapped.
- If user secrets are not read, run the command from the API project folder.
- If the worker exits immediately, inspect `ExecuteAsync` and logging output.
- If cache values seem stale, check expiration settings and keys.

Knowledge Check

1. Why is SQL Server usually the system of record while Redis is not?
2. What problem does a distributed cache solve in a load-balanced app?
3. Why is cache invalidation a design issue?
4. What problem does a queue solve?
5. Why can queue consumers receive the same work more than once?
6. What does idempotency mean for a message handler?
7. When should background work run outside the API process?
8. Why are timeouts important for external dependencies?
9. How can retries make an outage worse?

10. What metrics would you monitor for a queue-backed worker?

Summary

Distributed applications begin when behavior crosses process or service boundaries. Caches, queues, background workers, and external services all solve real problems, but each introduces new failure modes.

Redis can provide fast shared state and distributed caching. Queues decouple producers from consumers and buffer work. Background services process work outside the request path. Resilience policies help applications respond to transient failures with timeouts, retries, circuit breakers, and related strategies.

The discipline is to keep ownership clear. SQL Server holds durable truth. Caches hold temporary or derived data. Queues hold pending work. Workers perform asynchronous processing. Observability tells you whether those boundaries are healthy. The next chapter turns back to delivery, showing how CI/CD deployment pipelines safely move validated changes into production environments.

Sources

- [Caching in .NET](#)
- [Distributed caching in ASP.NET Core](#)
- [Background tasks with hosted services in ASP.NET Core](#)
- [Introduction to resilient app development](#)
- [Build resilient HTTP apps](#)
- [Redis documentation](#)
- [What is Redis?](#)

Further Reading

- [Worker Service template](#)
- [StackExchange.Redis documentation](#)
- [Azure Service Bus documentation](#)
- [Amazon SQS documentation](#)
- [RabbitMQ tutorials](#)
- [Polly documentation](#)

16

CI/CD Deployment Pipelines

Chapter Purpose

Chapter 8 introduced automated testing and CI basics: restore, build, test, and report status before merge. Chapter 16 extends that idea from validation to delivery.

Deployment pipelines exist because production releases are too important to depend on memory, heroics, or one person's deployment script. A professional team needs to know which commit was built, which tests passed, which artifact was produced, which environment received it, who approved it, what changed, and how to roll back if it fails.

If your earlier deployment model involved a build server, a release manager, manual copy steps, scheduled outage windows, and a checklist, the goal will feel familiar. The modern difference is that more of the release path is versioned, automated, observable, and attached to the same source-control workflow used by developers.

This chapter covers continuous deployment, release pipelines, Azure DevOps, container publishing, environment promotion, deployment approvals, and rollback strategies. It focuses on safely moving validated changes into production environments. It does not replace Chapter 17's infrastructure as code or Chapter 18's Kubernetes deployment model.

Where This Fits

A deployment pipeline connects CI output to runtime environments.

```
Pull request
|
v
CI: restore, build, test
|
v
Merge to main
|
v
Build deployable artifact
|
+-- publish folder
+-- package
+-- container image
|
v
Deploy to development or test
|
v
Promote to staging
|
v
```

```
Approval and release checks
|
v
Deploy to production
|
v
Monitor, verify, and roll back if needed
```

This chapter follows distributed applications because once a system includes APIs, workers, caches, queues, databases, and cloud resources, release safety becomes more important. A pipeline is not just a convenience. It is the system that coordinates change.

The most important distinction is:

```
CI proves the change can be built and tested.
CD moves a validated artifact through environments.
Release management controls when and how production changes happen.
```

Connection to the Reader's Existing Model

Older enterprise release processes often had real discipline.

There might have been a build number, a release candidate, a QA signoff, change advisory board approval, a deployment window, a rollback plan, and post-release verification. Those controls existed because production mattered.

Modern pipelines do not remove that discipline. They try to make it repeatable.

A release pipeline maps to a scripted version of the deployment checklist. It defines steps such as downloading an artifact, applying configuration, deploying to an environment, running smoke tests, and recording the result.

An environment maps to a named deployment target such as development, test, staging, or production. GitHub Actions and Azure Pipelines both support environment concepts with approvals, secrets, and deployment history.

A deployment approval maps to a human signoff or change-control step. The difference is that the approval is attached to the pipeline run and can control access to environment secrets.

A container registry maps to a package feed for runtime images. Instead of publishing only DLLs or zip files, the pipeline can publish a tagged image.

A rollback maps to restoring the previous known-good release. The modern expectation is that rollback should be practiced, documented, and tied to identifiable artifacts.

The analogy breaks down when a pipeline is treated as a magic release machine. If the pipeline deploys unreviewed schema changes, hides manual steps, lacks health checks, or cannot roll back, it has automated risk rather than reduced it.

Layer 1 — Conceptual Model

A deployment pipeline is an automated path that moves a known software artifact through one or more environments under controlled rules.

It solves these problems:

- making releases repeatable;
- tying deployments to commits and artifacts;
- reducing manual variation;
- controlling access to environment secrets;
- requiring approvals for sensitive environments;
- promoting the same artifact through stages;
- publishing container images;
- making rollback possible and traceable.

It does not solve these problems automatically:

- it does not make tests useful;
- it does not make database migrations safe;
- it does not choose the right deployment strategy;
- it does not replace production observability;
- it does not guarantee zero downtime;
- it does not fix bad architecture.

The conceptual model is:

```
Source commit -> build -> artifact -> environment -> verification -> promotion
```

The strongest pipeline habit is build once, promote many times. Build the artifact once from a known commit. Move that artifact through test, staging, and production with environment-specific configuration supplied by the environment, not by rebuilding different binaries.

Layer 2 — System Relationships

The repository owns source code, workflow definitions, Dockerfiles, scripts, and sometimes deployment manifests.

The CI system owns validation. It restores packages, builds, runs tests, and publishes a build result.

The artifact store owns deployable output. This might be a pipeline artifact, NuGet package feed, zip package, container registry, or cloud deployment package.

The container registry owns container images. GitHub Container Registry, Azure Container Registry, Docker Hub, Amazon ECR, and Google Artifact Registry are common examples.

The environment owns deployment configuration: URLs, app settings, secret references, deployment protections, approvals, and resource identity.

The deployment job owns change execution. It takes an artifact and applies it to a target environment.

The approval system owns human or automated gates. GitHub environments can use required reviewers, wait timers, branch restrictions, environment secrets, and custom protection rules. Azure Pipelines environments can use approvals and checks controlled by resource owners.

The monitoring system owns release verification. It tells the team whether health, error rate, latency, and business indicators look acceptable after deployment.

Failure boundaries include missing secrets, wrong environment, deploying the wrong artifact, mutable tags, skipped approvals, pipeline credential leaks, stuck deployments, failed database migrations, unavailable targets, partial deployments, and rollback plans that were never tested.

Layer 3 — Core Mechanics

A minimal deployment pipeline has stages.

```
Build
  restore, build, test, publish artifact

Package
  create zip package or container image

Deploy test
  deploy artifact to a nonproduction environment

Approve production
  require human or automated checks
```

```

Deploy production
  deploy the same artifact

Verify
  run smoke checks and observe health

```

A simple GitHub Actions workflow can build and publish a container image:

```

name: publish-container

on:
  push:
    branches: [ "main" ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v6

      - uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }

```

The important detail is the tag. A commit SHA tag identifies the image better than `latest`.

A deployment job can reference an environment:

```

jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: echo "Deploy the previously built artifact"

```

In GitHub Actions, environment protection rules can require approval before the job proceeds. Environment secrets are not available to the job until protection rules pass.

Azure Pipelines uses stages and environments:

```

stages:
- stage: Build
  jobs:
  - job: BuildAndTest
    steps:
    - script: dotnet build --configuration Release
    - script: dotnet test --configuration Release

```

```
- stage: DeployProduction
  dependsOn: Build
  jobs:
  - deployment: Deploy
    environment: production
    strategy:
      runOnce:
        deploy:
          steps:
            - script: echo Deploy artifact
```

In Azure DevOps, approvals and checks can be configured on environments and other protected resources outside the YAML file. That separation matters: someone who edits the pipeline YAML should not automatically be able to remove production approval rules.

Layer 4 — Developer Workflow

A developer-friendly deployment workflow begins before production.

Build locally:

```
dotnet restore
dotnet build -c Release
dotnet test -c Release
```

Build an image locally:

```
docker build -t orders-api:local .
```

Tag with a meaningful version for experiments:

```
docker tag orders-api:local orders-api:dev-$(git rev-parse --short HEAD)
```

In the pipeline, produce an artifact from the merge commit:

```
main branch commit
|
v
Release artifact: orders-api:<commit-sha>
```

Promote through environments:

```
development -> test -> staging -> production
```

At each environment, verify:

```
Did deployment finish?
Is the app healthy?
Do smoke tests pass?
Are error rates normal?
Did database migration complete?
Is rollback still possible?
```

A pull request should not deploy to production. It may deploy a preview environment or run integration tests. Production deployment should be tied to merge, tag, release, or a deliberate manual workflow.

The developer's responsibility is not to memorize every CI/CD product. It is to understand the release path well enough to diagnose when the pipeline fails and to design changes that can move safely through it.

Layer 5 — Production Usage

Production pipelines are security-sensitive systems.

Configuration should be environment-scoped. Development, test, staging, and production should have separate settings. A pipeline should not copy local configuration files into production.

Secrets should live in CI/CD secret stores, environment secrets, cloud secret stores, or federated identity. Prefer short-lived credentials and managed identity patterns when the platform supports them. Avoid long-lived credentials stored in repository variables.

Security includes least-privilege pipeline identities, protected branches, required reviews, environment approvals, audit trails, artifact integrity, secret masking, dependency scanning, and careful use of third-party actions or extensions.

Reliability depends on deterministic builds, reusable deployment scripts, health checks, smoke tests, rollback, and protection against simultaneous deployments. GitHub Actions concurrency groups and Azure environment locks can help prevent overlapping deployments.

Deployment should use identifiable artifacts. For containers, prefer immutable tags or digests. For zip or folder artifacts, keep build IDs, commit SHAs, and checksums.

Observability should be wired into release verification. A pipeline can deploy successfully while the application fails after startup. Health, logs, metrics, and alerts complete the release signal.

Scaling affects deployment strategy. A single instance can be replaced directly. Multiple instances may need rolling deployment, blue/green, canary, or traffic-splitting strategies.

Persistence affects release safety. Database migrations are often the hardest part of rollback. Use backward-compatible schema changes where possible and avoid coupling rollback to destructive data changes.

Cost includes build minutes, hosted runners, self-hosted runner maintenance, artifact storage, container registry storage, preview environments, and idle deployment targets.

Local development optimizes for fast iteration. Production pipelines optimize for traceability, controlled access, repeatability, verification, and recovery.

Layer 6 — Tradeoffs and Alternatives

Use GitHub Actions when the repository and team collaboration already live in GitHub, especially when pull requests, environment protection, packages, and deployment history should stay close to the code.

Use Azure DevOps when the organization already uses Azure Boards, Azure Repos, Azure Pipelines, release approvals, service connections, and enterprise governance around Azure DevOps.

Use GitLab CI, Jenkins, TeamCity, CircleCI, Buildkite, Octopus Deploy, Argo CD, Flux, or Spinnaker when the organization's platform, compliance, deployment model, or existing skills fit those tools better.

Continuous deployment to production is appropriate when tests, observability, rollback, and team maturity are strong. Continuous delivery with manual production approval is often better for regulated systems, early teams, or high-risk workloads.

Blue/green deployment reduces risk by running two environments and switching traffic. Canary deployment reduces risk by sending a small percentage of traffic to the new version first. Rolling deployment updates instances in groups. Direct replacement is simpler but riskier.

Common overengineering mistakes:

- building an elaborate pipeline before the deployment steps are understood;
- approving deployments without looking at what changed;
- rebuilding separately for each environment;
- using mutable production tags such as `latest`;
- storing production credentials as broad repository secrets;
- automating database migrations with no review;
- having no rollback beyond “redploy and hope”;
- making pipelines so slow that developers bypass them.

The state-of-the-art direction is secure, traceable, artifact-based delivery: short-lived credentials, protected environments, provenance and attestations, container scanning, policy gates, deployment health verification, and rollback paths that are tested before crisis.

Layer 7 — Interview Perspective

Interviewers use CI/CD questions to test whether you understand release risk.

Concepts commonly tested:

- CI versus CD;
- build artifacts;
- environment promotion;
- deployment approvals;
- environment secrets;
- container image tags;
- branch protection;
- rollback;
- blue/green and canary deployment;
- database migration safety.

Representative questions:

- “What is the difference between continuous integration and continuous deployment?”
- “Why build once and promote the same artifact?”
- “How should production secrets be handled in a pipeline?”
- “What is an environment approval?”
- “Why avoid `latest` tags in production?”
- “How would you roll back a bad release?”
- “How do database changes affect deployment strategy?”
- “What should a deployment pipeline verify after release?”

A strong answer connects automation to control:

“The pipeline should build and test once, produce a known artifact, deploy that artifact through environments, require approval for production, and verify health after deployment. Rollback should use a previous known-good artifact, but database migrations may require a separate compatibility plan.”

Common misconceptions:

- “CI/CD means every commit must go straight to production.”
- “A successful deployment job means the app is healthy.”
- “Approvals are anti-DevOps.”
- “Secrets are safe if they are hidden in YAML.”
- “`latest` is fine if the build just completed.”
- “Rollback is always simple.”
- “Deployment pipelines replace observability.”

Small design scenario:

You have an ASP.NET Core API, SQL Server database, Redis cache, and background worker. The team currently deploys by copying files and restarting services.

A good pipeline plan would build and test the solution, publish the API and worker artifacts or container images, tag them with the commit SHA, deploy to test, run smoke checks, require approval for production, deploy API and worker in a known

order, run compatibility-safe database migrations, monitor health and queue depth, and define rollback for both code and schema.

The strong answer treats deployment as a system change, not a file copy.

Hands-On Lab

Objective:

Create a simple deployment-oriented workflow that builds, tests, publishes, and records a container image tag.

Prerequisites:

- A GitHub repository.
- .NET 10 SDK or another supported SDK.
- Dockerfile from Chapter 10.
- Basic GitHub Actions knowledge from Chapter 8.

Steps:

1. Create `.github/workflows/release.yml`.
2. Add triggers:

```
on:
  push:
    branches: [ "main" ]
  workflow_dispatch:
```

3. Add a build job that restores, builds, and tests:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "10.0.x"
      - run: dotnet restore
      - run: dotnet build --no-restore --configuration Release
      - run: dotnet test --no-build --configuration Release
```

4. Add a package job that builds a container image tagged with the commit SHA.
5. Configure a GitHub environment named `production`.
6. Add required reviewers to the `production` environment.
7. Add a placeholder deployment job that references the environment:

```
deploy-production:
  needs: build
  runs-on: ubuntu-latest
  environment: production
  steps:
    - run: echo "Deploy artifact for ${github.sha}"
```

8. Trigger the workflow manually or by merging to `main`.

Expected results:

- The workflow validates the application.
- The workflow has a production deployment job gated by environment rules.
- The deployment step records the commit SHA it would deploy.

- You can explain what real deployment commands would replace the placeholder.

Validation commands:

```
dotnet restore
dotnet build -c Release
dotnet test -c Release
docker build -t app:${GITHUB_SHA:-local} .
```

Troubleshooting notes:

- If GitHub Actions cannot find the solution, check the repository root.
- If environment approval does not appear, confirm the job references the environment name exactly.
- If secrets are unavailable, remember that environment secrets are available only after environment protection rules pass.
- If Docker build fails, confirm the Dockerfile path and build context.

Knowledge Check

1. How is CI different from CD?
2. Why should the same artifact move through multiple environments?
3. What is an environment in a deployment pipeline?
4. Why should production deployments often require approvals?
5. How do environment secrets reduce risk?
6. Why are immutable image tags or digests safer than `latest`?
7. What should a pipeline verify after deployment?
8. Why are database migrations a rollback risk?
9. When would blue/green or canary deployment be useful?
10. What makes a deployment pipeline auditable?

Summary

Deployment pipelines turn release practice into a repeatable system. CI proves that a change can be restored, built, and tested. CD moves a known artifact through environments with controlled configuration, secrets, approvals, verification, and rollback.

GitHub Actions and Azure DevOps both support modern deployment workflows. GitHub uses environments, protection rules, environment secrets, and deployment reviews. Azure Pipelines uses stages, environments, approvals, checks, service connections, and protected resources. Other tools can implement the same core ideas.

The strongest habit is to build once and promote deliberately. Production deployment should use identifiable artifacts, protected secrets, clear approval rules, health verification, and a rollback path that accounts for data as well as code.

The next chapter moves one layer lower: infrastructure as code, where cloud resources and environment provisioning become versioned artifacts too.

Sources

- Deploying with GitHub Actions
- Deployments and environments
- Reviewing deployments
- Publishing Docker images with GitHub Actions

- Define approvals and checks in Azure Pipelines
- Deployment control using approvals in Azure Pipelines

Further Reading

- [GitHub Actions workflow syntax](#)
- [GitHub Actions security hardening](#)
- [Azure Pipelines documentation](#)
- [Docker build-push-action](#)
- [Azure Container Registry documentation](#)

17

Infrastructure as Code

Chapter Purpose

Chapter 16 showed how deployment pipelines move application artifacts through environments. Chapter 17 extends the same idea to the environments themselves.

Infrastructure as code, usually shortened to IaC, exists because modern applications depend on more than application files. An ASP.NET Core API may need an app service, container environment, database, storage account, virtual network, identity, secret store, monitoring workspace, container registry, and deployment permissions. If those resources are created by hand, the environment becomes a memory-based system.

That problem is familiar. In older Windows and IIS environments, a production server might have had settings that only one administrator understood: IIS bindings, application pool identities, Windows features, firewall rules, scheduled tasks, registry settings, service accounts, and SQL Server permissions. The server worked, but nobody wanted to recreate it from scratch. Infrastructure as code turns that implicit environment knowledge into versioned, reviewable, repeatable files. Instead of saying, “Create the database and storage account in the portal,” the team describes the desired resources in code and lets a tool provision or update them.

This chapter introduces why infrastructure became code, Terraform, Bicep, configuration management, and environment provisioning. It does not make you a cloud platform engineer in one chapter. It gives you the mental model needed to understand how modern .NET environments become repeatable.

Where This Fits

Infrastructure as code sits beside application code and deployment pipelines.

```
Application source
  |
  v
CI/CD pipeline
  |
  +-- builds application artifact
  |
  +-- applies infrastructure code
  |
  v
Cloud environment
  |
  +-- compute
  +-- database
  +-- storage
  +-- networking
  +-- identity
```

```
+-- secrets
+-- monitoring
|
v
Deployed ASP.NET Core application
```

Chapter 17 follows cloud platforms and deployment pipelines because IaC is most meaningful once you understand what resources exist and how releases move through environments. Chapter 18 introduces Kubernetes, where declarative infrastructure concepts appear again in manifests and desired state.

The central idea is:

```
Application code describes behavior.
Infrastructure code describes the environment where behavior runs.
Pipelines coordinate changes to both.
```

Connection to the Reader's Existing Model

You already understand configuration drift.

One server has the right IIS module installed. Another has a different TLS setting. A scheduled task exists in test but not production. A SQL login has extra permissions because of a production incident three years ago. A firewall rule was added manually and never documented. The system works until it must be recreated, audited, scaled, or compared.

Infrastructure as code addresses that class of problem.

An IaC file is like a deployment runbook that a tool can execute repeatedly. It is also like a source-controlled server build checklist, but more precise.

Terraform configuration maps to cross-platform infrastructure definitions. It can manage Azure, AWS, Google Cloud, on-premises systems, SaaS products, and many other APIs through providers.

Bicep maps to Azure Resource Manager. It is Azure-specific and designed as a cleaner declarative language for Azure resources than raw ARM template JSON.

Configuration management maps to server configuration tools such as Group Policy, Desired State Configuration, PowerShell scripts, Chef, Puppet, or Ansible. The difference is that cloud IaC often provisions managed services instead of configuring long-lived servers.

The analogy breaks down when infrastructure code is treated as a script. Most IaC is declarative. You describe the desired end state. The tool decides what actions are needed to reach it.

Layer 1 — Conceptual Model

Infrastructure as code is the practice of defining infrastructure resources in versioned files and applying those definitions with automation.

It solves these problems:

- recreating environments reliably;
- reviewing infrastructure changes before they happen;
- reducing portal-only or click-only configuration;
- comparing environments;
- automating environment provisioning;
- tying infrastructure changes to application releases;
- improving auditability and rollback planning;
- reducing configuration drift.

It does not solve these problems automatically:

- it does not choose the right architecture;
- it does not make cloud resources secure by default;
- it does not remove the need to understand networking and identity;
- it does not eliminate state, drift, or migration risk;
- it does not make destructive changes safe;
- it does not replace production review.

The core model is:

```
Desired state file
    |
    v
Plan or preview
    |
    v
Apply or deploy
    |
    v
Cloud resources
    |
    v
State, deployment record, or provider view
```

Terraform and Bicep both use declarative configuration, but they track reality differently. Terraform stores state that maps configuration to real resources. Bicep relies on Azure Resource Manager deployments and Azure’s resource model rather than a separate Terraform-style state file.

That difference shapes how teams collaborate and recover from manual changes.

Layer 2 — System Relationships

The infrastructure code repository owns resource definitions. It may live with the application code or in a separate platform repository, depending on team ownership and release coupling.

The IaC tool reads configuration files. Terraform reads `.tf` files written in HashiCorp Configuration Language. Bicep reads `.bicep` files and compiles them to ARM templates for Azure Resource Manager.

The provider or cloud control plane owns resource creation. Terraform uses providers to call APIs for Azure, AWS, Google Cloud, Kubernetes, GitHub, and many other systems. Bicep submits deployments to Azure Resource Manager.

The state or deployment record owns the mapping between desired configuration and real resources. Terraform state is critical and must be secured and backed up. Bicep does not use a separate state file; Azure Resource Manager compares the deployment with existing Azure resources.

The pipeline owns execution. It can run validation, plan, approval, and apply steps. Production infrastructure changes should not depend on a developer’s laptop.

The environment owns the resources: app host, database, storage, network, identity, role assignments, secret store, monitoring, and policy assignments.

Failure boundaries include missing permissions, wrong subscription or account, unsafe deletes, state file corruption, out-of-band portal changes, naming collisions, quota limits, provider bugs, secret exposure, policy violations, and infrastructure changes applied without application compatibility.

Layer 3 — Core Mechanics

A simple Terraform configuration declares a provider and resources:

```
terraform {
  required_providers {
```

```

    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "main" {
  name     = "rg-orders-dev"
  location = "eastus"
}

```

The basic Terraform workflow is:

```

terraform init
terraform plan
terraform apply

```

`init` installs providers and prepares the working directory. `plan` previews changes. `apply` performs changes.

A simple Bicep file declares Azure resources:

```

param location string = resourceGroup().location

resource storage 'Microsoft.Storage/storageAccounts@2025-01-01' = {
  name: 'ordersstorage${uniqueString(resourceGroup().id)}'
  location: location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
}

```

Deploy with Azure CLI:

```

az deployment group create \
  --resource-group rg-orders-dev \
  --template-file main.bicep

```

Preview with what-if:

```

az deployment group what-if \
  --resource-group rg-orders-dev \
  --template-file main.bicep

```

Both tools support parameters, variables, modules, outputs, and reusable patterns. The mechanics differ, but the discipline is the same: review the planned change before applying it.

Layer 4 — Developer Workflow

A practical IaC workflow begins with one environment and a small resource set.

```

Choose target environment
|
v
Define resources in code
|
v

```

```
Format and validate
  |
  v
Preview changes
  |
  v
Review pull request
  |
  v
Apply through pipeline
  |
  v
Verify resources and application deployment
```

For Terraform:

```
terraform fmt
terraform validate
terraform plan
```

For Bicep:

```
az bicep build --file main.bicep
az deployment group what-if \
  --resource-group rg-orders-dev \
  --template-file main.bicep
```

Keep environment-specific values out of copied files when possible. Use parameters:

```
dev.tfvars
test.tfvars
prod.tfvars
```

or Bicep parameter files:

```
main.dev.bicepparam
main.prod.bicepparam
```

The pull request should show:

```
What resources change?
Which environment changes?
What is the preview or plan?
Is anything destroyed?
Are permissions changing?
Does the app deployment depend on this change?
How do we recover if it fails?
```

For learning, running IaC locally is fine. For teams, production applies should move into CI/CD with approvals, secrets, and audit records.

Layer 5 — Production Usage

Production IaC begins with state and permissions.

Terraform state must be stored remotely, locked during changes, backed up, and protected as sensitive data. State can contain resource identifiers and secrets depending on providers and resources.

Bicep deployments rely on Azure Resource Manager, but production still needs clear deployment scopes, permissions, template history, parameter handling, and policy review.

Configuration should separate reusable modules from environment-specific values. Repeating entire infrastructure definitions for every environment invites drift.

Secrets should not be hard-coded in IaC files or parameter files. Prefer secret references, managed identities, Key Vault, CI/CD secret stores, or provider-supported secure inputs.

Security includes least-privilege deployment identities, protected state, reviewed role assignments, network boundaries, policy enforcement, and auditable change history.

Reliability depends on safe change ordering. Infrastructure changes can break applications: removing a subnet, renaming a storage account, rotating a key, or changing a database firewall can take production down.

Deployment pipelines should preview infrastructure changes, require approval for sensitive environments, and apply changes with identities scoped to the target environment.

Observability applies to infrastructure changes too. Resource health, deployment failures, policy violations, drift, quota usage, and cost changes should be visible.

Scaling IaC requires modules, naming conventions, tagging, repository structure, environment promotion, and ownership boundaries.

Persistence requires extra caution. Deleting a database, storage account, key vault, or network can be far more damaging than redeploying an app container. Production IaC should use deletion protection, locks, backups, and review for stateful resources where appropriate.

Cost is part of the design. IaC can create expensive resources quickly. Tags, budgets, policies, and cleanup automation should be part of the environment model.

Local IaC experimentation optimizes for learning. Production IaC optimizes for reviewability, least privilege, controlled change, and recovery.

Layer 6 — Tradeoffs and Alternatives

Use Terraform when the organization needs multi-cloud, hybrid-cloud, cross-provider automation, strong module ecosystems, or a consistent workflow across many APIs.

Use Bicep when the target is Azure, the team wants deep Azure Resource Manager integration, immediate Azure resource coverage, and a concise Azure-native language.

Use ARM templates when required by legacy systems or generated tooling, but prefer Bicep for hand-authored Azure templates in most new work.

Use Pulumi when the team wants to define infrastructure using general-purpose languages such as C#, TypeScript, Python, Go, or Java. Use Ansible, Chef, or Puppet when configuration management of servers is the central problem.

OpenTofu is an open-source Terraform fork created after Terraform licensing changes. Some organizations consider it when they want a Terraform-compatible open-source path.

Common overengineering mistakes:

- introducing IaC before naming and ownership conventions exist;
- putting secrets directly in code;
- applying production changes from a developer laptop;
- ignoring Terraform state security;
- treating plan output as unreadable noise;
- creating modules too early for one-off resources;
- letting portal changes drift away from code;
- using IaC to create resources no one understands.

The state-of-the-art direction is policy-aware, pipeline-driven IaC: modules for common patterns, secure remote state, plan or what-if review, environment approvals, drift detection, cost controls, and integration with deployment pipelines.

Layer 7 — Interview Perspective

Interviewers use IaC questions to test whether you understand repeatability and cloud responsibility.

Concepts commonly tested:

- why infrastructure became code;
- declarative versus imperative automation;
- Terraform providers;
- Terraform state;
- Bicep and ARM;
- plan versus apply;
- what-if previews;
- modules;
- environment parameterization;
- drift;
- secrets and state security.

Representative questions:

- “What problem does infrastructure as code solve?”
- “How is Terraform different from Bicep?”
- “Why is Terraform state important?”
- “What is a plan?”
- “What is Azure Bicep?”
- “How do you prevent dev and prod from drifting apart?”
- “Where should secrets live?”
- “Why should production infrastructure changes go through review?”

A strong answer names both benefits and risk:

“IaC makes environments repeatable and reviewable, but it can also destroy resources repeatably if used carelessly. I would use plans or what-if output, protect state, keep secrets out of code, apply through pipelines, and require extra review for stateful resources and permissions.”

Common misconceptions:

- “IaC means no one needs to understand cloud resources.”
- “Terraform state is just a cache.”
- “Bicep works across all clouds.”
- “If code review passed, applying infrastructure is always safe.”
- “Portal changes are harmless.”
- “Configuration management and provisioning are the same thing.”
- “IaC rollback is always easy.”

Small design scenario:

You need separate development, test, and production environments for an ASP.NET Core order API. Each environment needs app hosting, SQL database, storage, managed identity, Key Vault, and monitoring.

A good IaC design would create reusable modules, parameterize environment names and sizes, use separate state or deployment scopes per environment, store secrets in Key Vault, assign least-privilege identities, preview changes in pull requests, and apply production changes through an approved pipeline.

The strong answer makes environment creation boring and reviewable.

Hands-On Lab

Objective:

Create a simple infrastructure-as-code preview for an Azure resource using either Bicep or Terraform.

Prerequisites:

- Azure CLI installed if using Bicep.
- Terraform installed if using Terraform.
- A development Azure subscription or sandbox if you plan to apply changes.
- No production environment.

Steps:

1. Create a folder:

```
mkdir chapter17-iac
cd chapter17-iac
```

2. For Bicep, create `main.bicep` with a storage account resource.
3. Build the Bicep file:

```
az bicep build --file main.bicep
```

4. Preview the deployment:

```
az deployment group what-if \
  --resource-group rg-chapter17-dev \
  --template-file main.bicep
```

5. For Terraform, create `main.tf` with a resource group.
6. Initialize and validate:

```
terraform init
terraform fmt
terraform validate
```

7. Create a plan:

```
terraform plan
```

8. Do not apply to a shared or production environment.

Expected results:

- You can explain what resource the file describes.
- You can run a validation or preview command.
- You can describe what would change before applying it.
- You can explain where secrets and environment-specific values should not go.

Validation commands:

```
az bicep build --file main.bicep
az deployment group what-if --resource-group rg-chapter17-dev --template-file main.bicep
terraform init
terraform fmt
terraform validate
terraform plan
```

Troubleshooting notes:

- If Azure CLI is not logged in, run `az login`.

- If the resource group does not exist, create a sandbox group or keep the lab conceptual.
- If Terraform cannot authenticate, configure the provider for your cloud account.
- If a plan wants to destroy a resource you care about, stop.

Knowledge Check

1. Why did infrastructure become code?
2. What is the difference between declarative and imperative automation?
3. Why is Terraform state sensitive?
4. How does Bicep differ from Terraform?
5. What does a plan or what-if operation provide?
6. Why should production infrastructure changes go through a pipeline?
7. What causes drift?
8. Why should secrets not be stored in IaC files?
9. When is Terraform a better fit than Bicep?
10. When is Bicep a better fit than Terraform?

Summary

Infrastructure as code makes environments repeatable, reviewable, and automatable. It turns cloud resources, networks, identities, databases, storage, and monitoring into versioned definitions instead of portal memory and handwritten setup notes.

Terraform is a general-purpose IaC tool that manages many providers and tracks resources through state. Bicep is an Azure-native declarative language that deploys through Azure Resource Manager without a Terraform-style state file. Both support reusable modules, parameters, previews, and pipeline-driven deployment.

The important habit is caution with power. IaC can reliably create resources, but it can also reliably delete or misconfigure them. Plans, what-if previews, protected state, least-privilege identities, secrets management, approvals, and production review are part of responsible IaC.

The next chapter introduces Kubernetes, another desired-state system where declarative configuration describes how containerized workloads should run.

Sources

- What is Terraform?
- Terraform language documentation
- Bicep documentation
- Azure Resource Manager templates overview
- Get started with Infrastructure as Code on Azure
- Comparing Terraform and Bicep
- Azure resource reference for Bicep, ARM templates, and Terraform AzAPI

Further Reading

- Terraform tutorials

- [Terraform Registry](#)
- [Deploy and manage resources in Azure by using Bicep](#)
- [Azure Verified Modules](#)
- [OpenTofu documentation](#)

18

Kubernetes

Chapter Purpose

Kubernetes is where several threads of the book meet: Linux, containers, cloud platforms, deployment pipelines, infrastructure as code, observability, and distributed applications.

Kubernetes exists because running one container is not the same thing as operating many containers in production. A team may need to place containers across machines, restart failed instances, scale replicas, route traffic, perform rolling updates, attach configuration, manage secrets, and maintain a desired state over time. Doing that with hand-written scripts and individual servers becomes fragile quickly.

Kubernetes was originally created at Google and released as open source. It became the dominant container orchestration platform because it provided a portable API for describing and operating containerized workloads. It was not created for .NET, but modern .NET applications fit well into it when they are packaged as containers and designed for platform operation.

This chapter emphasizes concepts over operational expertise. You will learn why orchestration exists, what pods, deployments, services, scaling, health checks, and rolling deployments mean, and when Kubernetes is or is not the right choice.

Where This Fits

Kubernetes sits above individual containers and below application behavior.

```
Container image
  |
  v
Kubernetes Deployment
  |
  v
ReplicaSet
  |
  v
Pods
  |
  +--- ASP.NET Core container
  +--- environment variables
  +--- health probes
  +--- resource requests and limits
  |
  v
Nodes in a cluster
  |
  v
Service exposes stable network access
```

Chapter 18 follows infrastructure as code because Kubernetes is another desired-state system. You write resource definitions that describe what should exist, and controllers work to make the cluster match that description.

Chapter 19 follows with advanced observability because Kubernetes makes telemetry more important. If an app has many pods across nodes, local logs and manual server inspection are not enough.

Connection to the Reader's Existing Model

A load-balanced IIS farm is the closest familiar analogy.

In a web farm, multiple servers run copies of an application. A load balancer sends traffic to healthy instances. Administrators deploy new versions, restart unhealthy processes, patch servers, manage certificates, and watch capacity.

Kubernetes generalizes that operational model for containers.

A node is roughly like a server in the farm. It runs workloads.

A pod is the smallest deployable compute unit in Kubernetes. If you usually think in single containers, a pod often feels like “the running app container,” though a pod can contain multiple containers that must run together.

A deployment is roughly like the desired rollout rule for an application: run this container image, keep this many replicas, update them this way.

A service is roughly like a stable internal load-balancing name for a set of pods. Pods are replaceable and get new IP addresses. Services give other code a stable way to reach them.

Health probes map to load balancer health checks and service monitoring. Kubernetes can use probes to decide whether a container is alive or ready for traffic.

The analogy breaks down if you think Kubernetes is just a web farm manager. Kubernetes is an API-driven control plane for many kinds of containerized workloads: APIs, background workers, batch jobs, scheduled jobs, platform components, and operators. It also introduces its own security, networking, storage, upgrade, and cost responsibilities.

Layer 1 — Conceptual Model

Kubernetes is an open-source platform for managing containerized workloads and services using declarative configuration and automation.

It solves these problems:

- scheduling containers across machines;
- keeping desired replica counts running;
- restarting failed containers;
- exposing stable service names;
- rolling out new versions;
- scaling workloads;
- attaching configuration and secrets;
- providing a common API across environments.

It does not solve these problems automatically:

- it does not make applications stateless;
- it does not make databases easy to run;
- it does not remove the need for observability;
- it does not secure workloads without configuration;
- it does not eliminate cloud networking complexity;
- it does not make a small app simpler.

The conceptual model is:

```
You declare desired state.  
Kubernetes stores that desired state.  
Controllers compare desired state to actual state.  
Controllers act to close the gap.
```

This is the same kind of mental model you saw in infrastructure as code, but applied continuously to running workloads.

Layer 2 — System Relationships

The cluster is the whole Kubernetes environment. It includes the control plane and nodes.

The control plane exposes the Kubernetes API, stores cluster state, schedules pods, and runs controllers that reconcile desired state. In managed services such as Azure Kubernetes Service and Amazon EKS, the cloud provider manages much of the control plane.

Nodes are machines that run pods. They may be virtual machines, cloud-managed compute, or other host types. Nodes run a kubelet, networking components, and a container runtime.

Pods run one or more containers. A pod has its own network identity inside the cluster and shares storage volumes among containers in the pod.

Deployments manage replicated application pods. They create and update ReplicaSets so the desired number of pod replicas exists.

Services provide stable network access to pods. They select pods using labels and route traffic to matching pod endpoints.

Ingress or gateway resources expose HTTP traffic from outside the cluster into services. The exact implementation depends on the cluster and cloud provider.

ConfigMaps and Secrets provide configuration to pods. They are not a complete secrets-management strategy by themselves, especially without encryption and access controls.

Failure boundaries include bad manifests, unavailable images, crash loops, failed probes, insufficient CPU or memory, bad service selectors, DNS issues, network policies, secret misconfiguration, node failure, cluster upgrades, and applications that cannot shut down gracefully.

Layer 3 — Core Mechanics

A deployment describes the desired state for application replicas:

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: orders-api  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: orders-api  
  template:  
    metadata:  
      labels:  
        app: orders-api  
    spec:  
      containers:  
        - name: orders-api  
          image: registry.example.com/orders-api:2026.07.22  
          ports:  
            - containerPort: 8080  
          readinessProbe:  
            httpGet:
```

```

    path: /health
    port: 8080
  livenessProbe:
    httpGet:
      path: /health
      port: 8080
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"

```

A service gives stable access to pods with matching labels:

```

apiVersion: v1
kind: Service
metadata:
  name: orders-api
spec:
  selector:
    app: orders-api
  ports:
    - port: 80
      targetPort: 8080

```

Apply the files:

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

```

Inspect resources:

```

kubectl get pods
kubectl get deployments
kubectl get services

```

Update the image:

```

kubectl set image deployment/orders-api \
  orders-api=registry.example.com/orders-api:2026.07.23

```

Watch rollout status:

```

kubectl rollout status deployment/orders-api

```

Roll back:

```

kubectl rollout undo deployment/orders-api

```

These commands are mechanics. The more important concept is that the deployment controller tries to move the system from old state to new state without dropping all replicas at once.

Layer 4 — Developer Workflow

A .NET developer's Kubernetes workflow usually begins before the cluster.

```

Build ASP.NET Core app
|
v
Containerize app
|

```



```
      v
Push image to registry
      |
      v
Write Kubernetes manifests or Helm chart
      |
      v
Deploy to a development namespace
      |
      v
Inspect pods, logs, services, and rollout
```

Useful local commands:

```
kubectl version --client
kubectl config current-context
kubectl get namespaces
```

Check application status:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

Port-forward for local testing:

```
kubectl port-forward service/orders-api 8080:80
```

Call the app:

```
curl http://localhost:8080/health
```

For development, do not start by building a production cluster from scratch. Use a local cluster such as Docker Desktop Kubernetes, kind, or minikube, or a managed development cluster provided by the team.

The workflow habit is to connect app symptoms to Kubernetes resources:

```
App unavailable -> check service, pods, readiness
Container exits -> check logs and describe pod
New version stuck -> check rollout status
No traffic -> check labels, selectors, ingress, ports
Slow app -> check resources, metrics, dependencies
```

Layer 5 — Production Usage

Production Kubernetes begins with platform ownership.

Configuration should be environment-specific and declarative. Use manifests, Helm charts, Kustomize, GitOps tools, or platform templates rather than hand-editing live resources.

Secrets should be integrated with a real secret-management system when possible. Kubernetes Secrets require careful RBAC, encryption at rest, and access control.

Security includes image scanning, non-root containers, network policies, RBAC, service accounts, admission controls, pod security standards, workload identity, and least privilege.

Reliability depends on readiness probes, liveness probes, resource requests, limits, graceful shutdown, replica counts, disruption budgets, node health, cluster upgrades, and dependency resilience.

Deployment should use immutable image tags or digests. Rolling updates are common, but blue/green, canary, or GitOps-based approaches may be appropriate.

Observability must be centralized. Logs, metrics, events, traces, health, and cluster state need to be visible outside individual pods and nodes.

Scaling includes pod autoscaling, node autoscaling, event-driven autoscaling, and dependency scaling. More pods can increase database, cache, and queue load.

Persistence is difficult in Kubernetes. Stateless workloads are the natural fit. Stateful workloads can run in Kubernetes, but databases often remain better as managed services unless the team has strong operational capability.

Cost includes cluster control plane charges, node compute, load balancers, storage, monitoring, image registries, idle capacity, and platform labor.

Local Kubernetes optimizes for learning. Production Kubernetes optimizes for platform consistency, security, automation, and operational control.

Layer 6 — Tradeoffs and Alternatives

Use Kubernetes when the organization needs a common platform for many containerized workloads, portable deployment APIs, self-healing behavior, rolling updates, service discovery, autoscaling, and strong platform teams.

Do not use Kubernetes just to run one ASP.NET Core API. Azure App Service, Azure Container Apps, AWS App Runner, AWS ECS, Google Cloud Run, and other managed services are often simpler.

Managed Kubernetes services include Azure Kubernetes Service, Amazon EKS, and Google Kubernetes Engine. They reduce control-plane burden but do not remove application, workload, security, networking, cost, or cluster governance responsibilities.

AKS Automatic and EKS Auto Mode reflect a current trend: managed Kubernetes providers are trying to reduce node and platform operations so teams can get Kubernetes benefits with fewer manual decisions. That helps, but the workload model still matters.

Alternatives include Docker Compose for local development, managed container apps for simple services, serverless functions for event-driven workloads, VMs for legacy control needs, and PaaS hosting for straightforward web apps.

Common overengineering mistakes:

- choosing Kubernetes before understanding containers;
- deploying stateful databases without operational readiness;
- ignoring resource requests and limits;
- using `latest` image tags;
- exposing services publicly by accident;
- treating probes as optional;
- granting cluster-admin permissions broadly;
- building a custom platform before a managed service is considered.

The state-of-the-art direction is managed Kubernetes plus platform engineering: secure defaults, GitOps, policy controls, workload identity, autoscaling, integrated observability, and paved paths for application teams.

Layer 7 — Interview Perspective

Interviewers use Kubernetes questions to test whether you understand orchestration boundaries.

Concepts commonly tested:

- why orchestration exists;
- cluster, control plane, and node;
- pod;
- deployment;
- service;
- labels and selectors;
- liveness and readiness probes;

- rolling updates;
- scaling;
- Kubernetes versus Docker;
- Kubernetes versus managed app platforms.

Representative questions:

- “What problem does Kubernetes solve?”
- “What is a pod?”
- “How is a deployment different from a pod?”
- “What does a service do?”
- “Why do labels and selectors matter?”
- “What is the difference between readiness and liveness?”
- “How does a rolling deployment work?”
- “When would you avoid Kubernetes?”

A strong answer names both value and cost:

“Kubernetes lets us declare desired state for containerized workloads and have controllers maintain it. A Deployment keeps replicas running and rolls out updates, while a Service gives stable network access to changing pods. It is powerful, but it adds platform, security, networking, observability, and cost responsibilities.”

Common misconceptions:

- “Kubernetes replaces Docker.”
- “A pod is the same thing as a container.”
- “Kubernetes makes the app scalable automatically.”
- “Services are always public load balancers.”
- “A liveness probe and readiness probe answer the same question.”
- “Managed Kubernetes means no operations.”
- “Every cloud-native app should start on Kubernetes.”

Small design scenario:

You have five ASP.NET Core APIs, three background workers, Redis, and SQL Server. The organization already has a platform team and wants consistent container deployment across environments.

A good Kubernetes design might run the APIs and workers as deployments, expose internal services through ClusterIP services, use ingress for public HTTP, connect to managed SQL Server outside the cluster, use managed Redis or a carefully operated cache, configure readiness and liveness probes, set resource requests, and deploy through a GitOps or CI/CD process.

The strong answer puts stateless workloads in Kubernetes first and treats stateful dependencies deliberately.

Hands-On Lab

Objective:

Read and apply a basic Kubernetes deployment and service for an ASP.NET Core container image.

Prerequisites:

- `kubectl` installed.
- Access to a local or development Kubernetes cluster.
- A container image from Chapter 10.

Steps:

1. Confirm the current context:

```
kubectl config current-context
```

2. Create `deployment.yaml` for an ASP.NET Core image.
3. Include:

```
replicas: 2
containerPort: 8080
readinessProbe
livenessProbe
resource requests and limits
```

4. Create `service.yaml` selecting the deployment's label.
5. Apply the manifests:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

6. Inspect resources:

```
kubectl get pods
kubectl get deployments
kubectl get services
```

7. View logs:

```
kubectl logs deployment/orders-api
```

8. Port-forward:

```
kubectl port-forward service/orders-api 8080:80
```

9. Call the health endpoint:

```
curl http://localhost:8080/health
```

Expected results:

- Kubernetes creates pods for the deployment.
- The service provides stable access to the pods.
- The health endpoint responds through port-forwarding.
- You can inspect rollout, pod state, and logs.

Validation commands:

```
kubectl get pods
kubectl get deployments
kubectl get services
kubectl rollout status deployment/orders-api
kubectl logs deployment/orders-api
```

Troubleshooting notes:

- If pods show `ImagePullBackOff`, check image name, tag, and registry access.
- If pods show `CrashLoopBackOff`, inspect logs and environment variables.
- If the service cannot be reached, check labels and selectors.
- If readiness never succeeds, check the health endpoint path and port.
- If you are on a shared cluster, use a development namespace and clean up resources when finished.

Knowledge Check

1. Why does container orchestration exist?
2. What is the relationship between a cluster, node, pod, and container?
3. Why is a deployment usually preferred over creating pods directly?
4. What problem does a service solve?
5. Why do labels and selectors matter?
6. How are liveness and readiness probes different?
7. What happens during a rolling deployment?
8. Why are resource requests and limits important?
9. When is a managed app service simpler than Kubernetes?
10. Why are databases harder to operate in Kubernetes than stateless APIs?

Summary

Kubernetes is a declarative orchestration platform for containerized workloads. It lets teams describe desired state and relies on controllers to keep actual state aligned.

The core vocabulary is manageable. A cluster contains nodes. Nodes run pods. Pods contain containers. Deployments manage replicated pods and rollouts. Services provide stable network access. Probes tell the platform whether a container is alive or ready. Scaling changes replica counts and resource capacity.

Kubernetes is powerful, but it is not the default answer for every .NET application. It fits best when an organization needs a consistent platform for many containerized workloads and has the operational maturity to secure, observe, upgrade, and govern it.

The next chapter builds on this by introducing advanced observability for cloud-native and distributed systems, including OpenTelemetry, tracing, dashboards, alerting, and multi-service monitoring.

Sources

- [Kubernetes concepts](#)
- [Azure Kubernetes Service documentation](#)
- [Core concepts for Azure Kubernetes Service](#)
- [What is Azure Kubernetes Service?](#)
- [Kubernetes concepts in Amazon EKS](#)
- [What is Amazon EKS?](#)
- [Amazon EKS features](#)

Further Reading

- [Kubernetes workloads](#)
- [Kubernetes services, load balancing, and networking](#)
- [Kubernetes configuration](#)
- [Azure Well-Architected Framework for AKS](#)
- [Amazon EKS best practices guides](#)

19

Advanced Observability

Chapter Purpose

Chapter 14 introduced observability as the replacement for server-centered diagnostics. Chapter 18 showed why Kubernetes makes that shift unavoidable: the application may run as many short-lived pods across multiple nodes, and the specific process that handled a failed request may be gone by the time someone investigates.

Advanced observability exists because modern systems fail across boundaries. An ASP.NET Core API may call another API, publish a message, query a database, read from a cache, and run inside Kubernetes behind cloud networking. A user sees one slow checkout request, but the cause may be a retry storm, a saturated connection pool, a bad deployment, an overloaded pod, a slow SQL query, or a third-party dependency.

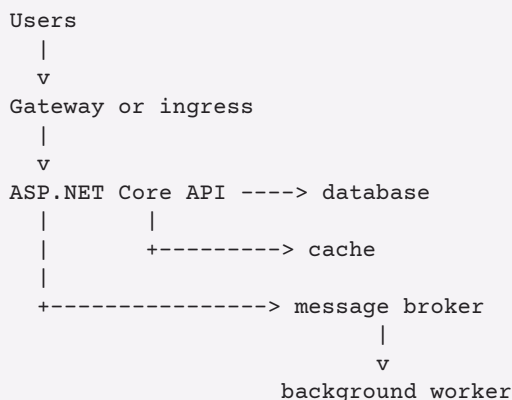
Older monitoring tools often focused on a machine or a process. Advanced observability focuses on a service, a request path, and the system behavior around it. The goal is not just to know that something is broken. The goal is to move from symptom to likely cause quickly enough to protect users.

OpenTelemetry is the central modern standard in this space. It grew out of the need for portable instrumentation across vendors and languages, combining ideas from earlier tracing and metrics projects into a common set of APIs, SDKs, semantic conventions, and protocols. It is not a monitoring product by itself. It is the instrumentation and transport layer that lets .NET applications emit telemetry to systems such as Azure Monitor, Grafana, Prometheus-compatible backends, Jaeger, Zipkin, Datadog, New Relic, Honeycomb, Elastic, and other observability platforms.

This chapter extends the earlier diagnostics chapter into production observability for cloud-native and distributed systems. You will learn how metrics, traces, logs, dashboards, alerts, and multi-service monitoring work together.

Where This Fits

Advanced observability sits across the whole running system.



```
Each component emits:
  logs    -> what happened?
  metrics -> how much, how fast, how many?
  traces  -> where did this request go?

Telemetry pipeline:
  application instrumentation
    |
    v
  OpenTelemetry SDK or automatic instrumentation
    |
    v
  collector or cloud agent
    |
    v
  observability backend
    |
    v
  dashboards, alerts, investigations
```

Chapter 19 belongs after Kubernetes because orchestration increases the number of moving parts. A single request can cross pods, services, nodes, queues, and managed dependencies. Local logs are no longer enough.

It also prepares for the AI chapters because AI-enabled systems often add expensive external calls, prompt pipelines, embeddings, vector stores, and background processing. Those systems require measurement and traceability from the beginning.

Connection to the Reader's Existing Model

The familiar model is a production support call for an IIS application.

You might check IIS logs, Windows Event Viewer, Performance Monitor counters, SQL Server activity, and application log files. If one server was misbehaving, you could log in, compare it with another server, recycle an application pool, or inspect the process.

Advanced observability keeps the same investigative intent but changes the unit of analysis.

Metrics are the modern equivalent of performance counters and SQL Server wait-stat trends. They show rates, latency, saturation, errors, and resource usage over time.

Traces are like a request timeline that follows the work across services. A trace can show that the API received a request, called another service, waited on SQL Server, retried a cache call, and returned an error.

Logs remain the event record, but they become more useful when correlated with trace identifiers, service names, deployment versions, user-safe business identifiers, and infrastructure metadata.

Dashboards are like operational consoles. They show whether the system is healthy and where to look next.

Alerts are like paging rules and service monitors, but the best alerts are tied to user impact rather than every low-level symptom.

The analogy breaks down when you expect one server to contain the answer. In a distributed system, the evidence is spread across services and collected after the fact. If telemetry is inconsistent, missing, too expensive, or uncorrelated, the team may know there was an incident without being able to explain it.

Layer 1 — Conceptual Model

Advanced observability is the practice of designing a system so production behavior can be understood across service, infrastructure, and dependency boundaries.

It solves these problems:

- connecting one user request across multiple services;

- separating application failures from infrastructure failures;
- detecting regressions after deployment;
- understanding latency and error distribution;
- finding dependency bottlenecks;
- supporting incident response;
- measuring service reliability;
- creating feedback for architecture and operations.

It does not solve these problems automatically:

- it does not replace good error handling;
- it does not make unreliable dependencies reliable;
- it does not remove the need for tests;
- it does not tell the team which reliability goals matter;
- it does not make every collected signal worth storing;
- it does not protect sensitive data unless telemetry is designed carefully.

The core model is:

```
Logs explain notable events.
Metrics quantify behavior over time.
Traces connect work across boundaries.
Dashboards organize signals for humans.
Alerts turn user-impacting signals into action.
```

The advanced part is correlation. Logs, metrics, and traces should not live as three unrelated piles of data. They should share enough context that an investigator can move from an error-rate spike to an example trace to the logs for the same operation.

Layer 2 — System Relationships

The application emits telemetry. ASP.NET Core, the .NET runtime, HTTP clients, database clients, messaging libraries, and application code can all emit signals.

Instrumentation is the code or agent behavior that creates telemetry. Manual instrumentation is written by the team. Automatic instrumentation comes from framework and library integrations.

The OpenTelemetry SDK receives telemetry from instrumentation and prepares it for export. In .NET, OpenTelemetry supports traces, metrics, and logs for supported .NET versions.

The OpenTelemetry Protocol, commonly shortened to OTLP, defines how telemetry is sent from applications or collectors to telemetry backends. This helps avoid hard-wiring application code to one vendor.

The collector is an optional but common pipeline component. It can receive telemetry, enrich it with Kubernetes or cloud metadata, filter it, sample it, batch it, and export it to one or more backends.

The backend stores, indexes, queries, visualizes, and alerts on telemetry. This may be a cloud-native platform, an open-source stack, or a commercial observability product.

Ownership boundaries matter:

- application teams own meaningful application instrumentation;
- platform teams often own collectors, agents, cluster metadata, and routing;
- operations or SRE teams own alert policies, incident processes, and service reliability goals;
- security teams influence retention, redaction, access control, and audit requirements.

Failure boundaries include missing instrumentation, broken context propagation, high-cardinality labels, excessive telemetry volume, sampling that hides rare failures, dashboards that do not match user journeys, alerts that page on symptoms instead of impact, and telemetry pipelines that fail during the same incident they should explain.

Layer 3 — Core Mechanics

OpenTelemetry organizes telemetry into signals.

A trace represents one logical operation, often one request. A trace contains spans. A span represents a timed unit of work, such as handling an HTTP request, calling another API, executing a SQL command, or processing a message.

```
Trace: checkout request
  Span: POST /checkout
    Span: validate cart
    Span: call pricing-api
    Span: write order to SQL
    Span: publish OrderSubmitted message
```

Metrics are numeric measurements over time. Common metric types include counters, gauges, and histograms. For production services, the most useful starting metrics usually describe request rate, error rate, duration, resource usage, dependency latency, queue depth, and saturation.

Logs are timestamped records of events. Logs are most useful when they include structured fields and correlation identifiers instead of unstructured text alone.

A small ASP.NET Core OpenTelemetry setup may look like this:

```
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddOpenTelemetry()
    .ConfigureResource(resource => resource
        .AddService(
            serviceName: "orders-api",
            serviceVersion: "2026.07.23"))
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddOtlpExporter())
    .WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddRuntimeInstrumentation()
        .AddOtlpExporter());

var app = builder.Build();

app.MapGet("/orders/{id:int}", async (int id, HttpClient http) =>
{
    var response = await http.GetAsync($"https://inventory-api/items/{id}");
    return Results.Ok(new { OrderId = id, InventoryStatus = response.StatusCode });
});

app.Run();
```

The service name is important. Without consistent service names, environment names, versions, and instance metadata, telemetry becomes difficult to group and compare.

Manual spans are useful when the framework cannot infer the business operation:

```
using System.Diagnostics;

private static readonly ActivitySource ActivitySource =
    new("Orders.Checkout");
```

```
using var activity = ActivitySource.StartActivity("ReserveInventory");
activity?.SetTag("order.id", orderId);
activity?.SetTag("inventory.items", itemCount);

await inventory.ReserveAsync(orderId, items);
```

Use tags carefully. A tag such as `order.id` can help with investigation, but high-cardinality or sensitive values can create cost, privacy, and performance problems. Do not put secrets, tokens, passwords, full payment information, or unnecessary personal data into telemetry.

Layer 4 — Developer Workflow

The developer workflow starts with a question, not a tool.

```
What should we be able to know in production?
  |
  v
Choose signals: logs, metrics, traces
  |
  v
Instrument application and dependencies
  |
  v
Run locally and verify telemetry
  |
  v
Deploy to test environment
  |
  v
Build dashboard and alert from real behavior
```

For a .NET service, start by adding OpenTelemetry packages for tracing, metrics, exporters, and the instrumentation libraries your app uses. Then verify that local requests produce traces and metrics before you deploy.

Local validation can use a development collector, console exporter, or a local observability stack. The exact backend is less important than proving that the service emits coherent telemetry with a service name, environment, version, and trace correlation.

Useful investigation flow:

```
Alert fires
  |
  v
Open user-impact dashboard
  |
  v
Identify affected service and time window
  |
  v
Find a representative trace
  |
  v
Inspect slow or failed span
  |
  v
Open correlated logs
  |
  v
Compare deployment, dependency, and infrastructure signals
```

For Kubernetes-hosted services, include pod, namespace, node, container, image, and deployment metadata when possible. That lets you answer questions such as “Did only the new version fail?”, “Was the problem isolated to one node?”, and “Did this begin during rollout?”

Developer-owned instrumentation should focus on business and dependency boundaries:

- order creation;
- payment authorization;
- inventory reservation;
- message publishing;
- background job processing;
- calls to external systems;
- retries, timeouts, and circuit-breaker decisions.

Avoid logging every line of code. The goal is a useful map of the system, not a transcript of the process.

Layer 5 — Production Usage

Production observability begins with reliability goals. A dashboard should show whether users are being served successfully, not merely whether servers are busy.

Common service-level indicators include:

- request success rate;
- request latency;
- availability;
- queue age or lag;
- background job success rate;
- dependency error rate;
- saturation of CPU, memory, threads, connections, and storage.

Dashboards should be organized by user journey and service ownership. A useful dashboard answers:

- Is the service healthy?
- Are users affected?
- What changed recently?
- Which dependency is slow or failing?
- Is the problem isolated to one version, region, node, or tenant?
- Where should the responder look next?

Alerting should be actionable. Page humans for user impact, fast-moving failure, data loss risk, security concerns, or exhausted capacity. Send lower priority signals to tickets, daily review, or dashboards.

Configuration should separate local, test, and production telemetry endpoints. Use environment variables, platform configuration, managed identity, or secret systems rather than hard-coded exporter credentials.

Security and privacy are production concerns. Telemetry may contain URLs, headers, user identifiers, exception details, and business data. Redact or drop sensitive values before export. Limit access to telemetry backends. Define retention based on business, legal, and cost needs.

Reliability of the telemetry pipeline matters. Use batching, retries, and backpressure-aware exporters. Avoid making request handling fail just because telemetry export is unavailable.

Scaling affects telemetry volume. More pods, more requests, and more dependencies produce more data. Sampling traces, reducing noisy logs, limiting high-cardinality attributes, and setting retention policies are normal production practices.

Persistence appears in two places. First, telemetry backends store large amounts of operational data. Second, traces and logs often point to persistent systems such as SQL Server, queues, and object storage. Observability should show both the application and the dependency behavior.

Cost is real. Logs are often the most expensive signal because they can be large and unbounded. Traces can become expensive under high traffic. Metrics are usually efficient, but high-cardinality labels can make them costly. Measure telemetry value the same way you measure application value.

Layer 6 — Tradeoffs and Alternatives

Use advanced observability when the system has multiple services, containers, background workers, queues, managed dependencies, cloud networking, or a production reliability obligation.

Do not start by instrumenting everything. Start with user journeys, service boundaries, dependencies, and the failure modes that would trigger an incident.

OpenTelemetry is a strong default because it is vendor-neutral and has broad ecosystem support. It reduces the risk of writing application code that only works with one backend.

Simpler alternatives include built-in cloud monitoring, ASP.NET Core logging, health checks, platform metrics, and a single logging provider. These may be enough for a small app or early prototype.

Open-source observability stacks commonly include Prometheus for metrics, Grafana for dashboards, Loki or Elasticsearch for logs, and Jaeger or Tempo for tracing. They can be powerful, but someone must operate, secure, scale, upgrade, and pay for the infrastructure.

Commercial and cloud-managed platforms reduce operational burden and often provide better integrated alerting, retention, search, correlation, anomaly detection, and support. They can also create cost and vendor-dependence concerns.

Common overengineering mistakes:

- collecting telemetry without defining investigation questions;
- paging on CPU instead of user impact;
- logging sensitive data;
- creating high-cardinality metric labels such as user IDs;
- sampling away the rare failures the team needs;
- building dashboards no one uses during incidents;
- treating OpenTelemetry as a backend instead of an instrumentation standard;
- ignoring telemetry cost until the bill arrives.

The state-of-the-art direction is correlated telemetry with OpenTelemetry, managed collection pipelines, service-level objectives, automatic instrumentation where appropriate, manual spans for business operations, trace-to-log correlation, policy-based redaction, and dashboards organized around user experience rather than server inventory.

Layer 7 — Interview Perspective

Interviewers use advanced observability questions to test whether you can operate distributed systems, not just write application code.

Concepts commonly tested:

- logs, metrics, and traces;
- spans and trace context;
- distributed tracing;
- OpenTelemetry;
- OTLP;
- collectors;

- dashboards;
- alerting;
- service-level indicators and objectives;
- cardinality;
- sampling;
- correlation;
- Kubernetes metadata;
- privacy and retention.

Representative questions:

- “How would you diagnose a slow request across three services?”
- “What is the difference between logs, metrics, and traces?”
- “What does OpenTelemetry provide?”
- “What is a span?”
- “Why does context propagation matter?”
- “What makes a dashboard useful?”
- “What should trigger a production alert?”
- “Why can high-cardinality metrics be dangerous?”
- “How do you avoid leaking sensitive data through telemetry?”

A strong answer connects signals:

“I would start with user-impact metrics to confirm scope and time window, then inspect traces from the affected service to find where latency or errors appear. From a failing span I would open correlated logs, compare the deployment version and Kubernetes metadata, and check dependency metrics. I would alert on symptoms users feel, not every internal warning.”

Common misconceptions:

- “Logs are enough if they are detailed.”
- “Distributed tracing replaces metrics.”
- “OpenTelemetry is a dashboard product.”
- “More telemetry is always better.”
- “Every error log should page someone.”
- “CPU alerts prove user impact.”
- “Sampling means observability is unreliable.”

Small design scenario:

You have an order system with an ASP.NET Core API, payment API, inventory API, SQL Server, Redis, and a background worker that processes messages. Users report intermittent slow checkout.

A good observability design would include request duration and error metrics for each service, distributed tracing across HTTP and messaging boundaries, structured logs correlated with trace IDs, dependency metrics for SQL Server and Redis, dashboard views for checkout success and latency, deployment version metadata, Kubernetes pod metadata, and alerts based on checkout error rate or latency objectives.

The strong answer does not begin by logging into a pod. It begins by finding the affected user journey and following correlated telemetry to the failing boundary.

Hands-On Lab

Objective:

Add basic OpenTelemetry tracing and metrics to an ASP.NET Core service and verify that one request can be followed through telemetry.

Prerequisites:

- A small ASP.NET Core API.
- .NET SDK installed.
- A local telemetry destination, development collector, or console exporter.
- Familiarity with the health endpoint and Docker workflow from earlier chapters.

Steps:

1. Add OpenTelemetry packages for ASP.NET Core instrumentation, HTTP client instrumentation, metrics, tracing, and an exporter suitable for your local environment.
2. Configure a service name and service version.
3. Enable ASP.NET Core and HTTP client tracing.
4. Enable ASP.NET Core, HTTP client, and runtime metrics.
5. Create one endpoint that calls another HTTP endpoint or a small local test service.
6. Send several requests to the endpoint.
7. Confirm that telemetry includes:

```
service name
request duration
HTTP status code
trace ID
span ID
dependency call span
runtime metrics
```

8. Add one structured log message during the request and confirm that it can be correlated with the trace when your backend supports log correlation.
9. Create a simple dashboard or query showing request rate, error rate, and latency for the service.

Expected results:

- Requests produce traces.
- The trace shows at least one server span and one dependency span.
- Metrics show request count and duration.
- Logs include useful structured values without sensitive data.
- The service name and version are visible in telemetry.

Validation commands:

```
dotnet run
curl http://localhost:5000/health
curl http://localhost:5000/orders/42
```

If using Kubernetes:

```
kubectl logs deployment/orders-api
kubectl get pods
kubectl port-forward service/orders-api 8080:80
curl http://localhost:8080/orders/42
```

Troubleshooting notes:

- If traces are missing, check instrumentation registration and exporter configuration.
- If service names are missing, check resource configuration.
- If dependency spans are missing, check whether the client library is instrumented.

- If metrics are noisy, review labels and cardinality.
- If telemetry export fails, check endpoint URLs, credentials, network access, and collector logs.
- If sensitive data appears, fix instrumentation before increasing retention or access.

Knowledge Check

1. Why do distributed systems need more than local logs?
2. How do logs, metrics, and traces answer different diagnostic questions?
3. What is a span, and how does it relate to a trace?
4. Why is OpenTelemetry useful even when a team has already chosen a backend?
5. What role does an OpenTelemetry Collector play?
6. Why does context propagation matter for multi-service monitoring?
7. What makes a production dashboard useful during an incident?
8. Why should alerts usually focus on user impact?
9. How can high-cardinality labels increase cost and reduce usefulness?
10. What telemetry should you add for a checkout flow that uses APIs, SQL Server, Redis, and a background worker?

Summary

Advanced observability is the production discipline of understanding a modern system across service and infrastructure boundaries. It builds on the basic signals from Chapter 14 and becomes essential once applications run as containers, Kubernetes workloads, distributed services, and cloud-managed dependencies.

The key shift is correlation. Logs describe events, metrics quantify behavior, and traces show how one operation moved through the system. When those signals share service names, versions, trace context, and infrastructure metadata, a team can move from a user-impact alert to a dashboard, then to an example trace, then to correlated logs and dependency metrics.

OpenTelemetry provides the modern vendor-neutral foundation for this work. It does not replace observability backends, dashboards, incident response, or engineering judgment. It gives .NET applications a standard way to emit telemetry that can survive changes in hosting platform and monitoring vendor.

The next chapter shifts from operating distributed systems to designing AI-enabled application architecture. The observability habits from this chapter will continue to matter because AI workflows add new dependencies, latency patterns, costs, and failure modes.

Sources

- OpenTelemetry .NET documentation
- OpenTelemetry specification overview
- OpenTelemetry specification status summary
- OpenTelemetry Metrics specification
- OpenTelemetry Logs specification
- OpenTelemetry Protocol specification
- Observability in .NET
- ASP.NET Core logging documentation

Further Reading

- [OpenTelemetry .NET GitHub repository](#)
- [OpenTelemetry Collector documentation](#)
- [OpenTelemetry semantic conventions](#)
- [Azure Monitor OpenTelemetry documentation](#)
- [Grafana OpenTelemetry documentation](#)

20

AI Application Architecture

Chapter Purpose

AI application architecture is the design discipline for adding language models, retrieval, tools, and human workflows to ordinary business software.

The technology exists because large language models changed what applications can do with unstructured input. A traditional application usually expects users to click a button, fill a form, call an API, or upload data in a known format. A model can accept natural language, summarize documents, draft text, classify messages, extract structure, answer questions, and decide when a business system should be queried.

That power creates a new architectural problem. A model is not a database, not a rules engine, not an API, and not a trusted employee. It is a probabilistic component that needs context, constraints, tools, evaluation, security, observability, and fallback behavior.

Modern AI application architecture grew quickly after the public release of chat-oriented large language models. Early applications often sent a prompt to a model and displayed the answer. Production systems moved beyond that pattern into retrieval-augmented generation, embeddings, vector databases, function calling, agent workflows, structured outputs, safety controls, and model-selection strategies.

The state of the art is changing rapidly, but the durable architecture is clear: keep business authority in your application and systems of record, use models for language and reasoning tasks, retrieve trusted context when needed, call tools through explicit contracts, observe behavior, evaluate quality, and control cost.

Where This Fits

AI architecture sits beside the application layer, not above the whole system as magic.

```
User or business process
|
v
ASP.NET Core application
|
+-- authorization and business rules
|
+-- AI orchestration layer
|   |
|   +-- prompt and instructions
|   +-- model selection
|   +-- retrieved context
|   +-- tool/function contracts
|   +-- response validation
|
+-- systems of record
```

```

|      |
|      +-- SQL Server
|      +-- document storage
|      +-- queues
|      +-- internal APIs
|
+-- vector index
|
v
Observability, evaluation, security, and cost controls

```

This chapter begins Part VI. The earlier chapters gave you the production platform: APIs, data access, security, testing, containers, cloud hosting, distributed systems, CI/CD, Kubernetes, and observability. AI applications use all of those foundations.

Chapter 21 will show .NET implementation options such as `Microsoft.Extensions.AI`, Semantic Kernel, and AI-powered APIs. This chapter answers the design questions that should come first: what should the model do, what context does it need, what tools may it call, how is the answer checked, and when should the system avoid AI entirely?

Connection to the Reader's Existing Model

The closest familiar analogy is a business application that calls a specialized external service.

You may have integrated payment gateways, address validation APIs, credit checks, document-generation services, search indexes, reporting databases, or rules engines. Those systems extend the application, but they do not own the business. Your application still controls authorization, validation, transactions, audit, and persistence.

An AI model should be treated the same way.

An LLM is like a language-capable service. It can transform, summarize, classify, draft, compare, and reason over text, images, or other supported inputs depending on the model. It should not become the system of record.

Embeddings are like search keys created from meaning rather than exact words. Traditional SQL indexes help find rows by values. Full-text indexes help find documents by words. Vector indexes help find content that is semantically similar.

Retrieval-augmented generation is like giving an employee the right folder before asking a question. The model is not expected to remember your company policy. The application retrieves relevant policy sections and asks the model to answer using that material.

Function calling is like letting a workflow engine call approved services through known interfaces. The model can request a tool call, but the application decides whether to execute it, with what identity, and how to validate the result.

Agents are like automated assistants that can take multiple steps toward a goal. The useful analogy is a supervised workflow, not an autonomous employee with unlimited access.

The analogy breaks down if you expect deterministic behavior. A stored procedure returns the same result for the same committed data. A language model may produce different wording, may misunderstand ambiguous instructions, and may generate confident but incorrect text. The architecture must expect that.

Layer 1 — Conceptual Model

An AI application uses a model as one component inside a larger software system.

It solves these problems:

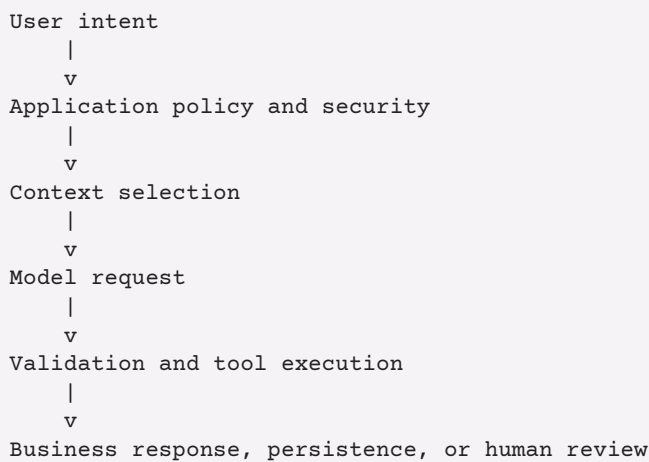
- understanding natural-language user input;
- summarizing long or messy content;
- drafting human-readable text;
- extracting structured data from unstructured content;
- classifying messages, documents, or records;

- answering questions from trusted sources;
- helping users navigate complex workflows;
- calling approved tools based on user intent.

It does not solve these problems automatically:

- it does not make unreliable data reliable;
- it does not replace authorization;
- it does not replace deterministic business rules;
- it does not remove the need for tests and evaluations;
- it does not guarantee truth;
- it does not protect secrets by default;
- it does not make every workflow cheaper or faster.

The conceptual model is:



The model is powerful, but the application remains responsible for what is allowed, what is trusted, what is stored, and what happens next.

Layer 2 — System Relationships

AI architecture has several major parts.

The user or calling process provides intent. That intent may be a chat message, support ticket, document, email, uploaded file, voice transcript, or internal workflow event.

The application layer authenticates the caller, authorizes the operation, loads tenant and user context, applies business rules, and decides whether AI is appropriate.

The prompt contains instructions and input for the model. A prompt is not just the user's text. It may include system instructions, developer instructions, conversation history, retrieved context, output format requirements, and tool definitions.

The model performs language, reasoning, generation, extraction, or tool-choice work. Different models have different strengths, costs, latencies, context limits, modalities, and operational characteristics.

The retrieval pipeline finds relevant external information. It may query SQL Server, a search engine, document storage, a vector database, or internal APIs.

The embedding model converts text or other supported content into vectors that can be compared by semantic similarity.

The vector database or vector index stores embeddings with document chunks, metadata, source identifiers, permissions, and freshness information.

Function calling gives the model a set of structured tool contracts. The model can request a function with arguments. The application validates the request, executes approved code or APIs, and returns results to the model or the user.

An agent workflow coordinates multiple model calls, retrieval steps, tool calls, validations, and decisions. Agents can be useful, but they expand the failure surface because the system may take several actions before returning.

Observability and evaluation measure behavior. Production AI systems should track latency, cost, token usage, tool calls, retrieval quality, failures, safety blocks, user feedback, and outcome quality.

Failure boundaries include prompt injection, stale retrieved context, incorrect model output, missing citations, tool misuse, permission leakage, model outages, rate limits, high latency, runaway cost, weak evaluation data, and user overtrust.

Layer 3 — Core Mechanics

An LLM is a model trained to predict and generate language-like output from input context. In application architecture, the important point is not the training math. The important point is that the model produces likely text or structured output based on its training, instructions, and supplied context.

A token is a unit of text used by the model. Cost, latency, and context limits are usually measured in tokens. Long documents, long chat histories, and large retrieved context increase token usage.

An embedding is a numeric representation of meaning. Similar pieces of content produce vectors that are near each other in vector space. That enables semantic search.

Retrieval-augmented generation, usually called RAG, uses retrieval to ground a model response in external content:

```
User asks question
      |
      v
Application rewrites or analyzes query
      |
      v
Search retrieves relevant chunks
      |
      v
Application builds prompt with sources
      |
      v
Model answers using retrieved context
      |
      v
Application validates, cites, logs, and returns
```

A document chunk is a smaller section of a larger source. Chunking exists because models and search systems work better when they receive focused context instead of entire documents.

Metadata describes the chunk: source, title, version, tenant, security label, date, author, product, or business category. Metadata lets the application filter before search and explain where an answer came from.

Function calling uses structured contracts. A model might be given a tool such as:

```
{
  "name": "get_order_status",
  "description": "Get the current status of an order by order ID.",
  "parameters": {
    "type": "object",
    "properties": {
      "orderId": {
        "type": "integer",
        "description": "The internal order identifier."
      }
    },
    "required": ["orderId"]
  }
}
```

The model does not directly query the database. It requests `get_order_status` with arguments. The application validates the arguments, checks authorization, calls the real service, and decides what result may be returned.

Structured outputs constrain model responses to a schema. They are useful when the application needs JSON that can be validated and processed.

Model selection is the practice of choosing a model based on task. A good architecture may use one model for complex reasoning, another for fast classification, another for embeddings, and another for multimodal input.

Layer 4 — Developer Workflow

The developer workflow should begin with a use case, not a model.

```
Define user problem
  |
  v
Decide whether AI is appropriate
  |
  v
Identify trusted data and allowed actions
  |
  v
Design prompt, retrieval, and tool boundaries
  |
  v
Build smallest evaluation set
  |
  v
Prototype
  |
  v
Measure quality, latency, cost, and safety
  |
  v
Harden for production
```

Start with narrow tasks:

- summarize a support ticket;
- classify an incoming email;
- draft a customer-safe response;
- answer questions from a known policy repository;
- extract fields from a contract;
- choose from a small set of approved workflow actions.

For RAG, the workflow is:

1. Identify trusted source documents.
2. Split documents into chunks.
3. Generate embeddings for chunks.
4. Store vectors, text, source identifiers, and metadata.
5. At query time, retrieve relevant chunks.
6. Build a prompt that includes the user question and retrieved context.
7. Ask the model for an answer with citations or source references.
8. Validate the answer and log enough telemetry to improve the system.

For function calling, the workflow is:

1. Define the business operation as a narrow tool.

2. Describe the tool clearly.
3. Validate model-proposed arguments.
4. Apply authorization using the user's identity.
5. Execute the real API or service.
6. Return a limited result to the model.
7. Audit the operation.

For agents, the workflow adds planning and limits:

1. Define the goal.
2. Define allowed tools.
3. Set step limits, timeouts, and budgets.
4. Require confirmation for important actions.
5. Store intermediate state.
6. Observe every step.
7. Provide human review where consequences are significant.

Do not wait until production to evaluate. Create a small test set of realistic inputs, expected outcomes, unacceptable outputs, and edge cases. AI tests are often less like exact unit tests and more like regression checks for behavior.

Layer 5 — Production Usage

Production AI systems need the same engineering discipline as any other distributed system, plus a few AI-specific controls. Configuration includes model names, deployment names, API endpoints, token budgets, timeouts, retry policies, sampling settings, retrieval settings, chunk sizes, similarity thresholds, and feature flags.

Secrets include API keys, managed identity configuration, search credentials, storage credentials, and tool access credentials. Do not place secrets in prompts, logs, retrieved context, or evaluation datasets.

Security includes authentication, authorization, data minimization, prompt injection defense, output validation, tool allowlists, tenant isolation, content filtering, audit logging, and human approval for consequential actions.

Reliability includes retries with backoff, circuit breakers, fallback responses, cached non-sensitive results, graceful degradation, model-provider failover where appropriate, queue-based background processing, and clear user messages when AI is unavailable.

Deployment should separate prompt changes, retrieval changes, model changes, and application code changes. A prompt is production behavior. Treat important prompt and tool-contract changes with review, versioning, testing, and rollback.

Observability should include latency, token usage, cost, model name, prompt version, retrieved document IDs, tool calls, validation failures, content filter results, user feedback, and downstream business outcome. Avoid logging full sensitive prompts and responses unless there is a controlled retention and privacy policy.

Scaling depends on rate limits, token volume, model latency, vector-search capacity, document-ingestion throughput, and tool-call throughput. More app instances do not automatically create more model capacity.

Persistence includes conversation history, retrieved source references, document indexes, embeddings, evaluation results, and audit records. Store the minimum needed for the business purpose, and apply retention rules.

Cost includes model input tokens, output tokens, embeddings, vector storage, search operations, tool calls, document ingestion, observability, human review, and engineering time. Cost should be visible before a feature is broadly enabled.

Local development can use small examples, local test documents, mocked tool calls, and development model deployments. Production needs real permissions, rate limits, redaction, monitoring, evaluations, fallback behavior, and audit.

Layer 6 — Tradeoffs and Alternatives

Use AI when the problem involves language, ambiguity, summarization, classification, extraction, semantic search, reasoning over documents, or a natural-language interface over existing systems.

Do not use AI when deterministic code, SQL, full-text search, a rules engine, a form, a report, or a simple workflow would be cheaper, clearer, faster, and more reliable.

RAG is useful when answers should come from private, current, or domain-specific documents. It is not necessary when the task is transformation, classification, or extraction from the user's provided input.

Fine-tuning may be useful for specialized style, classification, or repeated patterns, but it is not the first answer for adding private facts. Private facts usually belong in retrieval or tools, not in model weights.

Agents are useful when a task requires multiple steps and tool use. They are not appropriate when the workflow must be deterministic, fully auditable, or high consequence without approval gates.

Function calling is useful when the model needs to interact with real systems. It should be narrow, validated, authorized, and observable.

Open-source models can provide more deployment control, data locality, and cost predictability at scale. They also require model hosting, GPU capacity, serving infrastructure, evaluation, upgrades, and security patching.

Managed model APIs reduce hosting burden and provide fast access to advanced capabilities. They create dependency on provider availability, pricing, regions, rate limits, data handling terms, and model lifecycle changes.

Common overengineering mistakes:

- adding chat when a button would be better;
- using AI as the source of truth;
- skipping authorization on retrieved documents;
- giving an agent broad tool access;
- relying on prompt wording instead of validation;
- storing sensitive prompts without a policy;
- ignoring token cost and latency;
- changing models without evaluation;
- treating a demo-quality RAG pipeline as production-ready.

The state-of-the-art direction is task-specific model selection, grounded RAG, structured outputs, explicit tool contracts, agent workflows with guardrails, automated evaluation, privacy-aware telemetry, and human review for actions that affect money, legal status, safety, security, or customer trust.

Layer 7 — Interview Perspective

Interviewers use AI architecture questions to test whether you can integrate models responsibly into real applications.

Concepts commonly tested:

- LLMs;
- tokens and context windows;
- embeddings;
- vector databases;
- RAG;
- chunking;
- metadata filtering;
- prompt injection;
- function calling;
- structured outputs;
- agents;

- model selection;
- evaluation;
- latency, cost, and safety tradeoffs.

Representative questions:

- “When would you use RAG instead of fine-tuning?”
- “What is an embedding?”
- “What does a vector database store?”
- “How would you let an AI assistant check order status?”
- “How do you prevent a model from exposing documents a user cannot access?”
- “What is prompt injection?”
- “When should an agent ask for human confirmation?”
- “How would you choose between a fast model and a more capable model?”
- “What would you monitor in production?”

A strong answer keeps authority in the application:

“I would use the model for language understanding and generation, but keep authorization, business rules, tool execution, and persistence in the application. For company knowledge I would use RAG with permission-filtered retrieval. For actions I would expose narrow function calls, validate arguments, audit execution, and require approval for consequential changes.”

Common misconceptions:

- “The model remembers our documents after we send them once.”
- “RAG guarantees correct answers.”
- “A vector database replaces SQL Server.”
- “Prompt instructions are enough security.”
- “An agent should have access to every internal API.”
- “The most capable model is always the best model.”
- “If the model returns JSON, the application can trust it.”

Small design scenario:

You need to add an assistant to an internal order-management application. It should answer questions about order policies, summarize order history, and let authorized support staff check shipment status.

A good design would use RAG for policy documents, SQL Server or internal APIs for order history, function calling for shipment status, authorization filters before retrieval and tool execution, structured outputs for machine-readable results, audit logs for tool calls, prompt and model versioning, user feedback, and dashboards for latency, cost, errors, and answer quality.

The strong answer avoids making the model the database, the security boundary, or the final authority for consequential actions.

Hands-On Lab

Objective:

Design an AI architecture for a support assistant before writing implementation code.

Prerequisites:

- A basic understanding of ASP.NET Core APIs.
- Familiarity with SQL Server-backed business applications.
- Familiarity with observability concepts from Chapter 19.

Steps:

1. Choose a narrow support scenario, such as answering return-policy questions and checking order status.
2. Identify the trusted sources:

```
return policy documents
order database
shipment API
support knowledge base
```

3. Classify each source:

```
retrieved context
system of record
tool/function
human-only reference
```

4. Draw the request flow from user message to final answer.
5. Decide which parts use RAG and which parts use function calling.
6. Define at least three tool boundaries, such as:

```
get_order_summary(orderId)
get_shipment_status(orderId)
create_support_note(orderId, note)
```

7. Mark which tools require human confirmation.
8. Define what telemetry must be captured:

```
model name
prompt version
retrieved document IDs
tool calls
latency
token usage
validation failures
user feedback
```

9. Define five evaluation cases, including one prompt-injection attempt and one unauthorized-document attempt.
10. Decide when the assistant should refuse, ask for clarification, or hand off to a human.

Expected results:

- A clear architecture diagram.
- A list of trusted sources.
- A separation between retrieval, tools, and systems of record.
- Tool contracts with authorization and confirmation rules.
- An evaluation checklist.
- A telemetry plan.

Validation commands:

No command is required for this architecture lab. If you want to capture the design in the repository, create a Markdown design note in a scratch location or future sample folder after the chapter exercise is complete.

Troubleshooting notes:

- If the assistant needs private facts, use retrieval or tools.
- If the assistant needs to change business data, require explicit tool contracts and authorization.
- If users could be harmed by a wrong answer, add human review or refusal behavior.
- If cost is unclear, estimate token volume before implementation.
- If quality is subjective, create evaluation examples before choosing a model.

Knowledge Check

1. Why should an AI model not be treated as the system of record?
2. How are embeddings different from SQL indexes and full-text indexes?
3. What problem does RAG solve?
4. Why is metadata important in a vector index?
5. How does function calling differ from letting a model directly access a database?
6. When is an agent useful, and when is it risky?
7. Why is model selection an architectural decision rather than only a coding detail?
8. What should be evaluated before changing a prompt or model?
9. How can prompt injection affect a RAG application?
10. What production telemetry would you collect for an AI assistant?

Summary

AI application architecture is the careful integration of language models into ordinary business systems. The application still owns identity, authorization, business rules, persistence, transactions, audit, and user experience. The model contributes language understanding, generation, summarization, classification, extraction, and tool-choice capability.

The main architectural building blocks are LLMs, prompts, embeddings, vector indexes, RAG pipelines, function calls, structured outputs, agents, evaluations, and observability. Each has a specific job. RAG supplies trusted context. Function calling gives the model controlled access to actions. Agents coordinate multiple steps. Evaluation and telemetry keep the system honest after the demo.

The most important design habit is boundary keeping. Do not let the model become the database, the security layer, the workflow engine, or the final authority for consequential actions. Use it where language and reasoning help, then surround it with the same engineering discipline used for the rest of the modern .NET system.

Chapter 21 turns this architecture into .NET implementation patterns using `Microsoft.Extensions.AI`, Semantic Kernel, ASP.NET Core APIs, and integration with existing business applications.

Sources

- [OpenAI API developer quickstart](#)
- [OpenAI function calling guide](#)
- [OpenAI API data controls](#)
- [OpenAI model documentation](#)
- [Microsoft guidance on augmenting LLMs with RAG or fine-tuning](#)
- [Azure Architecture Center: RAG generate embeddings phase](#)
- [Semantic Kernel documentation](#)

Further Reading

- [OpenAI text generation documentation](#)
- [OpenAI embeddings documentation](#)
- [OpenAI structured outputs documentation](#)
- [OpenAI agents documentation](#)
- [Azure Architecture Center RAG guidance](#)

- Microsoft Semantic Kernel concepts

21

AI Development with .NET

Chapter Purpose

Chapter 20 explained the architecture of AI applications: keep business authority in the application, use models for language work, retrieve trusted context, expose approved tools, validate outputs, observe behavior, and control cost.

This chapter turns that architecture into .NET development practice.

.NET did not become an AI platform by replacing ASP.NET Core, dependency injection, logging, configuration, background services, or testing. It became a practical AI application platform by adding AI abstractions and orchestration libraries that fit into those existing patterns.

The problem this chapter resolves is tool choice. A modern .NET developer can call a model API directly, use `Microsoft.Extensions.AI` abstractions, use Semantic Kernel, use an agent framework, or call cloud-specific SDKs. Those choices are not interchangeable. They sit at different levels.

`Microsoft.Extensions.AI` exists to give .NET applications common abstractions for generative AI components such as chat clients and embedding generators. It follows familiar .NET patterns: dependency injection, middleware, telemetry, caching, and testable interfaces.

Semantic Kernel exists at a higher orchestration level. It helps organize prompts, plugins, services, memory, filters, and agent-style workflows. It is useful when an application needs more than one model call and a few lines of provider code.

The state-of-the-art direction in .NET AI development is provider-neutral abstractions for basic integration, orchestration frameworks for multi-step workflows, explicit tool contracts, RAG pipelines, OpenTelemetry, evaluation, and production security controls.

Where This Fits

AI development with .NET sits inside the application layer.

```
ASP.NET Core API or worker
|
+-- Controllers, endpoints, or jobs
|
+-- Application services
|   |
|   +-- IChatClient
|   +-- IEmbeddingGenerator
|   +-- prompt templates
|   +-- tool/function services
|
+-- Semantic Kernel or agent workflow
|
+-- SQL Server, search, storage, queues, APIs
```

|
v
Model provider, Azure OpenAI, OpenAI, local model, or other AI service

The model provider remains outside the application. Your .NET code owns identity, authorization, validation, tool execution, persistence, telemetry, and user experience.

This chapter follows AI architecture because implementation choices should be driven by architecture. It prepares for the architecture chapters that follow because AI features still live inside monoliths, modular monoliths, microservices, background workers, and distributed systems.

Connection to the Reader's Existing Model

The familiar model is dependency injection around an external service.

In a traditional .NET application, you might define an interface for payment processing, inject an implementation, configure credentials, log calls, retry transient failures, mock the interface in tests, and keep business rules in application services.

`Microsoft.Extensions.AI` brings that same habit to AI. Instead of scattering provider-specific model calls throughout controllers, code can depend on an abstraction such as `IClient`.

Semantic Kernel is closer to a workflow and integration layer. If `IClient` is like an HTTP client abstraction for model calls, Semantic Kernel is closer to a small orchestration runtime that knows about prompts, plugins, tool calls, model services, and conversations.

Native plugins in Semantic Kernel feel familiar if you have exposed C# methods through Web API controllers, background jobs, or service classes. You write ordinary C# methods, describe them clearly, and let the AI orchestration layer make them available as callable capabilities.

The analogy breaks down if you treat the model like a deterministic service. An address validation API either returns a valid result or an error. A model may return plausible but wrong output, ask to call the wrong tool, or require a clarifying question. The .NET code around the model must handle uncertainty.

Layer 1 — Conceptual Model

AI development with .NET is the practice of adding model-backed capabilities to .NET applications using familiar application patterns.

It solves these problems:

- calling AI providers from ASP.NET Core and worker services;
- isolating provider-specific SDK code;
- testing AI-facing application logic;
- generating embeddings for retrieval;
- adding prompts and structured outputs;
- exposing C# functions as model-callable tools;
- building RAG workflows;
- integrating AI into existing business applications.

It does not solve these problems automatically:

- it does not make model output correct;
- it does not remove the need for authorization;
- it does not choose the right architecture;
- it does not guarantee low cost or low latency;
- it does not make private data safe by default;
- it does not replace evaluations;

- it does not make every agent workflow production-ready.

The conceptual model is:

```
.NET application code
    |
    v
AI abstraction or orchestration library
    |
    v
Provider SDK or service endpoint
    |
    v
Model, embedding model, or agent runtime
```

Use the lowest layer that solves the problem. A simple classification endpoint may only need `IChatClient`. A multi-step support assistant with tools, retrieval, and conversation state may benefit from Semantic Kernel or an agent framework.

Layer 2 — System Relationships

The ASP.NET Core endpoint or background service receives the request. It authenticates the caller, authorizes the operation, loads application data, and calls an application service.

The application service owns the use case. It decides which prompt to use, which context to retrieve, which model capability is needed, what tools are allowed, and how to validate the result.

`IChatClient` represents a chat-capable model interaction. Application code can depend on this interface instead of a concrete provider client.

`IEmbeddingGenerator` represents embedding generation. It lets application code create vectors for text or other supported input and store them in a search or vector system.

Middleware in the AI client pipeline can add cross-cutting concerns such as logging, telemetry, caching, function invocation, rate limiting, or custom policy.

Semantic Kernel organizes services and plugins. A kernel can hold AI service configuration, prompt execution, plugin functions, filters, and workflow behavior.

A plugin exposes capabilities to the model. A plugin might wrap an order service, shipment API, policy lookup, calendar operation, or document tool.

The model provider executes the model request. It may be OpenAI, Azure OpenAI, Azure AI Foundry, a local model through Ollama, GitHub Models, Amazon Bedrock, Google Gemini, or another provider supported by the application's libraries.

Failure boundaries include provider outages, rate limits, invalid credentials, model changes, prompt regressions, malformed structured output, unsafe tool arguments, unauthorized context retrieval, missing embeddings, and telemetry blind spots.

Layer 3 — Core Mechanics

At the lowest useful level, application code asks a chat client for a response.

```
public sealed class SupportDraftService
{
    private readonly IChatClient _chatClient;

    public SupportDraftService(IChatClient chatClient)
    {
        _chatClient = chatClient;
    }

    public async Task<string> DraftReplyAsync(
        string customerMessage,
```

```

        CancellationToken cancellationToken)
    {
        var response = await _chatClient.GetResponseAsync(
            $"""
            Draft a concise support reply.

            Customer message:
            {customerMessage}
            """,
            cancellationToken: cancellationToken);

        return response.Text;
    }
}

```

The example is intentionally small. In production, the prompt would usually be versioned, the input would be validated, sensitive data would be minimized, and the output would be checked before returning it to a user.

For embeddings, application code sends text to an embedding generator and stores the resulting vector with source metadata:

```

public sealed class PolicyIndexingService
{
    private readonly IEmbeddingGenerator<string, Embedding<float>> _embeddings;
    private readonly IPolicyVectorStore _store;

    public PolicyIndexingService(
        IEmbeddingGenerator<string, Embedding<float>> embeddings,
        IPolicyVectorStore store)
    {
        _embeddings = embeddings;
        _store = store;
    }

    public async Task IndexChunkAsync(
        PolicyChunk chunk,
        CancellationToken cancellationToken)
    {
        var vector = await _embeddings.GenerateAsync(
            chunk.Text,
            cancellationToken: cancellationToken);

        await _store.UpsertAsync(
            chunk.Id,
            chunk.Text,
            vector.Vector,
            chunk.Metadata,
            cancellationToken);
    }
}

```

A RAG endpoint combines retrieval and chat:

```

API receives question
|
v
Authorize user
|
v
Retrieve allowed policy chunks
|
v
Build prompt with retrieved context
|

```

```

    v
Ask model for answer
|
v
Return answer with source references

```

Semantic Kernel adds an orchestration layer. A simplified native plugin might look like this:

```

public sealed class OrderPlugin
{
    private readonly IOrderService _orders;

    public OrderPlugin(IOrderService orders)
    {
        _orders = orders;
    }

    [KernelFunction]
    [Description("Gets the shipment status for an order the user is allowed to view.")]
    public Task<string> GetShipmentStatusAsync(
        [Description("The internal order ID.")] int orderId,
        CancellationToken cancellationToken)
    {
        return _orders.GetShipmentStatusAsync(orderId, cancellationToken);
    }
}

```

The method is still ordinary C#. The important production question is whether the underlying service checks authorization, validates the order ID, audits access, and returns only data the model should see.

Layer 4 — Developer Workflow

The .NET AI workflow begins with the smallest useful slice.

```

Choose one use case
|
v
Create an application service
|
v
Inject IChatClient or IEmbeddingGenerator
|
v
Add prompt and validation
|
v
Add retrieval or tools only when needed
|
v
Write tests and evaluation cases
|
v
Add telemetry and production controls

```

For a simple AI-powered API:

1. Create an ASP.NET Core project.
2. Add the AI client packages required by the selected provider.
3. Register the chat client in dependency injection.
4. Create an application service that depends on `IChatClient`.

5. Add an endpoint that calls the service.
6. Validate input length and content.
7. Add logging and telemetry.
8. Test normal, edge, and unsafe inputs.

For a RAG feature:

1. Build an ingestion path for trusted documents.
2. Chunk the documents.
3. Generate embeddings.
4. Store text, vectors, source IDs, and metadata.
5. Query the vector store at request time.
6. Filter results by tenant and authorization.
7. Build a prompt using retrieved context.
8. Return source references with the answer.

For a Semantic Kernel workflow:

1. Register model services.
2. Create or load prompt templates.
3. Add native plugins for approved business actions.
4. Add filters or middleware for logging, authorization, and validation.
5. Invoke the workflow from an application service.
6. Record tool calls, prompt version, model name, and outcome.

Useful commands depend on the chosen packages, but the development rhythm is ordinary .NET:

```
dotnet new webapi -n SupportAssistant
dotnet add package Microsoft.Extensions.AI
dotnet build
dotnet test
dotnet run
```

When using templates or samples, treat them as starting points. Before using them in production, review identity, authorization, data retention, model configuration, logging, cost, and deployment settings.

Layer 5 — Production Usage

Production .NET AI development should feel like production .NET development: configuration, security, reliability, observability, testing, deployment, and cost all matter.

Configuration includes provider endpoints, deployment names, model names, temperature or reasoning settings, token limits, timeouts, retry policy, prompt versions, retrieval settings, embedding model choices, and feature flags.

Secrets include API keys, managed identity settings, search credentials, storage connection strings, and tool credentials. Prefer managed identity or secure secret stores when available. Never put secrets in prompts.

Security includes normal ASP.NET Core authentication and authorization, plus AI-specific controls: prompt-injection defenses, tool allowlists, output validation, tenant-filtered retrieval, redaction, content filtering, and audit logging.

Reliability includes cancellation tokens, timeouts, retries with backoff, circuit breakers, graceful fallback responses, queue-based background work, provider health checks, and safe behavior when a model or vector store is unavailable.

Deployment should version prompts, tool contracts, retrieval pipelines, and model choices. A model change can alter behavior even when application code is unchanged.

Observability should use the Chapter 19 habits. Track request latency, model latency, token usage, cost, model name, prompt version, retrieved document IDs, tool calls, validation failures, safety decisions, and user feedback.

Scaling depends on both .NET application capacity and AI-provider capacity. Increasing pods or app instances does not remove model rate limits. Use queues, caching, throttling, and backpressure for expensive or bursty workloads.

Persistence includes conversation history, embeddings, source documents, evaluation cases, audit records, prompt versions, and user feedback. Store only what the business needs and apply retention rules.

Cost management includes token budgets, embedding costs, vector storage, search operations, tool-call costs, observability storage, and human review. Expose cost signals before the feature becomes widely used.

Local development may use test providers, small document sets, console exporters, fake tool implementations, and narrow evaluation cases. Production requires real identity, private networking where appropriate, secret management, monitoring, evaluation, rollout controls, and incident response.

Layer 6 — Tradeoffs and Alternatives

Use `Microsoft.Extensions.AI` when you want provider-neutral abstractions, dependency injection, testability, middleware, telemetry, caching, or the ability to swap concrete model providers without rewriting application services.

Use a provider SDK directly when the feature depends heavily on provider-specific capabilities and the portability layer would hide important behavior.

Use Semantic Kernel when the application needs prompt orchestration, plugins, function calling, multi-step workflows, memory concepts, filters, or agent-like composition.

Use a simpler direct call when the feature is a narrow one-step task and the extra orchestration layer would add complexity without value.

Use an agent framework when the workflow requires multi-step planning, tool use, conversation state, collaboration between agents, or reusable agent definitions. Avoid agents for deterministic workflows that should be ordinary C# code.

Use RAG when answers need private or current business context. Do not use RAG for every AI feature; classification, extraction, drafting, and summarization may only need the user's input.

Open-source or local model options can reduce data-sharing concerns and offer deployment control. They also require hosting, GPU capacity, model management, serving infrastructure, and evaluation.

Managed providers offer fast access to capable models and operational support. They introduce dependency on provider availability, pricing, rate limits, regions, data-handling terms, and model lifecycle.

Common overengineering mistakes:

- wrapping every model call in a custom framework before one feature exists;
- choosing an agent for a one-step task;
- exposing broad plugins without authorization checks;
- putting business rules in prompts instead of C#;
- skipping evaluation because the demo looked good;
- logging sensitive prompts and responses;
- assuming provider-neutral abstractions remove all provider differences;
- using production documents in local experiments without a data policy.

The state-of-the-art direction is layered development: provider-neutral interfaces for common calls, orchestration frameworks for workflows, explicit tools, RAG with permission-aware retrieval, OpenTelemetry, automated evaluations, secure deployment, and human approval for consequential actions.

Layer 7 — Interview Perspective

Interviewers use .NET AI questions to test whether you can build model-backed features without abandoning normal engineering discipline.

Concepts commonly tested:

- `IChatClient`;
- `IEmbeddingGenerator`;
- dependency injection;
- provider SDKs;
- Semantic Kernel;
- plugins;
- function calling;
- RAG implementation;
- prompt versioning;
- evaluation;
- telemetry;
- authorization around tools and retrieval;
- cost and latency controls.

Representative questions:

- “Why would you use `Microsoft.Extensions.AI`?”
- “When would you call the provider SDK directly?”
- “What problem does Semantic Kernel solve?”
- “How do you expose C# code as a tool safely?”
- “How would you build a RAG endpoint in ASP.NET Core?”
- “How do you test AI behavior?”
- “What should be logged or measured for an AI feature?”
- “How do you keep a model from seeing documents the user cannot access?”

A strong answer separates layers:

“I would keep the ASP.NET Core application responsible for authentication, authorization, validation, and persistence. For simple model calls I would depend on `IChatClient` so the application service is testable and less tied to one provider. For multi-step workflows I might use Semantic Kernel with narrow plugins, but every tool would still enforce authorization and audit access.”

Common misconceptions:

- “`Microsoft.Extensions.AI` is a model provider.”
- “Semantic Kernel replaces ASP.NET Core services.”
- “A plugin method is safe because it is only called by the model.”
- “RAG means the model has learned our documents.”
- “A prompt change is not a production change.”
- “Mocks are enough to prove answer quality.”
- “Provider-neutral code means all models behave the same.”

Small design scenario:

You need to add an AI feature to an existing ASP.NET Core order-management application. The assistant should summarize a customer conversation, answer questions from support policy documents, and let authorized users check shipment status. A good design would use `IChatClient` for model calls, an embedding generator and vector store for policy retrieval, normal SQL Server or internal APIs for order data, a narrow shipment-status plugin or function, authorization checks before retrieval and tool execution, prompt versioning, evaluation cases, OpenTelemetry, and cost tracking.

The strong answer keeps the model behind application services instead of letting controllers or prompts own business behavior.

Hands-On Lab

Objective:

Design and scaffold a small ASP.NET Core AI endpoint that can be implemented with `Microsoft.Extensions.AI` and later expanded with Semantic Kernel.

Prerequisites:

- .NET SDK installed.
- Basic ASP.NET Core knowledge.
- Access to a development AI provider or a test double.
- The architecture design from Chapter 20.

Steps:

1. Create a new web API project:

```
dotnet new webapi -n SupportAssistant
cd SupportAssistant
```

2. Add the common AI abstractions package:

```
dotnet add package Microsoft.Extensions.AI
```

3. Add the provider-specific package required by your chosen model provider.
4. Register an `IChatClient` in dependency injection.
5. Create a `SupportDraftService` that depends on `IChatClient`.
6. Add an endpoint such as:

```
POST /support/draft-reply
```

7. Validate input length and reject empty requests.
8. Add structured logging for request ID, prompt version, model name, latency, and validation result.
9. Write tests for:

```
empty input
long input
normal support request
unsafe or policy-breaking request
provider failure
```

10. Sketch the next step: either add RAG for policy documents or add a narrow Semantic Kernel plugin for shipment status.

Expected results:

- The endpoint calls an application service, not the provider directly.
- The service depends on an AI abstraction.
- Inputs are validated.
- AI-provider configuration is outside source code.
- Logs include operational context without sensitive prompt content.
- Tests cover behavior around the AI boundary.

Validation commands:

```
dotnet build
dotnet test
dotnet run
curl -X POST http://localhost:5000/support/draft-reply \
```

```
-H "Content-Type: application/json" \  
-d '{"message": "My order is late. Can someone help?"}'
```

Troubleshooting notes:

- If package APIs differ, check the current provider documentation and keep the application service boundary the same.
- If authentication fails, verify secrets or managed identity configuration.
- If responses are slow, measure model latency separately from application latency.
- If tests are brittle, mock the AI abstraction and evaluate model behavior separately with sample cases.
- If the endpoint leaks sensitive data, fix logging and prompt construction before adding more features.

Knowledge Check

1. Why is `IChatClient` useful in an ASP.NET Core application?
2. When is a direct provider SDK call better than a provider-neutral abstraction?
3. What problem does `IEmbeddingGenerator` solve?
4. How is Semantic Kernel different from `Microsoft.Extensions.AI`?
5. Why should C# plugin methods still enforce authorization?
6. What belongs in application code instead of a prompt?
7. How would you test an application service that depends on an AI model?
8. Why should prompt versions be observable in production?
9. What telemetry would you capture for a RAG endpoint?
10. When should an AI workflow use a background queue instead of a direct HTTP request?

Summary

AI development with .NET is not a separate universe from the rest of modern .NET. It uses the same foundation: ASP.NET Core, dependency injection, configuration, logging, testing, background services, security, deployment, and observability.

`Microsoft.Extensions.AI` provides common abstractions such as chat clients and embedding generators so application code can stay testable and less tightly coupled to one provider. Semantic Kernel sits above that layer when prompts, plugins, tool calls, memory, filters, and agent-style workflows need orchestration.

The implementation habit is to keep AI behind application services. Controllers and endpoints should not become prompt assembly scripts, and prompts should not become business-rule engines. Use models for language work, use C# for authority and validation, use retrieval for trusted context, use tools for approved actions, and use telemetry and evaluations to understand behavior.

The next chapter moves from AI-specific implementation back to broad architecture: monoliths, modular monoliths, microservices, scale, reliability, performance, and cost.

Sources

- Develop .NET apps with AI features
- `Microsoft.Extensions.AI` libraries
- AI apps for .NET developers
- Quickstart: Create a .NET AI app using the AI app template
- Semantic Kernel documentation
- Add native code as a Semantic Kernel plugin
- Semantic Kernel Agent Framework

- [OpenAI API developer quickstart](#)

Further Reading

- [Get started with .NET AI and the Model Context Protocol](#)
- [Microsoft.Extensions.AI API reference](#)
- [Semantic Kernel concepts](#)
- [OpenAI function calling guide](#)
- [OpenAI structured outputs documentation](#)

22

Modern Software Architecture and Scale

Chapter Purpose

Modern software architecture is the discipline of choosing a system shape that fits the business problem, team structure, reliability needs, data model, deployment model, and cost constraints.

The reason this chapter exists near the end of the book is deliberate. It is easy to talk about monoliths, microservices, Kubernetes, caching, and scale as abstract preferences. It is harder, and more useful, to discuss them after you understand ASP.NET Core, data access, authentication, testing, containers, cloud hosting, distributed applications, CI/CD, infrastructure as code, observability, Kubernetes, and AI integration.

Architecture became a central topic in modern .NET because deployment changed. Traditional enterprise applications often ran as one IIS application backed by one SQL Server database. That model could work for years. As teams moved to cloud hosting, containers, elastic capacity, managed services, multiple frontends, background workers, and faster deployment cycles, the question changed from “How do we deploy the application?” to “What shape should the system have so it can change and operate safely?”

Microservices became popular because large organizations needed independent deployment, scaling, and team ownership around business capabilities. But they also added distributed-system complexity. The current state of the art is not “everything should be microservices.” It is fit-for-purpose architecture: modular monoliths when a single deployable unit is enough, microservices when independent ownership and scaling justify the cost, managed cloud services where they simplify operations, and observability everywhere.

This chapter teaches how to choose among monoliths, modular monoliths, microservices, and distributed designs, and how to reason about horizontal scaling, statelessness, caching, performance, reliability, and cost.

Where This Fits

Architecture sits above individual implementation choices and below business strategy.

```

Business goals and constraints
    |
    v
Architecture style
    |
    +-- monolith
    +-- modular monolith
    +-- microservices
    +-- distributed system
    |
    v
Application components
    |

```

```
    +-- ASP.NET Core APIs
    +-- background workers
    +-- SQL Server and other data stores
    +-- queues
    +-- caches
    +-- AI services
    |
    v
Deployment and operations
    |
    +-- cloud hosting
    +-- containers
    +-- CI/CD
    +-- infrastructure as code
    +-- observability
```

Architecture decisions shape how the system is built, deployed, monitored, scaled, and paid for. A microservice boundary is not just a code boundary. It affects source control, builds, deployments, databases, transactions, monitoring, incident response, security, and team ownership.

Chapter 23 will go deeper into specific patterns such as repository, CQRS, event-driven systems, background processing, and domain events. This chapter sets the larger decision frame.

Connection to the Reader's Existing Model

The familiar model is a single enterprise application deployed to IIS with a SQL Server database.

That application might have multiple projects in one Visual Studio solution: web UI, business logic, data access, shared libraries, and tests. It might use stored procedures, scheduled tasks, Windows services, and a load-balanced IIS farm. Scaling often meant adding servers, increasing SQL Server capacity, moving background work elsewhere, or tuning queries.

A monolith is the modern name for a system deployed as one application unit. It is not automatically bad. Many successful systems are monoliths because the business domain, team size, and deployment needs fit a single deployable unit.

A modular monolith is closer to a well-structured enterprise application. It is still deployed as one unit, but the code is divided into clear modules with internal boundaries. Each module owns a business area and limits direct coupling to other modules. Microservices split the application into multiple independently deployable services. Each service owns a business capability and usually owns its data. This can resemble separate IIS applications or Windows services, but the operational expectations are much higher: independent pipelines, service communication, observability, versioning, failure handling, and data consistency.

Horizontal scaling is like adding more IIS servers behind a load balancer. The modern difference is that instances may be containers, platform-managed app instances, Kubernetes pods, or serverless workers.

Stateless applications are like web applications that do not keep important session state in process memory. If any instance can handle the next request, the system can scale and recover more easily.

The analogy breaks down when distributed data enters the picture. In a traditional application, one SQL Server transaction could protect several business changes. In microservices, data may be split across services, so consistency requires events, retries, compensation, idempotency, and careful workflow design.

Layer 1 — Conceptual Model

Architecture is the set of important decisions that are expensive to change later.

It solves these problems:

- organizing code around business capabilities;
- deciding deployment boundaries;

- assigning team ownership;
- scaling the right parts of the system;
- controlling reliability and failure isolation;
- managing data consistency;
- supporting change over time;
- balancing performance, cost, security, and operational complexity.

It does not solve these problems automatically:

- it does not fix unclear requirements;
- it does not make poor code maintainable;
- it does not remove the need for tests;
- it does not make distributed systems simple;
- it does not guarantee performance;
- it does not reduce cost unless cost is designed for;
- it does not replace team communication.

The core model is:

```
Choose boundaries.  
Assign ownership.  
Define communication.  
Control data.  
Observe behavior.  
Evolve deliberately.
```

A good architecture is not the most fashionable shape. It is the shape whose tradeoffs match the workload.

Layer 2 — System Relationships

A monolith has one deployable application boundary. Internal components may be well organized or messy, but deployment happens together. The main inputs are user requests, jobs, messages, files, or API calls. The main outputs are database changes, responses, messages, files, and integrations.

A modular monolith keeps the single deployment boundary but strengthens internal ownership. Modules communicate through explicit interfaces, events, or application services rather than arbitrary shared access. The database may be shared physically, but modules should avoid casually modifying each other's tables.

A microservice architecture creates multiple deployment boundaries. Each service owns a business capability and communicates through APIs, messages, or events. Ideally, each service owns its data, which reduces coupling but makes cross-service workflows harder.

A distributed system is any system where work and state are spread across multiple processes or machines. Microservices are distributed systems, but so are many monoliths that rely on queues, caches, search indexes, cloud storage, AI providers, and third-party APIs.

Ownership boundaries should match change boundaries. If the billing logic, data, team, release cycle, and scaling needs differ from order entry, billing may deserve a separate module or service. If they always change together, a separate microservice may create coordination cost without benefit.

Lifecycle boundaries include build, test, deploy, monitor, scale, version, roll back, and retire. A boundary that cannot be independently built, tested, deployed, and observed is not really an independent service boundary.

Failure boundaries include process crashes, database outages, slow dependencies, network partitions, bad deployments, cache failures, queue backlogs, rate limits, and partial data updates. Architecture decides whether a failure is contained or spreads through the system.

Layer 3 — Core Mechanics

The simplest architecture comparison is:

Monolith

- one deployable unit
- often one primary database
- simple calls in process
- simple deployment
- harder to scale or deploy pieces independently

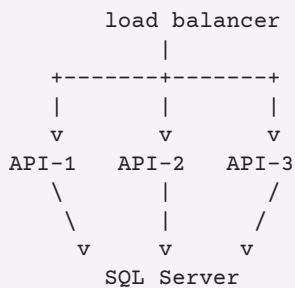
Modular monolith

- one deployable unit
- explicit internal modules
- strong code boundaries
- simpler operations than microservices
- requires discipline to preserve boundaries

Microservices

- many deployable units
- services own capabilities and often data
- independent deployment and scaling
- failure isolation potential
- requires distributed operations maturity

Horizontal scaling means adding more instances of a component instead of only making one instance larger.



This helps when the application tier is the bottleneck. It does not help if all instances are waiting on the same overloaded database.

Statelessness makes horizontal scaling possible. If request state is stored in process memory, the next request must return to the same instance. If state is stored in a database, distributed cache, signed token, queue, or durable workflow, any healthy instance can continue.

Caching stores a copy of data or computation results so later requests can be served faster or with less dependency load.

Common cache locations:

- in-process memory;
- distributed cache such as Redis;
- HTTP response cache;
- CDN or edge cache;
- database query cache;
- application-specific computed cache.

Caching improves performance when reads repeat and stale data is acceptable for a defined period. It creates bugs when invalidation, freshness, ownership, or tenant isolation are unclear.

Performance is the behavior users feel: latency, throughput, responsiveness, and consistency under load. Scaling is one way to improve performance, but query tuning, connection pooling, async I/O, caching, batching, indexing, payload reduction, and removing unnecessary remote calls often matter more.

Reliability is the ability to meet commitments despite failures. Modern reliability assumes failure will happen and designs to reduce impact.

Cost is an architectural constraint. More services, more databases, more clusters, more telemetry, more caches, and more redundancy all have ongoing cost.

Layer 4 — Developer Workflow

Architectural work should start with forces, not diagrams.

```
What must change independently?  
What must scale independently?  
What data must stay consistent?  
Who owns each capability?  
What failures are acceptable?  
What latency is acceptable?  
What can the team operate?  
What can the business afford?
```

For a new .NET application, a conservative workflow is:

1. Start with a clear domain model and use cases.
2. Build a modular monolith unless there is a strong reason not to.
3. Define module boundaries around business capabilities.
4. Keep application services, data access, and external integrations behind explicit interfaces.
5. Add tests around module behavior.
6. Add observability before scale becomes urgent.
7. Extract a service only when independent deployment, scaling, data ownership, or team ownership justifies the cost.

For an existing monolith:

1. Identify pain points with evidence.
2. Find modules that change for different reasons.
3. Remove accidental coupling inside the codebase.
4. Separate read-heavy, background, or integration-heavy workloads where useful.
5. Introduce queues for work that does not need to finish during the request.
6. Improve observability so service extraction can be measured.
7. Extract one boundary at a time.

Useful repository inspection commands:

```
dotnet sln list  
dotnet list package  
dotnet test  
dotnet build
```

Useful architecture questions during code review:

- Does this change cross module boundaries?
- Does this call need to be synchronous?
- Does this data need one transaction?
- Can this operation be retried safely?
- What happens if the dependency is slow?
- What metric or trace would show failure?

The workflow is evolutionary. Architecture is not a poster made once at the beginning. It is a set of decisions that should be revisited as the business, traffic, team, and reliability needs change.

Layer 5 — Production Usage

Production architecture has to survive real traffic, real incidents, and real budgets.

Configuration should be environment-specific and centralized enough to manage safely. In a monolith, configuration may be simpler. In microservices, configuration sprawl becomes a risk unless naming, ownership, secret handling, and rollout are standardized.

Secrets should be owned by the platform and exposed only to components that need them. Service boundaries can reduce blast radius, but only if credentials and permissions are scoped correctly.

Security architecture includes authentication, authorization, network boundaries, API gateways, service-to-service identity, least privilege, audit, data classification, and secure defaults. More services mean more endpoints to protect.

Reliability requires redundancy, timeouts, retries with backoff, circuit breakers, idempotency, graceful degradation, health checks, backups, disaster recovery, and clear ownership during incidents.

Deployment differs by style. A monolith has fewer deployment artifacts but larger releases. A modular monolith can keep deployment simple while improving code ownership. Microservices allow smaller independent deployments, but they require versioning, compatibility testing, deployment orchestration, and rollback strategy.

Observability is mandatory for distributed systems. Logs, metrics, traces, dashboards, and alerts must show user journeys, service health, dependency behavior, deployment versions, and cost signals.

Scaling should target the bottleneck. Scale the web tier when CPU or request concurrency is the problem. Scale reads with replicas, caches, or read models when data access is the bottleneck. Scale background work with queues and workers when processing can be asynchronous.

Persistence is the hardest boundary. A shared database simplifies reporting and transactions but couples services. Separate databases improve ownership but require eventual consistency and integration patterns.

Cost includes compute, storage, databases, network traffic, queues, caches, load balancers, telemetry, CI/CD, environments, licenses, and operations labor. Microservices can optimize cost at scale, but they often increase cost early.

Local development should remain convenient. A system that requires every developer to run twenty services, three databases, a message broker, a cache, and Kubernetes just to change one screen will slow the team unless tooling and defaults are excellent.

Layer 6 — Tradeoffs and Alternatives

Use a monolith when the team is small, the domain is still changing, the system has one primary deployment cadence, data consistency is important, and the operational cost of distributed systems is not justified.

Use a modular monolith when the application is significant enough to need clear boundaries but not yet complex enough to need independently deployable services. This is often the best default for modern business applications.

Use microservices when independent deployment, independent scaling, data ownership, team autonomy, resilience isolation, or technology diversity provides enough value to pay for distributed-system complexity.

Use a distributed architecture deliberately when the workload naturally involves queues, background processing, external APIs, search, caching, AI providers, or cloud-managed services. Do not pretend a system is simple just because the application code is in one repository.

Simpler alternatives include one ASP.NET Core application, one SQL Server database, background services in the same deployment, managed PaaS hosting, and careful modular code.

More advanced alternatives include microservices, event-driven architecture, CQRS, event sourcing, Kubernetes, service mesh, multi-region deployment, serverless workflows, and platform engineering.

Common overengineering mistakes:

- starting with microservices before boundaries are known;
- splitting services while sharing the same database tables casually;
- using queues to hide unclear ownership;
- adding caches without invalidation rules;

- optimizing for theoretical scale before real bottlenecks exist;
- treating Kubernetes as architecture;
- ignoring local developer experience;
- measuring service count instead of business outcomes;
- designing for reliability without funding operations.

Architecture always trades one kind of complexity for another. A monolith centralizes complexity. Microservices distribute it. Modular monoliths try to keep deployment simple while making code boundaries explicit.

Layer 7 — Interview Perspective

Interviewers use architecture questions to test judgment.

Concepts commonly tested:

- monolith;
- modular monolith;
- microservices;
- bounded contexts;
- distributed systems;
- horizontal scaling;
- stateless services;
- caching;
- eventual consistency;
- reliability;
- performance;
- cost;
- operational complexity.

Representative questions:

- “When would you choose a monolith?”
- “What is a modular monolith?”
- “What are the tradeoffs of microservices?”
- “How do you scale an ASP.NET Core API?”
- “Why is statelessness important?”
- “What can go wrong with caching?”
- “How do microservices affect data consistency?”
- “How would you decompose an existing application?”
- “What should you measure before changing architecture?”

A strong answer resists fashion:

“I would start by looking at change boundaries, data ownership, scaling needs, team ownership, reliability goals, and operational maturity. A modular monolith is often the right starting point. I would extract services when a boundary has clear ownership and independent deployment or scaling value, and I would plan for observability, versioning, data consistency, and operations before splitting.”

Common misconceptions:

- “Monolith means bad design.”
- “Microservices are automatically more scalable.”
- “Kubernetes means we have a microservice architecture.”
- “A shared database is harmless if services are separate.”

- “Caching fixes performance without tradeoffs.”
- “Horizontal scaling fixes database bottlenecks.”
- “Architecture is decided once at project kickoff.”

Small design scenario:

An order-management system has one ASP.NET Core application, SQL Server, background invoice generation, shipment integration, and a new AI support assistant. The team has six developers. Deployments happen weekly. The main pain points are slow invoice generation and fragile shipment integration.

A strong design might keep the core application as a modular monolith, move invoice generation to background processing, isolate shipment integration behind an anti-corruption layer or separate worker, add caching only for read-heavy safe data, and improve observability. It would not immediately split orders, customers, invoices, shipments, and AI into five microservices unless ownership, scaling, and deployment needs justify the cost.

Hands-On Lab

Objective:

Evaluate the architecture style for an existing or imagined .NET business application.

Prerequisites:

- Familiarity with ASP.NET Core, SQL Server, cloud hosting, distributed applications, and observability.
- A candidate application to analyze.

Steps:

1. List the main business capabilities:

```
orders
customers
billing
shipment
reporting
support
```

2. For each capability, answer:

```
Who owns it?
How often does it change?
Does it need separate scaling?
Does it own data?
Does it need one transaction with another capability?
What happens if it fails?
```

3. Identify current bottlenecks with evidence:

```
slow requests
SQL waits
deployment conflicts
queue backlog
high error rate
team coordination delay
```

4. Choose an architecture style:

```
monolith
modular monolith
selected service extraction
microservices
```

5. Draw the proposed system.

6. Mark synchronous calls, asynchronous messages, databases, caches, and external dependencies.
7. List reliability risks and how they will be observed.
8. List expected cost increases and cost reductions.
9. Decide what should not change yet.
10. Write one paragraph defending the choice.

Expected results:

- A capability map.
- A recommended architecture style.
- A list of boundaries that should stay together.
- A list of boundaries that may deserve separation.
- A reliability and observability plan.
- A cost tradeoff summary.

Validation commands:

If analyzing an existing .NET solution, use:

```
dotnet sln list
dotnet build
dotnet test
```

If the application is only hypothetical, validation is architectural: the design should explain why its boundaries match business capabilities, data ownership, scaling needs, reliability goals, team ownership, and cost.

Troubleshooting notes:

- If every module needs the same database transaction, microservices may be too early.
- If one background job causes user-facing slowness, separate processing before splitting the whole system.
- If teams frequently block each other, examine ownership boundaries.
- If no one can operate the proposed design, simplify it.
- If the design only says “microservices for scale,” identify the actual bottleneck first.

Knowledge Check

1. Why is a monolith not automatically a bad architecture?
2. How is a modular monolith different from an unstructured monolith?
3. What costs do microservices introduce?
4. Why does data ownership matter when splitting services?
5. How does statelessness support horizontal scaling?
6. Why might adding more API instances fail to improve performance?
7. What risks does caching introduce?
8. When should work move to a background process?
9. How do observability requirements change as systems become distributed?
10. What evidence would justify extracting a service from a monolith?

Summary

Modern architecture is not a contest between old and new labels. It is the practice of choosing boundaries that match the workload. A monolith can be the right answer when one deployable unit fits the team and domain. A modular monolith

improves maintainability while keeping operations simple. Microservices help when independent ownership, deployment, scaling, and failure isolation justify distributed complexity.

Scale also has layers. Horizontal scaling works best when applications are stateless and the bottleneck is the application tier. Caching helps when data can be reused safely. Background processing helps when work does not need to block the user. Reliability comes from designing for failure, not pretending failure can be avoided.

The architectural habit is to ask what should change independently, scale independently, fail independently, and be owned independently. Then choose the simplest system shape that satisfies those forces.

The next chapter narrows from broad architecture styles to common patterns: repository, CQRS, event-driven systems, background processing, and domain events.

Sources

- [Azure Architecture Center: Microservices architecture style](#)
- [Azure Architecture Center: Design patterns for microservices](#)
- [Azure Architecture Center: Cloud design patterns](#)
- [Azure Well-Architected Framework](#)
- [What is the Azure Well-Architected Framework?](#)
- [Martin Fowler: Monolith First](#)

Further Reading

- [Azure Architecture Center](#)
- [Azure Well-Architected Framework reliability pillar](#)
- [Azure Well-Architected Framework performance efficiency pillar](#)
- [Azure Well-Architected Framework cost optimization pillar](#)
- [Martin Fowler: Microservices Guide](#)

23

Common Architectural Patterns

Chapter Purpose

Chapter 22 explained architecture styles: monoliths, modular monoliths, microservices, distributed systems, scale, reliability, performance, and cost. This chapter narrows the focus to patterns that commonly appear inside those architectures.

Architectural patterns exist because the same design problems keep appearing: how to isolate data access, how to separate writes from reads, how to react to business events, how to move slow work out of request paths, and how to represent meaningful changes inside the domain.

Patterns are useful because they give names to recurring solutions. They let a team say “this should be a background job” or “this read model is CQRS” without explaining the whole design from scratch.

Patterns are dangerous when they become ceremony. A repository that only wraps Entity Framework Core without simplifying anything can make code worse. CQRS can be a clean separation or a pile of duplicate models. Events can decouple systems or hide control flow. Background jobs can improve responsiveness or create invisible failure. Domain events can clarify business meaning or turn every property change into noise.

This chapter teaches repository, CQRS, event-driven systems, background processing, and domain events by asking the most important pattern question: what problem is this pattern solving here?

Where This Fits

Patterns sit inside an architecture style.

```
Architecture style
|
+-- monolith
+-- modular monolith
+-- microservices
|
v
Application design patterns
|
+-- repository
+-- CQRS
+-- event-driven communication
+-- background processing
+-- domain events
|
v
Implementation
|
+-- ASP.NET Core endpoints
+-- application services
```

```
+-- EF Core
+-- SQL Server
+-- queues
+-- workers
+-- observability
```

A pattern does not decide the whole architecture. You can use background processing inside a monolith. You can use domain events inside a modular monolith. You can use CQRS without microservices. You can use repositories in some places and direct EF Core queries in others.

Chapter 23 is the final technical architecture chapter before interview and career preparation. It gives the reader vocabulary for architecture discussions without pretending every vocabulary word belongs in every system.

Connection to the Reader's Existing Model

The familiar model is an enterprise .NET application with layers.

You may have built applications with controllers, services, data access classes, stored procedures, DTOs, SQL Server Agent jobs, Windows services, message queues, and scheduled tasks. The names were different, but many of the pressures were the same.

The repository pattern resembles a data access layer. It provides a collection-like abstraction over persistence. The useful part is hiding persistence details when the rest of the application should not know how data is stored or queried.

CQRS resembles separating update procedures from reporting queries. A stored procedure that enforces an order workflow is not the same as a reporting view that joins orders, customers, and shipments for display. CQRS gives that separation an architectural name.

Event-driven systems resemble using SQL triggers, Service Broker, MSMQ, scheduled integration jobs, or publish-subscribe messaging to react after something happens. The modern version usually uses explicit events, brokers, queues, and handlers instead of hidden database behavior.

Background processing resembles Windows services, scheduled tasks, SQL Server Agent jobs, or batch processors. The modern version might be an ASP.NET Core hosted service, a worker service, a queue-triggered function, a containerized worker, or a cloud job.

Domain events resemble writing down that a business fact occurred: `OrderSubmitted`, `PaymentCaptured`, `ShipmentDelayed`, or `CustomerRegistered`. They are not merely technical notifications. They represent something the business cares about.

The analogy breaks down when old mechanisms hide too much. A SQL trigger may run invisibly. A modern event handler should be observable, testable, and owned. A scheduled task may retry by accident. A modern background worker should be idempotent and instrumented.

Layer 1 — Conceptual Model

An architectural pattern is a named solution to a recurring design problem.

Patterns solve these problems:

- giving teams shared vocabulary;
- separating responsibilities;
- making change safer;
- isolating persistence details;
- optimizing reads and writes differently;
- decoupling producers from consumers;
- moving slow work out of user requests;
- representing meaningful business events.

They do not solve these problems automatically:

- they do not replace understanding the domain;
- they do not make code clean by existing;
- they do not remove tradeoffs;
- they do not make distributed systems reliable by default;
- they do not guarantee performance;
- they do not excuse unclear ownership;
- they do not belong everywhere.

The conceptual model is:

```
Pressure appears.  
Pattern may fit.  
Tradeoffs are evaluated.  
Small implementation is tested.  
Operational behavior is observed.  
Pattern is kept only if it pays rent.
```

Patterns should reduce complexity where the system is under pressure. If a pattern adds more concepts than it removes, it is probably too early.

Layer 2 — System Relationships

The repository pattern sits between application logic and persistence. Inputs are domain-oriented queries or operations. Outputs are domain objects, aggregates, DTOs, or result sets. Its dependencies include EF Core, SQL Server, or another store. Its failure boundary is persistence access.

CQRS splits command handling from query handling. Commands change state. Queries read state. The write side owns validation, business rules, and consistency. The read side owns efficient projection and presentation-friendly data. The lifecycle boundary may be small, such as separate classes, or large, such as separate databases.

Event-driven systems connect producers, event channels, and consumers. A producer publishes that something happened. A channel stores or routes the event. Consumers react independently. The ownership boundary is important: the producer owns the meaning of the event, but consumers own their reactions.

Background processing moves work out of the interactive request path. A web request may enqueue work and return quickly. A worker later processes the job, updates state, sends notifications, or calls external systems. The lifecycle boundary includes queueing, retries, dead-letter handling, and status tracking.

Domain events live inside the business model or application layer. They mark important facts that occurred in the domain. They can trigger in-process handlers, outbox messages, notifications, projections, or integration events.

Failure boundaries include repository abstractions that leak provider details, CQRS models that drift, events that are published before database commits, handlers that run twice, queues that grow without visibility, jobs that cannot be retried safely, and domain events that become a dumping ground for every side effect.

Layer 3 — Core Mechanics

The repository pattern is most useful when it expresses domain intent:

```
public interface IOrderRepository  
{  
    Task<Order?> GetForCheckoutAsync(  
        int orderId,  
        CancellationToken cancellationToken);  
  
    Task SaveAsync(  
        Order order,
```

```

        CancellationToken cancellationToken);
    }

```

This is different from a generic wrapper that simply mirrors `DbSet<T>`. Entity Framework Core already provides a unit-of-work-like `DbContext` and set-like collections. A repository should add clarity, not hide useful EF Core features behind weaker copies.

CQRS begins with separating command intent from query intent:

```

public sealed record SubmitOrderCommand(int OrderId, int CustomerId);

public sealed class SubmitOrderHandler
{
    public Task HandleAsync(
        SubmitOrderCommand command,
        CancellationToken cancellationToken)
    {
        // Load order, validate business rules, change state, save.
        throw new NotImplementedException();
    }
}

public sealed record OrderSummaryDto(
    int OrderId,
    string CustomerName,
    string Status,
    DateTimeOffset SubmittedAt);

```

The command side protects business state. The query side can return exactly what the screen or API needs without dragging the whole domain model into the read path.

An event-driven flow has producers, channels, and consumers:

```

Order API
|
| publishes OrderSubmitted
v
Message broker or event channel
|
+--> Billing consumer
+--> Shipment consumer
+--> Notification consumer

```

Background processing often starts with a queue:

```

POST /orders
|
v
Save order
|
v
Enqueue invoice job
|
v
Return response

Worker later:
    read job -> generate invoice -> save result -> publish completion event

```

A domain event names something meaningful:

```

public sealed record OrderSubmitted(
    int OrderId,

```

```
int CustomerId,  
    DateTimeOffset OccurredAt);
```

The event should be past tense because it represents something that already happened. A command asks the system to do something. An event says something happened.

Layer 4 — Developer Workflow

The developer workflow starts by identifying the pressure.

```
Is data access too coupled?  
Consider repository or query objects.  
  
Are reads and writes pulling in opposite directions?  
Consider CQRS.  
  
Does another part of the system need to react?  
Consider an event.  
  
Is work slow or retry-prone?  
Consider background processing.  
  
Did something meaningful happen in the domain?  
Consider a domain event.
```

For repository decisions:

1. Start with direct EF Core when it is clear and local.
2. Add repositories around aggregate persistence or complex persistence rules.
3. Avoid generic repositories that only copy EF Core.
4. Keep query-heavy reporting paths free to use optimized queries or read models.

For CQRS decisions:

1. Separate command classes from query classes first.
2. Keep one database until separate stores are justified.
3. Add projections when queries become expensive or awkward.
4. Add separate read stores only when scale, security, or performance requires it.

For event-driven decisions:

1. Name events in business language.
2. Publish after state changes are durable.
3. Make handlers idempotent.
4. Add correlation IDs and trace context.
5. Monitor queue depth, failures, retries, and dead-letter messages.

For background processing:

1. Decide whether the user needs the result immediately.
2. Store job state or publish progress when users need visibility.
3. Use queues for bursty work.
4. Make jobs safe to retry.
5. Add cancellation, timeout, and failure handling.

Useful .NET commands remain ordinary:

```
dotnet build
dotnet test
dotnet run
```

The more important workflow is review. Each pattern should be reviewed with the question: what would be worse if we removed this pattern?

Layer 5 — Production Usage

Production patterns must be observable and recoverable.

Configuration includes queue names, topic names, retry counts, dead-letter settings, worker concurrency, cache duration, projection refresh intervals, and feature flags.

Secrets include broker credentials, storage credentials, database credentials, and service credentials used by workers. Workers should have the smallest permissions needed for their jobs.

Security includes authorization before commands, authorization-aware queries, tenant-safe read models, protected event payloads, safe handler permissions, and audit trails for business operations.

Reliability depends on idempotency. Background jobs and event handlers may run more than once. A handler should be able to detect that work is already done or apply the same logical change safely.

Deployment must account for versioning. A new event field, command shape, or read model can affect producers and consumers. In distributed systems, old and new versions often run at the same time.

Observability should include command duration, query duration, event publish counts, handler failures, queue depth, retry count, dead-letter messages, projection lag, and correlation IDs across request, event, and worker spans.

Scaling differs by pattern. Queries can often scale with read replicas, materialized views, or caches. Commands scale only as far as consistency rules allow. Workers can scale horizontally when jobs are independent. Event consumers can scale when partitions and ordering rules permit it.

Persistence is central. CQRS may use one database, two schemas, or separate stores. Event-driven systems need to avoid losing events when a database commit succeeds but publishing fails. The outbox pattern is a common answer: store the outgoing event in the same transaction as the business change, then publish it asynchronously.

Cost includes broker usage, extra storage, duplicate read models, worker compute, retries, telemetry, and development time. Pattern cost is not only cloud billing. It is also the mental load every developer carries.

Local development can use in-memory handlers, local containers, or development queues. Production needs durable queues, retries, dead-letter handling, monitoring, alerting, backup, access control, and runbooks.

Layer 6 — Tradeoffs and Alternatives

Use repositories when persistence complexity deserves a domain-oriented boundary, when tests benefit from substituting persistence behavior, or when a module should not know how storage is implemented.

Do not use repositories when they only wrap EF Core with methods such as `Add`, `Update`, `Delete`, and `GetById` without adding meaning.

Use CQRS when read and write needs differ significantly, when commands carry business intent, when read models need different shapes, or when read and write loads scale differently.

Do not use full CQRS with separate stores for simple CRUD screens.

Use event-driven architecture when producers and consumers should be decoupled, when multiple reactions follow one business fact, when work can be asynchronous, or when new consumers may be added without changing producers.

Do not use events when the workflow requires immediate consistency, straight-line user feedback, or simple in-process method calls.

Use background processing when work is slow, bursty, retry-prone, scheduled, CPU-intensive, I/O-intensive, or not required to finish before the user receives a response.

Do not use background jobs when the user must know the result immediately and the operation cannot be represented as pending.

Use domain events when the business cares that something happened and other parts of the system may react.

Do not use domain events for every setter, save operation, or technical notification.

Simpler alternatives include direct EF Core queries, application services, transaction scripts, synchronous method calls, scheduled tasks, and ordinary CRUD endpoints.

More advanced alternatives include event sourcing, sagas, workflow engines, materialized views, separate read stores, streaming platforms, service buses, and process managers.

Common overengineering mistakes:

- adding a generic repository to every entity;
- using CQRS before CRUD becomes painful;
- confusing commands and events;
- publishing events before the transaction commits;
- assuming queues guarantee exactly-once processing;
- building a service bus architecture for two method calls;
- hiding all control flow in handlers;
- adding background jobs without status, retries, or dead-letter handling;
- naming events after technical actions instead of business facts.

The best pattern is often the smallest version of the idea. Separate command and query classes before separate databases. Use an in-process domain event before a broker. Move one slow task to a worker before redesigning the whole system.

Layer 7 — Interview Perspective

Interviewers use pattern questions to test whether you understand tradeoffs.

Concepts commonly tested:

- repository pattern;
- EF Core and repositories;
- CQRS;
- commands versus queries;
- events versus commands;
- event-driven architecture;
- background jobs;
- queues;
- idempotency;
- outbox pattern;
- domain events;
- eventual consistency.

Representative questions:

- “Should every EF Core application use repositories?”
- “What is CQRS?”
- “When would you split read and write models?”
- “How is an event different from a command?”
- “Why do background jobs need idempotency?”
- “What happens if publishing an event fails after the database commit?”
- “When would you use a queue?”
- “What is a domain event?”

- “How do you monitor event-driven systems?”

A strong answer is conditional:

“I would use a repository only when it adds a useful domain boundary over persistence. EF Core already abstracts many data-access details. For CQRS, I would start by separating command and query handlers, then consider separate read models or stores only when performance, scale, or security requires it. For events and background jobs, I would design for retries, idempotency, correlation, and dead-letter handling.”

Common misconceptions:

- “Repository is mandatory with EF Core.”
- “CQRS means separate databases.”
- “Events are just async method calls.”
- “Queues guarantee exactly-once processing.”
- “Background jobs are simpler because users do not see them.”
- “Domain events should be raised for every property change.”
- “Event-driven architecture removes coupling entirely.”

Small design scenario:

An order system accepts orders, charges payments, generates invoices, sends emails, updates a reporting dashboard, and notifies the warehouse. Users need a fast response after submitting an order, but the invoice can be generated later.

A good design might handle `SubmitOrderCommand` synchronously enough to validate and save the order, raise an `OrderSubmitted` domain event, store an outbox message, publish an integration event after commit, process invoice and email work in background workers, and update a read model for reporting. The design should also include idempotency, retries, dead-letter monitoring, and correlation IDs.

The strong answer does not apply every pattern everywhere. It applies each pattern where the pressure exists.

Hands-On Lab

Objective:

Choose appropriate patterns for an order submission workflow.

Prerequisites:

- Familiarity with ASP.NET Core, EF Core, SQL Server, queues, and observability.
- The architecture decision habits from Chapter 22.

Steps:

1. Start with this workflow:

```
User submits order.  
System validates order.  
System saves order.  
System charges payment.  
System generates invoice.  
System sends confirmation email.  
System updates reporting dashboard.
```

2. Mark which steps must happen before the user gets a response.
3. Mark which steps can happen later.
4. Define one command:

```
SubmitOrderCommand
```


5. Define one domain event:

```
OrderSubmitted
```

6. Decide whether the first version needs CQRS with separate stores or only separate command and query handlers.

7. Decide whether a repository adds value over direct EF Core usage.

8. Choose background jobs for slow or retry-prone work.

9. Define what must be idempotent.

10. Define observability:

```
command duration
event publish count
queue depth
handler failures
retry count
dead-letter count
invoice generation duration
email send result
```

Expected results:

- A command definition.
- A domain event definition.
- A decision on repository usage.
- A decision on CQRS scope.
- A background processing plan.
- An idempotency plan.
- An observability checklist.

Validation commands:

If you implement a small prototype:

```
dotnet build
dotnet test
dotnet run
```

If you only design the workflow, validate it by explaining:

- what happens synchronously;
- what happens asynchronously;
- what is retried;
- what happens if a handler fails;
- how duplicate messages are handled;
- how a responder can diagnose a stuck order.

Troubleshooting notes:

- If the user must wait for every step, the workflow may need fewer background jobs or better status messaging.
- If a duplicate message would charge the customer twice, idempotency is missing.
- If an event can be lost after saving the order, consider an outbox.
- If reporting queries are slow, consider a read model before separating the whole service.
- If the pattern explanation is longer than the business workflow, simplify.

Knowledge Check

1. What problem should a repository solve in an EF Core application?
2. Why can a generic repository be harmful?
3. How are commands different from queries in CQRS?
4. Why does CQRS not always require separate databases?
5. How is a domain event different from an integration event?
6. Why should events usually be named in the past tense?
7. What kinds of work belong in background processing?
8. Why must event handlers and background jobs often be idempotent?
9. What does the outbox pattern protect against?
10. How can architectural patterns add unnecessary complexity?

Summary

Architectural patterns are named tools for recurring design pressure. They are not maturity badges. A pattern earns its place when it makes the system easier to change, understand, scale, test, or operate.

Repositories can provide a useful persistence boundary, but EF Core already solves many data-access problems. CQRS separates commands from queries and can grow from separate classes to separate stores when the pressure justifies it. Event-driven systems decouple producers and consumers, but require idempotency, observability, and eventual-consistency thinking. Background processing keeps slow or retry-prone work out of user requests. Domain events capture meaningful business facts.

The practical architecture habit is to begin with the smallest useful version of a pattern. Add complexity only when it removes a more painful complexity from the system.

The next chapter shifts from technical patterns to professional readiness: interviews, system design discussions, coding exercises, portfolio building, certifications, open-source contributions, and staying current in the .NET ecosystem.

Sources

- Azure Architecture Center: CQRS pattern
- Azure Architecture Center: Event-driven architecture style
- Azure Architecture Center: Best practices for background jobs
- Azure Architecture Center: Cloud design patterns
- Martin Fowler: Domain Event

Further Reading

- Azure Architecture Center: Design patterns for microservices
- Azure Architecture Center: Queue-Based Load Leveling pattern
- Azure Architecture Center: Competing Consumers pattern
- Azure Architecture Center: Scheduler Agent Supervisor pattern
- Martin Fowler: Event Sourcing

24

Interview Preparation and Career Roadmap

Chapter Purpose

This book has rebuilt your modern .NET mental model from the platform outward: .NET, ASP.NET Core, data access, security, testing, Linux, containers, cloud hosting, observability, distributed systems, CI/CD, infrastructure as code, Kubernetes, AI, and architecture.

This final chapter answers the practical career question: how do you convert that knowledge into interview performance, portfolio evidence, and a learning plan that keeps working after the book ends?

Interview preparation exists because professional evaluation is not the same as private understanding. You may know how to build a feature, but an interviewer needs to hear how you reason about tradeoffs. You may have years of C# and SQL Server experience, but a modern team may need evidence that you can work with Linux, containers, cloud deployments, CI/CD, observability, and AI-enabled workflows.

The hiring market changes, specific tools change, and certifications change. The durable expectation is broader: employers want developers who can build, ship, diagnose, secure, and evolve production software. For an experienced developer returning to modern .NET, the goal is not to pretend to be a beginner again. The goal is to connect your existing enterprise experience to the way modern systems are built and operated.

This chapter covers technical interview topics, architecture discussions, system design, behavioral expectations, coding exercises, skills employers expect, recommended learning order, portfolio building, certifications, open-source contributions, and staying current with the .NET ecosystem.

Where This Fits

Career readiness sits above the technical stack.

```
Technical knowledge
|
+-- .NET and ASP.NET Core
+-- SQL Server and EF Core
+-- security and authentication
+-- testing and CI/CD
+-- Linux, Docker, and cloud hosting
+-- observability and diagnostics
+-- distributed systems and architecture
+-- AI application development
|
v
Professional evidence
```

```
|
+-- interview explanations
+-- system design tradeoffs
+-- portfolio projects
+-- certifications
+-- open-source contributions
+-- production stories
|
v
Career outcomes
```

The final project after this chapter ties the technical material together. That project should become portfolio evidence: not just code that runs, but a small production-style system you can explain.

Connection to the Reader's Existing Model

You already have professional experience. You know what it means to support a business application, work with SQL Server, debug production issues, deploy to IIS, discuss requirements, and live with the consequences of design decisions.

Modern interviews still value that. What has changed is the vocabulary and the operational surface.

An IIS deployment story becomes a cloud deployment and container story.

Windows Event Viewer and Performance Monitor experience becomes observability, structured logging, metrics, traces, dashboards, and alerts.

SQL Server experience becomes data modeling, EF Core tradeoffs, migrations, query performance, transactions, concurrency, and managed database operations.

Windows services and scheduled tasks become worker services, background jobs, queues, containers, functions, and cloud schedulers.

Application pool scaling becomes stateless application design, horizontal scaling, load balancers, Kubernetes pods, managed app instances, and cache strategy.

The analogy breaks down if you describe old approaches as though nothing has changed. A strong interview answer should say, "In the older IIS model I would have done X; in a modern container or cloud model I would look at Y." That shows continuity and growth.

Layer 1 — Conceptual Model

Interview readiness is the ability to demonstrate judgment under limited time.

It solves these problems:

- translating experience into current terminology;
- explaining tradeoffs clearly;
- showing evidence of hands-on modern practice;
- identifying gaps before interviews;
- preparing for coding and system design exercises;
- discussing production incidents without defensiveness;
- showing learning momentum.

It does not solve these problems automatically:

- it does not replace hands-on practice;
- it does not hide shallow understanding;
- it does not guarantee a role;
- it does not make every certification valuable;

- it does not remove the need for communication skills;
- it does not let you skip fundamentals.

The conceptual model is:

```
Knowledge
+ practice
+ evidence
+ explanation
+ judgment
= interview readiness
```

The strongest candidates do not merely list technologies. They explain why a technology exists, when they would use it, when they would avoid it, and how they would operate it in production.

Layer 2 — System Relationships

Technical interviews evaluate multiple layers at once.

Coding exercises test whether you can write correct, readable code under constraints. For .NET roles, this may include C#, LINQ, async/await, APIs, collections, error handling, unit tests, and basic data structures.

Framework questions test whether you understand ASP.NET Core, dependency injection, configuration, middleware, authentication, authorization, EF Core, logging, and hosting.

Cloud and DevOps questions test whether you understand build pipelines, deployment, containers, environment configuration, secrets, observability, and rollback.

Architecture questions test whether you can reason about boundaries, monoliths, microservices, data consistency, scaling, caching, reliability, cost, and operational complexity.

Behavioral questions test whether you can work with others, learn, handle ambiguity, debug production issues, communicate risk, and take responsibility.

Portfolio evidence supports all of these. A repository with a deployed sample, tests, Docker Compose, CI/CD, authentication, logging, monitoring notes, and a README gives interviewers something concrete to inspect.

Certifications can support credibility, especially for cloud and DevOps roles, but they do not replace project evidence or practical explanation.

Open-source contributions show collaboration, code review, issue discussion, documentation skill, and comfort working in public.

Failure boundaries include overclaiming, memorizing definitions without experience, using outdated examples, being unable to explain tradeoffs, showing a portfolio that cannot run, and treating behavioral questions as less important than technical ones.

Layer 3 — Core Mechanics

A modern .NET technical interview commonly touches these areas:

```
C# fundamentals
ASP.NET Core request pipeline
dependency injection
configuration and options
logging and observability
EF Core and SQL Server
authentication and authorization
unit and integration testing
async programming
HTTP APIs
Docker and containers
```

```
cloud deployment
CI/CD
distributed systems
architecture tradeoffs
AI integration basics
```

A useful answer structure is:

```
State the principle.
Connect it to a familiar example.
Describe the tradeoff.
Name the production concern.
Give a concrete implementation detail.
```

Example:

Question: Why should an ASP.NET Core service be stateless?

Answer:

A stateless service does not depend on in-process memory for important request state. That lets any healthy instance handle the next request, which supports horizontal scaling and easier recovery after restarts. If state is needed, I would store it in SQL Server, a distributed cache, a token, a queue, or another durable system depending on the use case. The tradeoff is that external state adds latency and consistency concerns, so it should be designed deliberately.

System design answers should start with requirements:

```
Users and use cases
traffic and latency
data model
consistency requirements
security requirements
failure tolerance
deployment model
observability
cost constraints
```

Coding exercises should be approached with calm structure:

1. Restate the problem.
2. Ask about edge cases.
3. Choose a simple approach first.
4. Write readable code.
5. Test with normal and boundary cases.
6. Discuss complexity and tradeoffs.
7. Improve only if needed.

Behavioral answers should use specific stories. The STAR shape is useful: situation, task, action, result. For experienced candidates, add reflection: what changed in your judgment afterward?

Layer 4 — Developer Workflow

Your preparation workflow should be evidence-driven.

```
Assess gaps
|
v
Build one production-style project
|
```

```
      v
Practice explanations
      |
      v
Practice coding
      |
      v
Prepare architecture stories
      |
      v
Update resume and portfolio
      |
      v
Interview, learn, adjust
```

Recommended learning order after this book:

1. Strengthen ASP.NET Core and EF Core fundamentals.
2. Build and deploy one API with SQL Server.
3. Add tests, logging, health checks, and CI.
4. Containerize with Docker and Docker Compose.
5. Deploy to a cloud platform.
6. Add authentication and authorization.
7. Add observability with dashboards or documented telemetry.
8. Add a background worker or queue.
9. Add one AI feature with clear boundaries.
10. Write an architecture decision record explaining tradeoffs.

Portfolio project checklist:

- clear README;
- architecture diagram;
- setup instructions;
- local Docker Compose environment;
- ASP.NET Core API;
- SQL Server or compatible development database;
- EF Core migrations;
- authentication or documented security boundary;
- unit and integration tests;
- CI pipeline;
- deployment notes;
- logging and health checks;
- observability notes;
- one background process or asynchronous workflow;
- one AI integration if relevant;
- tradeoff discussion.

Useful commands for portfolio confidence:

```
dotnet build
dotnet test
docker compose up --build
git status --short
```

Practice explaining the project in three lengths:

- 30 seconds: what it is and why it exists;

- 3 minutes: architecture and tradeoffs;
- 15 minutes: code walkthrough, deployment, reliability, and next steps.

Layer 5 — Production Usage

Career preparation has production concerns too.

Configuration means keeping your resume, LinkedIn profile, GitHub profile, and portfolio README consistent. If the resume says you know Docker, the portfolio should prove it.

Secrets matter in portfolio projects. Never commit API keys, passwords, connection strings, private certificates, or cloud credentials. Use environment variables, sample configuration files, and clear setup notes.

Security matters in interviews. Be ready to discuss authentication, authorization, secret storage, least privilege, input validation, dependency updates, and sensitive logging.

Reliability matters in project demos. A portfolio project that builds and runs reliably is stronger than a larger project that requires undocumented steps.

Deployment matters because modern roles often expect developers to understand how code reaches production. Even if you are not a DevOps engineer, know the pipeline shape: restore, build, test, package, deploy, verify, roll back.

Observability matters because interviewers often ask what you would do when the system is slow or failing. Be ready to discuss logs, metrics, traces, health checks, dashboards, alerts, and incident response.

Scaling matters because many system design questions eventually ask what happens when usage grows. Discuss statelessness, database bottlenecks, caching, queues, background workers, read models, horizontal scaling, and cost.

Persistence matters because business systems revolve around data. Be ready to talk about schema design, EF Core, migrations, transactions, concurrency, indexes, backups, and reporting.

Cost matters because senior developers are expected to recognize that engineering choices have business consequences. A solution that is elegant but too expensive may not be a good solution.

Layer 6 — Tradeoffs and Alternatives

Certifications can help when they align with a target role. Azure Developer Associate can support developer roles that deploy and integrate with Azure. Azure Solutions Architect Expert can support architecture-oriented roles, but it expects broader infrastructure and governance knowledge. Azure DevOps Engineer Expert can support roles focused on pipelines, delivery, monitoring, and collaboration.

Do not collect certifications as a substitute for building. A certification can open a conversation. A working project and clear explanations carry the conversation.

Microsoft Applied Skills credentials can be useful for focused, task-based proof around real-world Microsoft technology scenarios. They may be a smaller step than a full role-based certification.

Open-source contributions can help, but they should be chosen thoughtfully. A small documentation fix, failing test reproduction, issue triage note, or targeted bug fix is better than an abandoned large pull request. Read the project's contribution guide before starting.

Portfolio projects are especially useful for career changers, returning developers, and developers whose current job does not expose them to the modern stack. The project should show production thinking, not just a feature demo.

Blogs, talks, and notes can help if they explain what you learned and why a decision was made. They are weaker when they merely repeat tutorial steps.

Common overengineering mistakes:

- building a portfolio project too large to finish;
- adding Kubernetes before the app has tests;
- adding microservices before one deployable API works;

- listing technologies you cannot explain;
- pursuing certifications unrelated to target roles;
- neglecting behavioral interview preparation;
- memorizing system design recipes without tradeoff reasoning;
- hiding older experience instead of connecting it to modern practice.

The best career strategy is cumulative. Build one coherent project, explain it well, add one capability at a time, and keep a visible record of your learning.

Layer 7 — Interview Perspective

Interviewers want to answer three questions:

- Can this person do the work?
- Can this person learn what they do not know?
- Can this person communicate and collaborate under pressure?

Concepts commonly tested:

- C# and async programming;
- ASP.NET Core and HTTP APIs;
- dependency injection and configuration;
- EF Core and SQL Server;
- authentication and authorization;
- testing;
- Docker and deployment;
- CI/CD;
- observability;
- distributed systems;
- caching and queues;
- architecture patterns;
- cloud fundamentals;
- AI integration boundaries.

Representative questions:

- “Walk me through a modern .NET application you built.”
- “How does middleware work in ASP.NET Core?”
- “How would you secure an API?”
- “How do you diagnose a slow production request?”
- “When would you use Docker Compose?”
- “What belongs in CI?”
- “How do you handle secrets?”
- “When would you choose a modular monolith over microservices?”
- “How would you add an AI feature safely?”
- “Tell me about a production issue you handled.”

A strong answer combines experience and current practice:

“In my earlier IIS and SQL Server work, I would start with server logs, Event Viewer, and database diagnostics. In a modern containerized ASP.NET Core system, I would start with centralized logs, metrics, traces, health checks, recent

deployments, and dependency signals. The diagnostic goal is the same, but the evidence has to be emitted before the instance disappears.”

Common misconceptions:

- “Interview prep is memorizing definitions.”
- “Older experience does not count.”
- “Certifications guarantee readiness.”
- “A portfolio project must be huge.”
- “System design answers should immediately pick microservices.”
- “Behavioral questions are filler.”
- “It is bad to admit a gap.”

Small design scenario:

An interviewer asks you to design an order-management system for a mid-sized business. A strong answer begins by asking about users, traffic, consistency, integrations, reporting, security, and deployment expectations. It might propose a modular monolith with ASP.NET Core, SQL Server, EF Core, background workers for invoice and notification work, Docker Compose for local development, CI/CD, cloud hosting, authentication, observability, and a clear path for extracting services later if ownership or scale requires it.

That answer is stronger than jumping immediately to Kubernetes and microservices because it shows judgment.

Hands-On Lab

Objective:

Create a personal interview readiness plan based on the book.

Prerequisites:

- Completion of the technical chapters.
- A target role or role family, such as senior .NET developer, cloud .NET developer, platform-oriented developer, or solution architect.

Steps:

1. Choose a target role.
2. List the top ten skills from job descriptions for that role.
3. Map each skill to chapters in this book.
4. Rate yourself:

```
1 = can define it
2 = can follow a tutorial
3 = can build a small feature
4 = can debug it
5 = can explain tradeoffs
```

5. Pick the three weakest high-value skills.
6. Add those skills to the final project.
7. Write three interview stories:

```
production issue
technical tradeoff
learning a new technology
```

8. Prepare a 3-minute portfolio walkthrough.
9. Choose whether a certification or Applied Skills credential supports your target role.

10. Schedule weekly review time to keep current.

Expected results:

- A target role.
- A skill gap map.
- A focused portfolio plan.
- Three interview stories.
- A certification decision.
- A continuing-learning routine.

Validation commands:

For the portfolio project:

```
dotnet build
dotnet test
docker compose up --build
```

For the career plan, validation is verbal:

- Can you explain your target role?
- Can you show evidence for the skills you claim?
- Can you explain one modern .NET system end to end?
- Can you discuss tradeoffs without reciting buzzwords?
- Can you describe what you are learning next?

Troubleshooting notes:

- If the plan is too broad, reduce it to one target role and one portfolio project.
- If the project does not run, fix setup before adding features.
- If you cannot explain a technology, remove it from the resume until you can.
- If interviews expose repeated gaps, update the learning plan instead of treating the interview as wasted.
- If you feel behind, connect new tools to what you already know. That is the whole point of this book.

Knowledge Check

1. Why is interview readiness different from private understanding?
2. How can older IIS and SQL Server experience be translated into modern .NET interview answers?
3. What should a portfolio project prove beyond “the app runs”?
4. Why should system design answers begin with requirements?
5. When can certifications help, and when are they a distraction?
6. What makes an open-source contribution useful career evidence?
7. Why should behavioral interview stories include reflection?
8. What production topics should a senior .NET developer be able to discuss?
9. How can you keep current without chasing every new tool?
10. What is the next concrete skill you should practice after finishing this book?

Summary

The modern .NET career story is not about abandoning your previous experience. It is about updating the frame. C#, SQL Server, production support, enterprise delivery, and business judgment still matter. The modern stack adds ASP.NET Core

hosting patterns, Linux, containers, cloud services, CI/CD, observability, distributed systems, AI integration, and architecture tradeoffs.

Interviewers listen for judgment. They want to know whether you can build the feature, ship it safely, diagnose it when it fails, secure it, scale it when needed, and explain why you chose one design over another.

The best evidence is a working project and clear explanation. Build a production-style application that shows the stack from this book. Keep the scope small enough to finish. Make it easy to run. Add tests, deployment notes, observability notes, and an architecture explanation. Then practice explaining it in terms of tradeoffs.

From here, the final project gives you the concrete artifact that pulls the book together.

Sources

- Microsoft Learn: Training for .NET
- .NET documentation
- Microsoft Applied Skills
- Microsoft Certified: Azure Solutions Architect Expert
- Exam AZ-400: Designing and Implementing Microsoft DevOps Solutions
- GitHub Docs: Contributing to open source

Further Reading

- Microsoft Learn credentials
- Azure Architecture Center
- Azure Well-Architected Framework
- GitHub Docs: About contributing to GitHub Docs
- Martin Fowler: Microservices Guide